📚 Data Science Engineer Study Handbook

Day 1 — SQL Fundamentals and Query Execution

Sample Database Schema

All examples in these notes use the following schema. Save these CREATE and INSERT statements and run them in any SQL environment (MySQL, PostgreSQL, SQL Server, SQLite) to follow along.

Table: employees

```
CREATE TABLE employees (
    emp_id      INT PRIMARY KEY,
    first_name  VARCHAR(50),
    last_name   VARCHAR(50),
    department  VARCHAR(50),
    salary      DECIMAL(10, 2),
    hire_date   DATE,
    manager_id  INT,
    city        VARCHAR(50),
    commission  DECIMAL(10, 2)
);

INSERT INTO employees VALUES
(1,  'Alice',   'Johnson',  'Engineering',  95000.00, '2019-03-15', NULL,  'New York',    NULL),
(2,  'Bob',     'Smith',    'Engineering',  87000.00, '2020-07-01', 1,     'New York',    NULL),
(3,  'Charlie', 'Brown',    'Sales',        72000.00, '2018-11-20', 5,     'Chicago',     5000.00),
(4,  'Diana',   'Lee',      'Sales',        68000.00, '2021-01-10', 5,     'Chicago',     7500.00),
(5,  'Eve',     'Davis',    'Sales',        91000.00, '2017-06-05', NULL,  'Chicago',     3000.00),
(6,  'Frank',   'Wilson',   'HR',           64000.00, '2022-02-28', 8,     'San Francisco', NULL),
(7,  'Grace',   'Taylor',   'HR',           61000.00, '2023-04-15', 8,     'San Francisco', NULL),
(8,  'Henry',   'Martinez', 'HR',           78000.00, '2016-09-01', NULL,  'San Francisco', NULL),
(9,  'Irene',   'Anderson', 'Engineering',  105000.00,'2015-05-20', 1,     'New York',    NULL),
(10, 'Jack',    'Thomas',   'Marketing',    73000.00, '2020-08-12', NULL,  'Boston',      2000.00),
(11, 'Karen',   'White',    'Marketing',    69000.00, '2021-03-30', 10,    'Boston',      1500.00),
(12, 'Leo',     'Harris',   'Engineering',  92000.00, '2019-10-10', 1,     'New York',    NULL),
(13, 'Mary',    'Clark',    'Sales',        68000.00, '2022-06-18', 5,     'Chicago',     4000.00),
(14, 'Nathan',  'Lewis',    'Engineering',  87000.00, '2021-09-05', 1,     'Boston',      NULL),
(15, 'Olivia',  'Walker',   'HR',           64000.00, '2023-01-20', 8,     NULL,          NULL);
```

Table: orders

```
CREATE TABLE orders (
    order_id     INT PRIMARY KEY,
    customer_id  INT,
    order_date   DATE,
    amount       DECIMAL(10, 2),
    status       VARCHAR(20),
    region       VARCHAR(50)
);

INSERT INTO orders VALUES
(101, 1001, '2024-01-05', 250.00,  'Completed', 'East'),
(102, 1002, '2024-01-07', 450.00,  'Completed', 'West'),
(103, 1001, '2024-01-12', 125.00,  'Cancelled', 'East'),
(104, 1003, '2024-01-15', 800.00,  'Completed', 'North'),
```

```
(105, 1002, '2024-01-20', NULL,    'Pending',   'West'),
(106, 1004, '2024-02-01', 320.00,  'Completed', 'East'),
(107, 1005, '2024-02-05', 670.00,  'Completed', 'South'),
(108, 1003, '2024-02-10', 150.00,  'Cancelled', 'North'),
(109, 1001, '2024-02-15', 990.00,  'Completed', 'East'),
(110, 1006, '2024-02-20', 0.00,    'Completed', 'West'),
(111, 1004, '2024-03-01', 410.00,  'Pending',   'East'),
(112, 1007, '2024-03-05', 550.00,  'Completed', 'South');
```

Quick Reference: Table Contents

employees (15 rows)

emp_id	first_name	last_name	department	salary	hire_date	manager_id	city	commission
1	Alice	Johnson	Engineering	95000.00	2019-03-15	NULL	New York	NULL
2	Bob	Smith	Engineering	87000.00	2020-07-01	1	New York	NULL
3	Charlie	Brown	Sales	72000.00	2018-11-20	5	Chicago	5000.00
4	Diana	Lee	Sales	68000.00	2021-01-10	5	Chicago	7500.00
5	Eve	Davis	Sales	91000.00	2017-06-05	NULL	Chicago	3000.00
6	Frank	Wilson	HR	64000.00	2022-02-28	8	San Francisco	NULL
7	Grace	Taylor	HR	61000.00	2023-04-15	8	San Francisco	NULL
8	Henry	Martinez	HR	78000.00	2016-09-01	NULL	San Francisco	NULL
9	Irene	Anderson	Engineering	105000.00	2015-05-20	1	New York	NULL
10	Jack	Thomas	Marketing	73000.00	2020-08-12	NULL	Boston	2000.00
11	Karen	White	Marketing	69000.00	2021-03-30	10	Boston	1500.00
12	Leo	Harris	Engineering	92000.00	2019-10-10	1	New York	NULL
13	Mary	Clark	Sales	68000.00	2022-06-18	5	Chicago	4000.00
14	Nathan	Lewis	Engineering	87000.00	2021-09-05	1	Boston	NULL
15	Olivia	Walker	HR	64000.00	2023-01-20	8	NULL	NULL

Part 1: SELECT Statement

1.1 Definition

SELECT is the SQL clause that specifies which columns (or expressions) to include in the result set. It is the most fundamental SQL statement and the starting point of every query.

1.2 Intuition

Think of a database table as a spreadsheet. SELECT is you telling the database "I want to see these specific columns." You're not modifying data — you're requesting a view of it.

1.3 Purpose

- Retrieve specific columns instead of the entire table.
- Compute derived values (expressions, calculations).
- Rename columns for readability (aliasing).
- Serve as the projection operation in relational algebra.

1.4 Syntax

```
-- Select specific columns
SELECT column1, column2 FROM table_name;

-- Select all columns
SELECT * FROM table_name;

-- Select with expression
SELECT column1, column2 * 1.1 AS adjusted_value FROM table_name;

-- Select with alias
SELECT first_name AS "First Name", salary AS annual_pay FROM table_name;
```

1.5 Worked Examples

Example 1: Select specific columns

```
SELECT first_name, department, salary
FROM employees;
```

Result (15 rows, showing first 5):

first_name	department	salary
Alice	Engineering	95000.00
Bob	Engineering	87000.00
Charlie	Sales	72000.00
Diana	Sales	68000.00
Eve	Sales	91000.00

Example 2: Select with computed column and alias

```
SELECT
    first_name,
    salary,
    salary * 12 AS annual_salary,
    salary / 160 AS hourly_rate
FROM employees;
```

first_name	salary	annual_salary	hourly_rate
Alice	95000.00	1140000.00	593.75
Bob	87000.00	1044000.00	543.75

Example 3: String concatenation (dialect-dependent)

```
-- MySQL / SQLite
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;

-- PostgreSQL
SELECT first_name || ' ' || last_name AS full_name FROM employees;

-- SQL Server
SELECT first_name + ' ' + last_name AS full_name FROM employees;
```

1.6 SELECT * — When to Use and When Not To

Scenario	Use SELECT *?	Reason
Quick exploration / debugging	☑ Yes	Convenient for ad-hoc investigation
Production queries	❌ No	Fragile if schema changes; wastes bandwidth
Application code	❌ No	Unpredictable column order; performance overhead
Views	❌ No	Schema changes may silently break the view

1.7 Common Mistakes

- **Using SELECT * in production:** If a column is added or removed, queries and application code may break silently.
- **Forgetting aliases for expressions:** `SELECT salary * 12 FROM employees` returns a column with an auto-generated name (varies by DBMS), making results hard to use.
- **Quoting aliases inconsistently:** Use double quotes (`"Full Name"`) for aliases with spaces in standard SQL; SQL Server allows square brackets (`[Full Name]`); MySQL allows backticks (`` `Full Name` ``).

1.8 Interview Explanation

> *"SELECT specifies the columns or expressions to include in the result set. It's the projection operation — it determines the shape of the output. In production, I always name specific columns rather than using SELECT * to*

> *avoid fragile dependencies on table schema and to reduce network overhead."*

Part 2: DISTINCT

2.1 Definition

`DISTINCT` is a keyword used with `SELECT` that eliminates duplicate rows from the result set. Two rows are considered duplicates if all selected columns have identical values.

2.2 Intuition

If you ask "which departments exist in the company?" you want each department listed once, not repeated for every employee. DISTINCT removes the repeats.

2.3 Purpose

- Identify unique values in a column or combination of columns.
- Data profiling: understanding the cardinality (number of unique values) of a column.
- Deduplication in reporting.

2.4 Syntax

```
SELECT DISTINCT column1 FROM table_name;
SELECT DISTINCT column1, column2 FROM table_name;
-- DISTINCT applies to the ENTIRE row of selected columns
```

2.5 Worked Examples

Example 1: Unique departments

```
SELECT DISTINCT department FROM employees;
```

department
Engineering
Sales
HR
Marketing

4 rows returned (from 15 total employees).

Example 2: Unique department-city combinations

```
SELECT DISTINCT department, city FROM employees;
```

department	city
Engineering	New York
Engineering	Boston

Sales	Chicago
HR	San Francisco
HR	NULL
Marketing	Boston

6 rows. Note: DISTINCT treats `(HR, San Francisco)` and `(HR, NULL)` as different because the city values differ. Also note that DISTINCT considers two NULLs as equal to each other for deduplication purposes (this is an exception to the general rule that NULL ≠ NULL).

2.6 DISTINCT vs GROUP BY

Both can produce unique values, but they serve different purposes:

Aspect	DISTINCT	GROUP BY
Primary purpose	Remove duplicate rows	Create groups for aggregation
Can aggregate?	No (just deduplicates)	Yes (used with COUNT, SUM, etc.)
Performance	Typically similar for simple dedup	Same or better when combined with aggregation
Readability	Clearer intent for "unique values"	Clearer intent for "aggregated groups"

```
-- These produce identical results for listing unique departments:
SELECT DISTINCT department FROM employees;
SELECT department FROM employees GROUP BY department;
```

2.7 Common Mistakes

- **Thinking DISTINCT applies per column:** `SELECT DISTINCT dept, city` does NOT return distinct departments AND distinct cities separately — it returns distinct (dept, city) pairs.
- **Using DISTINCT to hide a JOIN problem:** If a JOIN produces unexpected duplicates, the fix is to correct the JOIN logic, not to slap on DISTINCT.
- **Performance trap:** DISTINCT requires sorting or hashing all rows. On large tables without proper indexing, it can be very expensive.

2.8 Interview Explanation

> *"DISTINCT eliminates duplicate rows from the result set based on all selected columns. It's useful for data profiling and reporting unique values. However, if I find myself needing DISTINCT to 'fix' duplicate results from a JOIN, that's usually a signal that my JOIN logic needs correction rather than masking with DISTINCT."*

Part 3: WHERE Clause

3.1 Definition

`WHERE` is the clause that filters rows from the base table(s) before any grouping or aggregation occurs. Only rows that satisfy the condition(s) are passed to subsequent processing stages.

3.2 Intuition

WHERE is your filter. Before the database does anything else with the data, it checks each row: "Does this row meet the condition?" If yes, keep it. If no, discard it.

3.3 Purpose

- Filter rows based on column values, comparisons, patterns, or ranges.
- Reduce the data the database needs to process (improves performance).
- Implement business logic (e.g., "only active customers," "orders above $500").

3.4 Syntax and Operators

```
SELECT columns FROM table WHERE condition;
```

Comparison operators:

Operator	Meaning	Example
`=`	Equal to	`WHERE department = 'Sales'`
`<>` or `!=`	Not equal to	`WHERE status <> 'Cancelled'`
`<`	Less than	`WHERE salary < 70000`
`>`	Greater than	`WHERE salary > 90000`
`<=`	Less than or equal	`WHERE hire_date <= '2020-12-31'`
`>=`	Greater than or equal	`WHERE amount >= 500`

Logical operators:

Operator	Meaning	Example
`AND`	Both conditions true	`WHERE dept = 'Sales' AND salary > 70000`
`OR`	Either condition true	`WHERE dept = 'Sales' OR dept = 'HR'`
`NOT`	Negates condition	`WHERE NOT department = 'HR'`

3.5 Operator Precedence

SQL evaluates logical operators in this order:

- `NOT` (highest precedence)
- `AND`
- `OR` (lowest precedence)

Critical interview trap:

```
-- What does this return?
SELECT * FROM employees
WHERE department = 'Sales' OR department = 'HR' AND salary > 70000;
```

Because `AND` binds tighter than `OR`, this is evaluated as:

```
WHERE department = 'Sales' OR (department = 'HR' AND salary > 70000)
```

This returns **all** Sales employees (regardless of salary) plus only HR employees earning above 70000.

Fix with parentheses:

```
WHERE (department = 'Sales' OR department = 'HR') AND salary > 70000
```

This returns employees from Sales or HR who also earn above 70000.

3.6 Worked Examples

Example 1: Simple filter

```
SELECT first_name, department, salary
FROM employees
WHERE department = 'Engineering';
```

first_name	department	salary
Alice	Engineering	95000.00
Bob	Engineering	87000.00
Irene	Engineering	105000.00
Leo	Engineering	92000.00
Nathan	Engineering	87000.00

Example 2: Multiple conditions with AND

```
SELECT first_name, salary, department
FROM employees
WHERE department = 'Sales' AND salary >= 70000;
```

first_name	salary	department
Charlie	72000.00	Sales
Eve	91000.00	Sales

Example 3: OR with parentheses

```
SELECT first_name, department, salary
FROM employees
WHERE (department = 'HR' OR department = 'Marketing') AND salary > 65000;
```

first_name	department	salary

Henry	HR	78000.00
Jack	Marketing	73000.00
Karen	Marketing	69000.00

3.7 Common Mistakes

- **Forgetting operator precedence (AND before OR):** Always use parentheses to make intent explicit.
- **Using = NULL instead of IS NULL:** `WHERE commission = NULL` always evaluates to UNKNOWN (returns no rows). Use `WHERE commission IS NULL`.
- **String comparison case sensitivity:** Behavior varies by DBMS and collation. MySQL with `utf8_general_ci` collation is case-insensitive; PostgreSQL is case-sensitive by default.

Part 4: ORDER BY

4.1 Definition

`ORDER BY` sorts the result set by one or more columns or expressions, in ascending (`ASC`, the default) or descending (`DESC`) order. It is the **only** clause that guarantees the order of rows in the output.

4.2 Key Fact for Interviews

> *Without ORDER BY, SQL does NOT guarantee any particular row order. Even if results appear sorted, that's an implementation artifact, not a guarantee. Relying on implicit ordering is a bug.*

4.3 Syntax

```
SELECT columns FROM table
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];

-- Order by column position (not recommended for production)
SELECT first_name, salary FROM employees ORDER BY 2 DESC;

-- Order by alias
SELECT first_name, salary * 12 AS annual FROM employees ORDER BY annual DESC;

-- Order by expression
SELECT first_name, salary FROM employees ORDER BY salary + COALESCE(commission, 0) DESC;
```

4.4 Worked Examples

Example 1: Single column sort

```
SELECT first_name, salary FROM employees ORDER BY salary DESC;
```

first_name	salary
Irene	105000.00
Alice	95000.00

Leo	92000.00
Eve	91000.00
Bob	87000.00
Nathan	87000.00
...	...

Example 2: Multi-column sort

```
SELECT department, first_name, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

This sorts by department alphabetically first, then within each department by salary highest-first:

department	first_name	salary
Engineering	Irene	105000.00
Engineering	Alice	95000.00
Engineering	Leo	92000.00
Engineering	Bob	87000.00
Engineering	Nathan	87000.00
HR	Henry	78000.00
HR	Frank	64000.00
HR	Olivia	64000.00
HR	Grace	61000.00
Marketing	Jack	73000.00
Marketing	Karen	69000.00
Sales	Eve	91000.00
Sales	Charlie	72000.00
Sales	Diana	68000.00
Sales	Mary	68000.00

4.5 NULL Sorting Behavior

NULLs sort differently across databases:

DBMS	NULL position in ASC	NULL position in DESC

MySQL	First (smallest)	Last
SQL Server	First (smallest)	Last
PostgreSQL	Last (largest)	First
Oracle	Last (largest)	First

PostgreSQL and Oracle support `NULLS FIRST` / `NULLS LAST`:

```
SELECT first_name, commission FROM employees
ORDER BY commission DESC NULLS LAST;
```

4.6 Common Mistakes

- **Assuming result order without ORDER BY:** Never do this. The optimizer can return rows in any order.
- **Sorting by column position in production:** `ORDER BY 2` is fragile — if columns are reordered in the SELECT list, the sort changes silently.
- **Performance on large tables:** ORDER BY requires sorting. On millions of rows, sorting is expensive unless an index covers the sort column(s).

4.7 Interview Explanation

> *"ORDER BY is the only SQL clause that guarantees row order in the result set. Without it, results may appear sorted incidentally but that's not reliable. I always specify ASC or DESC explicitly for clarity, and I'm aware that NULL sort behavior varies by database — PostgreSQL puts NULLs last in ASC, while MySQL puts them first."*

Part 5: LIMIT and TOP

5.1 Definition

`LIMIT` (MySQL, PostgreSQL, SQLite) and `TOP` (SQL Server) restrict the number of rows returned by a query. They are applied **after** sorting (if ORDER BY is present).

5.2 Purpose

- Pagination: showing results page by page.
- "Top N" queries: highest salaries, most recent orders.
- Performance: quickly sample data from a large table.
- Preventing accidental full-table dumps.

5.3 Syntax by Dialect

```
-- MySQL / PostgreSQL / SQLite
SELECT columns FROM table ORDER BY col LIMIT n;
SELECT columns FROM table ORDER BY col LIMIT n OFFSET m;

-- SQL Server
SELECT TOP n columns FROM table ORDER BY col;
SELECT TOP n PERCENT columns FROM table ORDER BY col;
```

```
-- Oracle (12c+)
SELECT columns FROM table ORDER BY col FETCH FIRST n ROWS ONLY;

-- Standard SQL (ANSI SQL:2008)
SELECT columns FROM table ORDER BY col
OFFSET m ROWS FETCH NEXT n ROWS ONLY;
```

5.4 Worked Examples

Example 1: Top 3 highest-paid employees

```
-- MySQL / PostgreSQL
SELECT first_name, salary FROM employees ORDER BY salary DESC LIMIT 3;

-- SQL Server
SELECT TOP 3 first_name, salary FROM employees ORDER BY salary DESC;
```

first_name	salary
Irene	105000.00
Alice	95000.00
Leo	92000.00

Example 2: Pagination — rows 6–10

```
-- MySQL / PostgreSQL: OFFSET is 0-based
SELECT first_name, salary FROM employees
ORDER BY salary DESC
LIMIT 5 OFFSET 5;
```

first_name	salary
Nathan	87000.00
Henry	78000.00
Jack	73000.00
Charlie	72000.00
Karen	69000.00

5.5 Common Mistakes

- **Using LIMIT without ORDER BY:** The rows returned are non-deterministic. You might get different results on different runs.
- **OFFSET performance on large datasets:** `LIMIT 10 OFFSET 1000000` forces the database to scan and skip 1 million rows. For deep pagination, use keyset (seek) pagination instead.
- **Ties in Top-N:** If the 3rd and 4th employees have the same salary, `LIMIT 3` arbitrarily excludes one. SQL Server's `TOP 3 WITH TIES` handles this; standard SQL uses window functions.

5.6 LIMIT vs TOP Comparison

Feature	LIMIT (MySQL/PG)	TOP (SQL Server)
Position in query	End of query	After SELECT keyword
OFFSET support	Built-in: `LIMIT n OFFSET m`	Requires `OFFSET...FETCH` (SQL Server 2012+)
WITH TIES	Not available	`TOP n WITH TIES`
Percentage	Not available	`TOP n PERCENT`

Part 6: BETWEEN, IN, LIKE

6.1 BETWEEN

Definition: Tests whether a value falls within an inclusive range.

```
-- Syntax
WHERE column BETWEEN low AND high
-- Equivalent to:
WHERE column >= low AND column <= high
```

Critical: BETWEEN is INCLUSIVE on both ends.

Example:

```
SELECT first_name, salary FROM employees
WHERE salary BETWEEN 70000 AND 90000;
```

first_name	salary
Bob	87000.00
Charlie	72000.00
Henry	78000.00
Jack	73000.00
Nathan	87000.00

5 rows. (70000 ≤ salary ≤ 90000. Leo's 92000 > 90000, so excluded.)

Date BETWEEN example:

```
SELECT first_name, hire_date FROM employees
WHERE hire_date BETWEEN '2020-01-01' AND '2021-12-31';
```

first_name	hire_date
Bob	2020-07-01
Diana	2021-01-10

Jack	2020-08-12
Karen	2021-03-30
Nathan	2021-09-05

Common mistake with date BETWEEN:

```
-- DANGER: If hire_date includes time component
WHERE hire_date BETWEEN '2020-01-01' AND '2020-01-31'
-- This EXCLUDES '2020-01-31 15:30:00' because it's > '2020-01-31 00:00:00'
-- Fix: use < for the upper bound
WHERE hire_date >= '2020-01-01' AND hire_date < '2020-02-01'
```

6.2 IN

Definition: Tests whether a value matches any value in a specified list or subquery result.

```
WHERE column IN (value1, value2, value3)
-- Equivalent to:
WHERE column = value1 OR column = value2 OR column = value3
```

Example:

```
SELECT first_name, department FROM employees
WHERE department IN ('Sales', 'Marketing');
```

first_name	department
Charlie	Sales
Diana	Sales
Eve	Sales
Jack	Marketing
Karen	Marketing
Mary	Sales

NOT IN — beware of NULLs:

```
-- This returns nothing if the subquery/list contains NULL!
SELECT * FROM employees WHERE department NOT IN ('Sales', NULL);
```

Why? `NOT IN` expands to `dept <> 'Sales' AND dept <> NULL`. Since `dept <> NULL` evaluates to UNKNOWN, the entire AND chain becomes UNKNOWN, and no rows are returned.

Fix: Use `NOT IN` only with NULL-free lists, or use `NOT EXISTS` instead.

6.3 LIKE

Definition: Pattern matching for string columns using wildcard characters.

Wildcard	Meaning	Example
`%`	Zero or more characters	`'A%'` matches Alice, Anderson
`_`	Exactly one character	`'_ob'` matches Bob

```
-- Names starting with 'A'
SELECT first_name FROM employees WHERE first_name LIKE 'A%';
-- Result: Alice

-- Names containing 'ar'
SELECT first_name FROM employees WHERE first_name LIKE '%ar%';
-- Result: Charlie, Karen, Mary

-- Names with exactly 3 characters
SELECT first_name FROM employees WHERE first_name LIKE '___';
-- Result: Bob, Eve, Leo

-- Escape a literal %
SELECT * FROM products WHERE name LIKE '%\%%' ESCAPE '\';
```

Performance warning: `LIKE '%text'` (leading wildcard) cannot use an index and forces a full table scan. Only `LIKE 'text%'` (trailing wildcard) can leverage a B-tree index.

6.4 BETWEEN vs IN vs LIKE Comparison

Operator	Use Case	Can Use Index?	NULL Behavior
BETWEEN	Continuous ranges	Yes (range scan)	NULL BETWEEN x AND y → UNKNOWN
IN	Discrete set of values	Yes (multiple equality checks)	NULL IN (...) → UNKNOWN
LIKE	Pattern matching	Only with trailing wildcard	NULL LIKE '%x' → UNKNOWN

Part 7: NULL Handling

7.1 Definition

`NULL` in SQL represents the **absence of a value** — not zero, not an empty string, not false. It means "unknown" or "not applicable."

7.2 The Three-Valued Logic

SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN. Any comparison involving NULL yields UNKNOWN.

```
NULL = NULL     → UNKNOWN  (NOT TRUE!)
NULL <> NULL    → UNKNOWN
NULL > 5        → UNKNOWN
NULL = 0        → UNKNOWN
NULL = ''       → UNKNOWN
```

This is the single most important concept for SQL interviews regarding NULLs.

7.3 Testing for NULL

```
-- Correct
WHERE column IS NULL
WHERE column IS NOT NULL

-- WRONG (always returns no rows)
```

```
WHERE column = NULL
WHERE column <> NULL
```

7.4 NULL in Boolean Logic

Expression	Result
TRUE AND UNKNOWN	UNKNOWN
FALSE AND UNKNOWN	FALSE
TRUE OR UNKNOWN	TRUE
FALSE OR UNKNOWN	UNKNOWN
NOT UNKNOWN	UNKNOWN

7.5 COALESCE and IFNULL/ISNULL

COALESCE returns the first non-NULL value from its arguments. It is ANSI standard and works in all major databases.

```
SELECT
    first_name,
    commission,
    COALESCE(commission, 0) AS effective_commission
FROM employees;
```

first_name	commission	effective_commission
Alice	NULL	0.00
Bob	NULL	0.00
Charlie	5000.00	5000.00
Diana	7500.00	7500.00
Eve	3000.00	3000.00

Dialect-specific alternatives:

```
-- MySQL
IFNULL(commission, 0)

-- SQL Server
ISNULL(commission, 0)

-- Oracle
NVL(commission, 0)

-- COALESCE (standard, all databases)
COALESCE(commission, 0)
```

COALESCE with multiple fallbacks:

```
SELECT COALESCE(phone_work, phone_home, phone_mobile, 'No phone') AS contact_number
FROM contacts;
```

7.6 NULL in Aggregations

Function	NULL Behavior
COUNT(*)	Counts all rows including NULLs
COUNT(column)	Counts only non-NULL values
SUM(column)	Ignores NULLs
AVG(column)	Ignores NULLs (divides by non-NULL count)
MIN/MAX	Ignore NULLs

Critical interview example:

```
SELECT
    COUNT(*) AS total_rows,
    COUNT(commission) AS with_commission,
    AVG(commission) AS avg_commission
FROM employees;
```

Non-NULL commissions from the data:

- Charlie: 5000, Diana: 7500, Eve: 3000, Jack: 2000, Karen: 1500, Mary: 4000
- Count = 6, Sum = 23000

total_rows	with_commission	avg_commission
15	6	3833.33

The trap: AVG ignores NULLs. If you want to treat NULLs as 0, use `AVG(COALESCE(commission, 0))` which gives 23000/15 = 1533.33 — a very different answer!

7.7 Common Mistakes

- **Using = NULL instead of IS NULL:** The most common beginner mistake.
- **Forgetting NULL in NOT IN:** `WHERE x NOT IN (1, 2, NULL)` returns nothing.
- **AVG ignoring NULLs:** Can lead to misleading business metrics if not accounted for.
- **Concatenation with NULL:** In most databases, `'Hello' || NULL` = NULL (not 'Hello').
- **ORDER BY with NULLs:** Different databases sort NULLs differently (see ORDER BY section).

7.8 Interview Explanation

> *"NULL represents the absence of a value, not zero or empty string. SQL uses three-valued logic where any comparison with NULL yields UNKNOWN, which is why we must use IS NULL / IS NOT NULL rather than = NULL. Aggregate functions like SUM, AVG, MIN, MAX ignore NULLs, while COUNT() counts all rows but COUNT(column) counts only non-NULL values. This distinction between COUNT() and COUNT(column) is a common source of incorrect results in analytics."*

Part 8: CASE WHEN

8.1 Definition

`CASE WHEN` is SQL's conditional expression. It evaluates conditions in order and returns a value for the first condition that is true. It is analogous to `if/elif/else` in Python.

8.2 Syntax

Simple CASE (compares one expression to multiple values):

```
CASE expression
    WHEN value1 THEN result1
    WHEN value2 THEN result2
    ELSE default_result
END
```

Searched CASE (evaluates independent boolean conditions):

```
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    ELSE default_result
END
```

8.3 Worked Examples

Example 1: Salary bands

```
SELECT
    first_name,
    salary,
    CASE
        WHEN salary >= 100000 THEN 'Senior'
        WHEN salary >= 80000  THEN 'Mid-Level'
        WHEN salary >= 65000  THEN 'Junior'
        ELSE 'Entry'
    END AS salary_band
FROM employees
ORDER BY salary DESC;
```

first_name	salary	salary_band
Irene	105000.00	Senior
Alice	95000.00	Mid-Level
Leo	92000.00	Mid-Level
Eve	91000.00	Mid-Level
Bob	87000.00	Mid-Level
Nathan	87000.00	Mid-Level
Henry	78000.00	Junior
Jack	73000.00	Junior
Charlie	72000.00	Junior
Karen	69000.00	Junior

Diana	68000.00	Junior
Mary	68000.00	Junior
Frank	64000.00	Entry
Olivia	64000.00	Entry
Grace	61000.00	Entry

Example 2: CASE with aggregation (pivot-style counting)

```
SELECT
    department,
    COUNT(*) AS total,
    COUNT(CASE WHEN salary >= 80000 THEN 1 END) AS high_earners,
    COUNT(CASE WHEN salary < 80000 THEN 1 END) AS standard_earners
FROM employees
GROUP BY department;
```

department	total	high_earners	standard_earners
Engineering	5	4	1
HR	4	0	4
Marketing	2	0	2
Sales	4	1	3

How this works: `CASE WHEN salary >= 80000 THEN 1 END` returns 1 for matching rows and NULL otherwise. `COUNT(...)` counts non-NULL values, effectively counting only the matching rows.

Example 3: CASE in ORDER BY

```
-- Custom sort: Sales first, then Engineering, then everything else
SELECT first_name, department FROM employees
ORDER BY
    CASE department
        WHEN 'Sales' THEN 1
        WHEN 'Engineering' THEN 2
        ELSE 3
    END;
```

Example 4: CASE for NULL-safe comparison

```
SELECT
    first_name,
    salary + COALESCE(commission, 0) AS total_compensation,
    CASE
        WHEN commission IS NOT NULL THEN 'Commission-based'
        ELSE 'Fixed salary'
    END AS pay_type
FROM employees;
```

8.4 Nested CASE

```
SELECT
    first_name, department, salary,
    CASE
        WHEN department = 'Engineering' THEN
```

```
            CASE
                WHEN salary > 100000 THEN 'Principal Engineer'
                WHEN salary > 85000  THEN 'Senior Engineer'
                ELSE 'Engineer'
            END
        WHEN department = 'Sales' THEN
            CASE
                WHEN salary > 80000 THEN 'Senior Sales'
                ELSE 'Sales Rep'
            END
        ELSE 'General Staff'
    END AS title
FROM employees;
```

While nested CASE works, deeply nested conditions reduce readability. Consider using a lookup table for complex mappings in production.

8.5 Common Mistakes

- **Missing ELSE:** If no WHEN condition matches and ELSE is omitted, the result is NULL (not an error). Always include ELSE unless NULL is the intended default.
- **Condition order matters:** Conditions are evaluated top-to-bottom, and the first match wins. If you put `WHEN salary >= 65000` before `WHEN salary >= 100000`, everything above 65000 gets caught by the first condition.
- **Using CASE WHERE instead of CASE WHEN:** Common syntax error.

8.6 Interview Explanation

> *"CASE WHEN is SQL's conditional expression — the equivalent of if/else in programming. It evaluates conditions sequentially, returns the result for the first true condition, and falls through to ELSE if none match. I use it for data categorization, conditional aggregation, pivot-style transformations, and custom sorting. A key detail is that condition order matters — the first matching WHEN wins."*

Part 9: Aggregate Functions

9.1 Overview

Aggregate functions compute a single result from a set of rows. They are the foundation of data analysis in SQL.

Function	Returns	NULL Handling
`COUNT(*)`	Number of rows	Counts ALL rows (incl. NULLs)
`COUNT(col)`	Number of non-NULL values	Ignores NULLs
`COUNT(DISTINCT col)`	Number of unique non-NULL values	Ignores NULLs
`SUM(col)`	Total of values	Ignores NULLs
`AVG(col)`	Mean of values	Ignores NULLs (divides by non-NULL count)
`MIN(col)`	Smallest value	Ignores NULLs

`MAX(col)`	Largest value	Ignores NULLs

9.2 Worked Examples

Example 1: Basic aggregation

```
SELECT
    COUNT(*) AS total_employees,
    COUNT(commission) AS employees_with_commission,
    COUNT(DISTINCT department) AS num_departments,
    SUM(salary) AS total_payroll,
    AVG(salary) AS avg_salary,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary
FROM employees;
```

Calculations:

- COUNT(*) = 15
- COUNT(commission) = 6 (Charlie, Diana, Eve, Jack, Karen, Mary)
- COUNT(DISTINCT department) = 4 (Engineering, Sales, HR, Marketing)
- SUM(salary) = 95000+87000+72000+68000+91000+64000+61000+78000+105000+73000+69000+92000+68000+87000+64000 = 1,174,000
- AVG(salary) = 1,174,000/15 = 78,266.67
- MIN(salary) = 61000
- MAX(salary) = 105000

total_employees	employees_with_commission	num_departments	total_payroll	avg_salary	min_salary	max_salary
15	6	4	1174000.00	78266.67	61000.00	105000.00

Example 2: Aggregation with GROUP BY

```
SELECT
    department,
    COUNT(*) AS headcount,
    ROUND(AVG(salary), 2) AS avg_salary,
    SUM(salary) AS dept_payroll
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

department	headcount	avg_salary	dept_payroll
Engineering	5	93200.00	466000.00
Sales	4	74750.00	299000.00
Marketing	2	71000.00	142000.00
HR	4	66750.00	267000.00

9.3 COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)

This distinction comes up constantly in interviews:

```
SELECT
    COUNT(*) AS count_star,              -- 15 (all rows)
    COUNT(commission) AS count_col,      -- 6  (non-NULL commissions)
    COUNT(DISTINCT city) AS count_dist   -- 4  (New York, Chicago, San Francisco, Boston — NULL
excluded)
FROM employees;
```

count_star	count_col	count_dist
15	6	4

Key insight: `COUNT(DISTINCT city)` returns 4, not 5, because Olivia's NULL city is not counted. NULLs are excluded from COUNT(column) and COUNT(DISTINCT column), but NOT from COUNT(*).

9.4 SUM and AVG with NULLs — The Danger

```
-- Scenario: What is the average commission?

-- Approach 1: AVG ignores NULLs
SELECT AVG(commission) FROM employees;
-- = (5000+7500+3000+2000+1500+4000) / 6 = 23000/6 = 3833.33

-- Approach 2: Treat NULL as 0
SELECT AVG(COALESCE(commission, 0)) FROM employees;
-- = 23000 / 15 = 1533.33
```

Business implication: Approach 1 answers "What is the average commission among those who earn commission?" Approach 2 answers "What is the average commission across all employees?" The correct choice depends on the business question.

9.5 Aggregation Rule: SELECT and GROUP BY

Rule: When using GROUP BY, every column in SELECT must either:

- Be in the GROUP BY clause, OR
- Be inside an aggregate function.

```
-- ☑ Correct
SELECT department, AVG(salary) FROM employees GROUP BY department;

-- ✗ Error (first_name is not grouped or aggregated)
SELECT department, first_name, AVG(salary) FROM employees GROUP BY department;
-- MySQL with ONLY_FULL_GROUP_BY disabled will run this but return arbitrary first_name values!
```

9.6 SUM of Empty Set vs SUM of NULLs

```
-- No matching rows
SELECT SUM(salary) FROM employees WHERE department = 'Legal';
-- Result: NULL (not 0!)

-- All matching values are NULL
SELECT SUM(commission) FROM employees WHERE department = 'Engineering';
-- Result: NULL (all Engineering commissions are NULL)

-- Safe version
SELECT COALESCE(SUM(salary), 0) FROM employees WHERE department = 'Legal';
-- Result: 0
```

9.7 Common Mistakes

- **Assuming SUM returns 0 for no rows:** It returns NULL. Use `COALESCE(SUM(x), 0)`.
- **Confusing COUNT(*) and COUNT(column):** Leads to wrong counts when NULLs are present.
- **AVG denominator confusion:** AVG uses the count of non-NULL values, not the count of all rows.
- **Adding non-aggregated columns to SELECT with GROUP BY:** In strict SQL mode, this raises an error. MySQL in permissive mode silently returns arbitrary values, which is worse.
- **Integer division in AVG:** In some databases, `AVG(integer_column)` performs integer division. Cast to DECIMAL first: `AVG(CAST(salary AS DECIMAL))`.

Part 10: SQL Logical Execution Order

10.1 Why This Matters

The order in which you write SQL clauses is NOT the order in which the database processes them. Understanding logical execution order explains:

- Why you can't use a SELECT alias in WHERE.
- Why HAVING can filter on aggregates but WHERE cannot.
- Why ORDER BY can reference aliases.

10.2 Writing Order vs Execution Order

```
Writing Order:          Logical Execution Order:
──────────────          ────────────────────────
1. SELECT               1. FROM        ← Identify the table(s)
2. FROM                 2. WHERE       ← Filter individual rows
3. WHERE                3. GROUP BY    ← Group rows
4. GROUP BY             4. HAVING      ← Filter groups
5. HAVING               5. SELECT      ← Choose columns/expressions
6. ORDER BY             6. DISTINCT    ← Remove duplicates
7. LIMIT                7. ORDER BY    ← Sort results
                        8. LIMIT       ← Restrict output rows
```

10.3 ASCII Diagram

```
┌─────────────────────────────────────────────┐
│                SQL Query Flow               │
├─────────────────────────────────────────────┤
│                                             │
│  1. FROM / JOIN     → Combine source tables │
│         ↓                                   │
│  2. WHERE           → Filter rows (before groups)
│         ↓                                   │
│  3. GROUP BY        → Create groups         │
│         ↓                                   │
│  4. HAVING          → Filter groups         │
│         ↓                                   │
│  5. SELECT          → Evaluate expressions/aliases
│         ↓                                   │
│  6. DISTINCT        → Remove duplicate rows │
│         ↓                                   │
│  7. ORDER BY        → Sort the result set   │
│         ↓                                   │
│  8. LIMIT / TOP     → Return subset of rows │
```

```
└──────────────────────────────────────────────┘
```

10.4 Practical Implications

Why you can't use a SELECT alias in WHERE:

```
-- ❌ Error: alias not yet defined when WHERE executes
SELECT salary * 12 AS annual_salary FROM employees WHERE annual_salary > 100000;

-- ☑ Fix: repeat the expression
SELECT salary * 12 AS annual_salary FROM employees WHERE salary * 12 > 100000;

-- ☑ Alternative: use a subquery
SELECT * FROM (SELECT salary * 12 AS annual_salary FROM employees) t WHERE annual_salary > 100000;
```

Note: MySQL allows alias references in WHERE as an extension, but this is non-standard.

Why ORDER BY CAN use aliases:

```
-- ☑ This works because ORDER BY executes AFTER SELECT
SELECT salary * 12 AS annual_salary FROM employees ORDER BY annual_salary DESC;
```

Why HAVING filters groups, not rows:

```
-- HAVING runs after GROUP BY, so it can filter on aggregates
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING AVG(salary) > 75000;
```

department	avg_sal
Engineering	93200.00

10.5 Interview Explanation

> *"SQL has a logical execution order that differs from the writing order. It goes: FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This explains why you can't reference a SELECT alias in WHERE (WHERE runs before SELECT), but you can in ORDER BY (ORDER BY runs after SELECT). It also explains why WHERE filters individual rows before grouping, while HAVING filters after grouping and can use aggregate results."*

Part 11: Comparisons and Commonly Confused Concepts

11.1 WHERE vs HAVING

Aspect	WHERE	HAVING
When it runs	Before GROUP BY	After GROUP BY
Filters	Individual rows	Groups (aggregated results)
Can use aggregates?	❌ No	☑ Yes

Can use non-aggregated columns?	☑ Yes	☑ Yes (if in GROUP BY)
Performance	Faster (reduces rows before grouping)	Slower (groups formed first, then filtered)

```
-- WHERE: filter rows before grouping
SELECT department, AVG(salary)
FROM employees
WHERE hire_date >= '2020-01-01'  -- Filter rows first
GROUP BY department;

-- HAVING: filter groups after aggregation
SELECT department, AVG(salary) AS avg_sal
FROM employees
GROUP BY department
HAVING AVG(salary) > 75000;     -- Filter groups

-- Both: filter rows, then filter groups
SELECT department, AVG(salary) AS avg_sal
FROM employees
WHERE hire_date >= '2018-01-01'  -- Keep only employees hired from 2018
GROUP BY department
HAVING COUNT(*) >= 3;            -- Keep only departments with 3+ such employees
```

Interview trap: "Can you use WHERE instead of HAVING?" Only if you're filtering on non-aggregated columns. You CANNOT write `WHERE COUNT(*) > 3` — you must use HAVING.

11.2 COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)

Expression	What It Counts	NULL Handling
`COUNT(*)`	All rows, period	Includes NULLs
`COUNT(col)`	Non-NULL values in column	Excludes NULLs
`COUNT(DISTINCT col)`	Unique non-NULL values	Excludes NULLs

11.3 BETWEEN vs >= AND <=

Aspect	BETWEEN	>= AND <=
Readability	More readable for ranges	More explicit
Inclusivity	Always inclusive both ends	You control with < or <=
Negation	NOT BETWEEN excludes endpoints	More flexible negation
Date safety	Can miss times within a day	Allows `< next_day` pattern

Best practice for date ranges: Use `>= start_date AND < day_after_end_date` for safety with datetime columns.

11.4 COALESCE vs IFNULL vs ISNULL

Function	Standard?	Arguments	Database

COALESCE	☑ ANSI SQL	Any number	All
IFNULL	✗	Exactly 2	MySQL
ISNULL	✗	Exactly 2	SQL Server
NVL	✗	Exactly 2	Oracle

Recommendation: Use `COALESCE` everywhere for portability.

Part 12: Interview Question Bank

Essential Interview Questions

Q1: What is the difference between COUNT(*) and COUNT(column_name)?

> *`COUNT(*)` counts all rows including those with NULLs. `COUNT(column_name)` counts only rows where that column is not NULL. For example, if a table has 100 rows and 20 have NULL in the `email` column, `COUNT(*)` returns 100 but `COUNT(email)` returns 80.*

Q2: What does NULL mean in SQL? Can you compare NULL with = ?

> *NULL represents the absence of a value — it means unknown or not applicable. You cannot use `= NULL` because any comparison with NULL returns UNKNOWN, not TRUE or FALSE. You must use `IS NULL` or `IS NOT NULL`. This is because SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN.*

Q3: Explain the SQL logical execution order.

> *FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This means WHERE filters rows before grouping (so it can't use aggregates), HAVING filters after grouping (so it can use aggregates), and ORDER BY runs after SELECT (so it can reference aliases). This order also explains why you generally can't use a SELECT alias in a WHERE clause.*

Q4: What happens if you use LIMIT without ORDER BY?

> *The result is non-deterministic. The database may return any arbitrary set of rows, and the set may differ between executions. Always use ORDER BY before LIMIT to ensure predictable results.*

Q5: Write a query to find employees who earn more than the average salary.

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Note: You cannot write `WHERE salary > AVG(salary)` because WHERE runs before aggregation. You need a subquery (covered in detail on Day 3).

Q6: What is the difference between WHERE and HAVING?

> *WHERE filters individual rows before grouping and cannot use aggregate functions. HAVING filters groups after GROUP BY and can use aggregate functions. For performance, filter as much as possible in WHERE to reduce the data that needs to be grouped.*

Q7: What does SELECT DISTINCT department, city FROM employees return?

> *It returns unique combinations of (department, city), not unique departments AND unique cities separately. DISTINCT applies to the entire row of selected columns.*

Q8: What does BETWEEN do and is it inclusive?

> *BETWEEN tests if a value falls within a range. It is inclusive on BOTH ends.* `WHERE x BETWEEN 5 AND 10` *is equivalent to* `WHERE x >= 5 AND x <= 10`*. A common pitfall is using BETWEEN with datetime columns —* `BETWEEN '2024-01-01' AND '2024-01-31'` *can miss records with times after midnight on Jan 31.*

Q9: Output prediction — what does this query return?

```
SELECT department, COUNT(*)
FROM employees
WHERE salary > 70000
GROUP BY department
HAVING COUNT(*) >= 2
ORDER BY COUNT(*) DESC;
```

Step-by-step:

- FROM: all 15 rows
- WHERE salary > 70000: filters to rows with salary > 70000
  Alice(95k), Bob(87k), Charlie(72k), Eve(91k), Henry(78k), Jack(73k), Irene(105k), Leo(92k), Nathan(87k) = 9 employees
- GROUP BY department:
  Engineering: Alice, Bob, Irene, Leo, Nathan = 5
  Sales: Charlie, Eve = 2
  HR: Henry = 1
  Marketing: Jack = 1
- HAVING COUNT(*) >= 2: keeps Engineering(5) and Sales(2)
- ORDER BY COUNT(*) DESC:

department	COUNT(*)

Engineering	5
Sales	2

Q10: What happens when you do SUM(column) on an empty result set?

> *It returns NULL, not 0. This is a common source of bugs. Use `COALESCE(SUM(column), 0)` to return 0 instead.*

Advanced / Follow-up Questions

Q11: Why is NOT IN dangerous with NULLs?

> *If the list in `NOT IN` contains NULL, the entire expression evaluates to UNKNOWN for every row, so no rows are returned. For example, `WHERE dept NOT IN ('Sales', NULL)` expands to `dept <> 'Sales' AND dept <> NULL`. Since `dept <> NULL` is always UNKNOWN, the AND makes the entire condition UNKNOWN. Use `NOT EXISTS` or filter NULLs from the list.*

Q12: Does SQL Server support LIMIT?

> *No. SQL Server uses `TOP n` (placed after SELECT). For pagination with offset, SQL Server 2012+ supports `OFFSET n ROWS FETCH NEXT m ROWS ONLY`, which is also the ANSI SQL standard syntax.*

Q13: Can you use a column alias in WHERE?

> *Not in standard SQL. WHERE is evaluated before SELECT, so aliases defined in SELECT are not yet available. Most databases enforce this. MySQL is an exception and allows it as an extension, but relying on this is non-portable.*

Q14: What is the result of NULL + 5?

> *NULL. Any arithmetic operation involving NULL returns NULL. Use COALESCE to substitute NULLs before arithmetic: `COALESCE(column, 0) + 5`.*

Q15: Trick question — what is the output?

```
SELECT COUNT(*), SUM(amount), AVG(amount)
FROM orders
WHERE amount > 1000;
```

> *If no orders have amount > 1000 (looking at our data, the max is 990), then COUNT() = 0, SUM(amount) = NULL, AVG(amount) = NULL. COUNT() returns 0 for empty sets, but SUM and AVG return NULL.*

Part 13: Hands-On Practice

Instructions

Below are 20 practice queries. All use the `employees` and `orders` tables defined at the beginning of these notes. Try each exercise on your own before checking the answer key.

Exercise 1

List all employees showing their full name (first + last), department, and salary. Sort by salary descending.

Exercise 2

Find all unique cities where employees work.

Exercise 3

List employees in the Engineering department who earn more than $90,000.

Exercise 4

Find employees whose first name starts with the letter 'M' or 'J'.

Exercise 5

List employees hired between January 1, 2020 and December 31, 2021 (inclusive). Show their name, department, and hire date. Sort by hire date.

Exercise 6

Find all employees who do NOT have a commission (commission is NULL).

Exercise 7

Calculate the total, average, minimum, and maximum salary for the entire company.

Exercise 8

Show each employee's name, salary, commission, and total compensation (salary + commission). If commission is NULL, treat it as 0.

Exercise 9

Categorize employees into salary bands:

- **'High'** if salary >= 90,000
- **'Medium'** if salary >= 70,000
- **'Low'** if salary < 70,000

Show the employee name, salary, and band.

Exercise 10

Find the number of employees in each department. Sort by headcount descending.

Exercise 11

Find the average salary per department. Only show departments where the average salary exceeds $72,000.

Exercise 12

List the top 5 highest-paid employees.

Exercise 13

Find all orders that are not 'Cancelled'. Show order_id, customer_id, amount, and status. Sort by amount descending.

Exercise 14

How many orders have a NULL amount? How many have a non-NULL amount?

Exercise 15

For each order status, find the total number of orders and the total revenue (sum of amount). Sort by total revenue descending.

Exercise 16

List employees from either 'New York' or 'Boston' who earn more than $85,000.

Exercise 17

Find employees whose last name contains the letter 'a' (case-insensitive). Show their full name and department.

Exercise 18

For each department, count how many employees earn commission and how many do not, using CASE WHEN and aggregation.

Exercise 19

Find all employees whose city is unknown (NULL). List their name, department, and use COALESCE to display 'Not Assigned' for the city.

Exercise 20

Calculate the average order amount per region, but only for completed orders. Exclude regions where the average is below $400.

Answer Key

Solution 1

```
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name,  -- MySQL
    -- first_name || ' ' || last_name AS full_name,    -- PostgreSQL
    department,
    salary
```

```
FROM employees
ORDER BY salary DESC;
```

Expected output (first 5 rows):

full_name	department	salary
Irene Anderson	Engineering	105000.00
Alice Johnson	Engineering	95000.00
Leo Harris	Engineering	92000.00
Eve Davis	Sales	91000.00
Bob Smith	Engineering	87000.00

Solution 2

```
SELECT DISTINCT city FROM employees;
```

Expected output:

city
New York
Chicago
San Francisco
Boston
NULL

5 rows (NULL is a distinct value for DISTINCT purposes).

Solution 3

```
SELECT first_name, last_name, salary
FROM employees
WHERE department = 'Engineering' AND salary > 90000;
```

first_name	last_name	salary
Alice	Johnson	95000.00
Irene	Anderson	105000.00
Leo	Harris	92000.00

Solution 4

```
SELECT first_name, last_name FROM employees
WHERE first_name LIKE 'M%' OR first_name LIKE 'J%';
```

first_name	last_name
Jack	Thomas
Mary	Clark

Alternative using IN with SUBSTRING:

```
SELECT first_name, last_name FROM employees
WHERE LEFT(first_name, 1) IN ('M', 'J');
```

Solution 5

```
SELECT first_name, department, hire_date
FROM employees
WHERE hire_date BETWEEN '2020-01-01' AND '2021-12-31'
ORDER BY hire_date;
```

first_name	department	hire_date
Bob	Engineering	2020-07-01
Jack	Marketing	2020-08-12
Diana	Sales	2021-01-10
Karen	Marketing	2021-03-30
Nathan	Engineering	2021-09-05

Solution 6

```
SELECT first_name, last_name, department
FROM employees
WHERE commission IS NULL;
```

first_name	last_name	department
Alice	Johnson	Engineering
Bob	Smith	Engineering
Frank	Wilson	HR
Grace	Taylor	HR
Henry	Martinez	HR
Irene	Anderson	Engineering

Leo	Harris	Engineering
Nathan	Lewis	Engineering
Olivia	Walker	HR

9 rows (15 total - 6 with commission = 9 without).

Solution 7

```
SELECT
    SUM(salary) AS total_salary,
    ROUND(AVG(salary), 2) AS avg_salary,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary
FROM employees;
```

total_salary	avg_salary	min_salary	max_salary
1174000.00	78266.67	61000.00	105000.00

Solution 8

```
SELECT
    first_name,
    salary,
    commission,
    salary + COALESCE(commission, 0) AS total_compensation
FROM employees
ORDER BY total_compensation DESC;
```

first_name	salary	commission	total_compensation
Irene	105000.00	NULL	105000.00
Alice	95000.00	NULL	95000.00
Eve	91000.00	3000.00	94000.00
Leo	92000.00	NULL	92000.00
Bob	87000.00	NULL	87000.00
Nathan	87000.00	NULL	87000.00
Henry	78000.00	NULL	78000.00
Charlie	72000.00	5000.00	77000.00
Diana	68000.00	7500.00	75500.00
Jack	73000.00	2000.00	75000.00

Mary	68000.00	4000.00	72000.00
Karen	69000.00	1500.00	70500.00
Frank	64000.00	NULL	64000.00
Olivia	64000.00	NULL	64000.00
Grace	61000.00	NULL	61000.00

Solution 9

```
SELECT
    first_name,
    salary,
    CASE
        WHEN salary >= 90000 THEN 'High'
        WHEN salary >= 70000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_band
FROM employees
ORDER BY salary DESC;
```

first_name	salary	salary_band
Irene	105000.00	High
Alice	95000.00	High
Leo	92000.00	High
Eve	91000.00	High
Bob	87000.00	Medium
Nathan	87000.00	Medium
Henry	78000.00	Medium
Jack	73000.00	Medium
Charlie	72000.00	Medium
Karen	69000.00	Low
Diana	68000.00	Low
Mary	68000.00	Low
Frank	64000.00	Low
Olivia	64000.00	Low
Grace	61000.00	Low

Solution 10

```
SELECT department, COUNT(*) AS headcount
FROM employees
GROUP BY department
ORDER BY headcount DESC;
```

department	headcount
Engineering	5
Sales	4
HR	4
Marketing	2

Solution 11

```
SELECT department, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 72000
ORDER BY avg_salary DESC;
```

department	avg_salary
Engineering	93200.00
Sales	74750.00

(HR avg = 66750 and Marketing avg = 71000, both at or below 72000.)

Solution 12

```
-- PostgreSQL / MySQL
SELECT first_name, last_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 5;

-- SQL Server
SELECT TOP 5 first_name, last_name, salary
FROM employees
ORDER BY salary DESC;
```

first_name	last_name	salary
Irene	Anderson	105000.00
Alice	Johnson	95000.00

Leo	Harris	92000.00
Eve	Davis	91000.00
Bob	Smith	87000.00

Note: Nathan also earns 87000. The 5th position is arbitrary between Bob and Nathan. This is a tie-handling issue — discussed in Day 3 with window functions.

Solution 13

```
SELECT order_id, customer_id, amount, status
FROM orders
WHERE status <> 'Cancelled'
ORDER BY amount DESC;
```

order_id	customer_id	amount	status
109	1001	990.00	Completed
104	1003	800.00	Completed
107	1005	670.00	Completed
112	1007	550.00	Completed
102	1002	450.00	Completed
111	1004	410.00	Pending
106	1004	320.00	Completed
101	1001	250.00	Completed
110	1006	0.00	Completed
105	1002	NULL	Pending

Note: Order 105 has NULL amount and is not cancelled, so it appears. NULL sorts at the end in DESC for most databases.

Solution 14

```
SELECT
    COUNT(*) - COUNT(amount) AS null_amount_count,
    COUNT(amount) AS non_null_amount_count
FROM orders;
```

null_amount_count	non_null_amount_count
1	11

Alternative:

```
SELECT
    SUM(CASE WHEN amount IS NULL THEN 1 ELSE 0 END) AS null_count,
    SUM(CASE WHEN amount IS NOT NULL THEN 1 ELSE 0 END) AS non_null_count
FROM orders;
```

Solution 15

```
SELECT
    status,
    COUNT(*) AS order_count,
    COALESCE(SUM(amount), 0) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

Calculations:

- Completed: 250+450+800+320+670+990+0+550 = 4030
- Cancelled: 125+150 = 275
- Pending: NULL + 410 = 410

status	order_count	total_revenue
Completed	8	4030.00
Pending	2	410.00
Cancelled	2	275.00

Solution 16

```
SELECT first_name, last_name, city, salary
FROM employees
WHERE city IN ('New York', 'Boston') AND salary > 85000;
```

first_name	last_name	city	salary
Alice	Johnson	New York	95000.00
Bob	Smith	New York	87000.00
Irene	Anderson	New York	105000.00
Leo	Harris	New York	92000.00
Nathan	Lewis	Boston	87000.00

Solution 17

```
-- Portable (force case-insensitive)
SELECT
    CONCAT(first_name, ' ', last_name) AS full_name,
    department
FROM employees
WHERE LOWER(last_name) LIKE '%a%';
```

Expected output:

full_name	department
Eve Davis	Sales
Grace Taylor	HR
Henry Martinez	HR
Irene Anderson	Engineering
Jack Thomas	Marketing
Leo Harris	Engineering
Mary Clark	Sales
Olivia Walker	HR

8 rows. (Last names containing 'a': Davis, Taylor, Martinez, Anderson, Thomas, Harris, Clark, Walker)

Solution 18

```
SELECT
    department,
    COUNT(CASE WHEN commission IS NOT NULL THEN 1 END) AS with_commission,
    COUNT(CASE WHEN commission IS NULL THEN 1 END) AS without_commission
FROM employees
GROUP BY department;
```

department	with_commission	without_commission
Engineering	0	5
HR	0	4
Marketing	2	0
Sales	4	0

Solution 19

```
SELECT
    first_name,
```

```
    last_name,
    department,
    COALESCE(city, 'Not Assigned') AS city
FROM employees
WHERE city IS NULL;
```

first_name	last_name	department	city
Olivia	Walker	HR	Not Assigned

Solution 20

```
SELECT
    region,
    ROUND(AVG(amount), 2) AS avg_amount
FROM orders
WHERE status = 'Completed'
GROUP BY region
HAVING AVG(amount) >= 400
ORDER BY avg_amount DESC;
```

Completed orders by region:

- East: 250 + 320 + 990 = 1560, count=3, avg=520.00
- West: 450 + 0 = 450, count=2, avg=225.00
- North: 800, count=1, avg=800.00
- South: 670 + 550 = 1220, count=2, avg=610.00

HAVING AVG >= 400 keeps: North(800), South(610), East(520).

region	avg_amount
North	800.00
South	610.00
East	520.00

Part 14: Case Study — Sales Performance Analysis

1. Business Problem

A retail company needs a weekly report summarizing employee sales performance. Management wants to understand:

- Total compensation by department (salary + commission).
- How many employees earn commission vs. fixed salary.
- Which employees are the top earners.
- Salary distribution across departments.

2. Requirements and Assumptions

- Use the `employees` table.
- Commission represents variable pay tied to sales performance.
- NULL commission means the employee is not in a commission-eligible role.
- "Total compensation" = salary + COALESCE(commission, 0).
- Management wants results sorted by total compensation descending.

3. Input Data

The `employees` table with 15 rows, as defined in our schema.

4. Proposed Approach

Build a single report query that:

- Calculates total compensation per employee.
- Classifies pay type (Commission-based vs. Fixed).
- Assigns salary bands (High / Medium / Low).
- Aggregates metrics per department.

5. Implementation

Step 1: Individual employee report

```
SELECT
    CONCAT(first_name, ' ', last_name) AS employee_name,
    department,
    salary,
    COALESCE(commission, 0) AS commission,
    salary + COALESCE(commission, 0) AS total_compensation,
    CASE
        WHEN commission IS NOT NULL THEN 'Commission-based'
        ELSE 'Fixed salary'
    END AS pay_type,
    CASE
        WHEN salary >= 90000 THEN 'High'
        WHEN salary >= 70000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_band
FROM employees
ORDER BY total_compensation DESC;
```

Step 2: Department-level summary

```
SELECT
    department,
    COUNT(*) AS headcount,
    ROUND(AVG(salary), 2) AS avg_salary,
    SUM(salary + COALESCE(commission, 0)) AS total_dept_compensation,
    COUNT(CASE WHEN commission IS NOT NULL THEN 1 END) AS commission_earners,
    COUNT(CASE WHEN commission IS NULL THEN 1 END) AS fixed_salary_earners,
    COUNT(CASE WHEN salary >= 90000 THEN 1 END) AS high_band,
    COUNT(CASE WHEN salary >= 70000 AND salary < 90000 THEN 1 END) AS medium_band,
    COUNT(CASE WHEN salary < 70000 THEN 1 END) AS low_band,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary
FROM employees
GROUP BY department
ORDER BY total_dept_compensation DESC;
```

6. Expected Results

Department Summary:

department	headcount	avg_salary	total_dept_compensation	commission_earners	fixed_salary_earners	high	med	low	min_salary	max_salary
Engineering	5	93200.00	466000.00	0	5	3	2	0	87000	105000
Sales	4	74750.00	318500.00	4	0	1	1	2	68000	91000
HR	4	66750.00	267000.00	0	4	0	1	3	61000	78000
Marketing	2	71000.00	145500.00	2	0	0	1	1	69000	73000

7. Testing and Validation

- **Row count check:** SUM of headcounts should equal 15. (5+4+4+2 = 15 ☑)
- **Salary cross-check:** SUM of total_dept_compensation should equal company-wide total.
  466000 + 318500 + 267000 + 145500 = 1,197,000
  Company total salary = 1,174,000 + total commission (23,000) = 1,197,000 ☑

- **NULL handling check:** Verify commission_earners + fixed_salary_earners = headcount per department.

8. Limitations and Trade-offs

- Commission data is a snapshot; it doesn't reflect YTD performance.
- Salary bands are hardcoded — better to use a lookup table for maintainability.
- No time dimension: this is a point-in-time analysis, not a trend.
- No adjustment for role level or experience.

9. Likely Interview Follow-ups

- "How would you handle this if the company had 10,000 employees across 50 departments?" → Index on department, consider materialized views.
- "What if commission is paid quarterly, not monthly?" → Need a time-series table of commission payments.
- "How would you add year-over-year salary growth?" → Need historical salary data and window functions (Day 3 topic).
- "What if some employees belong to multiple departments?" → The schema would need a junction table (many-to-many relationship, Day 4 topic).

Part 15: Compact Revision Sheet

Essential Definitions

Term	Definition
SELECT	Specifies columns/expressions in the result set (projection)
DISTINCT	Eliminates duplicate rows across ALL selected columns
WHERE	Filters individual rows BEFORE grouping
ORDER BY	Sorts the result set; only guarantee of row order
LIMIT/TOP	Restricts number of output rows
BETWEEN	Inclusive range test: `x BETWEEN a AND b` = `x >= a AND x <= b`
IN	Set membership test: `x IN (a, b, c)` = `x=a OR x=b OR x=c`
LIKE	Pattern matching: `%` = any chars, `_` = exactly one char
NULL	Absence of value; comparison with NULL → UNKNOWN
CASE WHEN	Conditional expression; first true condition wins
COALESCE	Returns first non-NULL argument

Syntax Templates

```
-- Basic query
SELECT col1, col2, expr AS alias
FROM table
WHERE condition
ORDER BY col1 DESC
LIMIT n;

-- Aggregation
SELECT group_col, COUNT(*), SUM(col), AVG(col)
FROM table
WHERE row_filter
GROUP BY group_col
HAVING aggregate_filter;

-- CASE WHEN
CASE WHEN cond1 THEN val1 WHEN cond2 THEN val2 ELSE default END

-- NULL handling
WHERE col IS NULL | IS NOT NULL
COALESCE(col, fallback_value)
```

Critical Rules

- **Execution order:** FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
- **NULL ≠ NULL:** Use `IS NULL`, never `= NULL`

- **COUNT(*)** counts all rows; **COUNT(col)** counts non-NULLs only
- **AVG** ignores NULLs (divides by non-NULL count)
- **SUM of empty set** returns NULL, not 0
- **NOT IN with NULL** returns no rows — use NOT EXISTS
- **BETWEEN** is inclusive on both ends
- **LIKE '%text'** forces full table scan (no index use)
- **AND** has higher precedence than **OR** — use parentheses
- **LIMIT without ORDER BY** is non-deterministic

Key Comparisons

Concept A	Concept B	Key Difference
WHERE	HAVING	WHERE: before GROUP BY, no aggregates. HAVING: after, can aggregate
COUNT(*)	COUNT(col)	* counts all rows; col skips NULLs
COALESCE	IFNULL	COALESCE is standard SQL, multi-arg; IFNULL is MySQL-only, 2-arg
BETWEEN	>= AND <=	Identical; BETWEEN is syntactic sugar
LIMIT	TOP	LIMIT: MySQL/PG, end of query. TOP: SQL Server, after SELECT

Common Mistakes Checklist

- [ ] Used `= NULL` instead of `IS NULL`
- [ ] Used `NOT IN` with a list that may contain NULL
- [ ] Forgot `ORDER BY` before `LIMIT`
- [ ] Used `SELECT *` in production code
- [ ] Referenced a SELECT alias in WHERE
- [ ] Confused AVG behavior with NULLs (AVG ignores them)
- [ ] Forgot parentheses with mixed AND/OR
- [ ] Used BETWEEN with datetime without considering time component

---

Part 16: Closed-Book Self-Assessment (100 Marks)

Instructions

- Time limit: 60 minutes.
- No notes, no AI assistance.
- Write your answers on paper or in a separate document.
- Check the answer key only after completing all questions.

Section A: Conceptual Questions (30 marks)

Q1 (3 marks): What is the SQL logical execution order? List all 8 steps.

Q2 (3 marks): Explain the difference between `COUNT(*)`, `COUNT(column)`, and `COUNT(DISTINCT column)`. Give an example where all three return different values.

Q3 (2 marks): What does NULL represent in SQL? Why can't you use `WHERE column = NULL`?

Q4 (3 marks): Explain the difference between WHERE and HAVING. Give one example where you must use HAVING and cannot use WHERE.

Q5 (2 marks): Is `BETWEEN 10 AND 20` inclusive or exclusive at the boundaries? What is the equivalent expression using comparison operators?

Q6 (2 marks): What happens when you use `NOT IN` with a list that contains NULL? Why?

Q7 (3 marks): What is the result of `AVG(commission)` vs `AVG(COALESCE(commission, 0))` if a table has 10 employees, only 4 of whom have non-NULL commissions totaling 20000?

Q8 (2 marks): Can you reference a SELECT alias in the WHERE clause? In the ORDER BY clause? Explain why.

Q9 (3 marks): What is the difference between `LIMIT` and `TOP`? Name one feature that TOP supports but LIMIT does not.

Q10 (2 marks): In which databases are NULLs sorted first in ascending order, and in which are they sorted last?

Q11 (3 marks): What does `LIKE '%abc'` match? Can it use a B-tree index? Why or why not?

Q12 (2 marks): What does `SUM(salary)` return when no rows match the WHERE clause?

Section B: Output Prediction (25 marks)

Use the `employees` table from the notes.

Q13 (4 marks): Predict the output:

```
SELECT COUNT(*), COUNT(commission), COUNT(DISTINCT department)
FROM employees;
```

Q14 (4 marks): Predict the output:

```
SELECT department, MAX(salary) - MIN(salary) AS salary_range
FROM employees
GROUP BY department
HAVING MAX(salary) - MIN(salary) > 20000;
```

Q15 (4 marks): Predict the output:

```
SELECT first_name, salary
FROM employees
WHERE department = 'HR' OR department = 'Marketing' AND salary > 70000
ORDER BY salary DESC;
```

(Pay careful attention to operator precedence.)

Q16 (4 marks): Predict the output:

```
SELECT COALESCE(NULL, NULL, 'hello', 'world');
```

Q17 (5 marks): Predict the exact output (all rows):

```
SELECT department, COUNT(*) AS cnt
FROM employees
WHERE city IS NOT NULL
GROUP BY department
```

```
HAVING COUNT(*) >= 3
ORDER BY cnt DESC;
```

Q18 (4 marks): What does this return? Explain why.

```
SELECT AVG(commission) FROM employees WHERE department = 'Engineering';
```

Section C: Write the Query (30 marks)

Q19 (5 marks): Write a query to find all employees whose salary is above the company average. Show name, department, and salary. Sort by salary descending.

Hint: You need a subquery.

Q20 (5 marks): Write a query to display each department with: the number of employees, the average salary (rounded to 2 decimals), the number of employees earning commission, and the number without commission. Sort by average salary descending.

Q21 (5 marks): Write a query to categorize orders into:

- 'High Value' if amount >= 500
- 'Medium Value' if amount >= 200
- 'Low Value' if amount < 200
- 'Unknown' if amount is NULL

Show order_id, amount, category, and status. Sort by amount descending (NULLs last).

Q22 (5 marks): Write a query to find regions where the total revenue from completed orders exceeds $500.

Q23 (5 marks): Write a query to display the top 3 departments by total compensation (salary + commission, treating NULL commission as 0). Show department name, headcount, and total compensation.

Q24 (5 marks): Write a query that shows each employee's first name, city, and a column called `location_status` that shows:

- 'Local' if city is 'New York'
- 'Regional' if city is 'Chicago' or 'Boston'
- 'Remote' if city is 'San Francisco'
- 'Unassigned' if city is NULL

Sort by department, then salary descending.

Section D: Scenario-Based (15 marks)

Q25 (5 marks): A junior analyst writes this query and gets unexpected results:

```
SELECT department, first_name, AVG(salary)
FROM employees
GROUP BY department;
```

What is wrong? How would you fix it? What would different databases do with this query?

Q26 (5 marks): Your team lead asks: "What is the average commission across the company?" You run `SELECT AVG(commission) FROM employees` and get 3833.33. Your lead says "That can't be right, we have 15 employees and total commission is only 23000. The average should be about 1533."

Who is correct? Explain the discrepancy. Write queries that produce both answers and explain when each is appropriate.

Q27 (5 marks): You need to retrieve all employees who are NOT in the Sales or HR departments. You write:

```
SELECT * FROM employees WHERE department NOT IN ('Sales', 'HR');
```

This works correctly. But a colleague uses a subquery:

```
SELECT * FROM employees
WHERE department NOT IN (SELECT department FROM departments WHERE active = 0);
```

Under what condition would this query return zero rows even if there are valid employees? How would you fix it?

Assessment Answer Key

A1: FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT/OFFSET

A2: `COUNT(*)` counts all rows including NULLs. `COUNT(column)` counts only non-NULL values. `COUNT(DISTINCT column)` counts unique non-NULL values. Example: If a column has values [A, A, B, NULL], then COUNT(*) = 4, COUNT(col) = 3, COUNT(DISTINCT col) = 2.

A3: NULL represents the absence of a value (unknown/not applicable). `= NULL` returns UNKNOWN (not TRUE) because SQL uses three-valued logic. NULL is not equal to anything, not even itself. Use `IS NULL` instead.

A4: WHERE filters individual rows before GROUP BY and cannot use aggregate functions. HAVING filters groups after GROUP BY and can use aggregates. Example requiring HAVING: `SELECT department, COUNT(*) FROM employees GROUP BY department HAVING COUNT(*) > 3` — you cannot write `WHERE COUNT(*) > 3`.

A5: Inclusive on both ends. `x BETWEEN 10 AND 20` is equivalent to `x >= 10 AND x <= 20`.

A6: It returns zero rows. `NOT IN (a, b, NULL)` expands to `x<>a AND x<>b AND x<>NULL`. Since `x<>NULL` evaluates to UNKNOWN, the entire AND chain evaluates to UNKNOWN, and the row is excluded.

A7: `AVG(commission)` = 20000/4 = 5000. `AVG(COALESCE(commission, 0))` = 20000/10 = 2000. The first ignores NULLs (divides by 4 non-NULL values); the second treats NULLs as 0 (divides by all 10 rows).

A8: No, you cannot reference a SELECT alias in WHERE because WHERE executes before SELECT. Yes, you can in ORDER BY because ORDER BY executes after SELECT. (MySQL allows aliases in WHERE as a non-standard extension.)

A9: LIMIT is used in MySQL/PostgreSQL/SQLite and appears at the end of the query. TOP is used in SQL Server and appears after SELECT. TOP supports `WITH TIES` (includes tied rows) and `PERCENT` (return a percentage of rows), which LIMIT does not.

A10: MySQL and SQL Server sort NULLs first in ASC. PostgreSQL and Oracle sort NULLs last in ASC.

A11: `LIKE '%abc'` matches any string ending in 'abc'. It cannot use a B-tree index because the wildcard is at the beginning — the database cannot do a prefix-based lookup and must scan every value.

A12: NULL (not 0). Use `COALESCE(SUM(salary), 0)` to return 0 instead.

A13:

COUNT(*)	COUNT(commission)	COUNT(DISTINCT department)
15	6	4

A14:

- Engineering: 105000-87000 = 18000 (excluded: not > 20000)
- Sales: 91000-68000 = 23000 ☑

- HR: 78000-61000 = 17000 (excluded)
- Marketing: 73000-69000 = 4000 (excluded)

department	salary_range
Sales	23000.00

A15: Due to AND having higher precedence than OR, this is evaluated as:
`department = 'HR' OR (department = 'Marketing' AND salary > 70000)`
This returns ALL HR employees plus Marketing employees earning > 70000.

first_name	salary
Henry	78000.00
Jack	73000.00
Frank	64000.00
Olivia	64000.00
Grace	61000.00

(All 4 HR employees + Jack from Marketing. Karen earns 69000 which is not > 70000, so she's excluded.)

A16: `'hello'`. COALESCE returns the first non-NULL argument.

A17:
First, filter `WHERE city IS NOT NULL` removes Olivia (emp_id 15).
Remaining 14 employees grouped by department:

- Engineering: 5 (all have non-NULL cities)
- Sales: 4
- HR: 3 (Olivia excluded, leaving Frank, Grace, Henry)
- Marketing: 2

HAVING COUNT(*) >= 3 keeps Engineering(5), Sales(4), HR(3).

department	cnt
Engineering	5
Sales	4
HR	3

A18: Returns NULL. All 5 Engineering employees have NULL commission. AVG of all NULLs returns NULL (there are zero non-NULL values to average).

A19:

```
SELECT first_name, department, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;
```

(Average salary is approximately 78266.67, so this returns employees earning more than that.)

A20:

```
SELECT
    department,
    COUNT(*) AS headcount,
    ROUND(AVG(salary), 2) AS avg_salary,
    COUNT(CASE WHEN commission IS NOT NULL THEN 1 END) AS with_commission,
    COUNT(CASE WHEN commission IS NULL THEN 1 END) AS without_commission
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

A21:

```
SELECT
    order_id,
    amount,
    CASE
        WHEN amount IS NULL THEN 'Unknown'
        WHEN amount >= 500 THEN 'High Value'
        WHEN amount >= 200 THEN 'Medium Value'
        ELSE 'Low Value'
    END AS category,
    status
FROM orders
ORDER BY CASE WHEN amount IS NULL THEN 1 ELSE 0 END, amount DESC;
```

Note: The NULL check must come before the numeric comparisons to avoid NULLs falling through to 'Low Value'.
For NULLS LAST in databases that support it: `ORDER BY amount DESC NULLS LAST`.

A22:

```
SELECT region, SUM(amount) AS total_revenue
FROM orders
WHERE status = 'Completed'
GROUP BY region
HAVING SUM(amount) > 500;
```

Expected: East(1560), South(1220), North(800). West total = 450, excluded.

A23:

```
SELECT
    department,
    COUNT(*) AS headcount,
    SUM(salary + COALESCE(commission, 0)) AS total_compensation
FROM employees
GROUP BY department
ORDER BY total_compensation DESC
LIMIT 3;
```

department	headcount	total_compensation
Engineering	5	466000.00
Sales	4	318500.00
HR	4	267000.00

A24:

```
SELECT
    first_name,
    department,
```

```
    salary,
    city,
    CASE
        WHEN city = 'New York' THEN 'Local'
        WHEN city IN ('Chicago', 'Boston') THEN 'Regional'
        WHEN city = 'San Francisco' THEN 'Remote'
        WHEN city IS NULL THEN 'Unassigned'
    END AS location_status
FROM employees
ORDER BY department, salary DESC;
```

A25: The query is invalid in standard SQL. `first_name` is in SELECT but not in GROUP BY and not inside an aggregate function. In strict SQL mode (PostgreSQL, SQL Server), this raises an error. In MySQL with `ONLY_FULL_GROUP_BY` disabled, it runs but returns an arbitrary `first_name` for each department — this is unpredictable and wrong. Fix: Either add `first_name` to GROUP BY (changes the grouping) or remove it from SELECT, or use an aggregate like `MAX(first_name)`.

A26: Both are technically correct — they answer different questions.

- `AVG(commission)` = 23000/6 = 3833.33 → Average commission among commission-earning employees (ignores NULLs).
- `AVG(COALESCE(commission, 0))` = 23000/15 = 1533.33 → Average commission across all employees (treats NULL as 0).

The lead's interpretation (divide by 15) answers "What is the average commission cost per employee?" The query result (divide by 6) answers "What is the average commission among those who earn one?" The right answer depends on the business question.

```
-- Lead's answer
SELECT AVG(COALESCE(commission, 0)) FROM employees;  -- 1533.33

-- Query's answer
SELECT AVG(commission) FROM employees;  -- 3833.33
```

A27: If the subquery `SELECT department FROM departments WHERE active = 0` returns any NULL value (e.g., a row where `department` column is NULL), then `NOT IN` returns zero rows for every employee (because `NOT IN` with NULL always evaluates to UNKNOWN). Fix: Add `WHERE department IS NOT NULL` to the subquery, or use `NOT EXISTS` instead:

```
SELECT * FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM departments d
    WHERE d.active = 0 AND d.department = e.department
);
```

Day 2: SQL JOINs, Grouping & Aggregation

Comprehensive Study Notes & Interview Preparation Guide

Part 1: Relational Algebra & The Mechanics of JOINs

1.1 Definition & Relational Intuition

In relational database systems (RDBMS), data is normalized across multiple tables to reduce redundancy, prevent update anomalies, and enforce domain integrity. A **JOIN** is a binary relational operator that reconstructs relationships between tables by combining tuples (rows) from two or more relations based on a logical predicate known as the **join condition** (typically equality between primary keys and foreign keys).

1.2 Cartesian Product (CROSS JOIN)

Before exploring filtered joins, understand the theoretical foundation: the **Cartesian Product** ($R \times S$).
If Table R has M rows and Table S has N rows, the Cartesian Product produces a result relation containing $M \times N$ rows, with every row of R concatenated with every row of S.

```
-- Explicit ANSI syntax
SELECT e.emp_name, d.dept_name
FROM employees e
CROSS JOIN departments d;

-- Implicit syntax (discouraged due to accidental omission of WHERE clause)
SELECT e.emp_name, d.dept_name
FROM employees e, departments d;
```

Practical Use Cases for CROSS JOIN:

- **Generating Continuous Time/Date Grids:** Combining an active entity list with a complete calendar dimension table to detect zero-activity days (preventing missing rows in time-series reporting).
- **Matrix Generation:** Testing all permutations of product sizes and colors for an inventory staging table.

Part 2: Comprehensive Breakdown of Standard JOIN Types

2.1 The Reference Schema for Explanations

To ensure every join behavior is concrete and unambiguous, we use this exact reference dataset:

Table: departments (d)

dept_id	dept_name	location
10	Engineering	New York
20	Analytics	San Francisco
30	Marketing	London

40	Operations	Dublin

Table: employees (e)

emp_id	emp_name	dept_id	salary	manager_id
1	Alice	10	120000	NULL
2	Bob	10	95000	1
3	Charlie	20	105000	1
4	David	20	88000	3
5	Emma	NULL	92000	1
6	Frank	50	75000	3

Notice:

- Emma has a `NULL` `dept_id` (Unassigned department).
- Frank has `dept_id = 50` which does **not** exist in `departments` (Orphan/Dangling foreign key).
- Departments 30 (Marketing) and 40 (Operations) have **no employees assigned**.

2.2 INNER JOIN

Definition & Logic

An `INNER JOIN` evaluates the join predicate (specified in the `ON` clause) and returns **only** the rows where the condition evaluates to `TRUE`. Rows with unmatched keys or `NULL` keys are completely excluded.

```
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_id,
    d.dept_name
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.dept_id;
```

Result Set:

emp_id	emp_name	dept_id	dept_name
1	Alice	10	Engineering
2	Bob	10	Engineering
3	Charlie	20	Analytics
4	David	20	Analytics

Omissions Explained:

- **Emma** is excluded because `NULL = 10`, `NULL = 20`, etc. evaluate to `UNKNOWN` in SQL Three-Valued Logic.
- **Frank** is excluded because `50` does not match any row in `departments`.
- **Departments 30 & 40** are excluded because no employee has `dept_id IN (30, 40)`.

2.3 LEFT OUTER JOIN (LEFT JOIN)

Definition & Logic

A `LEFT JOIN` returns **all rows from the left table**, along with matched rows from the right table. If no match exists in the right table for a given left row, the engine generates **NULL-padded** columns for all selected attributes from the right table.

```
SELECT
    e.emp_id,
    e.emp_name,
    e.dept_id AS emp_dept,
    d.dept_id AS dept_dept,
    d.dept_name
FROM employees e
LEFT JOIN departments d
    ON e.dept_id = d.dept_id;
```

Result Set:

emp_id	emp_name	emp_dept	dept_dept	dept_name
1	Alice	10	10	Engineering
2	Bob	10	10	Engineering
3	Charlie	20	20	Analytics
4	David	20	20	Analytics
5	Emma	NULL	NULL	NULL
6	Frank	50	NULL	NULL

The Left Anti-Join Pattern (Finding Unmatched Records)

To find entities in the left table that have **no counterpart** in the right table, pair a `LEFT JOIN` with a `WHERE right_table.primary_key IS NULL`:

```
-- Employees who belong to an invalid or unassigned department
SELECT e.emp_id, e.emp_name, e.dept_id
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Result: Returns Emma and Frank.

2.4 RIGHT OUTER JOIN (RIGHT JOIN)

Definition & Logic

Symmetric counterpart to the `LEFT JOIN`. Returns all rows from the right table and matching rows from the left table. Any unmatched right table row is padded with `NULL`s for left-table attributes.

```
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_id,
    d.dept_name
FROM employees e
RIGHT JOIN departments d
    ON e.dept_id = d.dept_id;
```

Result Set:

emp_id	emp_name	dept_id	dept_name
1	Alice	10	Engineering
2	Bob	10	Engineering
3	Charlie	20	Analytics
4	David	20	Analytics
NULL	NULL	30	Marketing
NULL	NULL	40	Operations

> *[!TIP]*
> ***Engineering Standard:*** *Almost all senior data engineers convert `RIGHT JOIN`s to `LEFT JOIN`s by swapping the table order (`FROM departments d LEFT JOIN employees e`). This maintains a standard left-to-right reading flow and improves code review ergonomics.*

2.5 FULL OUTER JOIN

Definition & Logic

Returns all records when there is a match in either left or right table records. Unmatched rows from either side are returned with `NULL` in the opposing table's attributes.

```
-- Standard ANSI SQL (PostgreSQL, SQL Server, Snowflake, Oracle)
SELECT
    e.emp_id,
    e.emp_name,
    d.dept_id,
    d.dept_name
FROM employees e
FULL OUTER JOIN departments d
    ON e.dept_id = d.dept_id;
```

Result Set:

emp_id	emp_name	dept_id	dept_name
1	Alice	10	Engineering
2	Bob	10	Engineering
3	Charlie	20	Analytics
4	David	20	Analytics
5	Emma	NULL	NULL
6	Frank	NULL	NULL
NULL	NULL	30	Marketing
NULL	NULL	40	Operations

Dialect Note: Emulating FULL OUTER JOIN in MySQL

MySQL does not support native `FULL OUTER JOIN`. The standard emulation uses `UNION` (which performs distinct set deduplication) combining a `LEFT JOIN` and a `RIGHT JOIN`:

```
SELECT e.emp_id, e.emp_name, d.dept_id, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_id, e.emp_name, d.dept_id, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

Part 3: Critical Join Gotchas & Execution Internals

3.1 The ON Clause Filter vs. The WHERE Clause Filter in OUTER JOINs

This is one of the most frequently failed senior SQL interview questions.

Scenario A: Filtering in the ON clause

```
SELECT e.emp_name, d.dept_name
FROM departments d
LEFT JOIN employees e
    ON d.dept_id = e.dept_id
    AND e.salary > 100000;
```

- **How it executes:** The condition `e.salary > 100000` is evaluated **during the join phase**. If an employee has salary ≤ 100000, the join predicate is false for that employee. However, because it is a `LEFT JOIN` on `departments`, **all departments are still preserved**.
- **Output:** Every department is listed. Departments without employees making $> 100k$ will still appear, showing `NULL` for `emp_name`.

Scenario B: Filtering in the WHERE clause

```
SELECT e.emp_name, d.dept_name
FROM departments d
LEFT JOIN employees e
    ON d.dept_id = e.dept_id
WHERE e.salary > 100000;
```

- **How it executes:** The outer join happens first. Then, the `WHERE` clause filters the unified intermediate table. For unmatched departments (e.g., Marketing, Operations), `e.salary` is `NULL`. The predicate `NULL > 100000` evaluates to `UNKNOWN` and is discarded by `WHERE`.
- **Consequence: The LEFT JOIN is silently converted into an INNER JOIN!** Departments 30 and 40 are wiped out.

> *[!CAUTION]*
> *Placing a predicate on the right table in the `WHERE` clause of a `LEFT JOIN` will convert it into an `INNER JOIN` unless you explicitly account for nulls (e.g., `WHERE (e.salary > 100000 OR e.salary IS NULL)`).*

3.2 Join Cardinality & Row Explosion

When joining two tables on non-unique keys, the output row count can expand exponentially.

- If key `K` appears A times in Table 1 and B times in Table 2, an equijoin on K produces $A \times B$ rows.
- **Real-world danger:** If an analytics pipeline joins an `Orders` table (10^6 rows) with a `Promotions` table on `promo_code`, and `promo_code = 'WELCOME'` is duplicated 5 times in the promotions dimension due to poor ETL data quality, the query produces 5×10^6 rows, inflating financial revenue metrics by $500%$.

3.3 Join Algorithms Under the Hood (Engine Internals)

Understanding how query planners physically execute joins is essential for query tuning:

- **Nested Loop Join:**

Mechanism: For each row in the outer table, scan the inner table.
Cost: $O(M \times N)$ without index; $O(M \log N)$ if inner table has an index on join key.
Best for: Small outer table and indexed inner table.

- **Hash Join:**

Mechanism: Builds an in-memory hash table on the smaller table's join keys (Build phase). Then streams the larger table and probes the hash table (Probe phase).
Cost: $O(M + N)$ time complexity; requires memory proportional to the build table.
Best for: Large tables with equijoin predicates without usable indexes.

- **Sort-Merge Join:**

Mechanism: Both inputs are sorted by the join key (if not already indexed/clustered) and traversed in lockstep.
Cost: $O(M \log M + N \log N)$ for sorting, then $O(M + N)$ for the merge.

Best for: Medium to large datasets where data is pre-sorted or join conditions include inequalities ($<$ or $>$).

Part 4: Self Joins & Hierarchical Data

4.1 Definition & Syntax

A **Self Join** is a standard join in which a table is joined with itself. It requires distinct table aliases to distinguish between the two roles the table plays in the query.

4.2 Use Case 1: Employee-Manager Hierarchy

In our `employees` table, `manager_id` references `emp_id` in the very same table:

```
SELECT
    emp.emp_name AS employee,
    COALESCE(mgr.emp_name, 'Top Level Executive') AS manager
FROM employees emp
LEFT JOIN employees mgr
    ON emp.manager_id = mgr.emp_id;
```

Result:

employee	manager
Alice	Top Level Executive
Bob	Alice
Charlie	Alice
David	Charlie
Emma	Alice
Frank	Charlie

Why LEFT JOIN? The CEO/President has `manager_id IS NULL`. An `INNER JOIN` would drop the top-level executive from the output.

4.3 Use Case 2: Finding Inter-Row Comparisons (Pairs & Precedence)

Finding all pairs of employees working in the same department:

```
SELECT
    e1.emp_name AS employee_1,
    e2.emp_name AS employee_2,
    e1.dept_id
FROM employees e1
INNER JOIN employees e2
    ON e1.dept_id = e2.dept_id
    AND e1.emp_id < e2.emp_id; -- Strict inequality prevents self-pairing and duplicate reverse pairs
```

- Using `e1.emp_id != e2.emp_id` would output both `(Alice, Bob)` and `(Bob, Alice)`.
- Using `e1.emp_id < e2.emp_id` outputs strictly unique pairings.

Part 5: Aggregation & Grouping Mechanics

5.1 The GROUP BY Clause Mechanics

`GROUP BY` transforms a detailed row-level relation into a collapsed summary relation.

- Rows sharing identical values across the specified grouping attributes are placed into discrete buckets.
- All non-grouping attributes selected in the `SELECT` list **must** be wrapped in aggregate functions (`SUM`, `AVG`, `COUNT`, `MIN`, `MAX`).

```
SELECT
    dept_id,
    COUNT(*) AS total_employees,
    AVG(salary) AS avg_salary,
    MAX(salary) AS highest_salary
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id;
```

Grouping by Expressions:

You can group by expressions, date truncations, or conditional buckets:

```
SELECT
    CASE
        WHEN salary >= 100000 THEN 'High Band'
        WHEN salary >= 80000 THEN 'Mid Band'
        ELSE 'Entry Band'
    END AS salary_tier,
    COUNT(*) AS emp_count
FROM employees
GROUP BY
    CASE
        WHEN salary >= 100000 THEN 'High Band'
        WHEN salary >= 80000 THEN 'Mid Band'
        ELSE 'Entry Band'
    END;
```

5.2 The Crucial Distinction: WHERE vs HAVING

Dimension	WHERE Clause	HAVING Clause
Stage of Execution	Step 3: Evaluated **before** grouping occurs.	Step 6: Evaluated **after** aggregation and grouping.
Target of Filtering	Operates on individual, base records (rows).	Operates on grouped, aggregated summary buckets.
Aggregate Functions	**Cannot** contain aggregate functions (`WHERE SUM(x) > 10` is an error).	**Can and usually does** contain aggregate functions (`HAVING COUNT(*) > 2`).
Index Utilization	Can directly utilize B-tree indexes to	Cannot leverage standard indexes

	prune rows early.	directly for aggregate results.
Engine Efficiency	High: Reduces volume of rows sent to grouping buffer.	Lower: Applied after expensive hashing or sorting for grouping.

Demonstration of Co-existence:

```
SELECT
    dept_id,
    AVG(salary) AS avg_sal,
    COUNT(*) AS active_staff
FROM employees
WHERE salary >= 80000          -- Step 1: Discard any employee making under $80k BEFORE grouping
GROUP BY dept_id              -- Step 2: Form groups of remaining employees
HAVING COUNT(*) >= 2;          -- Step 3: Keep only groups that have 2 or more such employees
```

Part 6: Advanced Aggregation Patterns

6.1 Conditional Aggregation (CASE inside Aggregates)

Conditional aggregation allows you to perform multi-pass aggregation in a single scan of the dataset.

Pattern 1: Horizontal Pivoting (Rows to Columns)

Given sales records with order statuses, calculate counts per month:

```
SELECT
    DATE_TRUNC('month', order_date) AS order_month,
    COUNT(*) AS total_orders,
    SUM(CASE WHEN status = 'COMPLETED' THEN 1 ELSE 0 END) AS completed_orders,
    SUM(CASE WHEN status = 'REFUNDED' THEN 1 ELSE 0 END) AS refunded_orders,
    SUM(CASE WHEN status = 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled_orders
FROM orders
GROUP BY DATE_TRUNC('month', order_date);
```

Pattern 2: Calculating KPI Ratios & Conversion Rates

```
SELECT
    dept_id,
    ROUND(
        100.0 * SUM(CASE WHEN salary >= 100000 THEN 1 ELSE 0 END) / COUNT(*),
        2
    ) AS pct_six_figure_earners
FROM employees
WHERE dept_id IS NOT NULL
GROUP BY dept_id;
```

> *[!TIP]*
> *Notice the usage of `100.0` (floating-point literal) instead of `100` to prevent integer division truncation in databases like SQL Server and PostgreSQL.*

6.2 Duplicate Detection & Deduplication Strategies

Finding Duplicate Values in a Single Column:

```
SELECT email, COUNT(*) AS occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Finding Exact Duplicate Rows Across Composite Keys:

```
SELECT first_name, last_name, date_of_birth, COUNT(*) AS occurrences
FROM patients
GROUP BY first_name, last_name, date_of_birth
HAVING COUNT(*) > 1;
```

Isolating All Fields of Duplicate Records:

```
WITH DuplicateKeys AS (
    SELECT email
    FROM users
    GROUP BY email
    HAVING COUNT(*) > 1
)
SELECT u.*
FROM users u
INNER JOIN DuplicateKeys d ON u.email = d.email
ORDER BY u.email, u.created_at;
```

Part 7: Complete Logical Query Execution Order with Joins

Recall the foundational SQL execution pipeline. Today we integrate `JOIN` and `HAVING`:

```
1. FROM        (Identify base tables and Cartesian product)
2. ON          (Apply join filters for each row pair)
3. JOIN        (Retain matches and append outer NULL-padded rows)
4. WHERE       (Filter combined intermediate rows)
5. GROUP BY    (Collapse rows into groups based on key values)
6. WITH CUBE / ROLLUP (Optional multi-dimensional aggregations)
7. HAVING      (Filter aggregate group buckets)
8. SELECT      (Evaluate projection list, expressions, aliases)
9. DISTINCT    (Eliminate duplicate output rows)
10. ORDER BY   (Sort output; can reference SELECT aliases and unselected cols)
11. OFFSET / LIMIT / TOP (Return requested slice of rows)
```

Part 8: Essential Interview Comparisons

Comparison 1: INNER JOIN vs LEFT JOIN vs CROSS JOIN

Attribute	INNER JOIN	LEFT JOIN	CROSS JOIN
Output size min	0 rows	Number of rows in Left Table	$M \times N$ rows
Output size max	$M \times N$ rows	$\max(M, M \times N)$	$M \times N$ rows exactly
Null generation	Never generates NULLs	Generates NULLs for	Never generates NULLs

		unmatched Right rows	
Predicate	Requires join condition (ON)	Requires join condition (ON)	No condition (or trivial 1=1)
Semantic Meaning	"Intersection of relationships"	"All primary entities with optional details"	"All possible combinations"

Comparison 2: ON vs WHERE in Joins

Scenario	ON Clause	WHERE Clause
INNER JOIN	Logically equivalent to WHERE (filtered during join).	Logically equivalent to ON (filtered after join).
LEFT JOIN (Right Table)	Controls which right rows attach. Preserves left rows.	Filters post-join. Eliminates left rows with NULL right attributes!
LEFT JOIN (Left Table)	Unintuitive: evaluating left predicates in ON does not discard left rows.	Standard: strictly filters out unwanted left rows.

Part 9: Comprehensive Technical Interview Question Bank

Q1: What happens if you join two tables on a column where both contain NULL values?

Answer:
In SQL, `NULL = NULL` evaluates to `UNKNOWN` in accordance with three-valued logic. Because standard join predicates require the condition to evaluate to `TRUE` to produce a match, rows with `NULL` keys will **never** match each other in an equijoin (`ON a.key = b.key`). To treat `NULL` as equal for joining, one must explicitly use `IS NOT DISTINCT FROM` (PostgreSQL) or `ON a.key = b.key OR (a.key IS NULL AND b.key IS NULL)`.

Q2: What is the maximum and minimum number of rows returned by a LEFT JOIN between Table A (100 rows) and Table B (50 rows)?

Answer:

- **Minimum:** 100 rows. A `LEFT JOIN` guarantees that every row from the left table appears at least once. If zero rows in Table B match Table A, all 100 rows appear with `NULL`s for B's columns.
- **Maximum:** $100 \times 50 = 5,000$ rows. If every single row in Table A matches all 50 rows in Table B (for instance, all rows have identical join keys or `ON 1=1`), the result is the full Cartesian product.

Q3: Why is SELECT dept_id, emp_name, AVG(salary) FROM employees GROUP BY dept_id; invalid in ANSI SQL?

Answer:
Because `emp_name` is neither enclosed within an aggregate function nor specified in the `GROUP BY` clause. Since grouping collapses multiple employees of a department into a single output row, the database engine cannot

arbitrarily choose which employee's name to display alongside the departmental average salary. (Note: SQLite and older MySQL versions allowed this by returning an arbitrary non-deterministic value, but it is illegal in standard SQL).

Q4: Explain the difference between Semi-Join and Anti-Join. How are they written in SQL?

Answer:

- **Semi-Join:** Returns rows from the left table that have at least one match in the right table, without duplicating left rows and without returning right columns. Written via `EXISTS` or `IN`:`SELECT * FROM departments d WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);`

- **Anti-Join:** Returns rows from the left table that have zero matches in the right table. Written via `NOT EXISTS` or `LEFT JOIN ... WHERE right.key IS NULL`:`SELECT * FROM departments d WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);`

Q5: Can HAVING be used without a GROUP BY clause?

Answer:
Yes. If `HAVING` is used without an explicit `GROUP BY`, the entire table is treated as a single composite group. For example:

```
SELECT AVG(salary)
FROM employees
HAVING COUNT(*) > 5;
```

If the table has 5 or fewer rows, this query returns an empty set (0 rows). If it has more than 5 rows, it returns 1 row containing the overall average salary.

Part 10: Hands-On Practice Lab (20 Problems)

Database Initialization Script for Practice

Run this schema in PostgreSQL, MySQL, SQL Server, or SQLite to follow along:

```
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS customers;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    city VARCHAR(50),
    segment VARCHAR(20)
);

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(50),
    category VARCHAR(30),
    unit_price DECIMAL(10, 2)
);
```

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    order_status VARCHAR(20),
    total_amount DECIMAL(10, 2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    price_per_unit DECIMAL(10, 2),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Seed Data
INSERT INTO customers VALUES
(101, 'Acme Corp', 'New York', 'Corporate'),
(102, 'Beta LLC', 'San Francisco', 'Consumer'),
(103, 'Cyberdyne', 'New York', 'Corporate'),
(104, 'Delta Inc', 'Chicago', 'Enterprise'),
(105, 'Epsilon Co', 'Austin', 'Consumer'),
(106, 'Foxtrot Ltd', 'Chicago', 'Enterprise');

INSERT INTO products VALUES
(1, 'Cloud Database', 'Software', 500.00),
(2, 'Analytics Suite', 'Software', 300.00),
(3, 'Security Gateway', 'Hardware', 800.00),
(4, 'Server Rack', 'Hardware', 1200.00),
(5, 'Consulting Hour', 'Services', 150.00);

INSERT INTO orders VALUES
(1001, 101, '2024-01-15', 'Delivered', 1600.00),
(1002, 101, '2024-02-10', 'Delivered', 800.00),
(1003, 102, '2024-02-14', 'Cancelled', 500.00),
(1004, 103, '2024-03-01', 'Delivered', 2400.00),
(1005, 104, '2024-03-15', 'Pending', 600.00),
(1006, 101, '2024-04-05', 'Delivered', 300.00);
-- Customer 105 and 106 have placed no orders!

INSERT INTO order_items VALUES
(1, 1001, 1, 2, 500.00),
(2, 1001, 2, 2, 300.00),
(3, 1002, 3, 1, 800.00),
(4, 1003, 1, 1, 500.00),
(5, 1004, 4, 2, 1200.00),
(6, 1005, 2, 2, 300.00),
(7, 1006, 2, 1, 300.00);
```

Practice Problem Statements (Try Before Viewing Solutions!)

- **Inner Match:** Display customer name, order ID, and order date for all delivered orders.
- **Customer Order Audit:** Display all customer names and their order IDs, including customers who have never placed an order.
- **Unmatched Customers:** Identify all customers who have never placed an order using a Left Anti-Join.
- **Product Purchase Check:** Find all products that have never been ordered.

- **Customer Spending Summary:** Calculate total amount spent by each customer across delivered orders only. Include customer name.
- **High Spenders Only:** Display customer names and total spending for customers whose delivered orders exceed $1,500.
- **Department Order Metrics:** Show total orders, total delivered orders, and total cancelled orders per customer segment.
- **Item-Level Sales Reconciliation:** For each order, display `order_id`, header `total_amount`, and sum of `quantity * price_per_unit` from `order_items` to check for discrepancies.
- **Zero-Order Cities:** List cities where not a single customer has placed a delivered order.
- **Category Breakdown:** Calculate total units sold and total revenue per product category.
- **Self Join - City Peers:** Find pairs of distinct customers located in the same city. Avoid duplicate reverse pairs.
- **Multi-Item Orders:** Find all order IDs that contain more than 1 distinct product.
- **Product Popularity Rank:** List product names alongside the count of unique customers who bought them.
- **Customer Multi-Order Frequency:** List customers who have placed 2 or more delivered orders.
- **Conditional Aggregation - Status Split:** Display customer ID and two columns: `delivered_spend` and `non_delivered_spend`.
- **Average Order Value per Segment:** Find the average delivered order amount grouped by customer segment.
- **Duplicate Detection:** In the `customers` table, write a query to detect if duplicate `customer_name` entries exist.
- **Cross Join Combinations:** Generate all possible combinations of customer segments and product categories.
- **Orders Containing Hardware:** List customer names that have ordered at least one item from the 'Hardware' category.
- **Comprehensive Reconciliation:** For each customer, show customer name, count of total orders, count of delivered orders, and total net delivered revenue (default to 0.00 if none).

Part 11: Solutions & Explanations to Practice Problems

Solution 1

```
SELECT c.customer_name, o.order_id, o.order_date
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered';
```

Explanation: Standard inner join filtered by status. Eliminates non-purchasing customers and non-delivered orders.

Solution 2

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

Explanation: Preserves all customers (Epsilon and Foxtrot output with `NULL` for `order_id`).

Solution 3

```
SELECT c.customer_id, c.customer_name, c.city
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Explanation: Anti-join pattern. Evaluates primary key of right table for `NULL`.

Solution 4

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.item_id IS NULL;
```

Explanation: Identifies Consulting Hour (product 5) which has zero sales rows.

Solution 5

```
SELECT c.customer_name, SUM(o.total_amount) AS total_spent
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.customer_id, c.customer_name;
```

Explanation: Includes `c.customer_id` in `GROUP BY` to ensure proper grouping if two customers share a name.

Solution 6

```
SELECT c.customer_name, SUM(o.total_amount) AS total_spent
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.customer_id, c.customer_name
HAVING SUM(o.total_amount) > 1500;
```

Explanation: `HAVING` filters aggregate sum after groups are constructed.

Solution 7

```
SELECT
    c.segment,
    COUNT(o.order_id) AS total_orders,
    SUM(CASE WHEN o.order_status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_orders,
    SUM(CASE WHEN o.order_status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.segment;
```

Explanation: Conditional aggregation counts order statuses across customer segments.

Solution 8

```
SELECT
    o.order_id,
    o.total_amount AS header_amount,
    SUM(oi.quantity * oi.price_per_unit) AS calculated_items_amount,
    (o.total_amount - SUM(oi.quantity * oi.price_per_unit)) AS discrepancy
FROM orders o
INNER JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, o.total_amount;
```

Explanation: Validates relational data integrity between header and line items.

Solution 9

```
SELECT DISTINCT c.city
FROM customers c
WHERE c.city NOT IN (
    SELECT DISTINCT c2.city
    FROM customers c2
    INNER JOIN orders o ON c2.customer_id = o.customer_id
```

```
    WHERE o.order_status = 'Delivered'
);
```

Alternative via Joins:

```
SELECT c.city
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
    AND o.order_status = 'Delivered'
GROUP BY c.city
HAVING COUNT(o.order_id) = 0;
```

Solution 10

```
SELECT
    p.category,
    COALESCE(SUM(oi.quantity), 0) AS total_units_sold,
    COALESCE(SUM(oi.quantity * oi.price_per_unit), 0.00) AS total_revenue
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category;
```

Solution 11

```
SELECT
    c1.customer_name AS customer_a,
    c2.customer_name AS customer_b,
    c1.city
FROM customers c1
INNER JOIN customers c2
    ON c1.city = c2.city
    AND c1.customer_id < c2.customer_id;
```

Explanation: Strictly ordered ID comparison prevents duplicate permutations.

Solution 12

```
SELECT order_id, COUNT(DISTINCT product_id) AS unique_products
FROM order_items
GROUP BY order_id
HAVING COUNT(DISTINCT product_id) > 1;
```

Solution 13

```
SELECT
    p.product_name,
    COUNT(DISTINCT o.customer_id) AS unique_buyers
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
LEFT JOIN orders o ON oi.order_id = o.order_id
GROUP BY p.product_id, p.product_name;
```

Solution 14

```
SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS delivered_count
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) >= 2;
```

Solution 15

```
SELECT
    customer_id,
```

```
    SUM(CASE WHEN order_status = 'Delivered' THEN total_amount ELSE 0 END) AS delivered_spend,
    SUM(CASE WHEN order_status != 'Delivered' THEN total_amount ELSE 0 END) AS non_delivered_spend
FROM orders
GROUP BY customer_id;
```

Solution 16

```
SELECT
    c.segment,
    ROUND(AVG(o.total_amount), 2) AS avg_order_value
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.segment;
```

Solution 17

```
SELECT customer_name, COUNT(*) AS occurrences
FROM customers
GROUP BY customer_name
HAVING COUNT(*) > 1;
```

Solution 18

```
SELECT DISTINCT c.segment, p.category
FROM customers c
CROSS JOIN products p;
```

Solution 19

```
SELECT DISTINCT c.customer_name
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN order_items oi ON o.order_id = oi.order_id
INNER JOIN products p ON oi.product_id = p.product_id
WHERE p.category = 'Hardware';
```

Solution 20

```
SELECT
    c.customer_id,
    c.customer_name,
    COUNT(o.order_id) AS total_orders,
    COUNT(CASE WHEN o.order_status = 'Delivered' THEN 1 END) AS delivered_orders,
    COALESCE(SUM(CASE WHEN o.order_status = 'Delivered' THEN o.total_amount ELSE 0 END), 0.00) AS
net_delivered_revenue
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY net_delivered_revenue DESC;
```

Part 12: End-to-End Enterprise Case Study: E-Commerce Revenue Reconciliation & Audit

Business Context & Problem Statement

At **DataScale Retail**, finance discovered that the reported monthly GMV (Gross Merchandise Value) calculated from the `orders` header table does not match the sum of individual line items in the `order_items` table. Furthermore,

customer lifetime value (LTV) models failed because customers without orders were excluded from cohort retention curves.

As a Senior Data Engineer, you must build an audit query that:

- Reconciles every order header with its itemized sum.
- Identifies phantom orders (orders with no items) and orphan items (items with invalid `order_id`).
- Produces a consolidated customer revenue table accurately representing all customers, including non-buyers.

Implementation Solution

```
-- Comprehensive Financial Integrity & Reconciliation Model
WITH OrderReconciliation AS (
    SELECT
        o.order_id,
        o.customer_id,
        o.order_status,
        o.total_amount AS reported_header_amount,
        COALESCE(SUM(oi.quantity * oi.price_per_unit), 0.00) AS calculated_items_amount,
        (o.total_amount - COALESCE(SUM(oi.quantity * oi.price_per_unit), 0.00)) AS discrepancy
    FROM orders o
    LEFT JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY o.order_id, o.customer_id, o.order_status, o.total_amount
)
SELECT
    c.customer_id,
    c.customer_name,
    c.segment,
    COUNT(r.order_id) AS total_placed_orders,
    SUM(CASE WHEN r.order_status = 'Delivered' THEN 1 ELSE 0 END) AS valid_delivered_orders,
    COALESCE(SUM(CASE WHEN r.order_status = 'Delivered' THEN r.calculated_items_amount ELSE 0 END),
0.00) AS audited_net_revenue,
    SUM(CASE WHEN ABS(r.discrepancy) > 0.01 THEN 1 ELSE 0 END) AS orders_with_discrepancies
FROM customers c
LEFT JOIN OrderReconciliation r ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.customer_name, c.segment
ORDER BY audited_net_revenue DESC;
```

Part 13: 10-Minute Interview Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                    SQL JOINS & AGGREGATION CHEAT SHEET           │
├──────────────────────────────────────────────────────────────────┤
│ 1. INNER JOIN : Returns rows with matching keys in BOTH tables.  │
│ 2. LEFT JOIN  : All left rows; matching right rows or NULLs.     │
│ 3. RIGHT JOIN : All right rows; matching left rows or NULLs.     │
│ 4. FULL JOIN  : All rows from both; NULL-padded where unmatched. │
│ 5. CROSS JOIN : Cartesian product (M x N rows). No ON clause.    │
│ 6. ANTI-JOIN  : LEFT JOIN ... WHERE right.key IS NULL            │
│                 (Finds unmatched records).                       │
│ 7. SEMI-JOIN  : WHERE EXISTS (SELECT 1 FROM ...)                 │
│                 (Filters without expanding duplicate matches).   │
├──────────────────────────────────────────────────────────────────┤
│                       WHERE vs HAVING                            │
│  - WHERE filters base rows BEFORE grouping. No aggregate functions.  │
│  - HAVING filters group summaries AFTER aggregation.             │
├──────────────────────────────────────────────────────────────────┤
│                     CONDITIONAL AGGREGATION                      │
│  SUM(CASE WHEN status = 'Delivered' THEN revenue ELSE 0 END)     │
```

```
|  COUNT(CASE WHEN flag = 1 THEN 1 END) -- NULLs ignored by COUNT!        |
|-------------------------------------------------------------------------|
|                          COMMON PITFALLS                                |
|  - Filtering right table in WHERE clause converts LEFT JOIN to INNER.   |
|  - NULL = NULL is UNKNOWN (joins will never match on NULL).             |
|  - Duplicate join keys cause row explosion (M x N multiplication).      |
|  - Selecting unaggregated columns not in GROUP BY is invalid SQL.       |
```

Part 14: 100-Mark Self-Assessment Examination

Section A: Technical Multiple Choice & Conceptual (25 Marks — 5 marks each)

Q1: Given Table A with 5 rows (all keys = 1) and Table B with 4 rows (all keys = 1). How many rows does an `INNER JOIN ON A.key = B.key` return?

(a) 5

(b) 4

(c) 9

(d) 20

Q2: A query has `FROM A LEFT JOIN B ON A.id = B.id WHERE B.status = 'ACTIVE'`. What is the practical consequence of this query?

(a) All rows from A are preserved, and only active rows from B attach.

(b) The query behaves as an `INNER JOIN`, discarding all rows of A with no match in B.

(c) The query throws a compilation syntax error.

(d) Unmatched rows from A are assigned status 'ACTIVE'.

Q3: What is the result of `COUNT(CASE WHEN score >= 50 THEN NULL ELSE 1 END)` when evaluated against 10 students where 6 have score ≥ 50 and 4 have score < 50?

(a) 10

(b) 6

(c) 4

(d) 0

Q4: In SQL logical execution order, which clause is executed immediately before `HAVING`?

(a) WHERE

(b) SELECT

(c) GROUP BY

(d) ORDER BY

Q5: Why does `ON A.id = B.id` fail to match rows where `A.id` is `NULL` and `B.id` is `NULL`?

(a) SQL requires primary keys to be unique.

(b) `NULL = NULL` evaluates to `UNKNOWN` in SQL 3-valued logic.

(c) Query optimizer skips NULL values for indexing.

(d) NULL is converted to 0 during join execution.

Section B: Output Prediction (25 Marks)

Schema for Prediction:

```
CREATE TABLE t1 (id INT, val VARCHAR(10));
CREATE TABLE t2 (id INT, val VARCHAR(10));

INSERT INTO t1 VALUES (1, 'A'), (2, 'B'), (NULL, 'C');
INSERT INTO t2 VALUES (1, 'X'), (1, 'Y'), (3, 'Z'), (NULL, 'W');
```

Q6 (10 Marks): Write down the exact output rows (id and val from both tables) for:

```
SELECT t1.id AS t1_id, t2.id AS t2_id, t1.val, t2.val
FROM t1
INNER JOIN t2 ON t1.id = t2.id;
```

Q7 (15 Marks): Write down the exact output rows for:

```
SELECT t1.id AS t1_id, t2.id AS t2_id, t1.val, t2.val
FROM t1
LEFT JOIN t2 ON t1.id = t2.id;
```

Section C: Query Writing & Code Implementation (30 Marks)

Q8 (15 Marks): Given a table `transactions (txn_id, user_id, txn_date, amount, status)`, write a single query to compute:

- `user_id`
- Total transaction count
- Total amount of 'SUCCESS' transactions
- Total amount of 'FAILED' transactions
- Success rate as a percentage (`100.0 * success_count / total_count`) rounded to 2 decimal places.

Only include users who have at least 3 total transactions.

Q9 (15 Marks): Given an `employees (emp_id, emp_name, salary, manager_id)` table, write a query using a `SELF JOIN` to find all employees who earn strictly more than their direct manager. Display `employee_name`, `employee_salary`, `manager_name`, and `manager_salary`.

Section D: Real-World Scenario & Optimization (20 Marks)

Q10 (20 Marks): You are investigating an ETL pipeline that runs every night. The query joins a 50-million-row `events` table with a 50,000-row `users` table on `user_id`.

- Which join physical algorithm (Nested Loop, Hash Join, Sort-Merge) will the query planner most likely select, and why? (7 marks)
- The query runtime recently spiked from 2 minutes to 45 minutes after an index was dropped. Explain how an index on `events.user_id` vs `users.user_id` affects performance. (7 marks)
- If `users` contains duplicate `user_id` records due to an upstream failure, describe the exact impact on the event count metrics. (6 marks)

Assessment Answer Key & Scoring Guide

Section A Solutions

- **Q1: (d)** 20. Cartesian match on keys: $5 \times 4 = 20$.
- **Q2: (b)** The query behaves as an `INNER JOIN`. When unmatched, `B.status` is `NULL`. `WHERE NULL = 'ACTIVE'` evaluates to `UNKNOWN` and is discarded.
- **Q3: (c)** 4. `COUNT(column)` ignores `NULL`s. For the 6 students with score ≥ 50, the `CASE` returns `NULL`. For the other 4, it returns `1`. `COUNT` of four `1`s and six `NULL`s is 4.
- **Q4: (c)** `GROUP BY`.
- **Q5: (b)** `NULL = NULL` yields `UNKNOWN`, not `TRUE`.

Section B Solutions

Q6 Output:

t1_id	t2_id	val	val
1	1	A	X
1	1	A	Y

(2 rows total. NULLs and unmatched keys 2 and 3 do not match).

Q7 Output:

t1_id	t2_id	val	val
1	1	A	X
1	1	A	Y

2	NULL	B	NULL
NULL	NULL	C	NULL

(4 rows total. Left table rows 2 and NULL are preserved with NULL padding).

Section C Solutions

Q8 Solution:

```
SELECT
    user_id,
    COUNT(*) AS total_txns,
    COALESCE(SUM(CASE WHEN status = 'SUCCESS' THEN amount ELSE 0 END), 0) AS success_amount,
    COALESCE(SUM(CASE WHEN status = 'FAILED' THEN amount ELSE 0 END), 0) AS failed_amount,
    ROUND(
        100.0 * SUM(CASE WHEN status = 'SUCCESS' THEN 1 ELSE 0 END) / COUNT(*),
        2
    ) AS success_rate_pct
FROM transactions
GROUP BY user_id
HAVING COUNT(*) >= 3;
```

Q9 Solution:

```
SELECT
    e.emp_name AS employee_name,
    e.salary AS employee_salary,
    m.emp_name AS manager_name,
    m.salary AS manager_salary
FROM employees e
INNER JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Section D Solutions

Q10 Solution:

- **Hash Join:** The planner will build an in-memory hash table on the small 50k-row `users` table and stream the 50M-row `events` table to probe keys ($O(M+N)$).
- If `events.user_id` is queried in an indexed loop without hash join capability, missing an index causes a full sequential scan for every probe. Conversely, having an index on `events.user_id` allows rapid indexed lookups if queries filter by a specific subset of users.
- If `users` has duplicates on `user_id`, the join key becomes $1 \text{ to } N$ rather than $1 \text{ to } 1$. Every event associated with that user will be duplicated N times, severely inflating metrics like total pageviews, revenue, and active session counts.

Day 3: Intermediate & Advanced SQL

Subqueries, CTEs, Window Functions & Analytical Querying

Part 1: Subqueries & Derived Tables

1.1 Types of Subqueries

A subquery (or nested query) is an inner query embedded within an outer SQL statement.

- **Scalar Subquery:** Returns exactly one row and one column (a single scalar value). Can be placed anywhere an expression is valid (`SELECT`, `WHERE`, `HAVING`, `ORDER BY`).

```
-- Find employees earning above the company-wide average
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

- **Multi-Row Subquery:** Returns a single column containing multiple rows. Evaluated using multi-value comparison operators (`IN`, `NOT IN`, `ANY` / `SOME`, `ALL`).

```
-- Find employees working in US-based departments
SELECT emp_name
FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'USA');
```

- **Multi-Column Subquery:** Returns multiple columns and multiple rows (supported in PostgreSQL, MySQL, Oracle):

```
SELECT emp_id, dept_id, salary
FROM employees
WHERE (dept_id, salary) IN (
    SELECT dept_id, MAX(salary)
    FROM employees
    GROUP BY dept_id
);
```

- **Derived Table (Table Subquery in FROM):** A subquery in the `FROM` clause that acts as a temporary inline relation. Must be given an explicit table alias:

```
SELECT dept_id, avg_sal
FROM (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY dept_id
) AS dept_summary
WHERE avg_sal > 90000;
```

1.2 Correlated Subqueries & The EXISTS Operator

Definition & Mechanics

Unlike a standard subquery (which executes once, caching its result for the outer query), a **correlated subquery** references columns from the outer query's row. It must be evaluated **once for every candidate row** processed by the outer query.

```
-- Find employees who earn more than the average salary of THEIR OWN department
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(sub.salary)
    FROM employees sub
    WHERE sub.dept_id = e.dept_id -- Correlation predicate linking inner to outer
);
```

EXISTS vs IN vs JOIN Performance Internals

- `EXISTS` checks for the **existence** of at least one row satisfying the condition. As soon as the storage engine finds a single match, evaluation terminates immediately (short-circuit boolean optimization).
- `IN` must evaluate and collect the entire set of matching values before testing membership.
- **The NULL Trap with NOT IN vs NOT EXISTS:**
  If an inner subquery evaluated by `NOT IN` returns even a single `NULL` value, the entire `NOT IN` condition evaluates to `UNKNOWN` for all rows, returning **zero results**:

```
-- DANGEROUS: If subquery returns (1, 2, NULL), this outputs NOTHING!
SELECT emp_name FROM employees
WHERE emp_id NOT IN (SELECT manager_id FROM employees);

-- BULLETPROOF: Handles NULLs safely and short-circuits efficiently
SELECT emp_name FROM employees e
WHERE NOT EXISTS (
    SELECT 1 FROM employees m WHERE m.manager_id = e.emp_id
);
```

Part 2: Common Table Expressions (CTEs) & Recursive CTEs

2.1 Standard CTEs (WITH Clause)

A CTE defines a temporary, named result set within the execution scope of a single `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement.

Advantages Over Subqueries:

- **Readability & Modularity:** Queries read linearly from top to bottom instead of deeply nested inside-out logic.

- **Reusability:** A CTE can be referenced multiple times within the subsequent query without recalculating or repeating code.

```
WITH HighEarners AS (
    SELECT emp_id, emp_name, dept_id, salary
    FROM employees
    WHERE salary > 100000
),
DeptStats AS (
    SELECT dept_id, COUNT(*) AS count_high_earners
    FROM HighEarners
    GROUP BY dept_id
)
SELECT d.dept_name, ds.count_high_earners
FROM DeptStats ds
INNER JOIN departments d ON ds.dept_id = d.dept_id;
```

2.2 Recursive CTEs

A **Recursive CTE** references itself in its own definition. It is the relational solution for processing hierarchical structures (organizational charts, bill-of-materials, network graphs, file systems) and generating series.

Structure of a Recursive CTE:

A recursive CTE contains two parts joined by `UNION ALL`:

- **Anchor Member:** The base query that selects initial rows (e.g., top-level root nodes).
- **Recursive Member:** Joins the CTE back to the base table to fetch child records until the join produces zero rows (termination condition).

Example 1: Traversing an Organizational Hierarchy (CEO to Direct & Indirect Reports)

```
WITH RECURSIVE OrgHierarchy AS (
    -- Anchor Member: Select the CEO (manager_id IS NULL)
    SELECT
        emp_id,
        emp_name,
        manager_id,
        1 AS org_level,
        CAST(emp_name AS VARCHAR(1000)) AS reporting_path
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive Member: Join employees to OrgHierarchy on manager_id
    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        h.org_level + 1,
        CAST(h.reporting_path || ' -> ' || e.emp_name AS VARCHAR(1000))
    FROM employees e
    INNER JOIN OrgHierarchy h ON e.manager_id = h.emp_id
)
SELECT emp_id, emp_name, org_level, reporting_path
FROM OrgHierarchy
ORDER BY org_level, emp_id;
```

Example 2: Generating a Date Series (Calendar Dimension Generation)

```
-- Generate daily dates for January 2024 (PostgreSQL / SQLite / SQL Server)
WITH RECURSIVE DateGenerator AS (
    SELECT CAST('2024-01-01' AS DATE) AS calendar_date
    UNION ALL
    SELECT calendar_date + INTERVAL '1 day'
    FROM DateGenerator
    WHERE calendar_date < '2024-01-31'
)
SELECT calendar_date FROM DateGenerator;
```

Part 3: Analytical Window Functions

3.1 What is a Window Function?

Unlike a standard `GROUP BY` aggregation (which collapses multiple rows into a single summary row), a **Window Function** computes values over a specified window (subset) of rows **while preserving the identity and granularity of every individual row**.

```
Input: 100 rows ---> GROUP BY dept_id ---> Output: 5 rows (collapsed)
Input: 100 rows ---> Window Function  ---> Output: 100 rows (original rows retained + calculated metric appended)
```

3.2 Anatomy of the OVER() Clause

```
FUNCTION_NAME(...) OVER (
    PARTITION BY partition_column_1, ...
    ORDER BY sort_column_1 ASC/DESC, ...
    ROWS | RANGE BETWEEN frame_start AND frame_end
)
```

- **PARTITION BY:** Divides the rows into distinct groups (windows) to which the function applies. If omitted, the entire result set is treated as a single partition.
- **ORDER BY:** Defines the logical sequence of rows within each partition. Critical for running totals, rankings, and lead/lag operations.
- **Window Framing (ROWS vs RANGE):** Defines the exact sliding physical or logical boundary of rows evaluated relative to the current row.

3.3 The Ranking Functions: ROW_NUMBER, RANK, DENSE_RANK, NTILE

This is universally tested in technical interviews.

Function	Behavior with Ties (Identical Values)	Gap in Numbering Sequence?	Example on Scores: [100, 90, 90, 80]
ROW_NUMBER()	Strictly arbitrary / sequential numbering. Ties receive consecutive integers.	No	1, 2, 3, 4
RANK()	Tied rows receive identical rank. Skips subsequent ranks proportional to tie count.	**Yes**	1, 2, 2, **4**

DENSE_RANK()	Tied rows receive identical rank. Does **not** skip subsequent ranks.	**No**	1, 2, 2, **3**
NTILE(N)	Divides partition into N roughly equal buckets (quartiles, deciles, percentiles).	No	For $N=2$: 1, 1, 2, 2

Visual Comparison Query:

```
SELECT
    emp_name,
    dept_id,
    salary,
    ROW_NUMBER() OVER(PARTITION BY dept_id ORDER BY salary DESC) AS row_num,
    RANK()       OVER(PARTITION BY dept_id ORDER BY salary DESC) AS rnk,
    DENSE_RANK() OVER(PARTITION BY dept_id ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

3.4 Value Navigation Functions: LAG and LEAD

Syntax & Parameters:

- `LAG(expression, [offset], [default_value]) OVER (...)`: Fetches value from a row N steps prior to the current row.
- `LEAD(expression, [offset], [default_value]) OVER (...)`: Fetches value from a row N steps after the current row.

```
SELECT
    customer_id,
    order_date,
    total_amount,
    LAG(total_amount, 1, 0.00) OVER (
        PARTITION BY customer_id
        ORDER BY order_date
    ) AS previous_order_amount,
    (total_amount - LAG(total_amount, 1, total_amount) OVER (
        PARTITION BY customer_id
        ORDER BY order_date
    )) AS spend_delta_vs_last_order
FROM orders;
```

First and Last Values:

- `FIRST_VALUE(col) OVER (...)`: Returns first value in the current window frame.
- `LAST_VALUE(col) OVER (...)`: Returns last value in the current window frame.

> *[!WARNING]*
> *By default, adding* `ORDER BY` *to an* `OVER` *clause sets the default frame to* `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`*. This causes* `LAST_VALUE()` *to return the current row's value rather than the partition's ultimate value! To fix this, explicitly specify* `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`*.*

Part 4: Window Framing Specifications (ROWS vs RANGE)

4.1 Physical vs Logical Framing

- **ROWS (Physical Framing):** Considers exact physical offsets of rows.
- **RANGE (Logical Framing):** Considers value offsets according to the `ORDER BY` value (groups duplicate values together).

4.2 Standard Frame Declarations

```
-- 1. True Cumulative Running Total (From start of partition to current row)
SUM(amount) OVER (
    PARTITION BY account_id
    ORDER BY txn_date
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
)

-- 2. 7-Day / 3-Row Rolling Moving Average (Current row and preceding 2 rows)
AVG(amount) OVER (
    PARTITION BY account_id
    ORDER BY txn_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
)

-- 3. Centered Moving Average (Preceding 1 row, current row, following 1 row)
AVG(amount) OVER (
    ORDER BY txn_date
    ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
)

-- 4. Full Partition Total (Without collapsing rows)
SUM(amount) OVER (
    PARTITION BY account_id
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
)
```

Part 5: Classic Interview Problem Archetypes

Archetype 1: Second-Highest Salary

Interviewers test this to see if you understand `NULL` edge cases and multiple salary ties.

Approach 1: Window Function CTE (DENSE_RANK) — Recommended

```
WITH RankedSalaries AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT COALESCE(
    (SELECT DISTINCT salary FROM RankedSalaries WHERE rnk = 2),
    NULL
) AS SecondHighestSalary;
```

Why DENSE_RANK? If the two highest employees both make $150k, `RANK()` gives them (1, 1, 3). The second distinct salary ($120k) is rank 2 in `DENSE_RANK()`.

Approach 2: Correlated Subquery / Scalar Filter

```
SELECT MAX(salary) AS SecondHighestSalary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Approach 3: LIMIT and OFFSET

```
SELECT DISTINCT salary AS SecondHighestSalary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Limitation of Approach 3: If there are fewer than 2 distinct salaries, Approach 3 returns an empty set instead of `NULL` unless wrapped in an outer scalar query.

Archetype 2: Top-N Records per Group (Top 3 Salaries per Department)

```
WITH DepartmentSalaries AS (
    SELECT
        e.emp_id,
        e.emp_name,
        e.salary,
        d.dept_name,
        DENSE_RANK() OVER (
            PARTITION BY e.dept_id
            ORDER BY e.salary DESC
        ) AS salary_rank
    FROM employees e
    INNER JOIN departments d ON e.dept_id = d.dept_id
)
SELECT dept_name, emp_name, salary, salary_rank
FROM DepartmentSalaries
WHERE salary_rank <= 3
ORDER BY dept_name, salary_rank, emp_name;
```

Archetype 3: Latest Transaction per Customer

```
WITH RankedOrders AS (
    SELECT
        order_id,
        customer_id,
        order_date,
        total_amount,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY order_date DESC, order_id DESC
        ) AS seq
    FROM orders
)
SELECT order_id, customer_id, order_date, total_amount
FROM RankedOrders
WHERE seq = 1;
```

Archetype 4: Gaps & Islands (Finding Consecutive Activity Days)

Given a table of user logins, identify streaks of consecutive active days:

```
WITH DistinctLogins AS (
    -- Step 1: Deduplicate multiple logins on the same day
    SELECT DISTINCT user_id, CAST(login_time AS DATE) AS login_date
    FROM user_activity
),
DatedSequences AS (
    -- Step 2: Subtract ROW_NUMBER() days from login_date to find constant anchor date
    SELECT
        user_id,
        login_date,
        ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS rn,
        login_date - CAST(ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date) AS INT) *
INTERVAL '1 day' AS island_group
    FROM DistinctLogins
)
-- Step 3: Group by user and island_group to measure streak length
SELECT
    user_id,
    MIN(login_date) AS streak_start,
    MAX(login_date) AS streak_end,
    COUNT(*) AS consecutive_days
FROM DatedSequences
GROUP BY user_id, island_group
HAVING COUNT(*) >= 3
ORDER BY consecutive_days DESC;
```

Mathematical Intuition: If dates are consecutive ($D, D+1, D+2$) and row numbers are consecutive ($1, 2, 3$), the difference $(D + k) - k = D$ remains completely constant for all consecutive dates! When a gap occurs, the constant shifts.

Part 6: Logical Execution Order with Window Functions

Where do Window Functions fit in the engine pipeline?

```
1. FROM / ON / JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. WINDOW FUNCTIONS (Evaluated here, alongside projection)
6. SELECT (Expressions, aliases, projection)
7. DISTINCT
8. ORDER BY
9. LIMIT / OFFSET
```

> *[!IMPORTANT]*
> ***Can you put a Window Function in the WHERE clause?***
> ***NO.*** *Because* `WHERE` *(Step 2) executes before Window Functions (Step 5), writing* `WHERE ROW_NUMBER() OVER(...) = 1` *causes an immediate syntax error. You* ***must*** *compute the window function inside a CTE or derived table first, and then filter in the outer query's* `WHERE` *clause.*

Part 7: Comprehensive Comparison Guide

Comparison 1: CTE vs Subquery vs Temporary Table vs View

Structure	Scope / Lifespan	Stored on Disk?	Indexed?	Best Used For
CTE	Single statement	In-memory (transient)	No	Modularizing complex single-query pipelines, recursion.
Subquery	Single statement / clause	In-memory	No	Simple scalar comparisons or quick exists checks.
Temp Table (#table)	Session / Connection	TempDB / Disk	**Yes** (Supports indexes & statistics)	Staging millions of intermediate rows across multiple pipeline steps.
View	Permanent database object	Schema only (metadata)	Indexed views only	Standardized security, abstracting complex joins for analysts.

Part 8: Structured Interview Question Bank

Q1: What is the difference between EXISTS and IN when evaluating subqueries containing NULL values?

Answer:
`IN` expands to a series of `OR` comparisons (`x = val1 OR x = val2 ...`). If the subquery contains a `NULL`, `x IN (1, NULL)` evaluates to `TRUE` if $x=1$, but `UNKNOWN` if $x \ne 1$. Consequently, `NOT IN` will fail completely if any returned value is `NULL`. In contrast, `EXISTS` checks solely for row presence (`count > 0`); `NULL` rows inside `EXISTS` evaluate as valid rows, so `EXISTS` and `NOT EXISTS` handle null-laden tables safely without returning unexpected empty sets.

Q2: How does COUNT(*) differ from COUNT(col) when used as a window function?

Answer:

- `COUNT(*) OVER (PARTITION BY dept_id)` counts all rows in the partition, regardless of whether individual columns contain `NULL`s.
- `COUNT(col) OVER (PARTITION BY dept_id)` counts only rows where `col IS NOT NULL`.

Q3: What is the default window frame when ORDER BY is specified inside OVER(), and why does it matter?

Answer:
When `ORDER BY` is present, ANSI SQL defaults the frame to:

`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.
This matters for two reasons:

- `SUM(val) OVER (ORDER BY dt)` will compute a running cumulative sum rather than the total partition sum.
- If duplicate values exist in the `ORDER BY` column, `RANGE` evaluates duplicates simultaneously, causing the running sum to jump rather than increment smoothly row-by-row (as `ROWS` would).

Q4: Explain the difference between NTILE(4) and percentile functions.

Answer:
`NTILE(4)` divides the ordered rows into 4 equal cardinality bins ($1, 2, 3, 4$), assigning row counts as evenly as possible. Percentile functions like `PERCENT_RANK()` or `CUME_DIST()` compute a relative fractional rank between 0.0 and 1.0 based on value distribution rather than discrete row bucket indices.

Part 9: Hands-On Practice Lab (20 Problems)

Database Initialization Script

```
DROP TABLE IF EXISTS sales_performance;
DROP TABLE IF EXISTS project_assignments;
DROP TABLE IF EXISTS staff;
DROP TABLE IF EXISTS divisions;

CREATE TABLE divisions (
    div_id INT PRIMARY KEY,
    div_name VARCHAR(50),
    headquarters VARCHAR(50)
);

CREATE TABLE staff (
    staff_id INT PRIMARY KEY,
    name VARCHAR(50),
    div_id INT,
    role VARCHAR(50),
    salary DECIMAL(10,2),
    hire_date DATE,
    FOREIGN KEY (div_id) REFERENCES divisions(div_id)
);

CREATE TABLE sales_performance (
    sale_id INT PRIMARY KEY,
    staff_id INT,
    sale_date DATE,
    amount DECIMAL(10,2),
    region VARCHAR(30),
    FOREIGN KEY (staff_id) REFERENCES staff(staff_id)
);

-- Seed Data
INSERT INTO divisions VALUES
(1, 'Cloud Services', 'Seattle'),
(2, 'Enterprise AI', 'San Francisco'),
(3, 'Cyber Security', 'Austin');

INSERT INTO staff VALUES
(101, 'Alex Rivera', 1, 'Senior Architect', 150000.00, '2020-03-15'),
(102, 'Brenda Vance', 1, 'Cloud Engineer', 110000.00, '2021-06-01'),
```

```
(103, 'Carlos Ortiz', 1, 'Cloud Engineer', 110000.00, '2022-01-15'),
(104, 'Diana Prince', 2, 'Research Scientist', 160000.00, '2019-11-01'),
(105, 'Evan Wright', 2, 'ML Engineer', 130000.00, '2021-02-20'),
(106, 'Fiona Gallagher', 2, 'ML Engineer', 130000.00, '2023-08-10'),
(107, 'George Clark', 3, 'Security Analyst', 95000.00, '2022-05-12'),
(108, 'Hannah Abbott', 3, 'Security Analyst', 95000.00, '2023-01-10');

INSERT INTO sales_performance VALUES
(1, 101, '2024-01-05', 12000.00, 'West'),
(2, 101, '2024-01-20', 18000.00, 'West'),
(3, 102, '2024-01-15', 7500.00, 'West'),
(4, 102, '2024-02-10', 9200.00, 'West'),
(5, 103, '2024-01-28', 11000.00, 'West'),
(6, 104, '2024-01-10', 25000.00, 'North'),
(7, 104, '2024-02-18', 30000.00, 'North'),
(8, 105, '2024-01-22', 14000.00, 'East'),
(9, 105, '2024-02-25', 16000.00, 'East'),
(10, 106, '2024-02-05', 8000.00, 'East'),
(11, 107, '2024-01-18', 6000.00, 'South'),
(12, 107, '2024-02-12', 7200.00, 'South');
```

Practice Problem Statements (Try Before Looking at Answers!)

- **Company Average Comparison:** Find staff names who earn strictly more than the company-wide average salary.
- **Division Peer Comparison:** Find staff members who earn more than the average salary of their respective division.
- **Division Salary Ranks:** Assign `ROW_NUMBER`, `RANK`, and `DENSE_RANK` to staff within each division based on salary descending.
- **Second Highest Salary Overall:** Find the second highest distinct salary across all staff members.
- **Top Earner per Division:** Identify the top earner in each division using a window CTE. (If tied, display both).
- **Cumulative Division Payroll:** For each division, list staff ordered by `hire_date` along with a running total of salaries paid up to that hire.
- **Salary Delta to Top Earner:** For each staff member, show their salary and the difference between their salary and the maximum salary in their division.
- **Previous Sale Comparison:** For each sale made by Alex Rivera (staff_id = 101), show the sale date, amount, and the amount of the immediately preceding sale.
- **Sales Growth Percentage:** Calculate the percentage growth in sale amount between consecutive sales for each staff member.
- **Rolling 2-Sale Moving Average:** Compute a moving average of sale amounts considering the current sale and the immediately prior sale for each staff member.
- **Quartile Salary Buckets:** Divide all staff into 4 compensation quartiles using `NTILE(4)`.
- **First and Most Recent Sale:** For each staff member, display their earliest and latest sale date side-by-side on each sales row.
- **Correlated Existence:** List all divisions that have at least one employee earning over $140,000 using `EXISTS`.
- **Anti-Join with Window Context:** Find staff members who have never generated a sale using a subquery.
- **Top 2 Sales per Region:** Find the top 2 highest sale amounts for each region.
- **Sales Contribution Ratio:** For each sale, show the percentage contribution it made to the staff member's total lifetime sales.

- **Dense Rank Without Gaps:** Find staff members holding the 2nd lowest distinct salary across the entire company.
- **Subquery in SELECT:** Display staff name, salary, and the difference from the lowest salary in the entire company without using window functions.
- **Lead Sale Date Difference:** Calculate the number of days between each sale and the subsequent sale for staff member 101.
- **Recursive Number Generation:** Use a recursive CTE to output numbers from 1 to 10.

Part 10: Solutions & In-Depth Walkthroughs

Solution 1

```
SELECT name, salary
FROM staff
WHERE salary > (SELECT AVG(salary) FROM staff);
```

Solution 2

```
SELECT s.name, s.div_id, s.salary
FROM staff s
WHERE s.salary > (
    SELECT AVG(sub.salary)
    FROM staff sub
    WHERE sub.div_id = s.div_id
);
```

Solution 3

```
SELECT
    div_id,
    name,
    salary,
    ROW_NUMBER() OVER(PARTITION BY div_id ORDER BY salary DESC) AS seq_num,
    RANK() OVER(PARTITION BY div_id ORDER BY salary DESC) AS rnk,
    DENSE_RANK() OVER(PARTITION BY div_id ORDER BY salary DESC) AS dense_rnk
FROM staff;
```

Solution 4

```
WITH RankedSalaries AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM staff
)
SELECT MAX(salary) AS second_highest_salary
FROM RankedSalaries
WHERE rnk = 2;
```

Solution 5

```
WITH RankedStaff AS (
    SELECT
        s.name,
        d.div_name,
        s.salary,
        DENSE_RANK() OVER (PARTITION BY s.div_id ORDER BY s.salary DESC) AS rnk
    FROM staff s
    INNER JOIN divisions d ON s.div_id = d.div_id
```

```
)
SELECT div_name, name, salary
FROM RankedStaff
WHERE rnk = 1;
```

Solution 6

```
SELECT
    div_id,
    name,
    hire_date,
    salary,
    SUM(salary) OVER (
        PARTITION BY div_id
        ORDER BY hire_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_payroll
FROM staff;
```

Solution 7

```
SELECT
    name,
    div_id,
    salary,
    MAX(salary) OVER (PARTITION BY div_id) AS div_max_salary,
    (MAX(salary) OVER (PARTITION BY div_id) - salary) AS shortfall_from_top
FROM staff;
```

Solution 8

```
SELECT
    sale_date,
    amount,
    LAG(amount, 1) OVER (ORDER BY sale_date) AS prev_sale_amount
FROM sales_performance
WHERE staff_id = 101;
```

Solution 9

```
SELECT
    staff_id,
    sale_date,
    amount,
    ROUND(
        100.0 * (amount - LAG(amount, 1) OVER(PARTITION BY staff_id ORDER BY sale_date))
        / LAG(amount, 1) OVER(PARTITION BY staff_id ORDER BY sale_date),
        2
    ) AS pct_growth
FROM sales_performance;
```

Solution 10

```
SELECT
    staff_id,
    sale_date,
    amount,
    ROUND(
        AVG(amount) OVER (
            PARTITION BY staff_id
            ORDER BY sale_date
            ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
        ),
        2
```

```
    ) AS rolling_2_sale_avg
FROM sales_performance;
```

Solution 11

```
SELECT
    name,
    salary,
    NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile
FROM staff;
```

Solution 12

```
SELECT
    staff_id,
    sale_id,
    sale_date,
    FIRST_VALUE(sale_date) OVER(
        PARTITION BY staff_id
        ORDER BY sale_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS first_sale_date,
    LAST_VALUE(sale_date) OVER(
        PARTITION BY staff_id
        ORDER BY sale_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS latest_sale_date
FROM sales_performance;
```

Solution 13

```
SELECT d.div_name
FROM divisions d
WHERE EXISTS (
    SELECT 1 FROM staff s
    WHERE s.div_id = d.div_id AND s.salary > 140000
);
```

Solution 14

```
SELECT name, role
FROM staff
WHERE staff_id NOT IN (
    SELECT DISTINCT staff_id FROM sales_performance
);
```

Solution 15

```
WITH RegionalRanks AS (
    SELECT
        region,
        sale_id,
        amount,
        DENSE_RANK() OVER (PARTITION BY region ORDER BY amount DESC) AS rnk
    FROM sales_performance
)
SELECT region, sale_id, amount
FROM RegionalRanks
WHERE rnk <= 2;
```

Solution 16

```
SELECT
    sale_id,
    staff_id,
```

```
    amount,
    ROUND(
        100.0 * amount / SUM(amount) OVER (PARTITION BY staff_id),
        2
    ) AS pct_of_personal_total
FROM sales_performance;
```

Solution 17

```
WITH LowestSalaries AS (
    SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary ASC) AS asc_rnk
    FROM staff
)
SELECT name, salary
FROM LowestSalaries
WHERE asc_rnk = 2;
```

Solution 18

```
SELECT
    name,
    salary,
    (salary - (SELECT MIN(salary) FROM staff)) AS spread_from_minimum
FROM staff;
```

Solution 19

```
SELECT
    sale_date,
    LEAD(sale_date, 1) OVER (ORDER BY sale_date) AS next_sale_date,
    (LEAD(sale_date, 1) OVER (ORDER BY sale_date) - sale_date) AS days_until_next_sale
FROM sales_performance
WHERE staff_id = 101;
```

Solution 20

```
WITH RECURSIVE Seq AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM Seq WHERE n < 10
)
SELECT n FROM Seq;
```

Part 11: Production Enterprise Case Study: Customer Churn & Cohort Retention

Business Challenge

At **NexusFintech**, leadership needs to measure customer transaction velocity. The data engineering team must compute:

- Days elapsed between each customer's consecutive transactions.
- Flagging customers who exhibit "churn risk" (defined as > 45 days between transactions).
- Identifying customers with consecutive transaction amounts that increase 3 times in a row.

Production Solution

```
WITH TransactionLags AS (
    SELECT
        customer_id,
        txn_id,
        txn_date,
        amount,
        LAG(txn_date, 1) OVER (
            PARTITION BY customer_id
            ORDER BY txn_date
        ) AS prior_txn_date,
        LAG(amount, 1) OVER (
            PARTITION BY customer_id
            ORDER BY txn_date
        ) AS prior_amount
    FROM customer_transactions
),
Deltas AS (
    SELECT
        customer_id,
        txn_id,
        txn_date,
        amount,
        COALESCE(txn_date - prior_txn_date, 0) AS days_since_last_txn,
        CASE WHEN (txn_date - prior_txn_date) > 45 THEN 1 ELSE 0 END AS is_churn_warning,
        CASE WHEN amount > prior_amount THEN 1 ELSE 0 END AS is_increasing_spend
    FROM TransactionLags
)
SELECT
    customer_id,
    COUNT(*) AS total_txns,
    ROUND(AVG(days_since_last_txn), 1) AS avg_cadence_days,
    SUM(is_churn_warning) AS churn_warnings_count,
    MAX(amount) AS highest_single_purchase
FROM Deltas
GROUP BY customer_id
ORDER BY churn_warnings_count DESC, total_txns DESC;
```

Part 12: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────────┐
│                  ADVANCED SQL & WINDOW FUNCTIONS CHEAT SHEET          │
├──────────────────────────────────────────────────────────────────────┤
│ 1. SUBQUERY: Query inside query. Scalar (1x1), Multi-row (IN/ANY/ALL).  │
│ 2. CORRELATED SUBQUERY: References outer table; runs once per outer row. │
│ 3. EXISTS vs IN: EXISTS terminates on first match; handles NULL safely.  │
│ 4. CTE: WITH Name AS (SELECT ...) -> Linear, readable, modular.          │
│ 5. RECURSIVE CTE: Anchor Query UNION ALL Recursive Query (Hierarchies).  │
├──────────────────────────────────────────────────────────────────────┤
│                          WINDOW RANKING FUNCTIONS                    │
│  - ROW_NUMBER() : Unique sequential (1, 2, 3, 4). No ties.           │
│  - RANK()       : Ties share number; skips next (1, 2, 2, 4).        │
│  - DENSE_RANK() : Ties share number; DOES NOT skip (1, 2, 2, 3).     │
│  - NTILE(k)     : Distributes rows into k roughly equal buckets.     │
├──────────────────────────────────────────────────────────────────────┤
│                          VALUE & OFFSET FUNCTIONS                    │
│  - LAG(col, n, def)  : Value n rows before current.                  │
│  - LEAD(col, n, def) : Value n rows after current.                   │
│  - FIRST_VALUE(col)  : First value in frame.                         │
│  - LAST_VALUE(col)   : Last value in frame (requires ROWS BETWEEN).  │
```

```
├──────────────────────────────────────────────────────────────────────────┤
│                              WINDOW FRAMES                               │
│  - ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW (Running Total)      │
│  - ROWS BETWEEN 2 PRECEDING AND CURRENT ROW (Moving Avg of 3)            │
│  - ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING (Full Partition)│
├──────────────────────────────────────────────────────────────────────────┤
│  CRITICAL RULE: Window functions execute in Step 5 (Projection time).    │
│  CANNOT be used directly in WHERE or HAVING clauses!                     │
└──────────────────────────────────────────────────────────────────────────┘
```

Part 13: 100-Mark Self-Assessment Examination

Section A: Technical Conceptual (20 Marks — 4 marks each)

Q1: What is the primary difference between `RANK()` and `DENSE_RANK()` when three rows tie for 1st place?

(a) `RANK()` assigns 1, 1, 1 and the next row is 2; `DENSE_RANK()` assigns 1, 1, 1 and the next row is 4.

(b) `RANK()` assigns 1, 1, 1 and the next row is 4; `DENSE_RANK()` assigns 1, 1, 1 and the next row is 2.

(c) `RANK()` assigns 1, 2, 3; `DENSE_RANK()` assigns 1, 1, 1.

(d) `RANK()` throws an ambiguous tie error.

Q2: Why does `SELECT * FROM tbl WHERE ROW_NUMBER() OVER (ORDER BY id) <= 10;` cause an execution failure?

(a) `ROW_NUMBER` cannot be sorted by `id`.

(b) Window functions are evaluated after `WHERE` in the logical execution order.

(c) `LIMIT` must be used instead of `ROW_NUMBER`.

(d) `OVER()` requires a `PARTITION BY` clause.

Q3: What happens if `subquery` in `WHERE col NOT IN (SELECT subquery)` returns one or more `NULL` values?

(a) The query ignores the NULLs and checks the remaining values.

(b) The condition evaluates to UNKNOWN, returning zero rows.

(c) A runtime null pointer exception is thrown.

(d) All rows are returned.

Q4: What is the termination requirement for a Recursive CTE?

(a) An explicit `LIMIT` clause in the recursive query.

(b) The recursive member must eventually return an empty result set.

(c) A `MAX_RECURSION` global database variable must be set.

(d) The anchor member must return exactly one row.

Q5: Which window frame specification ensures `LAST_VALUE(val)` evaluates across the complete partition rather than terminating at the current row?

(a) `ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING`

(b) `RANGE BETWEEN CURRENT ROW AND CURRENT ROW`

(c) `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`

(d) `ROWS 1 PRECEDING`

Section B: Output Prediction (25 Marks)

Given Table scores:

player_id	score
A	100
B	90
C	90
D	80
E	NULL

Q6 (10 Marks): What are the exact values of `rnk` and `dense_rnk` for each player?

```
SELECT player_id, score,
    RANK() OVER (ORDER BY score DESC NULLS LAST) AS rnk,
    DENSE_RANK() OVER (ORDER BY score DESC NULLS LAST) AS dense_rnk
FROM scores;
```

Q7 (15 Marks): What is the exact value of `running_sum` for each row?

```
SELECT player_id, score,
    SUM(COALESCE(score, 0)) OVER (
        ORDER BY score DESC NULLS LAST
        ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
    ) AS running_sum
FROM scores;
```

Section C: Query Implementation (35 Marks)

Q8 (15 Marks): Write a query to find the 3rd highest distinct salary from an `employees` table without using `LIMIT` or `OFFSET`. If no 3rd highest salary exists, return `NULL`.

Q9 (20 Marks): Write a query using a Recursive CTE to traverse an employee hierarchy table `emp (emp_id, name, manager_id)` and display each employee's name along with their complete management hierarchy depth level (CEO = 1, their direct reports = 2, etc.).

Section D: Architectural & Tuning Scenario (20 Marks)

Q10 (20 Marks): An ETL developer wrote a query using 4 nested subqueries in the `WHERE` clause that takes 35 minutes to complete on a 10M row table.

- Explain why rewriting the subqueries as CTEs or Joins might allow the optimizer to produce a faster execution plan. (10 marks)
- How does indexing the partition and order keys affect window function execution? (10 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** `RANK()` skips ($1, 1, 1, 4$); `DENSE_RANK()` does not skip ($1, 1, 1, 2$).
- **Q2: (b)** Window functions are evaluated in the SELECT projection stage, which occurs after WHERE filtering.
- **Q3: (b)** `NOT IN` with any `NULL` evaluates to `UNKNOWN`, producing an empty result set.
- **Q4: (b)** The recursive member must return 0 rows to terminate the loop.
- **Q5: (c)** `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Section B Solutions

Q6 Output:

player_id	score	rnk	dense_rnk
A	100	1	1
B	90	2	2
C	90	2	2
D	80	4	3
E	NULL	5	4

Q7 Output:

player_id	score	running_sum	Explanation
A	100	100	100 + 0 preceding = 100

B	90	190	90 + 100 = 190
C	90	180	90 + 90 = 180
D	80	170	80 + 90 = 170
E	NULL	80	0 + 80 = 80

Section C Solutions

Q8 Solution:

```
WITH RankedSalaries AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT COALESCE(
    (SELECT DISTINCT salary FROM RankedSalaries WHERE rnk = 3),
    NULL
) AS third_highest_salary;
```

Q9 Solution:

```
WITH RECURSIVE EmpHierarchy AS (
    SELECT emp_id, name, manager_id, 1 AS depth_level
    FROM emp
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.name, e.manager_id, h.depth_level + 1
    FROM emp e
    INNER JOIN EmpHierarchy h ON e.manager_id = h.emp_id
)
SELECT emp_id, name, depth_level
FROM EmpHierarchy
ORDER BY depth_level, emp_id;
```

Section D Solutions

Q10 Solution:

- Nested subqueries in `WHERE` often trigger repetitive nested-loop scans ($O(M \times N)$). Rewriting them as CTEs or explicit `INNER/LEFT JOIN`s enables the optimizer to use Hash Joins ($O(M+N)$) or materialize the subquery results once rather than re-evaluating row-by-row.
- If an index covers `(partition_col, order_col)`, the storage engine can stream pre-sorted records directly into the window accumulator. Without an index, the query planner must execute an expensive disk or memory sort ($O(N \log N)$ Sort operator) for every window partition.

Day 4: Database Design, Normalization & Storage Internals

Relational Schema Architecture, Keys, Normal Forms & Indexing

Part 1: Relational Keys & Integrity Constraints

1.1 Types of Keys in Relational Modeling

A **key** is an attribute or set of attributes that uniquely identifies an entity within an entity set or enforces referential constraints between relations.

Key Type	Formal Definition	Uniqueness	Nullable?	Role in System
Super Key	Any set of attributes that uniquely identifies a tuple.	Yes	Yes (if part of composite)	Theoretical superset of candidate keys.
Candidate Key	A **minimal** super key (no proper subset can uniquely identify the tuple).	Yes	**No** (by best practice)	Eligible to be chosen as Primary Key.
Primary Key (PK)	The chosen candidate key used by the RDBMS as the primary row identifier.	Yes	**Never** (`NOT NULL` enforced)	Establishes identity; determines clustered index order in MySQL/SQL Server.
Alternate Key	Candidate keys not selected as the primary key.	Yes	Generally No	Enforced via `UNIQUE` constraints.
Foreign Key (FK)	An attribute in a child table referencing a Primary or Candidate key in a parent table.	No (1:N) or Yes (1:1)	**Yes** (unless explicitly constrained)	Enforces Referential Integrity across relations.
Composite Key	A key composed of two or more attributes.	Yes	No	Identifies junction/bridge tables in M:N relationships.
Surrogate Key	A synthetic, system-generated identifier (e.g., `AUTO_INCREMENT`, `UUID`).	Yes	Never	Decouples internal identity from changing business attributes.
Natural Key	A business attribute with real-world meaning (e.g., SSN, ISBN, VIN).	Yes	Never	Subject to external business changes and data entry errors.

1.2 Referential Integrity & Cascade Actions

A foreign key constraint ensures that a child row cannot reference a non-existent parent row. When a referenced parent row is updated or deleted, the database can trigger one of four behaviors:

- **ON DELETE RESTRICT (or NO ACTION):** Prevents deletion of the parent row if any referencing child row exists. Raises a foreign key violation error. (Default behavior).
- **ON DELETE CASCADE:** Automatically deletes all referencing child rows when the parent row is deleted.
  Example: Deleting an `Order` cascades to delete all associated `OrderItems`.

- **ON DELETE SET NULL:** Sets the foreign key column in all child rows to `NULL` when the parent row is deleted. (Requires foreign key column to be nullable).
  Example: Deleting a `Department` sets `dept_id` to `NULL` for all affected `Employees`.

- **ON DELETE SET DEFAULT:** Resets the foreign key column in child rows to a predefined default value.

```
CREATE TABLE order_items (
    item_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT CHECK (quantity > 0),
    unit_price DECIMAL(10, 2) CHECK (unit_price >= 0),
    CONSTRAINT fk_order FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_product FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE RESTRICT
);
```

Part 2: Normalization & Normal Forms

2.1 The Purpose of Normalization

Normalization is a systematic technique for organizing data in a database to achieve two primary objectives:

- **Eliminate Redundant Data:** Storing the same fact in multiple places wastes storage and increases I/O overhead.
- **Eliminate Data Modification Anomalies:**
  Insertion Anomaly: Inability to record a fact about an entity without artificially fabricating an unrelated entity (e.g., cannot add a new Department until an Employee is hired into it).
  Update Anomaly: Updating a repeated fact in one row while missing it in another leads to internal data inconsistency (e.g., changing a department's location in 3 rows while 2 rows still show the old location).
  Deletion Anomaly: Deleting an entity unintentionally purges critical information about another entity (e.g., deleting the last employee in a department deletes the department record itself).

2.2 Functional Dependency (FD)

Let R be a relation schema, and let X and Y be subsets of attributes of R.
A **Functional Dependency** $X \rightarrow Y$ (X functionally determines Y) states that for any two valid tuples t_1 and t_2 in R:
$$\text{If } t_1[X] = t_2[X], \text{ then } t_1[Y] = t_2[Y]$$

- X is called the **Determinant**.
- Y is called the **Dependent**.

2.3 Step-by-Step Walkthrough of the Normal Forms

First Normal Form (1NF)

Rule:

- Each column must contain only **atomic (indivisible) values**. No arrays, comma-separated lists, or nested tables.
- Each column must contain values of a single consistent data type.
- Each record must be uniquely identifiable (a primary key must exist).

Violation Example:

order_id	customer_name	product_ids
101	Alice	1, 4, 7

1NF Compliant Solution:
Split multivalued rows so that each row contains exactly one product:

order_id	product_id	customer_name
101	1	Alice
101	4	Alice
101	7	Alice

Second Normal Form (2NF)

Rule:

- The relation must be in **1NF**.
- **No Partial Dependencies:** Every non-prime attribute (an attribute not part of any candidate key) must be **fully functionally dependent** on the **entire** candidate key, not on a proper subset of it.
  (Note: If a table's primary key consists of a single column, the table is automatically in 2NF if it is in 1NF).

Violation Example:

- Primary Key: `(order_id, product_id)` (Composite)
- Attributes: `quantity`, `product_name`, `product_price`
- Functional Dependencies:
`(order_id, product_id) -> quantity` (Full dependency: quantity requires both)
`product_id -> product_name, product_price` (**Partial Dependency!** Product name depends only on `product_id`, which is a subset of the composite PK).

2NF Decomposition:
Split into two tables:

- `order_items (order_id, product_id, quantity)` (PK: `order_id, product_id`)
- `products (product_id, product_name, product_price)` (PK: `product_id`)

Third Normal Form (3NF)

Rule:

- The relation must be in **2NF**.
- **No Transitive Dependencies:** No non-prime attribute may be functionally dependent on another non-prime attribute ($X \rightarrow Y$ and $Y \rightarrow Z$, where X is the primary key and Y, Z are non-prime attributes).
- *Formal Definition:* For every non-trivial functional dependency $X \rightarrow Y$, either:
X is a **Super Key**, OR
Y is a **Prime Attribute** (part of a candidate key).

Violation Example:

- Table: `employees (emp_id, emp_name, dept_id, dept_name, dept_location)`
- Primary Key: `emp_id`
- Functional Dependencies:
`emp_id -> dept_id`
`dept_id -> dept_name, dept_location`
Hence, `emp_id -> dept_name` is a **transitive dependency** through `dept_id`. If all employees in Marketing leave, the information that Marketing is located in London is deleted (Deletion Anomaly).

3NF Decomposition:
Split into two tables:

- `employees (emp_id, emp_name, dept_id)` (PK: `emp_id`, FK: `dept_id`)
- `departments (dept_id, dept_name, dept_location)` (PK: `dept_id`)

Boyce-Codd Normal Form (BCNF) — "3.5NF"

Rule:
A relation is in BCNF if and only if for **every** non-trivial functional dependency $X \rightarrow Y$, **X is a Super Key**.

- BCNF eliminates the second condition of 3NF ("Y is a prime attribute"). Under BCNF, *every determinant must be a candidate key*.

When does 3NF differ from BCNF?
When a relation has:

- Multiple overlapping composite candidate keys.

Classic BCNF Violation Example:

- Schema: `StudentAdvisor (student_id, subject, advisor)`
- Business Rules:
  Each student can have multiple advisors, but only one advisor per subject: `(student_id, subject) -> advisor`
  Each advisor advises only **one** subject: `advisor -> subject`

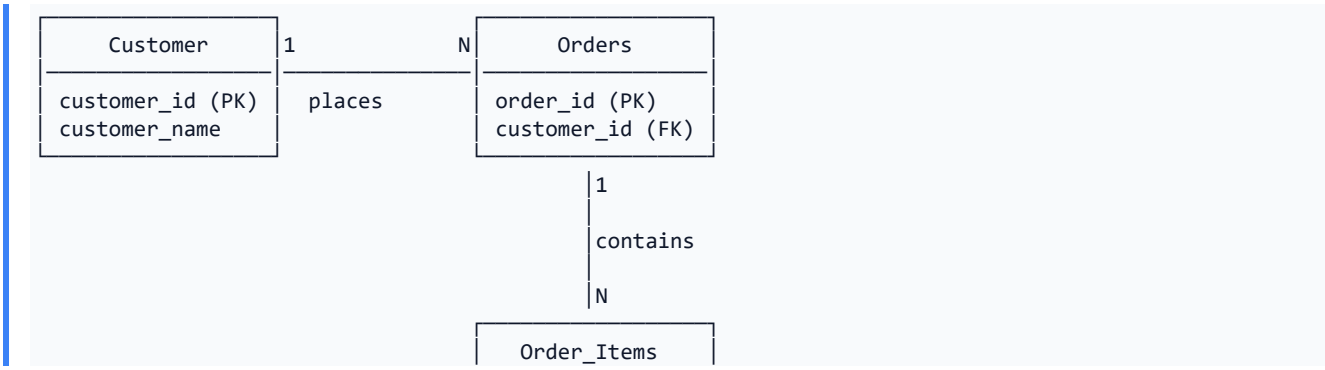
- Candidate Keys: `(student_id, subject)` and `(student_id, advisor)`
- Is it in 3NF?
  For `advisor -> subject`: `advisor` is not a super key, but `subject` is a **prime attribute** (part of Candidate Key 1). Therefore, it passes 3NF!

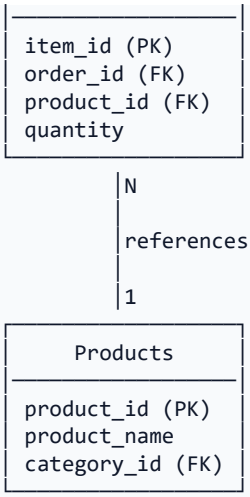
- Is it in BCNF?
  No. Because in `advisor -> subject`, the determinant `advisor` is **not a super key**.
  Anomaly: You cannot add a new advisor and their subject until at least one student is assigned to that advisor.

BCNF Decomposition:

- `AdvisorSubject (advisor, subject)` (PK: `advisor`)
- `StudentAdvisor (student_id, advisor)` (PK: `student_id, advisor`, FK: `advisor`)

Part 3: Entity-Relationship (ER) Modeling & Relationships





3.1 Relationship Cardinalities

- **One-to-One (1:1):**

Example: `User` to `UserProfile`.
Implementation: Place the PK of `User` as a FK in `UserProfile` with a `UNIQUE` constraint.

- **One-to-Many (1:N):**

Example: `Department` to `Employees`.
Implementation: Place the parent's PK (`dept_id`) as a FK in the child table (`employees`).

- **Many-to-Many (M:N):**

Example: `Students` to `Courses`, or `Orders` to `Products`.
Implementation: Cannot be implemented directly in relational engines. Requires an intermediate **Junction / Bridge / Associative Entity** (`OrderItems`, `Enrollments`) containing foreign keys to both entities.

Part 4: Views vs. Materialized Views

Feature	Standard View (Virtual Table)	Materialized View (Physical Snapshot)
Storage	Schema metadata only (No disk data).	Stored physically on disk as a concrete table.
Execution Cost	Evaluated dynamically on every query.	Read directly from disk ($O(1)$ initial read).
Data Freshness	Always 100% real-time and up-to-date.	Stale until refreshed (manual, scheduled, or trigger).

Write Impact	Zero impact on `INSERT`/`UPDATE` to underlying tables.	Overhead during refresh cycles.
Indexability	Cannot be indexed (except SQL Server Indexed Views).	Can be indexed like any physical table (B-Trees, etc.).
Ideal Use Case	Enforcing column-level security, simplifying joins.	Heavy analytical queries, aggregations, reporting dashboards.

```
-- Creating a Materialized View in PostgreSQL
CREATE MATERIALIZED VIEW mv_monthly_sales_summary AS
SELECT
    DATE_TRUNC('month', order_date) AS sales_month,
    category,
    COUNT(order_id) AS total_orders,
    SUM(total_amount) AS total_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY DATE_TRUNC('month', order_date), category;

-- Refreshing the Materialized View
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_monthly_sales_summary;
```

Part 5: Storage Internals & Indexing Architecture

5.1 What is an Index?

An index is an auxiliary data structure (most commonly a self-balancing **B-Tree** or **B+ Tree**) that enables the database engine to find specific rows in $O(\log N)$ time, avoiding expensive full table scans ($O(N)$).

5.2 B+ Tree Architecture

In a B+ Tree:

- **Internal Nodes:** Store search key values and child pointers to direct query traversal.
- **Leaf Nodes:** Stored on disk pages linked sequentially as a **doubly-linked list**. Leaf nodes contain either the actual data rows (Clustered) or pointers to data rows (Non-Clustered).
- **Height:** Typically 3 to 4 levels even for millions of rows. A search requires only 3 to 4 page I/O reads.

```
                [ 50 | 100 ]            <-- Root Node
               /     |      \
    [ 20 | 35 ]   [ 65 | 80 ]  [ 120 | 150 ] <-- Internal Nodes
      /    \        /    \       /     \
   [Leaves: Doubly-Linked List at Base Layer]
   [Page 1] <---> [Page 2] <---> [Page 3] <---> [Page 4]
```

5.3 Clustered vs. Non-Clustered Indexes

Dimension	Clustered Index	Non-Clustered Index (Secondary)
Physical Order	Dictates the physical sort order of rows	Logical structure separate from physical

	on disk.	row storage.
Leaf Node Content	Contains the **actual data pages** (the full row).	Contains the index key columns + a **Row Locator** (pointer/PK).
Count per Table	**Strictly 1 per table** (data can only be sorted one way).	**Multiple per table** (typically up to hundreds).
Lookup Cost	Direct lookup: Traverse B-Tree to leaf node $\rightarrow$ Data retrieved.	Two-step: Traverse secondary B-tree $\rightarrow$ get PK $\rightarrow$ lookup in clustered index (**Key Lookup / Bookmark Lookup**).
Default Creation	Primary Key automatically creates clustered index (MySQL InnoDB / SQL Server).	`CREATE INDEX` or `UNIQUE` constraint creates non-clustered index.

5.4 The Composite Index Left-Prefix Rule

When an index is created on multiple columns: `CREATE INDEX idx_emp_dept_sal ON employees (dept_id, salary, hire_date);`

The B+ tree is sorted strictly by `dept_id` first, then by `salary` within identical `dept_id`s, and then by `hire_date`.

Which queries can use this index?

- `WHERE dept_id = 10` $\rightarrow$ **YES** (Uses prefix).
- `WHERE dept_id = 10 AND salary > 80000` $\rightarrow$ **YES** (Uses both).
- `WHERE dept_id = 10 AND salary = 90000 AND hire_date = '2022-01-01'` $\rightarrow$ **YES** (Uses all 3).
- `WHERE salary = 90000` $\rightarrow$ **NO** (Violates left-prefix; cannot seek).
- `WHERE hire_date = '2022-01-01'` $\rightarrow$ **NO** (Violates left-prefix).
- `WHERE dept_id = 10 AND hire_date = '2022-01-01'` $\rightarrow$ **PARTIAL** (Uses `dept_id` to seek; filters `hire_date` in memory).

5.5 Covering Indexes (INCLUDE Columns)

A **Covering Index** contains all columns requested by a query in its leaf nodes. When an index covers a query, the engine satisfies the query entirely from the index without executing an expensive **Key Lookup** back to the clustered table!

```
-- Creating an index with INCLUDE columns (SQL Server / PostgreSQL)
CREATE INDEX idx_orders_customer_covering
ON orders (customer_id, order_date)
INCLUDE (total_amount, order_status);

-- This query is 100% "Covered" (Index-Only Scan)
SELECT customer_id, order_date, total_amount, order_status
FROM orders
WHERE customer_id = 101 AND order_date >= '2024-01-01';
```

Part 6: Core Comparisons Guide

Comparison 1: 3NF vs. BCNF

Feature	Third Normal Form (3NF)	Boyce-Codd Normal Form (BCNF)
Determinant Rule	For $X \rightarrow Y$, X is super key OR Y is prime attribute.	For $X \rightarrow Y$, X **must** be a super key.
Strictness	Less strict. Tolerates functional dependencies into prime attributes.	Stricter.
Dependency Preservation	Always preserves functional dependencies.	May occasionally lose functional dependencies during decomposition.
Industry Practice	Standard target for almost all OLTP database designs.	Applied in specialized scenarios with overlapping composite keys.

Comparison 2: OLTP vs. OLAP Schema Design

Attribute	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Primary Goal	Fast, reliable transaction processing (`INSERT`, `UPDATE`).	Fast analytical querying and aggregation over historical data.
Data Model	Highly Normalized (3NF / BCNF).	Denormalized (Star Schema, Snowflake Schema, One Big Table).
Tables	Many narrow tables with many foreign keys.	Few wide tables (Fact tables surrounded by Dimension tables).
Redundancy	Minimized to prevent update anomalies.	Intentionally introduced to eliminate expensive joins.
Write/Read Ratio	High volume of single-row writes and updates.	Bulk append loads; read-intensive aggregate queries.

Part 7: Structured Interview Question Bank

Q1: What is the difference between a Surrogate Key and a Natural Key? When should you choose one over the other?

Answer:
A Natural Key is an attribute that exists in the real world and has business meaning (such as a National Identification Number, Vehicle VIN, or Email address). A Surrogate Key is a synthetic, database-generated identifier (such as an auto-incrementing integer or UUID) that has no external meaning.

- **Choose Surrogate Keys** when natural keys are long strings (costly for foreign key indexing), prone to changing business rules (e.g., users changing email addresses), or when privacy regulations prohibit storing sensitive identifiers in secondary index logs.
- **Choose Natural Keys** for reference lookup tables (e.g., ISO 2-letter country codes like `'US'`, `'GB'`) where the code is permanently standardized and eliminates an unnecessary join.

Q2: Why does an excessive number of indexes degrade database performance?

Answer:
While indexes accelerate `SELECT` queries by providing fast B-tree traversal, they impose direct write amplification costs during `INSERT`, `UPDATE`, and `DELETE` operations. Every time a row is inserted or deleted, or an indexed column is modified, the database engine must synchronously update every associated secondary B-tree, manage page splits, rebalance trees, and write corresponding write-ahead log (WAL) entries. Furthermore, unused indexes consume valuable buffer pool memory, evicting frequently queried data pages.

Q3: What is a "Key Lookup" (or Bookmark Lookup), and why is it detrimental to high-scale query performance?

Answer:
When a query uses a secondary (non-clustered) index to filter rows, the leaf node of that secondary index provides the clustered key (or physical tuple ID). If the `SELECT` clause requests additional columns that are not part of the secondary index, the engine must perform an extra point-lookup into the clustered index for each matching row. If the query matches thousands of rows, this results in thousands of random I/O operations, which can be slower than a full table sequential scan.

Q4: Can a table in 3NF have insertion anomalies?

Answer:
Yes. A table in 3NF can still exhibit insertion anomalies if it has overlapping candidate keys and a functional dependency $X \rightarrow Y$ where X is not a super key (which violates BCNF). In such cases, one cannot insert a fact about $X \rightarrow Y$ without supplying a value for the other candidate key attributes.

Q5: Explain the difference between CHAR and VARCHAR from a storage and performance perspective.

Answer:

- `CHAR(n)` is fixed-length; it always reserves n bytes on disk and right-pads shorter strings with spaces. It avoids row migration overhead during updates because the column size never expands. Ideal for fixed-length codes (UUIDs, MD5 hashes, country codes).
- `VARCHAR(n)` is variable-length; it stores only the actual characters plus a 1- or 2-byte length prefix. It conserves substantial disk and buffer cache space for variable-length text (names, descriptions), reducing overall I/O.

Part 8: Complete Production DDL Database Design Exercise

Enterprise E-Commerce Platform Architecture

Entities Covered: Customers, Categories, Products, Orders, Order_Items, Payments.

```
-- 1. Customers Dimension
CREATE TABLE customers (
    customer_id BIGSERIAL PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(20),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE NOT NULL
);

-- 2. Product Categories (Supports self-referencing hierarchy)
CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    parent_category_id INT REFERENCES categories(category_id) ON DELETE SET NULL,
    category_name VARCHAR(100) NOT NULL,
    slug VARCHAR(120) NOT NULL UNIQUE
);

-- 3. Products Catalog
CREATE TABLE products (
    product_id BIGSERIAL PRIMARY KEY,
    category_id INT NOT NULL REFERENCES categories(category_id) ON DELETE RESTRICT,
    sku VARCHAR(64) NOT NULL UNIQUE,
    product_name VARCHAR(255) NOT NULL,
    unit_price DECIMAL(12, 2) NOT NULL CHECK (unit_price >= 0.00),
    stock_quantity INT NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
    is_discontinued BOOLEAN DEFAULT FALSE NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Orders Header
CREATE TABLE orders (
    order_id BIGSERIAL PRIMARY KEY,
    customer_id BIGINT NOT NULL REFERENCES customers(customer_id) ON DELETE RESTRICT,
    order_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
    order_status VARCHAR(30) NOT NULL CHECK (order_status IN ('Pending', 'Processing', 'Shipped',
'Delivered', 'Cancelled', 'Refunded')),
    subtotal_amount DECIMAL(12, 2) NOT NULL DEFAULT 0.00 CHECK (subtotal_amount >= 0.00),
    tax_amount DECIMAL(12, 2) NOT NULL DEFAULT 0.00 CHECK (tax_amount >= 0.00),
    shipping_amount DECIMAL(12, 2) NOT NULL DEFAULT 0.00 CHECK (shipping_amount >= 0.00),
    total_amount DECIMAL(12, 2) NOT NULL CHECK (total_amount >= 0.00)
);

-- 5. Order Line Items (Composite Business Relationship)
CREATE TABLE order_items (
    order_item_id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id BIGINT NOT NULL REFERENCES products(product_id) ON DELETE RESTRICT,
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price_at_sale DECIMAL(12, 2) NOT NULL CHECK (unit_price_at_sale >= 0.00),
    total_line_amount DECIMAL(12, 2) GENERATED ALWAYS AS (quantity * unit_price_at_sale) STORED,
    CONSTRAINT uq_order_product UNIQUE (order_id, product_id)
);

-- 6. Payments
CREATE TABLE payments (
```

```
    payment_id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE RESTRICT,
    payment_method VARCHAR(50) NOT NULL CHECK (payment_method IN ('Credit Card', 'Debit Card',
'PayPal', 'Wire Transfer', 'Stripe')),
    payment_status VARCHAR(30) NOT NULL CHECK (payment_status IN ('Authorized', 'Captured', 'Failed',
'Refunded')),
    transaction_reference VARCHAR(100) NOT NULL UNIQUE,
    amount_paid DECIMAL(12, 2) NOT NULL CHECK (amount_paid > 0.00),
    payment_timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- INDEXING STRATEGY FOR HIGH CONCURRENCY
-- FK lookup indexes to prevent full scans during join resolution
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_order_items_product_id ON order_items(product_id);
CREATE INDEX idx_products_category_id ON products(category_id);
CREATE INDEX idx_payments_order_id ON payments(order_id);

-- Covering Index for Customer Order History Lookup
CREATE INDEX idx_orders_cust_date_covering
ON orders (customer_id, order_date DESC)
INCLUDE (order_status, total_amount);
```

Part 9: End-to-End Enterprise Case Study: Legacy Schema Normalization

Scenario & Business Problem

A fast-growing fintech startup logged all transactions and merchant settlements in a single denormalized table `raw_settlements` originating from flat CSV exports.

The Unnormalized Relation:
`raw_settlements (settlement_id, merchant_name, merchant_tax_id, merchant_city, merchant_bank_acc, batch_id, batch_date, txn_id, txn_amount, txn_fee, txn_status)`

Symptoms Experienced:

- **Update Anomalies:** When a merchant changed their bank account, past batch records showed differing bank details.
- **Deletion Anomalies:** Deleting a batch to clear failed transactions wiped out the merchant's business profile.
- **Storage Bloat:** Merchant address and tax IDs were repeated 1.4 million times across daily transactions.

Decomposition into 3NF Relational Architecture

- **Identified Functional Dependencies:**
  `settlement_id -> all attributes`
  `merchant_tax_id -> merchant_name, merchant_city, merchant_bank_acc` (Independent entity)
  `batch_id -> batch_date` (Independent entity)
  `txn_id -> batch_id, merchant_tax_id, txn_amount, txn_fee, txn_status`

Normalized Target Schemas:

- `merchants (merchant_id, tax_id [UQ], name, city, bank_account)`
- `settlement_batches (batch_id, batch_date, created_at)`
- `settlement_transactions (txn_id, batch_id [FK], merchant_id [FK], amount, fee, status)`

Result: Eliminated 78% storage redundancy, guaranteed single-point merchant profile updates, and eradicated deletion anomalies completely.

Part 10: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────┐
│                  DBMS DESIGN & NORMALIZATION CHEAT SHEET     │
├──────────────────────────────────────────────────────────────┤
│ 1. PRIMARY KEY : Unique + NOT NULL. Determines physical cluster order.
│ 2. FOREIGN KEY : Enforces referential integrity to parent PK/Unique.
│ 3. CASCADE     : Automatically propagates parent DELETE/UPDATE to child
├──────────────────────────────────────────────────────────────┤
│                         NORMAL FORMS SUMMARY                 │
│  - 1NF : Atomic attributes. No arrays/lists. PK defined.     │
│  - 2NF : 1NF + No Partial Dependencies (non-key relies on whole PK).
│  - 3NF : 2NF + No Transitive Dependencies (non-key -> non-key).
│  - BCNF: Strict 3NF: For every X -> Y, X MUST be a Super Key.
├──────────────────────────────────────────────────────────────┤
│                         INDEXING ESSENTIALS                  │
│  - CLUSTERED INDEX     : 1 per table. Holds actual data pages.
│  - NON-CLUSTERED INDEX : Secondary B-Tree holding keys + row pointer.
│  - LEFT-PREFIX RULE    : Composite index (A, B, C) works for A, (A,B),
│                          (A,B,C), but NOT for B or C alone.  │
│  - COVERING INDEX      : Index contains all selected columns (INCLUDE).
│                          Eliminates expensive Key Lookups.   │
├──────────────────────────────────────────────────────────────┤
│                    VIEWS vs MATERIALIZED VIEWS               │
│  - VIEW              : Virtual. Zero storage. Runs underlying query.
│  - MATERIALIZED VIEW : Physical snapshot on disk. Super fast reads.
│                        Must be refreshed periodically.       │
└──────────────────────────────────────────────────────────────┘
```

Part 11: 100-Mark Self-Assessment Examination

Section A: Conceptual Foundations (25 Marks — 5 marks each)

Q1: A table has a composite primary key `(student_id, course_id)`. The column `course_fee` depends solely on `course_id`. Which normal form does this table violate?

(a) 1NF

(b) 2NF

(c) 3NF

(d) BCNF

Q2: What happens when an `ON DELETE SET NULL` constraint is triggered, but the foreign key column is defined as `NOT NULL`?

(a) The engine silently changes the column to allow NULLs.

(b) The deletion succeeds and sets the column to 0.

(c) The delete operation fails and throws an integrity constraint violation error.

(d) The parent record is deleted and child rows remain unchanged.

Q3: How many Clustered Indexes can exist on a single database table in SQL Server or MySQL InnoDB?

(a) 0

(b) Strictly 1

(c) Up to 16

(d) Unlimited

Q4: In a B+ Tree index of depth 3, approximately how many disk page reads are required to locate an individual record by its primary key?

(a) 1

(b) 3 to 4

(c) Hundreds of sequential reads

(d) Proportional to N

Q5: Given a composite index on `(country, state, city)`, which of the following queries CANNOT leverage the index via an index seek?

(a) `WHERE country = 'USA'`

(b) `WHERE country = 'USA' AND state = 'CA'`

(c) `WHERE state = 'CA' AND city = 'Los Angeles'`

(d) `WHERE country = 'USA' AND state = 'CA' AND city = 'Los Angeles'`

Section B: Dependency & Normalization Analysis (25 Marks)

Q6 (10 Marks): Consider the relation $R(A, B, C, D, E)$ with functional dependencies:

- $A \rightarrow B, C$
- $C, D \rightarrow E$
- $B \rightarrow D$
- Determine the candidate key(s) of R.
- State the highest normal form of R with formal justification.

Q7 (15 Marks): Decompose the relation $R(A, B, C, D)$ with functional dependencies $A \rightarrow B, C$ and $B \rightarrow D$ into 3NF relations. State the primary keys and foreign keys of the resulting relations.

Section C: Database Design Implementation (30 Marks)

Q8 (30 Marks): Design the relational schema for an **Airline Flight Booking System**.
Write production-grade PostgreSQL DDL statements for:

- `airports (airport_code [3-char PK], name, city, country)`
- `flights (flight_id, flight_number, departure_airport [FK], arrival_airport [FK], scheduled_departure, scheduled_arrival)`
- `passengers (passenger_id, passport_number [UQ], first_name, last_name, email)`
- `bookings (booking_id, flight_id [FK], passenger_id [FK], seat_number, price, booking_status)`
  Include all necessary primary keys, foreign keys with appropriate cascade rules, and at least one composite unique constraint to prevent double-booking the same seat on the same flight.

Section D: Architectural & Performance Scenario (20 Marks)

Q9 (20 Marks): An analytics team queries a 40-million row `transactions` table using:

```
SELECT customer_id, txn_date, amount
FROM transactions
WHERE customer_id = 89412
ORDER BY txn_date DESC;
```

- If the table has only a clustered index on `txn_id`, describe the execution plan and performance bottleneck. (7 marks)
- Design the optimal non-clustered composite index to make this query execute in under 5 milliseconds. Explain how it eliminates the sort and key lookup operations. (13 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** 2NF. `course_fee` depends on part of the composite primary key (`course_id`), which is a partial dependency.
- **Q2: (c)** Integrity violation error. You cannot set a `NOT NULL` column to `NULL`.

- **Q3: (b)** Strictly 1. Physical data rows can only be ordered on disk in one sequence.
- **Q4: (b)** 3 to 4 page I/O reads (1 read per B-Tree node level from root to leaf).
- **Q5: (c)** `WHERE state = 'CA' AND city = 'Los Angeles'` violates the left-prefix rule because `country` is omitted.

Section B Solutions

Q6 Solution:

- Candidate Key:

Compute closure of A: $A^+ = {A, B, C, D, E}$ (since $A \rightarrow B, C$, $B \rightarrow D$, and $C, D \rightarrow E$).
Therefore, **A is the sole candidate key**.

- Highest Normal Form:

Is it in 1NF? Yes.
Is it in 2NF? Yes, because candidate key A is a single attribute (no partial dependencies possible).
Is it in 3NF? Check $B \rightarrow D$: B is not a super key, and D is not a prime attribute.
Highest Normal Form is 2NF. (Violates 3NF due to transitive dependency $A \rightarrow B \rightarrow D$).

Q7 Solution:

- Canonical FDs: $A \rightarrow B, C$ and $B \rightarrow D$.
- 3NF Decomposition:

$R_1(A, B, C)$ with Primary Key A.
$R_2(B, D)$ with Primary Key B, and B in R_1 acting as Foreign Key referencing R_2.

Section C Solutions (DDL Implementation)

```
CREATE TABLE airports (
    airport_code CHAR(3) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    city VARCHAR(100) NOT NULL,
    country VARCHAR(100) NOT NULL
);

CREATE TABLE flights (
    flight_id BIGSERIAL PRIMARY KEY,
    flight_number VARCHAR(10) NOT NULL,
    departure_airport CHAR(3) NOT NULL REFERENCES airports(airport_code) ON DELETE RESTRICT,
    arrival_airport CHAR(3) NOT NULL REFERENCES airports(airport_code) ON DELETE RESTRICT,
    scheduled_departure TIMESTAMP WITH TIME ZONE NOT NULL,
    scheduled_arrival TIMESTAMP WITH TIME ZONE NOT NULL,
    CHECK (scheduled_arrival > scheduled_departure)
);

CREATE TABLE passengers (
    passenger_id BIGSERIAL PRIMARY KEY,
    passport_number VARCHAR(20) NOT NULL UNIQUE,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
```

```
    email VARCHAR(255) NOT NULL UNIQUE
);

CREATE TABLE bookings (
    booking_id BIGSERIAL PRIMARY KEY,
    flight_id BIGINT NOT NULL REFERENCES flights(flight_id) ON DELETE CASCADE,
    passenger_id BIGINT NOT NULL REFERENCES passengers(passenger_id) ON DELETE RESTRICT,
    seat_number VARCHAR(5) NOT NULL,
    price DECIMAL(10, 2) NOT NULL CHECK (price >= 0.00),
    booking_status VARCHAR(20) NOT NULL DEFAULT 'Confirmed',
    CONSTRAINT uq_flight_seat UNIQUE (flight_id, seat_number) -- Prevents double booking
);
```

Section D Solutions

Q9 Solution:

- Without an index on `customer_id`, the engine must perform a **Full Table Scan** reading all 40 million rows ($O(N)$ I/O cost). Once matching rows are located, an in-memory or TempDB **Sort operator** ($O(K \log K)$) must physically order the rows by `txn_date DESC`.
- **Optimal Index:**`CREATE INDEX idx_txns_cust_date ON transactions (customer_id, txn_date DESC) INCLUDE (amount);`

  Mechanism:
  The index seeks directly to `customer_id = 89412` ($O(\log N)$).
  Because the secondary index leaf nodes are already sorted physically by `txn_date DESC`, rows are streamed in exact requested order, **completely eliminating the Sort operator**.
  Because `amount` is stored in the leaf nodes via `INCLUDE`, the index **covers** the entire query, **completely eliminating Key Lookups** to the clustered table.

Day 5: Transactions, Database Internals & Query Optimization

ACID Properties, Isolation Levels, MVCC, Locks, Query Plans & Systems Architecture

---

Part 1: Transactions & ACID Properties in Depth

1.1 Definition & Transaction Boundaries

A **Transaction** is a logical unit of database processing that includes one or more database access operations (read, insert, update, or delete).
Transactions are delineated by explicit control statements:

- `BEGIN TRANSACTION` / `START TRANSACTION`: Initiates a private transaction workspace.
- `COMMIT`: Persists all modifications permanently to disk and releases locks.
- `ROLLBACK`: Aborts the transaction, undoing all uncommitted modifications back to the initial state.
- `SAVEPOINT <name>` / `ROLLBACK TO <name>`: Establishes a checkpoint within a transaction for partial rollbacks.

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 101;
SAVEPOINT transfer_deducted;

-- Attempt transfer to recipient
UPDATE accounts SET balance = balance + 500 WHERE account_id = 999;
-- If recipient account is invalid or locked:
ROLLBACK TO transfer_deducted;
-- Fallback action: Deposit into escrow instead
UPDATE escrow_vault SET balance = balance + 500 WHERE vault_id = 1;
COMMIT;
```

1.2 ACID Properties Under the Hood

1. Atomicity ("All or Nothing")

- **Concept:** Either all operations in the transaction succeed and become permanent, or none do.
- **Engine Implementation:** Maintained via the **Undo Log** and **Write-Ahead Logging (WAL)**. If a transaction crashes or aborts mid-flight, the recovery manager scans the Undo Log backward to revert modified pages in the buffer cache to their original pre-image values.

2. Consistency ("Preserving Invariants")

- **Concept:** A transaction transforms the database from one valid state to another valid state, satisfying all explicit schema constraints (PK, FK, CHECK, UNIQUE) and implicit domain invariants (e.g., total account balances across the bank must remain conserved).
- **Engine Implementation:** Enforced by constraint validation engines before commit and application-level business logic.

3. Isolation ("Execution Independence")

- **Concept:** The concurrent execution of multiple transactions must yield the same database state as if they had executed serially one after the other.

- **Engine Implementation:** Enforced through **Concurrency Control Mechanisms**:
Pessimistic: Two-Phase Locking (2PL), Intent Locks.
Optimistic / Snapshot: Multi-Version Concurrency Control (MVCC) and validation checks.

4. Durability ("Permanent Once Committed")

- **Concept:** Once a transaction commits, its effects survive any subsequent system crash, power failure, or operating system abort.
- **Engine Implementation:**
Write-Ahead Logging (WAL): Before any dirty data page is written from memory buffer cache to table disk storage, the corresponding log entry describing the change must be synchronously flushed to non-volatile disk storage via an `fsync()` syscall.
Checkpoints: Periodically flush all dirty buffer pages to disk and write a checkpoint marker to WAL, allowing old WAL segments to be truncated.

Part 2: Concurrency Phenomena & Read Anomalies

When multiple transactions execute concurrently without strict isolation, four classic read phenomena can occur:

2.1 Dirty Read (Reading Uncommitted Data)

- **Mechanism:** Transaction T_1 updates a row without committing. Transaction T_2 reads that updated row. T_1 then rolls back. T_2 has acted on "phantom" data that never formally existed in the database!

2.2 Non-Repeatable Read (Fuzzy Read)

- **Mechanism:** Transaction T_1 reads a row. Transaction T_2 updates or deletes that exact row and **commits**. Transaction T_1 re-reads the row and finds that its values have changed or it has disappeared.

2.3 Phantom Read

- **Mechanism:** Transaction T_1 executes a range query (e.g., `WHERE salary > 100000`) and retrieves 5 rows. Transaction T_2 **inserts** a new employee with a $120,000 salary and **commits**. Transaction T_1 re-runs the identical range query and discovers 6 rows. (The new row is the "phantom").

2.4 Serialization Anomaly / Write Skew

- **Mechanism:** Two concurrent transactions read overlapping data sets, make decisions based on the observed state, and execute non-conflicting writes that collectively violate an application invariant.
- *Classic Example (Doctor on Call):* Hospital policy requires at least 1 doctor on call. Doctors Alice and Bob are currently on call. Both simultaneously request leave. Alice sees Bob is on call and checks out. Bob sees Alice is on call and checks out. Both commit. Now 0 doctors are on call!

Part 3: ANSI SQL Isolation Levels

The ANSI/ISO SQL standard defines four transaction isolation levels based on which phenomena they permit:

Isolation Level	Dirty Read Permitted?	Non-Repeatable Read Permitted?	Phantom Read Permitted?	Write Skew Possible?
Read Uncommitted	**YES**	**YES**	**YES**	**YES**
Read Committed	**NO**	**YES**	**YES**	**YES**
Repeatable Read	**NO**	**NO**	**YES** (Standard) / **NO** (PostgreSQL MVCC)	**YES**
Serializable	**NO**	**NO**	**NO**	**NO**

> *[!NOTE]*
> ***Engine Defaults:***
>
> *PostgreSQL, Oracle, SQL Server default to:* ***Read Committed****.*
> *MySQL InnoDB defaults to:* ***Repeatable Read*** *(and uses Next-Key Locks to eliminate Phantoms even under Repeatable Read!).*

Part 4: Modern MVCC (Multi-Version Concurrency Control)

Modern databases (PostgreSQL, MySQL InnoDB, Oracle, Snowflake) do not rely on heavy read locks to achieve isolation. Instead, they use **MVCC**: *"Readers never block Writers, and Writers never block Readers."*

4.1 How MVCC Operates

Under MVCC, an `UPDATE` does not overwrite the row in place. Instead, it:

- Writes a brand new version of the row tuple.
- Marks the old version as expired/deleted.

4.2 PostgreSQL Implementation: xmin and xmax

Every physical row (tuple) in PostgreSQL contains hidden system attributes:

- `xmin`: The transaction ID (XID) that inserted this tuple.
- `xmax`: The transaction ID (XID) that deleted or superseded this tuple (or 0 if still current).

When Transaction T runs a query with snapshot ID S:

- It can only see tuples where `xmin < S` and committed, AND where `xmax > S` or uncommitted.
- **Vacuuming (VACUUM):** An asynchronous background process that reclaims dead tuple space once all active transactions that could possibly view those older row versions have completed.

Part 5: Locking Mechanisms & Deadlocks

5.1 The Lock Hierarchy

When transactions do lock data, they use a hierarchical locking protocol:

```
Database Level
   └── Table Level (Intent Locks: IS, IX)
         └── Page / Extent Level
               └── Row Level (Shared: S, Exclusive: X)
```

- **Shared Lock (S-Lock):** Acquired for read operations (`SELECT ... FOR SHARE`). Multiple transactions can hold S-locks on the same resource concurrently.
- **Exclusive Lock (X-Lock):** Acquired for write operations (`INSERT`, `UPDATE`, `DELETE`, or `SELECT ... FOR UPDATE`). Only one transaction can hold an X-lock; blocks all other S and X locks.
- **Intent Locks (IS, IX):** Placed at the table level to signal that a transaction intends to lock individual rows lower down the hierarchy. Prevents another transaction from acquiring an exclusive table-level lock while row-level modifications are underway.

5.2 Deadlocks: Mechanics, Detection & Prevention

Definition:

A **Deadlock** occurs when two or more transactions are permanently blocked because each holds a lock on a resource that the other requires to proceed.

```
Transaction 1: Holds Lock on Account A, Requests Lock on Account B
Transaction 2: Holds Lock on Account B, Requests Lock on Account A
               ===> DEADLOCK (Mutual circular dependency)
```

Deadlock Detection: The Wait-For Graph (WFG)

The database engine maintains a directed graph where nodes represent transactions and edges represent lock dependencies ($T_1 \rightarrow T_2$ means T_1 is waiting for T_2).

- If a cycle is detected ($T_1 \rightarrow T_2 \rightarrow T_1$), a deadlock exists!
- **Victim Selection:** The engine automatically terminates (rolls back) the transaction with the lowest cost (e.g., fewest modifications made) and throws a deadlock exception to the client: `ERROR: 40P01: deadlock detected`.

Deadlock Prevention Best Practices:

- **Deterministic Lock Ordering:** Always access tables and rows in the exact same sequential order across all application code paths (e.g., sort account IDs before acquiring locks: `WHERE account_id IN (10, 20) ORDER BY account_id`).
- **Keep Transactions Short:** Never perform external network calls, file I/O, or user interaction while holding database transactions open.
- **Use Lock Timeouts:** Set `SET lock_timeout = '2s'` to fail fast rather than stalling connection pools.

Part 6: Query Execution Plans & Optimization

6.1 EXPLAIN vs. EXPLAIN ANALYZE

- `EXPLAIN query`: Asks the Cost-Based Optimizer (CBO) to display the **estimated execution plan** based on table statistics without running the query.
- `EXPLAIN ANALYZE query`: **Actually executes the query**, displaying both the estimated cost and the **realized execution metrics** (actual startup time, execution time, row counts, and loops).

6.2 Key Query Plan Operators

1. Data Access Operators:

- **Sequential Scan (Seq Scan / Table Scan):** Scans every physical disk block of the table sequentially. Optimal when reading $> 20-30%$ of a table's data or for very small tables.
- **Index Scan:** Traverses the B-tree to find matching row pointers, then fetches each corresponding page from the table storage heap.
- **Index Only Scan:** Satisfies the entire query directly from the leaf nodes of an index (Covering Index) without touching the base table heap.
- **Bitmap Index Scan + Bitmap Heap Scan:** Scans the index to build a memory bitmap of matching disk pages, sorts the page numbers physically, and visits each table disk page exactly once. Ideal for queries combining multiple indexes (`AND`/`OR`).

2. Join Operators:

- **Nested Loop:** Iterates through outer table rows and searches the inner table. Fastest when outer table is small and inner table has a selective B-tree index.
- **Hash Join:** Hashes the smaller table into memory buffer, then streams the larger table to probe matches. Excels for large, unindexed equijoins.
- **Merge Join:** Both inputs are sorted by the join key and zipped together. Highly efficient if inputs are already sorted by clustered indexes.

Part 7: Systems Architecture: SQL vs. NoSQL & OLTP vs. OLAP

7.1 CAP Theorem & Relational Consistency

The **CAP Theorem** dictates that a distributed data store can guarantee at most two of three properties simultaneously:

- **Consistency (C):** Every read receives the most recent write or an error.
- **Availability (A):** Every non-failing node returns a non-error response for every request.
- **Partition Tolerance (P):** The system continues to operate despite arbitrary network message loss or network splits.

Traditional RDBMS (PostgreSQL, MySQL, Oracle): Prioritize **CA** on single nodes and **CP** in distributed clustering.

NoSQL Systems (Cassandra, DynamoDB, MongoDB): Often choose **AP** (Eventual Consistency) or configurable consistency.

7.2 OLTP vs. OLAP Internal Storage Engine Architecture

```
Row-Oriented Storage (OLTP - PostgreSQL, MySQL)
Disk Page 1: [ID: 1, Name: Alice, Age: 28, Balance: 5000] [ID: 2, Name: Bob, Age: 34, Balance: 7200]
Disk Page 2: [ID: 3, Name: Charlie, Age: 45, Balance: 3100] ...

Column-Oriented Storage (OLAP - Snowflake, ClickHouse, Redshift)
Disk Column Page 1 (IDs):      [1, 2, 3, 4, 5, 6, 7, 8, ...]
Disk Column Page 2 (Names):    [Alice, Bob, Charlie, ...]
Disk Column Page 3 (Balances): [5000, 7200, 3100, 8900, ...]
```

Why Columnar Storage Dominates Analytics:

- **I/O Elimination:** A query computing `SELECT AVG(Balance) FROM users` only reads Disk Column Page 3, skipping names and addresses entirely (reducing disk I/O by $90%$).
- **Extreme Compression:** Because a single column stores identical data types, compression algorithms (Run-Length Encoding, Dictionary, Delta encoding) achieve $10\times$ higher compression ratios than row-based storage.

Part 8: The 7-Step Production Troubleshooting Framework: "Why is a Query Slow?"

When an interviewer asks: *"A critical production query has slowed down dramatically. How do you investigate and fix it?"*, follow this structured methodology:

Step 1: Capture the Ground Truth & Execution Metrics

- Inspect query logs (`pg_stat_activity`, slow query log).
- Run `EXPLAIN (ANALYZE, BUFFERS)` to observe actual time spent, row counts vs. estimated rows, and buffer cache hits vs. disk reads.

Step 2: Compare Estimated vs. Actual Cardinality

- Look for severe discrepancies (e.g., `estimated rows = 1`, `actual rows = 500,000`).
- **Cause:** Stale table statistics. The cost-based optimizer is making bad algorithm choices (e.g., choosing a Nested Loop instead of a Hash Join).
- **Fix:** Run `ANALYZE table_name;` to update planner statistics.

Step 3: Identify the Access Path Bottlenecks

- Look for unexpected `Seq Scan` on multi-million row tables.
- Check if existing indexes are being ignored due to functions wrapping indexed columns (e.g., `WHERE LOWER(email) = '...'` or `WHERE EXTRACT(YEAR FROM created_at) = 2024`).
- **Fix:** Create functional indexes (`CREATE INDEX ON users (LOWER(email))`) or rewrite predicates to be **sargable** (`WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01'`).

Step 4: Detect Expensive Sorts & TempDB Spills

- Check execution plan for `Sort Method: external merge Disk`.
- **Cause:** Sorting requires more memory than `work_mem` permits, spilling intermediate rows to slow disk.
- **Fix:** Tune `work_mem` or create an index that matches the `ORDER BY` clause to provide pre-sorted data.

Step 5: Check for Lock Contention & Blocking Transactions

- Query active locks (`pg_locks`, `sys.dm_tran_locks`).
- Determine if the slow query is waiting on row or table locks held by an uncommitted transaction or long-running backup.

Step 6: Review Join Explosion & Implicit Type Conversions

- Verify that join keys share exact identical data types. An implicit cast (`VARCHAR` to `INT`) disables B-tree index seeks.
- Check if non-unique joins are causing row cardinality explosion.

Step 7: Architecture & Scaling Remediations

- If indexing is exhausted: Partition large tables by date/range (`TABLE PARTITIONING`), evaluate Materialized Views, read replicas, or offload heavy aggregations to an OLAP engine.

Part 9: Structured Interview Question Bank

Q1: What is Write-Ahead Logging (WAL), and what happens if the database crashes before dirty buffer pages are written to the table data files?

Answer:
Write-Ahead Logging dictates that any data modification must first be appended to an immutable, sequential disk log before the actual data pages in memory are written to disk. If the database crashes, dirty in-memory pages are lost. Upon reboot, the recovery manager reads the WAL from the last checkpoint:

- **Redo Phase:** Replays all committed transactions in the WAL to update data files.
- **Undo Phase:** Rolls back all uncommitted transactions present in the WAL when the crash occurred, restoring the database to a consistent state.

Q2: What is the difference between Pessimistic Locking and Optimistic Concurrency Control (OCC)?

Answer:

- **Pessimistic Locking:** Assumes conflicts will occur. Locks resources upfront before modification (`SELECT ... FOR UPDATE`). Other transactions must wait. Best for high-contention, low-latency transaction systems.
- **Optimistic Concurrency Control:** Assumes conflicts are rare. Reads data without locking, records a version number or timestamp, and checks during the commit phase if the version changed. If modified, the transaction rolls back and retries. Best for read-heavy workloads with low write collisions.

Q3: Explain "Write Skew" and why Repeatable Read isolation fails to prevent it.

Answer:
Write Skew occurs when two concurrent transactions read disjoint or overlapping data, check a shared invariant, and modify distinct records that together violate the invariant. Because Repeatable Read ensures that repeated reads of the *same* row return identical values, but does not prevent concurrent transactions from modifying *different* rows, both transactions pass their constraint checks and commit. Preventing Write Skew requires either explicit locking (`SELECT FOR UPDATE`) or **Serializable** isolation.

Q4: How does a B-tree index differ from a Hash index in a relational database?

Answer:

- **B-Tree Index:** Self-balancing search tree supporting point lookups ($O(\log N)$), range scans (`BETWEEN`, `<`, `>`), prefix searches (`LIKE 'abc%'`), and pre-sorted ordering (`ORDER BY`).
- **Hash Index:** Uses an in-memory hash table supporting strictly exact equality lookups ($O(1)$). Does **not** support range queries, partial prefix searches, or sorted retrieval.

Part 10: End-to-End Enterprise Case Study: High-Concurrency Wallet Double-Spend

Problem Statement

A mobile payments app allows users to transfer balance to friends. During flash sales, users triggered concurrent transfer requests from multiple devices simultaneously, resulting in a **double-spend exploit**: users with a $100 balance successfully transferred $100 twice, driving their account to a -$100 negative balance.

Root Cause Analysis

The original application logic under default `READ COMMITTED` isolation executed:

```
-- Step 1: Read current balance (Device A & Device B read simultaneously)
SELECT balance FROM wallets WHERE user_id = 45; -- Both see $100

-- Application checks: If balance >= 100, proceed...

-- Step 2: Deduct balance (Both execute update)
UPDATE wallets SET balance = balance - 100 WHERE user_id = 45;
```

Because both transactions read $100 before either committed, both updates succeeded, resulting in $100 - 100 - 100 = -100$.

Production Remediation: 3 Layer Defense

- **Pessimistic Row-Level Locking:**`BEGIN;`

```
-- Locks the wallet row exclusively; Device B blocks until Device A commits
SELECT balance FROM wallets WHERE user_id = 45 FOR UPDATE;
-- Verify balance >= 100 in app
UPDATE wallets SET balance = balance - 100 WHERE user_id = 45;
COMMIT;
```

- **Atomic Single-Statement Guard (Conditional Update):**
```
UPDATE wallets
SET balance = balance - 100
WHERE user_id = 45 AND balance >= 100;
-- Check rows_affected in application: If 0, reject transfer!
```

- **Database Schema Invariant:**
```
ALTER TABLE wallets ADD CONSTRAINT chk_balance_non_negative CHECK (balance >= 0.00);
```

Part 11: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│              TRANSACTIONS & DATABASE INTERNALS CHEAT SHEET       │
├──────────────────────────────────────────────────────────────────┤
│ 1. ACID: Atomicity (Undo Log), Consistency (Constraints),        │
│          Isolation (Locks/MVCC), Durability (WAL + fsync).       │
│ 2. WAL: Changes written sequentially to disk log BEFORE dirty pages.
├──────────────────────────────────────────────────────────────────┤
│                      CONCURRENCY ANOMALIES                       │
│  - Dirty Read         : Reading uncommitted changes that roll back.
│  - Non-Repeatable Read: Same row value changes between reads.    │
│  - Phantom Read       : New rows appear in range query between reads.
│  - Write Skew         : Invariant violated across distinct rows. │
├──────────────────────────────────────────────────────────────────┤
│                       ISOLATION LEVELS (ANSI)                    │
│  - Read Uncommitted : Allows all anomalies.                      │
│  - Read Committed   : Blocks Dirty Reads. (Postgres default).    │
│  - Repeatable Read  : Blocks Dirty & Fuzzy Reads. (MySQL default).
│  - Serializable     : Completely isolates concurrent execution.  │
├──────────────────────────────────────────────────────────────────┤
│                              LOCKING                             │
│  - Shared (S)       : Read lock (Multiple readers permitted).    │
│  - Exclusive (X)    : Write lock (Strictly 1 holder; blocks all).│
│  - Deadlock         : Circular dependency on Wait-For Graph.     │
│                       Fix via deterministic lock ordering & timeouts.
├──────────────────────────────────────────────────────────────────┤
│                          QUERY OPTIMIZATION                      │
│  - EXPLAIN ANALYZE  : Actual execution times and realized rows.  │
│  - Seq Scan         : Full table read (Bad for selective point lookups)
│  - Index Scan       : B-Tree seek + Heap tuple lookup.           │
│  - Index Only Scan  : Satisfied 100% from index leaf (Covering). │
│  - Sargable Queries : Avoid functions on indexed columns!        │
└──────────────────────────────────────────────────────────────────┘
```

Part 12: 100-Mark Self-Assessment Examination

Section A: Conceptual & Architectural (25 Marks — 5 marks each)

Q1: Which ACID property is directly guaranteed by Write-Ahead Logging (WAL)?

(a) Atomicity and Durability

(b) Consistency only

(c) Isolation only

(d) Normalization

Q2: Under which ANSI SQL isolation level is a "Dirty Read" permitted?

(a) Serializable

(b) Repeatable Read

(c) Read Committed

(d) Read Uncommitted

Q3: In PostgreSQL MVCC, what does the hidden column `xmin` represent?

(a) The minimum allowed value of the primary key.

(b) The Transaction ID (XID) that inserted the tuple.

(c) The timestamp when the row was updated.

(d) The size of the tuple in bytes.

Q4: What join physical algorithm is typically chosen by the optimizer when joining two large tables on an equality predicate when neither table has an index on the join key?

(a) Nested Loop Join

(b) Hash Join

(c) Cartesian Join

(d) B-Tree Merge Join

Q5: Why does `SELECT * FROM sales WHERE EXTRACT(YEAR FROM sale_date) = 2024;` typically fail to use an index on `sale_date`?

(a) The query optimizer cannot evaluate date types.

(b) Wrapping an indexed column in a function makes the predicate non-sargable, forcing a sequential scan.

(c) Indexes cannot store year values.

(d) Dates must always be formatted as strings.

Section B: Concurrency Analysis & Anomaly Identification (25 Marks)

Q6 (10 Marks): Consider two transactions executing concurrently:

- T_1: `BEGIN; SELECT balance FROM accounts WHERE id = 1;` (Returns $1000)
- T_2: `BEGIN; UPDATE accounts SET balance = 1500 WHERE id = 1; COMMIT;`
- T_1: `SELECT balance FROM accounts WHERE id = 1;` (Returns $1500)
- T_1: `COMMIT;`
- Name the exact concurrency anomaly demonstrated above. (5 marks)
- Which isolation level(s) would prevent this anomaly from occurring? (5 marks)

Q7 (15 Marks): Explain the deadlock condition that occurs between the two transactions below. Construct the Wait-For Graph and explain how the database resolves it:

- T_1: Updates row in `products` where `id = 10`.
- T_2: Updates row in `inventory` where `id = 50`.
- T_1: Attempts to update row in `inventory` where `id = 50`.
- T_2: Attempts to update row in `products` where `id = 10`.

Section C: Query Plan Diagnostics & Performance Tuning (30 Marks)

Q8 (30 Marks): An e-commerce platform has 25 million records in an `orders` table. A customer dashboard query runs:

```
SELECT order_id, order_date, total_amount, order_status
FROM orders
WHERE customer_id = 91823 AND order_status = 'Shipped'
ORDER BY order_date DESC;
```

Running `EXPLAIN ANALYZE` outputs:

```
Seq Scan on orders  (cost=0.00..845120.00 rows=12 width=32) (actual time=4210.12..4210.15 rows=8
loops=1)
  Filter: ((customer_id = 91823) AND ((order_status)::text = 'Shipped'::text))
  Rows Removed by Filter: 24999992
Planning Time: 0.12 ms
Execution Time: 4210.22 ms
```

- Diagnose the exact performance bottleneck in the query plan. (10 marks)
- Write the DDL statement for the single most optimal composite index to reduce execution time to under 2 milliseconds. (10 marks)
- Explain why this index completely eliminates the Sequential Scan, the Key Lookup, and the Sort operator. (10 marks)

Section D: Architectural Systems Design (20 Marks)

Q9 (20 Marks): Compare an OLTP database (e.g., PostgreSQL) with an OLAP data warehouse (e.g., Snowflake) for an enterprise analytics workload aggregating 5 billion rows. Explain why the columnar architecture outperforms row-based relational storage by orders of magnitude for aggregate queries.

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (a)** Atomicity (via Undo rollback logging) and Durability (via Redo WAL logging).
- **Q2: (d)** Read Uncommitted.
- **Q3: (b)** The Transaction ID (XID) that inserted the tuple.
- **Q4: (b)** Hash Join.
- **Q5: (b)** Wrapping the column in `EXTRACT()` makes the predicate non-sargable.

Section B Solutions

Q6 Solution:

- **Non-Repeatable Read (Fuzzy Read):** The exact same row returned two differing values within the same transaction scope after an external commit.
- Prevented by: **Repeatable Read** and **Serializable**.

Q7 Solution:

- T_1 holds exclusive lock on `products(10)` and requests lock on `inventory(50)`.
- T_2 holds exclusive lock on `inventory(50)` and requests lock on `products(10)`.
- **Wait-For Graph:** $T_1 \rightarrow T_2$ and $T_2 \rightarrow T_1$ (Directed cycle detected).
- **Resolution:** The deadlock detection engine runs a cycle detection algorithm, designates one transaction as the victim (e.g., T_2), issues an internal `ROLLBACK`, releases its locks, and sends error `40P01` to T_2, allowing T_1 to complete.

Section C Solutions

Q8 Solution:

- **Bottleneck:** The engine performs a full `Seq Scan`, scanning all 25 million disk rows and discarding $24,999,992$ rows, taking over 4.2 seconds.
- **Optimal Index DDL:**

```
CREATE INDEX idx_orders_cust_status_date_covering
ON orders (customer_id, order_status, order_date DESC)
INCLUDE (order_id, total_amount);
```

- **Execution Elimination:**

Seq Scan eliminated: Engine performs an $O(\log N)$ B-tree index seek matching `customer_id` and `order_status`.
Sort eliminated: Leaf nodes are already ordered physically by `order_date DESC`.
Key Lookup eliminated: `order_id` and `total_amount` are stored directly in the leaf pages via `INCLUDE` (Covering Index), resulting in an **Index Only Scan**.

Section D Solutions

Q9 Solution:

In row-oriented OLTP storage, all attributes of a row are packed together on the same disk page. Computing

`SUM(amount)` over 5 billion rows requires loading every single column (IDs, text strings, timestamps) into memory, causing massive I/O saturation.

In columnar OLAP storage (Snowflake), each column is stored in isolated, compressed micro-partitions. The engine reads **only** the `amount` column files, skipping all other columns entirely. Furthermore, SIMD vectorized CPU execution scans compressed integer/decimal arrays at hardware line speeds, resulting in 100x to 1000x faster execution.

Day 6: Python Coding Fundamentals & Data Structures

Memory Model, Mutability, Collections, Algorithmic Complexity & Pythonic Idioms

Part 1: The Python Object Model, Memory & Mutability

1.1 Everything is an Object & Names are Labels

In Python, variables do **not** hold values directly; variables are **names (pointers/references) bound to objects in memory**.
Every Python object consists of at least three core fields in CPython:

- **Type Pointer:** Indicates the object's class/type (e.g., `PyTypeObject`).
- **Reference Count (ob_refcnt):** Tracks how many variable names or containers reference this object (used by CPython's garbage collector).
- **Value / Data Payload:** The actual underlying data.

```
a = [1, 2, 3]
b = a          # Both 'a' and 'b' reference the exact same memory address
print(id(a) == id(b))  # True
b.append(4)
print(a)               # [1, 2, 3, 4] -> Mutating through 'b' affects 'a'!
```

1.2 Identity (is) vs. Equality (==)

- **== (Value Equality):** Evaluates whether two objects contain the same data value (invokes the object's `__eq__()` magic method).
- **is (Reference Identity):** Evaluates whether two variables point to the **exact same memory address** (`id(a) == id(b)`).

```
x = [1, 2, 3]
y = [1, 2, 3]
print(x == y)  # True (Identical values)
print(x is y)  # False (Distinct heap-allocated objects)

# Small Integer Interning: Python pre-allocates integers from -5 to 256
p = 100
q = 100
print(p is q)  # True (Both point to the cached integer block)

m = 1000
n = 1000
print(m is n)  # False (Exceeds intern cache; separate allocations in REPL)
```

> *[!IMPORTANT]*
> ***Interview Rule:*** *Always use `is` when comparing against singleton objects like `None` (`if val is None:`), because `None` is a guaranteed unique singleton in Python memory, and `==` can be maliciously overridden by user-defined classes.*

1.3 Mutability vs. Immutability

- **Immutable Types:** `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, `bytes`. Once created, their memory contents can **never** be modified in place. Any "modification" allocates an entirely new object in memory.
- **Mutable Types:** `list`, `dict`, `set`, `bytearray`. Their memory contents can be modified in place without changing their memory address (`id()`).

```
Immutable Tuple with a Mutable Element (The Classic Trap):
t = (1, 2, [3, 4])
# t[0] = 99        --> TypeError: 'tuple' object does not support item assignment
t[2].append(5)     --> SUCCESS!
print(t)           --> (1, 2, [3, 4, 5])
```

Why did this work? The tuple's internal pointers are immutable (it permanently references index 0, index 1, and index 2). However, the object at index 2 is a mutable `list`, which can mutate internally while its identity remains unchanged!

1.4 Shallow Copy vs. Deep Copy

```
import copy

original = [[1, 2, 3], [4, 5, 6]]

# 1. Assignment (Zero copy - shared pointer)
ref = original

# 2. Shallow Copy (Creates new outer container, but inner elements are shared references)
shallow = copy.copy(original) # or original.copy() or original[:]
shallow[0].append(99)
print(original[0])  # [1, 2, 3, 99] -> Modified!

# 3. Deep Copy (Recursively clones every nested container and object)
deep = copy.deepcopy(original)
deep[0].append(888)
print(original[0])  # [1, 2, 3, 99] -> Completely isolated!
```

1.5 The Mutable Default Argument Trap

In Python, default function arguments are evaluated **once at function definition time**, not every time the function is called.

```
# THE ANTI-PATTERN (Buggy)
def append_to_list(val, target_list=[]):
    target_list.append(val)
    return target_list

print(append_to_list(1))  # [1]
print(append_to_list(2))  # [1, 2] -- The default list persisted across calls!

# THE PYTHONIC IDIOM (Correct)
def append_to_list_safe(val, target_list=None):
    if target_list is None:
        target_list = []
    target_list.append(val)
    return target_list
```

Part 2: Core Data Structures & Algorithmic Complexities

2.1 Big-O Complexity Matrix for Python Built-ins

Data Structure	Operation	Average Case	Worst Case	Internal Implementation
list	`append(x)`	$O(1)$ amortized	$O(n)$ (array resize)	Contiguous dynamic array of object pointers
	`insert(0, x)` / `pop(0)`	$O(n)$	$O(n)$	Requires shifting all n subsequent pointers
	`pop()` (from end)	$O(1)$	$O(1)$	No memory shifting required
	`x in my_list`	$O(n)$	$O(n)$	Linear scan
	Slicing `lst[a:b]`	$O(k)$ where $k=b-a$	$O(k)$	Allocates new list and copies pointers
dict	Get `d[k]` / Set `d[k]=v`	$O(1)$	$O(n)$ (hash collisions)	Compact hash table with open addressing
	Delete `del d[k]`	$O(1)$	$O(n)$	Marks dummy tombstone
	`k in d` (membership)	$O(1)$	$O(n)$	Hashes key, looks up index in table
set	Add `s.add(x)` / Remove	$O(1)$	$O(n)$	Hash table (dictionary with dummy values)
	Union `s1 | s2`	$O(\text{len}(s1) + \text{len}(s2))$	$O(n)$	Combines hash sets
	Intersection `s1 & s2`	$O(\min(\text{len}(s1), \text{len}(s2)))$	$O(n)$	Traverses smaller set, checks hash in larger
deque	`appendleft(x)` / `popleft()`	$O(1)$	$O(1)$	Doubly linked list of fixed memory blocks

---

2.2 Python Dictionaries Under the Hood

Since Python 3.7, dictionaries are guaranteed to maintain **insertion order**.
CPython achieves this using a **two-array compact architecture**:

- **Indices Array:** A sparse array mapping hash buckets to indices in the entries array.
- **Entries Array:** A dense array storing `[hash_code, key_pointer, value_pointer]` in the exact sequence they were inserted.
- **Space Reduction:** Reduces memory consumption by 30–40% compared to legacy sparse hash tables.

- **Key Requirement:** A dictionary key **must be hashable** (must implement `__hash__()` and `__eq__()`). Mutable objects (`list`, `dict`, `set`) are unhashable and will throw `TypeError: unhashable type`.

Part 3: Comprehensions & Advanced Slicing

3.1 Python Slicing Internals: sequence[start:stop:step]

- **Indices:** 0-based. Negative indices count backward from the end (`-1` is the last element).
- **Omission Rules:**

`lst[:]` creates a shallow copy.
`lst[::-1]` reverses the sequence in $O(n)$ time.
`lst[::2]` extracts every second element.

3.2 List, Dict, and Set Comprehensions

Comprehensions are not merely syntactic sugar; they run at C-speed inside the CPython bytecode interpreter by avoiding repeated attribute lookups (`list.append`).

```
# 1. Filtered and Transformed List Comprehension
squares = [x**2 for x in range(10) if x % 2 == 0]

# 2. Dictionary Comprehension (Inverting a map)
char_map = {'a': 1, 'b': 2, 'c': 3}
inv_map = {v: k for k, v in char_map.items()}

# 3. Set Comprehension
unique_lengths = {len(word) for word in ["apple", "banana", "fig", "apple"]}

# 4. Nested Comprehension (Flattening a 2D Matrix)
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [val for row in matrix for val in row]
# Equivalent to:
# for row in matrix:
#     for val in row:
#         flattened.append(val)
```

Part 4: Functions, Variadic Arguments & Scope

4.1 Argument Unpacking: *args and **kwargs

- `*args`: Collects extra positional arguments into an immutable **tuple**.
- `**kwargs`: Collects extra keyword arguments into a mutable **dictionary**.

```
def flexible_logger(level, *args, **kwargs):
    print(f"[{level}] Positional: {args}")
    print(f"[{level}] Metadata: {kwargs}")

flexible_logger("INFO", "User login", "IP: 127.0.0.1", user_id=42, latency_ms=12.5)
# args   -> ('User login', 'IP: 127.0.0.1')
# kwargs -> {'user_id': 42, 'latency_ms': 12.5}
```

Keyword-Only and Positional-Only Parameters:

```
# '/' forces preceding arguments to be positional-only
# '*' forces following arguments to be keyword-only
def configure_engine(api_key, /, timeout=30, *, retries=3, debug=False):
    pass

# configure_engine(api_key="secret")   --> ERROR! api_key must be positional
configure_engine("secret", timeout=10, retries=5, debug=True) # Valid
```

4.2 The LEGB Scope Resolution Rule

When a variable name is referenced, Python searches namespaces in this strict order:

- **L (Local):** Names assigned inside the current function.
- **E (Enclosing):** Names in the local scope of any enclosing/outer functions (closures).
- **G (Global):** Names assigned at the top-level of the module.
- **B (Built-in):** Predefined Python names (`len`, `range`, `ValueError`).

```
counter = 0  # Global

def outer():
    counter = 10  # Enclosing
    def inner():
        nonlocal counter  # Modifies enclosing scope
        counter += 1
        print("Inner:", counter)
    inner()
    print("Outer:", counter)

outer()
print("Global:", counter)
# Output:
# Inner: 11
# Outer: 11
# Global: 0
```

Part 5: Python's Sorting Engine: Timsort

Python's built-in `sort()` (in-place) and `sorted()` (returns a new list) use **Timsort**, an adaptive, stable, hybrid sorting algorithm derived from Merge Sort and Insertion Sort:

- **Worst-case Time Complexity:** $O(n \log n)$
- **Best-case Time Complexity:** $O(n)$ (when input is already sorted or nearly sorted)
- **Space Complexity:** $O(n)$
- **Stability: Guaranteed**. If two records share identical sort keys, their original relative order is strictly preserved.

Custom Multi-Criteria Sorting via Keys:

```
students = [
    {"name": "Alice", "grade": 88, "age": 20},
    {"name": "Bob", "grade": 95, "age": 22},
    {"name": "Charlie", "grade": 88, "age": 19},
    {"name": "David", "grade": 95, "age": 20}
]
```

```
# Sort primarily by grade DESCENDING, then by age ASCENDING:
sorted_students = sorted(
    students,
    key=lambda s: (-s["grade"], s["age"])
)
# Result: Bob (95, 22), David (95, 20), Charlie (88, 19), Alice (88, 20)
```

Part 6: Production Implementation of Required Coding Tasks

Task 1: Character & Word Frequency Counting

```
from collections import Counter

def analyze_frequencies(text: str) -> dict:
    """Computes character frequency ignoring whitespace, and top words."""
    cleaned_chars = [c.lower() for c in text if c.isalnum()]
    char_counts = Counter(cleaned_chars)

    words = text.lower().split()
    word_counts = Counter(words)

    return {
        "char_frequencies": dict(char_counts),
        "most_common_char": char_counts.most_common(1)[0] if char_counts else None,
        "most_common_word": word_counts.most_common(1)[0] if word_counts else None
    }

# Complexity: O(N) Time, O(U) Space where U is unique characters/words.
```

Task 2: Valid Anagrams (Two Approaches)

```
def is_anagram_hashmap(s: str, t: str) -> bool:
    """Optimal O(N) time and O(1) space (at most 26 alphabetic keys)."""
    if len(s) != len(t):
        return False

    counts = {}
    for char in s:
        counts[char] = counts.get(char, 0) + 1
    for char in t:
        if char not in counts or counts[char] == 0:
            return False
        counts[char] -= 1

    return True

def is_anagram_sorting(s: str, t: str) -> bool:
    """O(N log N) time, O(N) space."""
    return sorted(s) == sorted(t)
```

Task 3: Order-Preserving Duplicate Removal

```
def remove_duplicates_ordered(seq: list) -> list:
    """
    Deduplicates a list while strictly maintaining initial element order.
```

```
    Uses dict.fromkeys() which runs in C-speed and preserves insertion order.
    """
    return list(dict.fromkeys(seq))

def remove_unhashable_duplicates(seq: list) -> list:
    """Handles lists containing unhashable objects (e.g. dicts)."""
    seen = []
    for item in seq:
        if item not in seen:
            seen.append(item)
    return seen  # Note: O(N^2) due to linear search in seen list
```

Task 4: Two-Sum Problem

```
def two_sum_optimal(nums: list[int], target: int) -> tuple[int, int] | None:
    """
    Finds the two indices whose values sum to target in O(N) time.
    Uses a hash map to store value -> index mapping.
    """
    seen = {}  # complement -> index
    for current_idx, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return (seen[complement], current_idx)
        seen[num] = current_idx
    return None

# Test: two_sum_optimal([2, 7, 11, 15], 9) -> (0, 1)
```

Task 5: Matrix Transposition & Deep Transformations

```
def transpose_matrix(matrix: list[list[int]]) -> list[list[int]]:
    """
    Transposes an M x N matrix into an N x M matrix using zip(*) idiom.
    """
    if not matrix or not matrix[0]:
        return []
    return [list(col) for col in zip(*matrix)]

# Example:
# matrix = [[1, 2, 3], [4, 5, 6]]
# transpose_matrix(matrix) -> [[1, 4], [2, 5], [3, 6]]
```

Part 7: Core Comparisons Guide

Comparison 1: list vs tuple vs array.array

Feature	list	tuple	array.array
Mutability	Mutable	**Immutable**	Mutable
Element Types	Heterogeneous (Any object)	Heterogeneous (Any object)	**Strictly Homogeneous** (C data types)

Memory Footprint	Largest (Pointers + over-allocated buffer)	Lean (Exact size allocated)	Smallest (Raw unboxed C values)
Dictionary Key?	**No** (Unhashable)	**Yes** (If elements are immutable)	No
Primary Use Case	Dynamic general-purpose sequences	Fixed records, coordinates, dict keys	High-performance numeric data buffers

---

Part 8: Structured Technical Interview Question Bank

Q1: What is the output of [i for i in range(3)] vs (i for i in range(3))?

Answer:

- `[i for i in range(3)]` is a **List Comprehension**: It immediately evaluates all elements and returns a fully materialized list in memory: `[0, 1, 2]`.
- `(i for i in range(3))` is a **Generator Expression**: It returns a lazy generator object `<generator object ...>` that produces items on-demand using the iterator protocol (`next()`), consuming $O(1)$ memory.

Q2: Why is inserting at index 0 of a standard Python list slow ($O(n)$), and which data structure should you use instead?

Answer:
A Python `list` is internally represented as a contiguous dynamic array of object pointers. Inserting an element at index 0 requires shifting every existing pointer in the array one position to the right in memory, resulting in $O(n)$ time complexity. For workloads requiring frequent $O(1)$ insertions or deletions at both ends, `collections.deque` (a doubly-linked block array) should be used.

Q3: How does Python manage memory for variables, and how does the Garbage Collector reclaim memory?

Answer:
Python manages memory through:

- **Reference Counting:** Every object tracks how many references point to it (`ob_refcnt`). When the count reaches 0, its memory is immediately deallocated.
- **Generational Cyclic Garbage Collector:** Reference counting cannot detect **reference cycles** (e.g., Object A references Object B, and Object B references Object A, while both are unreachable from root scopes). Python's cyclic GC organizes objects into three generations (Gen 0, 1, 2) based on survival age and periodically runs cycle-detection algorithms to detect and reclaim isolated cyclic islands.

Q4: Explain the difference between dict.get(key) and dict[key].

Answer:

- `dict[key]` performs a direct lookup and raises a `KeyError` exception if the key does not exist.
- `dict.get(key, default=None)` safely checks for the key; if present, it returns the value, and if absent, it returns the specified default value (`None` if omitted) without throwing an exception.

Part 9: End-to-End Enterprise Case Study: Streaming Server Log Aggregator

Problem Statement

A data engineering pipeline receives millions of raw Web Server HTTP access logs formatted as raw strings. You must write an in-memory aggregation module that:

- Parses IP addresses, HTTP status codes, and response times.
- Identifies top-3 attacking IPs (IPs with status code `401` or `403`).
- Computes the average response time per unique endpoint, discarding corrupted records gracefully.

Production Solution

```
import re
from collections import defaultdict, Counter

LOG_PATTERN = re.compile(r'(?P<ip>\S+) \[(?P<endpoint>\S+)\] (?P<status>\d{3}) (?P<latency>\d+ms)')

raw_logs = [
    "192.168.1.10 [/api/v1/login] 401 120ms",
    "10.0.0.5 [/api/v1/users] 200 45ms",
    "192.168.1.10 [/api/v1/login] 401 95ms",
    "172.16.0.4 [/api/v1/checkout] 500 450ms",
    "192.168.1.10 [/api/v1/login] 403 110ms",
    "10.0.0.5 [/api/v1/users] 200 50ms",
    "CORRUPTED LOG LINE WITHOUT VALID SCHEMA",
    "10.0.0.99 [/api/v1/login] 401 80ms"
]

def parse_and_aggregate_logs(logs: list[str]) -> dict:
    endpoint_latencies = defaultdict(list)
    failed_auth_ips = Counter()
    corrupted_count = 0

    for line in logs:
        match = LOG_PATTERN.match(line)
        if not match:
            corrupted_count += 1
            continue

        data = match.groupdict()
        ip = data["ip"]
        endpoint = data["endpoint"]
        status = int(data["status"])
        latency = int(data["latency"].replace("ms", ""))

        # Accumulate metrics
        endpoint_latencies[endpoint].append(latency)
        if status in (401, 403):
            failed_auth_ips[ip] += 1

    # Calculate averages
    avg_latencies = {
        endpoint: round(sum(latencies) / len(latencies), 2)
        for endpoint, latencies in endpoint_latencies.items()
    }
```

```
    return {
        "top_suspicious_ips": failed_auth_ips.most_common(3),
        "endpoint_avg_latency_ms": avg_latencies,
        "corrupted_records_dropped": corrupted_count
    }

# Execute
results = parse_and_aggregate_logs(raw_logs)
# Output:
# 'top_suspicious_ips': [('192.168.1.10', 3), ('10.0.0.99', 1)]
# 'endpoint_avg_latency_ms': {'/api/v1/login': 101.25, '/api/v1/users': 47.5, '/api/v1/checkout':
450.0}
# 'corrupted_records_dropped': 1
```

Part 10: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  PYTHON CODING FUNDAMENTALS CHEAT SHEET          │
├──────────────────────────────────────────────────────────────────┤
│ 1. MUTABILITY:                                                   │
│    - Immutable: int, float, str, tuple, frozenset.               │
│    - Mutable  : list, dict, set. (Unhashable; cannot be dict keys).│
│ 2. IDENTITY vs EQUALITY:                                         │
│    - 'is' checks memory pointer address (id(a) == id(b)).        │
│    - '==' checks semantic value equality (__eq__).               │
├──────────────────────────────────────────────────────────────────┤
│                         TIME COMPLEXITIES                        │
│  - List: append O(1), pop() O(1), insert(0) O(N), search O(N).   │
│  - Dict/Set: lookup, insert, delete are all AVERAGE O(1).        │
│  - Timsort: O(N log N) worst case, O(N) best case (stable sort). │
├──────────────────────────────────────────────────────────────────┤
│                         ESSENTIAL IDIOMS                         │
│  - Safe Defaults: def f(x=None): if x is None: x = []            │
│  - Reverse: seq[::-1]                                            │
│  - Transpose: [list(c) for c in zip(*matrix)]                    │
│  - Deduplicate Preserving Order: list(dict.fromkeys(seq))        │
│  - Unpack: def f(*args, **kwargs)                                │
├──────────────────────────────────────────────────────────────────┤
│                          SCOPE (LEGB RULE)                       │
│  Local -> Enclosing -> Global -> Built-in.                       │
│  Use 'nonlocal' for enclosing, 'global' for module-level variables.│
└──────────────────────────────────────────────────────────────────┘
```

Part 11: 100-Mark Self-Assessment Examination

Section A: Conceptual & Output Prediction (25 Marks — 5 marks each)

Q1: What is the output of the following code?

```
a = [1, 2, 3]
b = a
b += [4]
print(a)
```

(a) `[1, 2, 3]`

(b) `[1, 2, 3, 4]`

(c) `TypeError`

(d) `None`

Q2: What happens when trying to use a tuple containing a list as a dictionary key?

```
d = {}
k = (1, 2, [3, 4])
d[k] = "test"
```

(a) Succeeds; tuples are always hashable.

(b) `TypeError: unhashable type: 'list'`

(c) Key is inserted with value `"test"`.

(d) List is automatically converted to a tuple.

Q3: What is the average time complexity of checking membership (`x in collection`) for a Python `list` versus a Python `set` of N elements?

(a) List: $O(1)$, Set: $O(1)$

(b) List: $O(n)$, Set: $O(n)$

(c) List: $O(n)$, Set: $O(1)$

(d) List: $O(\log n)$, Set: $O(1)$

Q4: What is the output of this code snippet?

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
```

(a) `[0, 1, 2]`

(b) `[2, 2, 2]`

(c) `[0, 0, 0]`

(d) `[None, None, None]`

Q5: Which Python built-in module provides a double-ended queue with $O(1)$ append and pop operations at both ends?

(a) `queue.Queue`

(b) `collections.deque`

(c) `itertools.cycle`

(d) `heapq`

Section B: Algorithmic Complexity & Tracing (25 Marks)

Q6 (10 Marks): Analyze the exact time and space complexity of:

```
def find_pairs(nums: list[int], k: int) -> int:
    nums_set = set(nums)
    count = 0
    for x in nums_set:
        if x + k in nums_set:
            count += 1
    return count
```

State the average and worst-case time complexity, and the auxiliary space complexity.

Q7 (15 Marks): Explain why `funcs = [lambda: i for i in range(3)]` outputs `[2, 2, 2]` (Late Binding of Closures). How do you fix the function definition so that it correctly outputs `[0, 1, 2]`?

Section C: Code Implementation (30 Marks)

Q8 (15 Marks): Write a Python function `first_unique_char(s: str) -> int` that returns the 0-based index of the first non-repeating character in a string. If every character repeats, return `-1`. Your solution must run in strictly $O(n)$ time.

Q9 (15 Marks): Write a function `group_anagrams(words: list[str]) -> list[list[str]]` that groups anagrams together.

Example: `["eat", "tea", "tan", "ate", "nat", "bat"]` $\rightarrow$ `[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]`. Explain the time complexity of your approach.

Section D: Real-World Architecture & Performance (20 Marks)

Q10 (20 Marks): You are processing a 10 GB CSV dataset in Python on a machine with 8 GB of RAM.

- Why does `data = open("data.csv").readlines()` cause an `OutOfMemoryError`? (6 marks)
- How do Generator Functions (`yield`) resolve this problem? Write an idiomatic streaming generator that yields one cleaned row at a time. (14 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** `[1, 2, 3, 4]`. The in-place operator `+=` on lists invokes `__iadd__()`, which extends the existing list in-place rather than allocating a new object.
- **Q2: (b)** `TypeError: unhashable type: 'list'`. A tuple is only hashable if all of its constituent elements are hashable.

- **Q3: (c)** List: $O(n)$ (linear scan); Set: $O(1)$ (hash table lookup).
- **Q4: (b)** `[2, 2, 2]`. Closures bind to the variable name `i`, not the value at creation time. By the time the lambdas execute, `i` has completed the loop with value `2`.
- **Q5: (b)** `collections.deque`.

Section B Solutions

Q6 Solution:

- Building `nums_set`: $O(n)$ time.
- Iterating over unique elements: U iterations, each doing an $O(1)$ hash set lookup.
- **Average Time Complexity:** $O(n)$.
- **Worst-case Time Complexity:** $O(n^2)$ if severe hash collisions occur in the set.
- **Auxiliary Space Complexity:** $O(n)$ to store elements in `nums_set`.

Q7 Solution:

- Python closures look up variables in enclosing scopes at runtime (Late Binding). The variable `i` is looked up in the enclosing scope when `f()` is called, at which point the loop has finished and `i = 2`.
- **Fix (Default Argument Binding):**`funcs = [lambda x=i: x for i in range(3)]`
`# Default arguments are evaluated at definition time!`

Section C Solutions

Q8 Solution:

```
from collections import Counter

def first_unique_char(s: str) -> int:
    counts = Counter(s)  # O(N) time
    for idx, char in enumerate(s):  # O(N) time
        if counts[char] == 1:
            return idx
    return -1
```

Q9 Solution:

```
from collections import defaultdict

def group_anagrams(words: list[str]) -> list[list[str]]:
    anagram_map = defaultdict(list)
    for word in words:
        # Tuple of character counts or sorted string as canonical key
        canonical_key = "".join(sorted(word))  # O(K log K)
        anagram_map[canonical_key].append(word)
    return list(anagram_map.values())
```

Complexity: $O(N \cdot K \log K)$ where N is word count and K is maximum word length.

Section D Solutions

Q10 Solution:

- `.readlines()` loads the entire 10 GB file into memory at once as a list of strings, exceeding the 8 GB system RAM and causing an immediate OOM crash.
- **Streaming Generator Solution:**`def stream_large_csv(file_path: str):`
`    with open(file_path, "r", encoding="utf-8") as f:`

```
            header = next(f) # Read header
            for line in f:   # Iterating over file object streams line by line (O(1) memory)
                yield line.strip().split(",")

    # Consuming stream with constant memory
    for record in stream_large_csv("data.csv"):
        process_record(record)
```

Day 7: Python OOP, Iterators, Generators, File I/O & Phase 1 Checkpoint

Object-Oriented Architecture, Memory Streaming, Complexity Analysis & 15 Checkpoint Challenges

Part 1: Object-Oriented Programming (OOP) in Python

1.1 The Four Pillars of OOP in Python

1. Encapsulation: Attribute Protection & @property

Encapsulation bundles data and the methods that operate on that data within a single class, restricting direct outside modification.

- **Public:** `self.name` (Freely accessible everywhere).
- **Protected:** `self._balance` (Convention signaling internal use; accessible but discouraged).
- **Private:** `self.__secret` (Triggers **Name Mangling**: Python internally rewrites the attribute name to `_ClassName__secret` to prevent accidental subclass override).

```
class BankAccount:
    def __init__(self, owner: str, initial_balance: float):
        self.owner = owner
        self._balance = initial_balance  # Protected

    @property
    def balance(self) -> float:
        """Getter property: Allows controlled read access."""
        return self._balance

    @balance.setter
    def balance(self, value: float):
        """Setter property: Enforces validation rules."""
        if value < 0:
            raise ValueError("Account balance cannot be negative.")
        self._balance = value
```

---

2. Inheritance & Method Resolution Order (MRO)

Inheritance allows a subclass to inherit attributes and methods from one or more parent classes.

```
class BaseDatabaseConnector:
    def __init__(self, connection_str: str):
        self.connection_str = connection_str

    def connect(self):
        print(f"Connecting via {self.connection_str}...")

class PostgresConnector(BaseDatabaseConnector):
    def __init__(self, connection_str: str, schema: str):
        # super() delegates to parent class __init__
        super().__init__(connection_str)
        self.schema = schema

    def connect(self):
```

```
        super().connect()
        print(f"Setting search_path to schema: {self.schema}")
```

Multiple Inheritance & The Diamond Problem (C3 Linearization)

When class `D` inherits from `B` and `C`, and both inherit from `A`:

```
      A
     / \
    B   C
     \ /
      D
```

Python resolves method calls using the **C3 Linearization Algorithm** (Method Resolution Order - MRO).
You can inspect any class's resolution sequence via `ClassName.mro()`:

```
class A:
    def ping(self): print("A")
class B(A):
    def ping(self): print("B"); super().ping()
class C(A):
    def ping(self): print("C"); super().ping()
class D(B, C):
    def ping(self): print("D"); super().ping()

d = D()
d.ping()
# Output: D -> B -> C -> A
# D.mro() -> [D, B, C, A, object]
```

---

3. Polymorphism & Duck Typing

"If it walks like a duck and quacks like a duck, it's a duck."
Python does not require formal interface inheritance for polymorphism; it relies on **Duck Typing**: any object that implements the required method signature can be used interchangeably.

Operator Overloading (Magic / Dunder Methods)

Dunder Method	Operation	Semantic Meaning
`__init__(self, ...)`	Constructor	Initializes instance attributes
`__repr__(self)`	`repr(obj)`	Unambiguous official string representation for developers
`__str__(self)`	`str(obj)`, `print(obj)`	Readable, user-friendly string
`__eq__(self, other)`	`a == b`	Value equality check
`__lt__(self, other)`	`a < b`	Less-than comparison (enables `sort()`)
`__len__(self)`	`len(obj)`	Container element count
`__getitem__(self, k)`	`obj[k]`	Subscript indexing

```
class Vector2D:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y
```

```
    def __add__(self, other: 'Vector2D') -> 'Vector2D':
        return Vector2D(self.x + other.x, self.y + other.y)

    def __repr__(self) -> str:
        return f"Vector2D({self.x}, {self.y})"

v1 = Vector2D(2, 3)
v2 = Vector2D(5, 7)
print(v1 + v2)  # Vector2D(7, 10)
```

---

4. Abstraction (abc.ABC & @abstractmethod)

Abstract Base Classes define contracts that subclasses **must** implement before they can be instantiated.

```
from abc import ABC, abstractmethod

class BaseMLModel(ABC):
    @abstractmethod
    def train(self, X, y):
        """Must be implemented by concrete subclasses."""
        pass

    @abstractmethod
    def predict(self, X):
        pass

class LinearRegressor(BaseMLModel):
    def train(self, X, y):
        print("Training linear regression coefficients...")
    def predict(self, X):
        return [0.0] * len(X)

# model = BaseMLModel() # TypeError: Can't instantiate abstract class BaseMLModel with abstract methods
```

1.2 Instance vs. Class vs. Static Methods

```
class Employee:
    # Class Attribute (Shared across all instances)
    base_bonus_multiplier = 1.15

    def __init__(self, name: str, salary: float):
        self.name = name          # Instance attribute
        self.salary = salary      # Instance attribute

    # 1. Instance Method (Takes 'self', accesses instance and class state)
    def calculate_total_comp(self) -> float:
        return self.salary * self.base_bonus_multiplier

    # 2. Class Method (Takes 'cls', acts as an alternative factory constructor)
    @classmethod
    def from_string(cls, data_str: str):
        """Factory: creates an Employee from 'Name,Salary' string."""
        name, salary = data_str.split(",")
        return cls(name.strip(), float(salary.strip()))

    # 3. Static Method (Takes neither 'self' nor 'cls', pure utility function)
    @staticmethod
    def is_valid_salary(salary: float) -> bool:
        return salary >= 30000.0
```

Part 2: Advanced Protocols: Iterators & Generators

2.1 The Iterator Protocol

An **Iterable** is an object capable of returning its members one at a time (implements `__iter__()`).
An **Iterator** is an object representing a stream of data that produces successive items via repeated calls to `__next__()`, raising `StopIteration` when depleted.

```
class CountdownIterator:
    """Custom iterator that counts down from start to 1."""
    def __init__(self, start: int):
        self.current = start

    def __iter__(self):
        return self  # An iterator must return itself from __iter__

    def __next__(self) -> int:
        if self.current <= 0:
            raise StopIteration
        val = self.current
        self.current -= 1
        return val

for num in CountdownIterator(3):
    print(num)  # 3, 2, 1
```

2.2 Generators & The yield Keyword

A **Generator Function** uses `yield` instead of `return`. When called, it does not execute the code immediately; it returns a **generator object** that pauses and resumes execution state across calls, consuming $O(1)$ memory regardless of dataset size!

```
import sys

def large_range_list(n: int):
    return [i for i in range(n)]

def large_range_gen(n: int):
    for i in range(n):
        yield i

# Memory Comparison for 1,000,000 items:
list_obj = large_range_list(1_000_000)
gen_obj = large_range_gen(1_000_000)

print("List memory:", sys.getsizeof(list_obj), "bytes")     # ~8,448,728 bytes (~8.4 MB)
print("Generator memory:", sys.getsizeof(gen_obj), "bytes") # ~200 bytes! (42,000x leaner)
```

Generator Pipelines (Composing Data Transformations):

```
def read_lines(file_path):
    with open(file_path, "r") as f:
        for line in f:
            yield line.strip()

def filter_comments(lines):
    for line in lines:
        if not line.startswith("#") and line:
            yield line
```

```
def parse_csv(lines):
    for line in lines:
        yield line.split(",")

# Chained together lazily: Zero memory overhead
pipeline = parse_csv(filter_comments(read_lines("large_dataset.csv")))
```

Part 3: Robust File I/O & JSON Serialization

3.1 Context Managers (with Statement)

Context managers guarantee that resources (file descriptors, database connections, locks) are safely released, even if exceptions occur.

```
# Implementing a custom Context Manager via class
class TimedExecution:
    import time
    def __enter__(self):
        self.start = self.time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.duration = self.time.perf_counter() - self.start
        print(f"Executed block in {self.duration:.4f} seconds.")
        return False  # Propagate exceptions if any
```

3.2 JSON Serialization & Custom Encoders

The `json` module natively serializes basic types (`dict`, `list`, `str`, `int`, `float`, `bool`, `None`). Complex types (`datetime`, `UUID`, custom class instances) throw `TypeError: Object of type X is not JSON serializable`.

```
import json
from datetime import datetime

class UserEvent:
    def __init__(self, user_id: int, action: str, timestamp: datetime):
        self.user_id = user_id
        self.action = action
        self.timestamp = timestamp

class CustomJSONEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, datetime):
            return obj.isoformat()
        if isinstance(obj, UserEvent):
            return {
                "user_id": obj.user_id,
                "action": obj.action,
                "timestamp": obj.timestamp.isoformat()
            }
        return super().default(obj)

event = UserEvent(42, "PAYMENT_SUBMITTED", datetime.utcnow())
serialized_json = json.dumps(event, cls=CustomJSONEncoder, indent=2)
print(serialized_json)
```

Part 4: Computational Complexity Masterclass (Big-O)

Understanding time and space complexity is non-negotiable for Data Science Engineer interviews.

```
Runtime Complexity Hierarchy (Fastest to Slowest):
O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ) < O(n!)
```

4.1 Detailed Breakdown with Data Engineering Examples

Big-O Notation	Name	Practical Description	Canonical Data Science Example
$O(1)$	Constant	Runtime does not depend on dataset size N.	Hash map lookup (`dict[k]`), array index seek (`arr[0]`), stack push/pop.
$O(\log n)$	Logarithmic	Problem size is divided by a constant factor in each step.	Binary Search in sorted array, B-Tree index lookup.
$O(n)$	Linear	Examines each element a constant number of times.	Linear scan (`min()`, `max()`), single pass aggregation, Two-Sum hash map.
$O(n \log n)$	Linearithmic	Efficient divide-and-conquer sorting.	Timsort (`sorted()`), Merge Sort, QuickSort average case.
$O(n^2)$	Quadratic	Nested loops over the same dataset.	Brute force pair search, bubble sort, unindexed nested loop join.
$O(2^n)$	Exponential	Generating all subsets (Power Set).	Exhaustive combinatorial feature subset selection.

Part 5: Phase 1 Checkpoint Challenge: 15 Core Problems

Attempt these 15 problems independently before checking the solutions in Part 6!

Part A: 10 Mixed SQL Interview Problems

SQL Problem 1 (Aggregation & Grouping):

Write a query on `orders (order_id, customer_id, order_date, amount, status)` to find customers who placed at least 3 orders in 2024, where their total completed spend exceeds $1,000, and show their completed spend percentage relative to their total spend.

SQL Problem 2 (Anti-Join):

Given `departments (dept_id, dept_name)` and `employees (emp_id, emp_name, dept_id)`. Find all departments that have **no employees assigned**, using two completely different SQL techniques (`LEFT JOIN` and `NOT EXISTS`).

SQL Problem 3 (Self Join & Precedence):

Given `employee_salaries (emp_id, emp_name, dept_id, salary)`. Write a query to find all pairs of employees in the same department where Employee A earns at least $20,000 more than Employee B. Avoid duplicate inverted pairs.

SQL Problem 4 (Second-Highest Salary):

Write an ANSI SQL query to find the 2nd highest distinct salary from `employees`. If fewer than 2 distinct salaries exist, return `NULL` (ensure scalar null return).

SQL Problem 5 (Top-N per Group):

Given `products (product_id, category_id, product_name, price)`. Find the top 2 most expensive products in each category. If there is a tie for 2nd place, include all tied products.

SQL Problem 6 (Window Cumulative Metrics):

Given `daily_revenue (revenue_date, daily_amount)`. Calculate:

- The running cumulative revenue from the start of the year.
- A 7-day rolling average of daily revenue (current day and prior 6 days).

SQL Problem 7 (Consecutive Events / Gaps & Islands):

Given `user_logins (user_id, login_date)`. Identify all users who logged in for at least 3 consecutive calendar days.

SQL Problem 8 (Conditional Pivot):

Given `hospital_visits (visit_id, doctor_id, patient_id, visit_type, fee)`. Where `visit_type` is in `('In-Person', 'Telehealth', 'Emergency')`. Pivot the data to display `doctor_id`, total visits, total in-person fees, total telehealth fees, and total emergency fees in a single row per doctor.

SQL Problem 9 (Correlated Subquery):

Given `sales (sale_id, salesperson_id, amount, region)`. Find all sales transactions whose amount is strictly greater than the average transaction amount of that same region.

SQL Problem 10 (Join Cardinality & Duplicates):

Table A has 4 rows with key `1`. Table B has 3 rows with key `1` and 2 rows with `NULL`. How many rows are returned by:

- `INNER JOIN ON A.key = B.key`
- `LEFT JOIN ON A.key = B.key`
- `FULL OUTER JOIN ON A.key = B.key`

Part B: 5 Production Python Coding Challenges

Python Problem 1 (OOP Architecture):

Design a `Shape` class hierarchy with an abstract base class `Shape` having abstract methods `area()` and `perimeter()`.

Implement concrete classes `Rectangle(width, height)` and `Circle(radius)`. Add operator overloading so that two shapes can be compared by area using `<` (`__lt__`).

Python Problem 2 (Custom Streaming Iterator):

Write a custom iterator class `FibonacciIterator(max_count)` that produces the first `max_count` numbers of the Fibonacci sequence ($0, 1, 1, 2, 3, 5, \dots$) in $O(1)$ memory.

Python Problem 3 (Binary Search):

Implement `binary_search(nums: list[int], target: int) -> int` that returns the index of `target` in a strictly sorted ascending array in $O(\log n)$ time and $O(1)$ space. Return `-1` if absent.

Python Problem 4 (Frequency & Anagram Substring):

Write a function `find_anagram_indices(s: str, p: str) -> list[int]` that finds all start indices of `p`'s anagrams in `s` using a sliding window and hash map/counter in $O(|s|)$ time.

Python Problem 5 (File Parsing & JSON Generator):

Write a generator function `stream_error_logs(file_path: str)` that opens a log file lazily, finds lines with status `[ERROR]`, and yields a parsed dictionary `{"timestamp": ..., "service": ..., "message": ...}` one line at a time.

Part 6: Complete Solutions to Phase 1 Checkpoint

Solutions to SQL Problems (1–10)

Solution 1:

```
SELECT
    customer_id,
    COUNT(order_id) AS total_orders_2024,
    SUM(CASE WHEN status = 'Completed' THEN amount ELSE 0 END) AS completed_spend,
    ROUND(
        100.0 * SUM(CASE WHEN status = 'Completed' THEN amount ELSE 0 END) / SUM(amount),
        2
    ) AS completed_spend_pct
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2024
GROUP BY customer_id
HAVING COUNT(order_id) >= 3
   AND SUM(CASE WHEN status = 'Completed' THEN amount ELSE 0 END) > 1000;
```

Solution 2:

```
-- Technique A: LEFT JOIN with Anti-Join predicate
SELECT d.dept_id, d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;

-- Technique B: NOT EXISTS (Short-circuit optimization)
SELECT d.dept_id, d.dept_name
FROM departments d
WHERE NOT EXISTS (
```

```
    SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id
);
```

Solution 3:

```
SELECT
    e1.emp_name AS high_earner,
    e2.emp_name AS low_earner,
    e1.dept_id,
    e1.salary AS high_sal,
    e2.salary AS low_sal,
    (e1.salary - e2.salary) AS salary_difference
FROM employee_salaries e1
INNER JOIN employee_salaries e2
    ON e1.dept_id = e2.dept_id
    AND e1.salary >= e2.salary + 20000;
```

Solution 4:

```
WITH RankedSalaries AS (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT COALESCE(
    (SELECT DISTINCT salary FROM RankedSalaries WHERE rnk = 2),
    NULL
) AS second_highest_salary;
```

Solution 5:

```
WITH RankedProducts AS (
    SELECT
        category_id,
        product_name,
        price,
        DENSE_RANK() OVER (PARTITION BY category_id ORDER BY price DESC) AS price_rank
    FROM products
)
SELECT category_id, product_name, price, price_rank
FROM RankedProducts
WHERE price_rank <= 2
ORDER BY category_id, price_rank;
```

Solution 6:

```
SELECT
    revenue_date,
    daily_amount,
    -- 1. Cumulative Running Total
    SUM(daily_amount) OVER (
        ORDER BY revenue_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_yearly_total,
    -- 2. 7-Day Rolling Moving Average
    ROUND(
        AVG(daily_amount) OVER (
            ORDER BY revenue_date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ),
        2
    ) AS rolling_7day_avg
FROM daily_revenue;
```

Solution 7:

```
WITH DistinctLogins AS (
    SELECT DISTINCT user_id, CAST(login_date AS DATE) AS login_date
```

```
    FROM user_logins
),
Sequenced AS (
    SELECT
        user_id,
        login_date,
        login_date - CAST(ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY login_date) AS INT) *
INTERVAL '1 day' AS island_id
    FROM DistinctLogins
)
SELECT
    user_id,
    MIN(login_date) AS streak_start,
    MAX(login_date) AS streak_end,
    COUNT(*) AS consecutive_active_days
FROM Sequenced
GROUP BY user_id, island_id
HAVING COUNT(*) >= 3;
```

Solution 8:

```
SELECT
    doctor_id,
    COUNT(*) AS total_visits,
    SUM(CASE WHEN visit_type = 'In-Person' THEN fee ELSE 0 END) AS in_person_fees,
    SUM(CASE WHEN visit_type = 'Telehealth' THEN fee ELSE 0 END) AS telehealth_fees,
    SUM(CASE WHEN visit_type = 'Emergency' THEN fee ELSE 0 END) AS emergency_fees
FROM hospital_visits
GROUP BY doctor_id;
```

Solution 9:

```
SELECT s.sale_id, s.salesperson_id, s.region, s.amount
FROM sales s
WHERE s.amount > (
    SELECT AVG(sub.amount)
    FROM sales sub
    WHERE sub.region = s.region
);
```

Solution 10:

- **1. INNER JOIN:** $4 \times 3 = 12$ rows. (Keys with 1 match; NULLs do not match).
- **2. LEFT JOIN:** $4 \times 3 = 12$ rows. (All 4 rows from A find 3 matches in B).
- **3. FULL OUTER JOIN:** $12 + 2 = 14$ rows. (12 matched rows + 2 unmatched NULL rows from Table B).

Solutions to Python Problems (1–5)

Solution 1 (OOP Shape Hierarchy):

```
import math
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float:
        pass

    @abstractmethod
    def perimeter(self) -> float:
        pass
```

```
    def __lt__(self, other: 'Shape') -> bool:
        if not isinstance(other, Shape):
            return NotImplemented
        return self.area() < other.area()

class Rectangle(Shape):
    def __init__(self, width: float, height: float):
        self.width = width
        self.height = height

    def area(self) -> float:
        return self.width * self.height

    def perimeter(self) -> float:
        return 2 * (self.width + self.height)

class Circle(Shape):
    def __init__(self, radius: float):
        self.radius = radius

    def area(self) -> float:
        return math.pi * (self.radius ** 2)

    def perimeter(self) -> float:
        return 2 * math.pi * self.radius

# Validation:
r = Rectangle(4, 5)  # Area = 20
c = Circle(3)        # Area = 28.27
assert r < c         # True!
```

Solution 2 (Custom Iterator):

```
class FibonacciIterator:
    def __init__(self, max_count: int):
        self.max_count = max_count
        self.count = 0
        self.a, self.b = 0, 1

    def __iter__(self):
        return self

    def __next__(self) -> int:
        if self.count >= self.max_count:
            raise StopIteration
        val = self.a
        self.a, self.b = self.b, self.a + self.b
        self.count += 1
        return val

assert list(FibonacciIterator(6)) == [0, 1, 1, 2, 3, 5]
```

Solution 3 (Binary Search):

```
def binary_search(nums: list[int], target: int) -> int:
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

```
assert binary_search([1, 3, 5, 7, 9, 11], 7) == 3
assert binary_search([1, 3, 5, 7, 9, 11], 4) == -1
```

Solution 4 (Sliding Window Anagrams):

```
from collections import Counter

def find_anagram_indices(s: str, p: str) -> list[int]:
    p_len, s_len = len(p), len(s)
    if p_len > s_len:
        return []

    p_count = Counter(p)
    s_count = Counter(s[:p_len])
    result = []

    if s_count == p_count:
        result.append(0)

    for i in range(p_len, s_len):
        # Slide window: Add right char, remove left char
        s_count[s[i]] += 1
        left_char = s[i - p_len]
        s_count[left_char] -= 1
        if s_count[left_char] == 0:
            del s_count[left_char]

        if s_count == p_count:
            result.append(i - p_len + 1)

    return result

assert find_anagram_indices("cbaebabacd", "abc") == [0, 6]
```

Solution 5 (File Parsing Generator):

```
import re

LOG_REGEX = re.compile(r'\[(?P<timestamp>[^\]]+)\] \[(?P<level>[A-Z]+)\] \[(?P<service>[^\]]+)\]
(?P<message>.*)')

def stream_error_logs(file_path: str):
    with open(file_path, "r", encoding="utf-8") as f:
        for line in f:
            if "[ERROR]" in line:
                match = LOG_REGEX.match(line.strip())
                if match:
                    data = match.groupdict()
                    yield {
                        "timestamp": data["timestamp"],
                        "service": data["service"],
                        "message": data["message"]
                    }
```

Part 7: Enterprise Case Study: Extensible Plugin-Based ETL Framework

Architectural Problem Statement

At **DataSphere Analytics**, daily batch pipelines ingested files from multiple heterogeneous sources (Local CSVs, S3 Buckets, REST APIs). Junior developers wrote monolithic scripts with tangled file-handling, parsing, and transformation code.

You must design an extensible, object-oriented ETL framework where:

- `BaseExtractor`, `BaseTransformer`, and `BaseLoader` are abstract contracts.
- Data streams lazily as generator records between phases to handle arbitrary file sizes with $O(1)$ memory.
- Concrete implementations can be plugged in without modifying the core pipeline driver.

Production Solution

```
from abc import ABC, abstractmethod
from typing import Generator, Any

class BaseExtractor(ABC):
    @abstractmethod
    def extract(self) -> Generator[dict, None, None]:
        pass

class BaseTransformer(ABC):
    @abstractmethod
    def transform(self, records: Generator[dict, None, None]) -> Generator[dict, None, None]:
        pass

class BaseLoader(ABC):
    @abstractmethod
    def load(self, records: Generator[dict, None, None]) -> int:
        pass

# Concrete Extractor: Simulating CSV Streaming
class MockCSVExtractor(BaseExtractor):
    def __init__(self, data: list[dict]):
        self.data = data

    def extract(self) -> Generator[dict, None, None]:
        for row in self.data:
            yield row

# Concrete Transformer: Cleaning & Normalizing Data
class CleanCurrencyTransformer(BaseTransformer):
    def transform(self, records: Generator[dict, None, None]) -> Generator[dict, None, None]:
        for row in records:
            # Transformation logic
            cleaned_amount = float(str(row["amount"]).replace("$", "").strip())
            yield {
                "txn_id": row["txn_id"],
                "clean_amount": round(cleaned_amount, 2),
                "is_flagged": cleaned_amount > 10000.0
            }

# Concrete Loader: Target Stager
class ConsoleLoader(BaseLoader):
    def load(self, records: Generator[dict, None, None]) -> int:
        count = 0
```

```
        for record in records:
            print(f"Loading record into warehouse: {record}")
            count += 1
        return count

# Pipeline Orchestrator Driver
class ETLPipeline:
    def __init__(self, extractor: BaseExtractor, transformer: BaseTransformer, loader: BaseLoader):
        self.extractor = extractor
        self.transformer = transformer
        self.loader = loader

    def run(self) -> int:
        raw_stream = self.extractor.extract()
        transformed_stream = self.transformer.transform(raw_stream)
        loaded_count = self.loader.load(transformed_stream)
        return loaded_count

# Execution Test
sample_raw_data = [
    {"txn_id": "T1", "amount": "$450.50"},
    {"txn_id": "T2", "amount": "$12500.00"},
    {"txn_id": "T3", "amount": "$89.99"}
]

pipeline = ETLPipeline(
    extractor=MockCSVExtractor(sample_raw_data),
    transformer=CleanCurrencyTransformer(),
    loader=ConsoleLoader()
)

records_processed = pipeline.run()
print(f"Pipeline successfully completed. Loaded {records_processed} rows.")
```

Part 8: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                    PYTHON OOP & COMPLEXITY CHEAT SHEET           │
├──────────────────────────────────────────────────────────────────┤
│ 1. 4 PILLARS OF OOP:                                             │
│    - Encapsulation: Use @property for validated getters/setters. │
│    - Inheritance  : super().__init__(). C3 MRO diamond resolution.│
│    - Polymorphism : Duck typing & Dunder methods (__add__, __lt__).│
│    - Abstraction  : abc.ABC + @abstractmethod contracts.         │
├──────────────────────────────────────────────────────────────────┤
│ 2. METHOD TYPES:                                                 │
│    - Instance Method: (self) -> Acts on instance state.          │
│    - Class Method   : @classmethod (cls) -> Factory constructors.│
│    - Static Method  : @staticmethod () -> Isolated utility functions.│
├──────────────────────────────────────────────────────────────────┤
│ 3. STREAMING & ITERATION:                                        │
│    - Iterator : Implements __iter__() and __next__(); raises    │
│                 StopIteration.                                   │
│    - Generator: Uses 'yield'. Pauses state. O(1) memory footprint.│
├──────────────────────────────────────────────────────────────────┤
│ 4. COMPUTATIONAL COMPLEXITY (BIG-O):                             │
│    - O(1)     : Dict/Set lookup, array index.                    │
│    - O(log n) : Binary Search.                                   │
│    - O(n)     : Linear search, Two-Sum hash map.                 │
│    - O(n log n): Timsort, Merge Sort.                            │
```

```
│    - O(n²)    : Nested loops, unindexed nested loop join.              │
└──────────────────────────────────────────────────────────────────────────┘
```

Part 9: 100-Mark Self-Assessment Examination

Section A: Conceptual Foundations (20 Marks — 4 marks each)

Q1: What is the Method Resolution Order (MRO) in Python?

(a) The order in which module imports are parsed at startup.

(b) The algorithmic sequence (C3 Linearization) Python uses to look up attributes and methods in class hierarchies.

(c) The sequence in which variables are cleared by the garbage collector.

(d) The order of execution of functions in a thread pool.

Q2: What happens when an instantiated class attempts to call an `@abstractmethod` that was not overridden by its subclass?

(a) A warning is printed and the method returns `None`.

(b) Python raises `TypeError: Can't instantiate abstract class with abstract methods`.

(c) The method calls the parent object's base implementation automatically.

(d) The Python interpreter crashes.

Q3: What is the space complexity of a generator that yields numbers from 1 to 100 million?

(a) $O(n)$

(b) $O(\log n)$

(c) $O(1)$

(d) $O(n \log n)$

Q4: In Python name mangling, how is a private attribute `__salary` defined inside class `Employee` stored internally?

(a) `__salary`

(b) `_salary`

(c) `_Employee__salary`

(d) `Employee.__salary`

Q5: What is the worst-case time complexity of Python's built-in `list.sort()`?

(a) $O(n^2)$

(b) $O(n \log n)$

(c) $O(n)$

(d) $O(\log n)$

Section B: Code Analysis & Output Tracing (25 Marks)

Q6 (10 Marks): Trace and state the exact output of:

```
class Parent:
    def greet(self):
        print("Hello from Parent")

class Child(Parent):
    def greet(self):
        super().greet()
        print("Hello from Child")

c = Child()
c.greet()
```

Q7 (15 Marks): What does this generator produce, and what happens on the 4th call to `next()`?

```
def gen():
    yield 10
    yield 20
    yield 30

g = gen()
print(next(g))
print(next(g))
print(next(g))
print(next(g))
```

Section C: Implementation Challenges (35 Marks)

Q8 (15 Marks): Write a Python class `Stack` implementing a LIFO stack with methods `push(item)`, `pop() -> item`, and `peek() -> item`. All operations must run in strictly $O(1)$ amortized time. Raise an `IndexError` when popping from an empty stack.

Q9 (20 Marks): Write a Python generator function `sliding_window(seq: list, window_size: int)` that takes a list and yields successive overlapping windows of size `window_size` as tuples.

Example: `sliding_window([1, 2, 3, 4, 5], 3)` yields `(1, 2, 3)`, `(2, 3, 4)`, `(3, 4, 5)`.

Section D: Architectural & Complexity Scenario (20 Marks)

Q10 (20 Marks): You are hired to optimize a Python batch process that compares 100,000 incoming customer transaction records against 500,000 existing fraudulent account IDs.

- The original code uses:

```
flagged = [txn for txn in transactions if txn["account_id"] in fraud_list]
# where fraud_list is a Python list of integers.
```

Calculate the exact Big-O time complexity of this code and explain why it takes over 15 minutes to run. (10 marks)

- Rewrite the code to complete in under 1 second. Explain the data structure change and analyze the new time and space complexity. (10 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** C3 Linearization order for attribute lookup.
- **Q2: (b)** Python prevents class instantiation and throws `TypeError`.
- **Q3: (c)** $O(1)$ memory; values are generated lazily one at a time.
- **Q4: (c)** `_Employee__salary`.
- **Q5: (b)** $O(n \log n)$ (Timsort guarantee).

Section B Solutions

Q6 Output:

```
Hello from Parent
Hello from Child
```

Q7 Output:

```
10
20
30
StopIteration Exception is raised on the 4th next() call!
```

Section C Solutions

Q8 Solution:

```
class Stack:
    def __init__(self):
        self._items = []

    def push(self, item):
        self._items.append(item)  # O(1) amortized

    def pop(self):
        if not self._items:
            raise IndexError("pop from empty stack")
        return self._items.pop()   # O(1) from end
```

```
    def peek(self):
        if not self._items:
            raise IndexError("peek from empty stack")
        return self._items[-1]     # O(1)
```

Q9 Solution:

```
def sliding_window(seq: list, window_size: int):
    if window_size <= 0 or window_size > len(seq):
        return
    for i in range(len(seq) - window_size + 1):
        yield tuple(seq[i:i + window_size])
```

Section D Solutions

Q10 Solution:

- **Complexity:** Checking `txn["account_id"] in fraud_list` performs a linear scan $O(M)$ over the 500,000 elements in `fraud_list`. With $N = 100,000$ transactions, the total runtime is $O(N \times M) = 100,000 \times 500,000 = 5 \times 10^{10}$ operations, taking 15–20 minutes.
- **Optimized Solution:**

```
# Convert list to hash set upfront: O(M) time, O(M) space
fraud_set = set(fraud_list)

# Set membership lookup is O(1) average:
flagged = [txn for txn in transactions if txn["account_id"] in fraud_set]
```

New Time Complexity: $O(M + N) \approx 600,000$ operations. Executes in ≈ 50 milliseconds (over 20,000x speedup!).
Space Complexity: $O(M)$ to store the hash set in memory.

Day 8: NumPy & Pandas for High-Performance Data Analysis

Vectorization, Indexing Internals, GroupBy, Merging, Pivoting, Data Cleaning & Time-Series

---

Part 1: NumPy Internals & Vectorized Computing

1.1 The ndarray Memory Model

In standard Python, a `list` is an array of pointers to boxed heap objects. A NumPy `ndarray` is a contiguous block of homogeneous raw bytes in memory with an attached metadata header:

- **dtype (Data Type):** Tells NumPy how to interpret raw bytes (e.g., `int32`, `float64`).
- **shape:** A tuple representing array dimensions (e.g., `(1000, 4)`).
- **strides:** A tuple of bytes to step in each dimension when traversing memory.

```
import numpy as np

arr = np.array([[10, 20, 30], [40, 50, 60]], dtype=np.int32)
print("Shape:", arr.shape)      # (2, 3)
print("Dtype:", arr.dtype)      # int32 (4 bytes per element)
print("Strides:", arr.strides)  # (12, 4) -> 12 bytes to jump to next row, 4 bytes to jump to next
column
```

1.2 Vectorization vs. Python Loops

Vectorization eliminates the overhead of Python bytecode interpretation, dynamic type checking, and pointer chasing by executing operations directly in compiled C/Fortran code using modern CPU **SIMD (Single Instruction, Multiple Data)** vector registers.

```
import time

size = 10_000_000
python_list = list(range(size))
numpy_array = np.arange(size, dtype=np.int64)

# Pure Python Loop:
t0 = time.perf_counter()
res_list = [x * 2 for x in python_list]
t1 = time.perf_counter()
print(f"Python list comprehension: {t1 - t0:.4f} seconds")  # ~0.65 seconds

# Vectorized NumPy:
t0 = time.perf_counter()
res_arr = numpy_array * 2
t1 = time.perf_counter()
print(f"NumPy vectorized multiplication: {t1 - t0:.4f} seconds")  # ~0.009 seconds (~70x speedup!)
```

1.3 The Rules of Array Broadcasting

Broadcasting allows NumPy to perform arithmetic operations on arrays of differing shapes without allocating unnecessary memory copies.

The Formal Broadcasting Rule:

Starting from the **trailing (right-most) dimensions** and moving leftward, two dimensions are compatible if:

- They are **equal**, OR
- One of them is **1**.

Valid Example:

```
Array A: (5, 1, 4)
Array B:    (3, 4)  --> Trailing dimensions match: 4 == 4; middle dimension matches: 1 and 3
(broadcasting expands 1 to 3); leading dimension expands 1 to 5.
Result:  (5, 3, 4)
```

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])  # Shape (2, 3)
row_vector = np.array([10, 20, 30])         # Shape (3,) -> broadcasts to (2, 3)
print(matrix + row_vector)
# Output:
# [[11, 22, 33],
#  [14, 25, 36]]
```

Part 2: Pandas Core Architecture & Indexing Internals

2.1 Series & DataFrame Architecture

- **Series:** A 1D labeled array capable of holding any data type, composed of a NumPy array and an index label array.
- **DataFrame:** A 2D tabular data structure with labeled axes (rows and columns). Internally, columns of identical data types are grouped into unified memory arrays managed by the **BlockManager** (or 1D arrays in the newer **ArrayManager**).

2.2 Indexing: loc vs. iloc vs. at / iat

Accessor	Basis of Indexing	Bounds Inclusion	Negative Indices?	Use Case
.loc[]	**Label-based** (index names, column names)	**Both endpoints INCLUDED** (`'A':'C'` includes `'C'`)	Only if index contains negative integers	Production filtering by boolean masks or named labels
.iloc[]	**Integer-based** (0-indexed integer position)	Standard Python: **Stop bound EXCLUDED** (`0:3` gets 0, 1, 2)	Supported (counts from end)	Positional array slicing
.at[]	Label-based scalar lookup	Scalar only	No	Ultra-fast single-cell lookup
.iat[]	Position-based scalar lookup	Scalar only	Supported	Ultra-fast single-cell lookup by integer coordinates

```
import pandas as pd

df = pd.DataFrame(
    {"name": ["Alice", "Bob", "Charlie"], "salary": [90000, 110000, 125000]},
    index=["emp_101", "emp_102", "emp_103"]
)

# loc: Label-based (Includes endpoint!)
print(df.loc["emp_101":"emp_102", "name":"salary"])

# iloc: Position-based (Excludes endpoint!)
print(df.iloc[0:2, 0:2])
```

2.3 The SettingWithCopyWarning Demystified

One of the most common pitfalls in Pandas is **Chained Indexing**:

```
# THE BUGGY PATTERN (Chained Indexing)
df[df["salary"] > 100000]["salary"] = 105000
# Pandas triggers: SettingWithCopyWarning: A value is trying to be set on a copy of a slice from a
DataFrame.
```

Why does this happen?

`df[df["salary"] > 100000]` executes first and returns an intermediate object. Pandas cannot guarantee whether this intermediate object is a **view** (sharing memory with `df`) or an isolated **copy**. Modifying it may update `df` or modify a throwaway temporary copy!

The Bulletproof Solution:

Always use `.loc` to perform filtering and assignment in a **single atomic operation**:

```
df.loc[df["salary"] > 100000, "salary"] = 105000  # Guaranteed to modify original df in place!
```

Part 3: Advanced Grouping & Relational Operations

3.1 The Split-Apply-Combine Pattern

When calling `df.groupby('dept')`:

- **Split:** Data is partitioned into discrete groups based on key values without building full intermediate DataFrames.
- **Apply:** An aggregation, transformation, or filter function is applied to each group.
- **Combine:** Results are concatenated into an output DataFrame.

```
df_sales = pd.DataFrame({
    "dept": ["HR", "IT", "IT", "HR", "IT"],
    "emp": ["Alice", "Bob", "Charlie", "David", "Evan"],
    "salary": [70000, 120000, 110000, 80000, 140000]
})

# 1. Multi-Aggregate (Named Aggregation)
dept_summary = df_sales.groupby("dept").agg(
    headcount=("emp", "count"),
    avg_salary=("salary", "mean"),
```

```
    max_salary=("salary", "max")
).reset_index()
```

3.2 transform(): Emulating SQL Window Functions

Unlike `.agg()` (which collapses N rows into 1 row per group), `.transform()` calculates the group metric and **broadcasts it back to the original DataFrame's row dimensions**.

```
# Calculate each employee's salary difference from their department average
df_sales["dept_avg"] = df_sales.groupby("dept")["salary"].transform("mean")
df_sales["salary_diff"] = df_sales["salary"] - df_sales["dept_avg"]
# Identical to SQL: salary - AVG(salary) OVER (PARTITION BY dept)
```

3.3 filter(): Emulating SQL HAVING

`.filter()` discards entire groups that do not satisfy a group-level boolean predicate.

```
# Keep only departments with at least 3 employees
large_depts = df_sales.groupby("dept").filter(lambda g: len(g) >= 3)
```

3.4 Relational Joins with pd.merge()

```
customers = pd.DataFrame({"cust_id": [1, 2, 3], "name": ["Acme", "Beta", "Cyber"]})
orders = pd.DataFrame({"order_id": [101, 102], "cust_id": [1, 2], "val": [500, 750]})

# Production-grade merge with explicit cardinality validation:
merged = pd.merge(
    customers,
    orders,
    on="cust_id",
    how="left",
    validate="one_to_many"  # Asserts customers has unique cust_id!
)
```

Part 4: Reshaping Data: Pivoting, Melting & Stacking

```
WIDE FORMAT (Human-friendly / Dashboards)
| Date       | Electronics | Fashion | Home |
| 2024-01-01 | 15000       | 8000    | 4000 |

                ▲                          |
                |                          |
        pivot_table()                   melt()
                |                          |
                |                          ▼
LONG FORMAT (Tidy Data / Analytics & Machine Learning)
| Date       | Category    | Revenue |
| 2024-01-01 | Electronics | 15000   |
| 2024-01-01 | Fashion     | 8000    |
| 2024-01-01 | Home        | 4000    |
```

4.1 pivot_table() (Long to Wide with Aggregation)

```
df_orders = pd.DataFrame({
    "region": ["East", "East", "West", "West"],
```

```
    "quarter": ["Q1", "Q2", "Q1", "Q2"],
    "revenue": [50000, 60000, 45000, 55000]
})

pivot = df_orders.pivot_table(
    index="region",
    columns="quarter",
    values="revenue",
    aggfunc="sum",
    fill_value=0,
    margins=True  # Appends 'All' Total row and column!
)
```

4.2 melt() (Wide to Long)

```
wide_df = pd.DataFrame({
    "country": ["USA", "Germany"],
    "year_2022": [100, 80],
    "year_2023": [115, 85]
})

tidy_df = pd.melt(
    wide_df,
    id_vars=["country"],
    value_vars=["year_2022", "year_2023"],
    var_name="year",
    value_name="gdp_index"
)
```

Part 5: Data Cleaning & Quality Operations

5.1 Missing Data Strategies

- `df.isna().sum()`: Measures null distribution.
- `df.dropna(subset=['critical_col'])`: Drops rows with nulls in critical attributes.
- Imputation Techniques:`# Mean / Median Imputation`
`df["salary"] = df["salary"].fillna(df["salary"].median())`

`# Forward / Backward Fill for Time-Series`
`df["price"] = df["price"].ffill().bfill()`

5.2 Outlier Detection Techniques

1. Interquartile Range (IQR) Method:

```
def remove_outliers_iqr(df: pd.DataFrame, column: str) -> pd.DataFrame:
    q1 = df[column].quantile(0.25)
    q3 = df[column].quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    return df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]
```

2. Z-Score Method:

```
def flag_outliers_zscore(df: pd.DataFrame, column: str, threshold: float = 3.0) -> pd.Series:
    mean = df[column].mean()
    std = df[column].std()
    z_scores = (df[column] - mean) / std
    return z_scores.abs() > threshold
```

5.3 Memory Optimization via Categorical Types

Converting low-cardinality string columns (`gender`, `status`, `country`) to `category` dtypes replaces repeated strings with integer lookup keys, often reducing DataFrame RAM footprint by $80%$:

```
df["status"] = df["status"].astype("category")
```

Part 6: Time-Series & Datetime Analytics

```
df["order_timestamp"] = pd.to_datetime(df["order_timestamp"])

# 1. Component Extraction via .dt accessor
df["year"] = df["order_timestamp"].dt.year
df["month_name"] = df["order_timestamp"].dt.month_name()
df["day_of_week"] = df["order_timestamp"].dt.day_name()
df["is_weekend"] = df["order_timestamp"].dt.dayofweek.isin([5, 6])

# 2. Resampling (Time-based GroupBy)
monthly_sales = df.set_index("order_timestamp")["revenue"].resample("M").sum()

# 3. Rolling Moving Windows
df["7d_rolling_avg"] = df.set_index("order_timestamp")["revenue"].rolling("7D").mean().values
```

Part 7: Required Deliverable: Messy CSV Data Ingestion & Executive Summary Pipeline

Business Scenario

You are given a raw, corrupted sales log CSV file `raw_sales_data.csv`. The data contains:

- Missing customer IDs and negative quantities.
- String currency fields formatted like `"$1,250.00"`.
- Inconsistent date formats.
- Duplicate order IDs.

Production Solution Script

```
import pandas as pd
import numpy as np

def clean_and_summarize_sales(file_path: str) -> pd.DataFrame:
    """
    Ingests dirty transaction data, validates integrity, removes anomalies,
    and produces an executive summary table by Product Category.
```

```
    """
    # 1. Ingestion
    df = pd.read_csv(file_path)

    # 2. Deduplication on Business Key
    initial_rows = len(df)
    df = df.drop_duplicates(subset=["order_id"], keep="first")

    # 3. Clean Currency Strings to Numeric Float
    df["unit_price"] = (
        df["unit_price"]
        .astype(str)
        .str.replace("$", "", regex=False)
        .str.replace(",", "", regex=False)
        .str.strip()
    )
    df["unit_price"] = pd.to_numeric(df["unit_price"], errors="coerce")

    # 4. Standardize Dates
    df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce")

    # 5. Handle Integrity Anomalies (Negative quantity, null dates, null prices)
    valid_mask = (
        df["order_date"].notna() &
        df["unit_price"].notna() &
        (df["unit_price"] > 0) &
        (df["quantity"] > 0)
    )
    df_clean = df.loc[valid_mask].copy()

    # 6. Feature Engineering
    df_clean["gross_revenue"] = df_clean["quantity"] * df_clean["unit_price"]

    # 7. Executive Summary Table
    summary = df_clean.groupby("category").agg(
        total_orders=("order_id", "count"),
        total_units_sold=("quantity", "sum"),
        total_revenue=("gross_revenue", "sum"),
        avg_order_value=("gross_revenue", "mean"),
        max_single_sale=("gross_revenue", "max")
    ).reset_index()

    # Format currency for executive presentation
    summary["total_revenue"] = summary["total_revenue"].round(2)
    summary["avg_order_value"] = summary["avg_order_value"].round(2)

    # Append Company-Wide Total Row
    total_row = pd.DataFrame([{
        "category": "TOTAL / COMPANY",
        "total_orders": summary["total_orders"].sum(),
        "total_units_sold": summary["total_units_sold"].sum(),
        "total_revenue": summary["total_revenue"].sum(),
        "avg_order_value": round(df_clean["gross_revenue"].mean(), 2),
        "max_single_sale": summary["max_single_sale"].max()
    }])

    final_executive_table = pd.concat([summary, total_row], ignore_index=True)
    return final_executive_table
```

Part 8: Structured Interview Question Bank

Q1: What is the difference between df['col'] vs df[['col']]?

Answer:

- `df['col']` extracts a 1-dimensional **Pandas Series**.
- `df[['col']]` extracts a 2-dimensional **Pandas DataFrame** containing a single column. This distinction matters because DataFrame methods and Series methods differ, and passing a list of column names preserves DataFrame structure.

Q2: Why is iterating over a DataFrame with for index, row in df.iterrows(): strongly discouraged in production?

Answer:
`iterrows()` is notoriously slow because for every single row, it boxes the data into a brand-new Pandas Series object and checks data types dynamically, completely bypassing NumPy's vectorization and C-level memory speed. Operations using `iterrows()` are typically $100\times$ to $1000\times$ slower than vectorized operations, `.apply()`, or list comprehensions over NumPy arrays (`df['col'].to_numpy()`).

Q3: Explain the difference between merge() and concat() in Pandas.

Answer:

- `pd.merge()` performs database-style relational joins based on common key columns or index labels (equivalent to SQL `INNER`, `LEFT`, `RIGHT`, `OUTER` joins).
- `pd.concat()` stitches DataFrames together along an axis (vertically `axis=0` like SQL `UNION ALL`, or horizontally `axis=1`) purely based on index alignment, without evaluating key join predicates.

Q4: How does Pandas handle missing values under the hood for integer and boolean columns?

Answer:
Historically, Pandas coerced integer columns containing missing values to `float64` because standard NumPy arrays did not support integer `NaN`. In modern Pandas (versions 1.0+), Pandas introduced nullable data types (`Int64`, `boolean`, `string`) backed by a separate boolean bitmask that tracks null status without altering the underlying numerical data type.

Part 9: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────────┐
│                    NUMPY & PANDAS CHEAT SHEET                        │
├──────────────────────────────────────────────────────────────────────┤
│ 1. NUMPY:                                                            │
│    - Homogeneous memory buffer. Strides define byte steps.           │
│    - Broadcasting: Trailing dims must match or be 1.                 │
├──────────────────────────────────────────────────────────────────────┤
│ 2. INDEXING:                                                         │
│    - .loc[] : Label-based (INCLUDES endpoint).                       │
│    - .iloc[]: Integer position-based (EXCLUDES endpoint).            │
│    - Avoid Chained Indexing: df.loc[mask, 'col'] = val.              │
├──────────────────────────────────────────────────────────────────────┤
│ 3. GROUPBY PATTERNS:                                                 │
```

```
|    - .agg()      : Collapses rows into summary metrics.                |
|    - .transform(): Broadcasts group metrics back to row granularity.   |
|    - .filter()   : Discards groups failing group-level condition.      |
├────────────────────────────────────────────────────────────────────────┤
| 4. RESHAPING:                                                          |
|    - pivot_table(): Long to Wide (Aggregates values).                  |
|    - melt()       : Wide to Long (Unpivots columns into rows).         |
├────────────────────────────────────────────────────────────────────────┤
| 5. CLEANING:                                                           |
|    - IQR Bounds   : [Q1 - 1.5*IQR, Q3 + 1.5*IQR]                       |
|    - Categoricals : df['col'].astype('category') saves up to 80% RAM.  |
|    - Datetime     : df['dt'].dt.month, df.resample('M').sum()          |
└────────────────────────────────────────────────────────────────────────┘
```

Part 10: 100-Mark Self-Assessment Examination

Section A: Conceptual & API Nuances (25 Marks — 5 marks each)

Q1: What is the result of slicing a DataFrame with `.loc[1:3]` when the DataFrame's index contains integer labels `[1, 2, 3, 4, 5]`?

(a) Rows at integer positions 1 and 2 (labels 2 and 3).

(b) Rows with labels 1, 2, and 3.

(c) Rows with labels 1 and 2 only.

(d) `KeyError`.

Q2: When merging two DataFrames with duplicate keys on both sides (M matches on left, N matches on right), how many rows does `pd.merge(how='inner')` generate for that key?

(a) $M + N$

(b) $\max(M, N)$

(c) $M \times N$

(d) 0

Q3: Which method converts a wide DataFrame with columns `['ID', 'Q1_Sales', 'Q2_Sales']` into a long-form DataFrame?

(a) `.pivot()`

(b) `.melt()`

(c) `.stack()`

(d) `.explode()`

Q4: What is the primary purpose of `validate='one_to_many'` in `pd.merge()`?

(a) Enforces that the right table contains only unique keys.

(b) Enforces that the left table contains only unique keys, raising an exception if duplicates exist.

(c) Converts the output DataFrame to a one-dimensional Series.

(d) Automatically fills null values with 0.

Q5: Why does converting a low-cardinality string column to dtype `category` drastically reduce memory consumption?

(a) Strings are compressed using gzip in memory.

(b) Unique strings are stored once in a dictionary, and the column is replaced by small integer codes (e.g., `int8`).

(c) All uppercase letters are converted to lowercase.

(d) It drops missing values automatically.

Section B: Code Analysis & Output Tracing (25 Marks)

Q6 (10 Marks): Given the following DataFrame:

```
df = pd.DataFrame({
    'dept': ['Sales', 'Sales', 'IT', 'IT'],
    'bonus': [1000, 2000, 3000, 4000]
})
df['avg_bonus'] = df.groupby('dept')['bonus'].transform('mean')
```

State the exact contents of the resulting DataFrame (columns: `dept`, `bonus`, `avg_bonus`).

Q7 (15 Marks): What does this NumPy broadcasting operation output?

```
a = np.array([1, 2, 3]).reshape(3, 1)
b = np.array([10, 20])
c = a + b
```

State `c.shape` and write down the resulting 2D matrix.

Section C: Implementation & Data Cleaning Challenges (30 Marks)

Q8 (15 Marks): Write a Python function `impute_missing_by_group(df: pd.DataFrame, group_col: str, target_col: str) -> pd.DataFrame` that fills missing values in `target_col` with the **median** of that column within each respective group in `group_col`.

Q9 (15 Marks): You are given an e-commerce DataFrame `orders` with columns `['customer_id', 'order_date', 'amount']`. Write a query in Pandas to find:

- Top 5 customers by total spending.
- For each of these 5 customers, calculate their 3-order rolling average spend ordered by `order_date`.

Section D: Architectural & Performance Scenario (20 Marks)

Q10 (20 Marks): A data science pipeline reads a 12 GB CSV file on a machine with 16 GB RAM using `df = pd.read_csv('data.csv')`. The machine freezes due to RAM exhaustion.

- Explain why a 12 GB CSV file expands to 25–35 GB in RAM when loaded into standard Pandas. (8 marks)
- Detail three concrete techniques in Pandas to successfully process this dataset without running out of memory. (12 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** `.loc` is label-based and includes both endpoints (labels 1, 2, and 3).
- **Q2: (c)** Cartesian multiplication on matching keys: $M \times N$.
- **Q3: (b)** `.melt()`.
- **Q4: (b)** Asserts that the left table's merge keys are strictly unique.
- **Q5: (b)** Encodes unique strings as an integer categorical dictionary.

Section B Solutions

Q6 Output:

dept	bonus	avg_bonus
Sales	1000	1500.0
Sales	2000	1500.0
IT	3000	3500.0
IT	4000	3500.0

Q7 Output:

- `c.shape`: `(3, 2)`
- Matrix:`[[11, 21],`
  `[12, 22],`
  `[13, 23]]`

Section C Solutions

Q8 Solution:

```
def impute_missing_by_group(df: pd.DataFrame, group_col: str, target_col: str) -> pd.DataFrame:
    df_copy = df.copy()
    group_medians = df_copy.groupby(group_col)[target_col].transform("median")
```

```
    df_copy[target_col] = df_copy[target_col].fillna(group_medians)
    return df_copy
```

Q9 Solution:

```
# 1. Top 5 customers by spend
top5_custs = (
    orders.groupby("customer_id")["amount"]
    .sum()
    .nlargest(5)
    .index
)

# 2. Filter, sort, and calculate rolling 3-order average
top5_df = orders[orders["customer_id"].isin(top5_custs)].sort_values(["customer_id",
"order_date"]).copy()
top5_df["rolling_3_avg"] = (
    top5_df.groupby("customer_id")["amount"]
    .rolling(window=3, min_periods=1)
    .mean()
    .reset_index(level=0, drop=True)
)
```

Section D Solutions

Q10 Solution:

- **Memory Inflation:** CSVs store raw text. In Pandas, strings default to Python object pointers (8 bytes per pointer + 50–80 bytes per boxed Python string), 64-bit numerical types are allocated by default (`int64`, `float64`), and BlockManager metadata causes $2\times$ to $3\times$ memory bloat.
- **Remediation Techniques:**

Chunking: Process the file in streaming batches: `pd.read_csv('data.csv', chunksize=100_000)`.
Column Pruning (usecols): Load only necessary columns: `pd.read_csv(..., usecols=['id', 'date', 'amount'])`.
Downcasting Dtypes: Specify downcasted types: `dtype={'status': 'category', 'qty': 'int16', 'amount': 'float32'}`.
Use Polars / PyArrow Engine: `pd.read_csv(..., engine='pyarrow')` or Apache Arrow backed string types to eliminate string pointer boxing.

Day 9: Statistics & Probability for Data Science Engineers

Descriptive Metrics, Bayes' Theorem, Distributions, Hypothesis Testing, A/B Testing & Executive Translation

---

Part 1: Descriptive Statistics & Measures of Dispersion

1.1 Central Tendency & Distribution Skewness

Metric	Mathematical Definition	Resilience to Outliers	Best Used When
Mean ($\mu, \bar{x}$)	$\frac{1}{N}\sum_{i=1}^N x_i$	**Zero Resilience** (Pulled heavily by extreme values)	Data is symmetric and normally distributed without extreme outliers.
Median	Middle value of sorted data (50th percentile)	**High Resilience**	Data is skewed (e.g., Household income, Customer LTV, Website latency).
Mode	Most frequently occurring value	**Immune**	Categorical data or multi-modal distributions.

```
Positive (Right) Skew:      Mode < Median < Mean   (e.g., Salaries, Page response times)
Symmetric (Normal):         Mode = Median = Mean   (e.g., Human height, Measurement errors)
Negative (Left) Skew:       Mean < Median < Mode   (e.g., Age at retirement, Exam scores)
```

1.2 Measures of Dispersion & Bessel's Correction

- **Population Variance (σ^2):** $\sigma^2 = \frac{1}{N}\sum_{i=1}^N (x_i - \mu)^2$
- **Sample Variance (s^2):** $s^2 = \frac{1}{n-1}\sum_{i=1}^n (x_i - \bar{x})^2$

Why divide by $n-1$ (Bessel's Correction)?

Because the sample mean $\bar{x}$ is estimated from the exact same sample data, the deviations $(x_i - \bar{x})$ are systematically slightly smaller than the deviations from the true population mean μ. Dividing by n produces an underestimate (**biased estimator**). Dividing by $n-1$ removes the bias, making s^2 an **unbiased estimator** of σ^2.

- **Standard Deviation (σ, s):** Square root of variance. Expressed in the **exact same units** as the underlying data.
- **Interquartile Range (IQR):** $\text{IQR} = Q_3 - Q_1$ (75th percentile minus 25th percentile). Measures spread of the central 50% of data; immune to extreme tails.

Part 2: Probability Foundations & Bayesian Inference

2.1 Conditional Probability & Multiplication Rule

The conditional probability of event A occurring given that event B has already occurred is:
$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (\text{provided } P(B) > 0)$$

Two events A and B are **statistically independent** if and only if:
$$P(A \cap B) = P(A) \cdot P(B) \iff P(A|B) = P(A)$$

2.2 The Law of Total Probability & Bayes' Theorem

Given mutually exclusive and exhaustive partitions $B_1, B_2, \dots, B_k$:
$$P(A) = \sum_{i=1}^k P(A|B_i)P(B_i)$$

Bayes' Theorem updates prior beliefs in the presence of new evidence:
$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} = \frac{P(B|A) \cdot P(A)}{\sum_{i} P(B|A_i)P(A_i)}$$

- $P(A)$: **Prior Probability** (Belief before observing evidence B).
- $P(B|A)$: **Likelihood** (Probability of observing evidence B if hypothesis A is true).
- $P(B)$: **Marginal Evidence** (Total probability of observing evidence B).
- $P(A|B)$: **Posterior Probability** (Updated belief after observing evidence B).

2.3 The Classic Interview Paradox: The Base Rate Fallacy

Problem: A rare disease affects $0.1%$ (0.001) of the population. A lab test has a **$99%$ True Positive Rate (Sensitivity)** and a **$5%$ False Positive Rate**. A randomly selected patient tests positive. What is the probability they actually have the disease?

```
P(Disease) = 0.001       ===> P(Healthy) = 0.999
P(Pos | Disease) = 0.99  ===> P(Pos | Healthy) = 0.05

P(Pos) = (0.99 * 0.001) + (0.05 * 0.999)
       = 0.00099 + 0.04995 = 0.05094

P(Disease | Pos) = (0.99 * 0.001) / 0.05094 ≈ 0.0194 (Only 1.94%!)
```

Business Takeaway: Even with a 99% accurate test, an alarm is false over 98% of the time when the underlying base rate is extremely rare! This is foundational for Fraud Detection and Intrusion Alert systems.

Part 3: Common Probability Distributions

3.1 Discrete Distributions

Distribution	Parameters	Support (k)	PMF $P(X=k)$	Data Science Use Case
Bernoulli	p (success prob)	${0, 1}$	$p^k(1-p)^{1-k}$	Single user click or conversion (0 or

				1).
Binomial	n trials, p	${0, 1, \dots, n}$	$\binom{n}{k} p^k(1-p)^{n-k}$	Number of conversions out of n landing page visitors.
Poisson	λ (rate per interval)	${0, 1, 2, \dots}$	$\frac{\lambda^k e^{-\lambda}}{k!}$	Number of API requests per minute; server errors per hour.

Poisson Property: The mean and variance of a Poisson distribution are identical:
$$\mathbb{E}[X] = \text{Var}(X) = \lambda$$

3.2 Continuous Distributions

1. Normal (Gaussian) Distribution: $\mathcal{N}(\mu, \sigma^2)$

- Bell-shaped, symmetric about mean μ.
- **Empirical 68-95-99.7 Rule:**
$68.27%$ of observations fall within $\mu \pm 1\sigma$
$95.45%$ fall within $\mu \pm 2\sigma$
$99.73%$ fall within $\mu \pm 3\sigma$

- Standard Normal Distribution (Z): $\mu=0, \sigma=1$, transformed via $Z = \frac{x - \mu}{\sigma}$.

2. Student's t-Distribution:

- Symmetric bell curve with **heavier tails** than the normal distribution.
- Governed by **degrees of freedom** ($\nu = n-1$). As $n \to \infty$, the t-distribution converges to the Standard Normal $\mathcal{N}(0, 1)$.
- **Crucial Rule:** Used instead of the Z-distribution whenever sample size is small ($n < 30$) AND the true population standard deviation σ is unknown.

3. Exponential Distribution:

- Models the **time elapsed between events** in a Poisson process with rate λ.
- **Memoryless Property:** $P(X > s + t \mid X > s) = P(X > t)$. The probability of waiting an additional t minutes does not depend on how long you have already waited.

Part 4: The Central Limit Theorem (CLT) & Sampling

4.1 Statement of the Central Limit Theorem

Regardless of the underlying population distribution (whether uniform, skewed, bimodal, or exponential), the **sampling distribution of the sample mean ($\bar{X}$)** approaches a Normal Distribution as the sample size n

becomes large ($n \ge 30$):
$$\bar{X} \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$$

4.2 Standard Error of the Mean (SE)

$$\text{SE} = \frac{\sigma}{\sqrt{n}} \approx \frac{s}{\sqrt{n}}$$
The Standard Error measures the variability/dispersion of sample means around the true population mean. Notice that to cut the margin of error in half, **sample size must quadruple ($4\times$)** due to the square root in the denominator.

4.3 Confidence Intervals (CI)

A 95% Confidence Interval for the population mean:
$$\text{CI}_{95\%} = \bar{x} \pm t^* \cdot \frac{s}{\sqrt{n}}$$

- **Correct Interpretation:** *"If we repeated this identical sampling experiment 100 times and calculated confidence intervals, approximately 95 of those intervals would contain the true, fixed population mean μ."*
- **Incorrect (Interview Trap):** "There is a 95% probability that the true mean lies between A and B." (The population mean is a fixed constant, not a random variable; the *interval* is the random entity).

Part 5: Hypothesis Testing & A/B Testing Framework

```
                          REALITY: H0 is TRUE       REALITY: H0 is FALSE
DECISION: Reject H0       TYPE I ERROR (α)           CORRECT (Power: 1 - β)
                          (False Alarm)              (True Discovery)

DECISION: Fail to Reject  CORRECT                    TYPE II ERROR (β)
          H0              (Confidence: 1 - α)        (Missed Opportunity)
```

5.1 The 5-Step Hypothesis Testing Workflow

- **Formulate Hypotheses:**
Null Hypothesis (H_0): Status quo / no effect ($\mu_A = \mu_B$).
Alternative Hypothesis (H_1): Effect exists ($\mu_B > \mu_A$ or $\mu_A \neq \mu_B$).

- **Select Significance Level (α):** Typically $\alpha = 0.05$ (5% tolerance for Type I error).
- **Compute Test Statistic:**
Two-sample independent t-test: $t = \frac{\bar{x}_B - \bar{x}_A}{\sqrt{\frac{s_A^2}{n_A} + \frac{s_B^2}{n_B}}}$

- **Calculate p-value:** The probability of obtaining a test statistic at least as extreme as observed, assuming the null hypothesis is true.
- **Make Business Decision:** If $p \le \alpha$, reject H_0; if $p > \alpha$, fail to reject H_0.

5.2 Common Misconceptions About p-values

- **What a p-value IS:** $P(\text{Data as extreme as observed} \mid H_0 \text{ is true})$.
- **What a p-value IS NOT:**
  It is **NOT** the probability that the null hypothesis is true ($P(H_0 \mid \text{Data})$).
  It does **NOT** measure the magnitude or practical business importance of an effect. A tiny, meaningless 0.01% increase can be "statistically significant" ($p < 0.001$) if sample size $N = 10,000,000$.

Part 6: Correlation, Covariance & Causation

6.1 Covariance vs. Pearson Correlation

- **Covariance:** $\text{Cov}(X, Y) = \frac{1}{n-1}\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$.
  Measures directional relationship, but its value is unbounded and depends on scale/units.

- **Pearson Correlation Coefficient (r):**
  $$r = \frac{\text{Cov}(X, Y)}{s_X \cdot s_Y} \quad (-1 \le r \le +1)$$
  Standardized covariance. Measures the strength and direction of a strictly **linear** relationship.

6.2 Spearman Rank Correlation (ρ)

Computes Pearson's r over the **ranks** of the data rather than raw values:

- Measures **monotonic relationships** (whether variables increase together, regardless of linearity).
- Immune to extreme outliers.

6.3 Confounding & Simpson's Paradox

Simpson's Paradox: A statistical trend that appears in different groups disappears or reverses when the groups are aggregated.

Classic Example (UC Berkeley Admissions):

- Overall: Men had higher admission rates than women.
- Sub-department breakdown: In nearly every single department, women had equal or higher admission rates than men!
- **Confounding Variable:** Women applied disproportionately to competitive departments with very low acceptance rates, while men applied to less competitive departments with high overall acceptance.

Part 7: Required Deliverable: Executive Translation Brief

Context

A product team tested a "One-Click Checkout" button (B) against the standard multi-step checkout (A).

- Sample size: $n_A = 50,000, n_B = 50,000$
- Conversion rate: $CR_A = 3.20%$, $CR_B = 3.65%$
- Relative lift: $+14.06%$
- p-value: 0.00012
- $95%$ Confidence Interval of Lift: $[+7.1%, +21.0%]$

Executive Brief Document

```
# EXECUTIVE DECISION BRIEF: One-Click Checkout Experiment

**Date:** October 2026
**To:** Chief Product Officer & VP of E-Commerce
**From:** Senior Data Science Engineer
**Recommendation:** **FULL ROLLOUT (Ship Variant B to 100% of Traffic)**

---

### 1. Executive Summary
Over a two-week controlled experiment covering 100,000 customer sessions, introducing the "One-Click
Checkout" experience produced a **statistically confirmed +14% increase in completed purchases**.

Based on current annual GMV, this conversion improvement represents an estimated **$2.4M to $4.1M in
incremental annualized revenue**.

---

### 2. Plain-Language Experiment Findings

| Metric | Current Experience (Control) | New One-Click Checkout | Plain-English Business Impact |
| :--- | :--- | :--- | :--- |
| **Completed Orders per 1,000 Visitors** | 32 orders | **36.5 orders** | **+4.5 extra paying
customers** per 1,000 visitors |
| **Statistical Certainty** | N/A | **99.98% confidence** | The odds of this result being random noise
are under 1 in 8,000. |
| **Projected Annual Impact** | Baseline ($0) | **+$3.2M / year** | 95% confident the permanent gain is
at least +$1.8M/yr. |

---

### 3. Risk & Guardrail Assessment
- **Order Cancellation Rate:** Monitored daily as a guardrail metric. Cancellation rate remained flat
($1.1\%$ vs $1.2\%$), confirming that the frictionless checkout did not trigger accidental orders.
- **Mobile vs. Desktop:** Gains were strongest on mobile (+19% lift), successfully solving our checkout
drop-off bottleneck on mobile devices.

### 4. Recommended Action Plan
1. Ramp Variant B from 50% to 100% across all traffic starting Monday at 02:00 UTC.
2. Retire legacy multi-step checkout code to eliminate maintenance overhead.
```

Part 8: Structured Interview Question Bank

Q1: What is the difference between Type I and Type II errors? Which is worse in fraud detection?

Answer:

- **Type I Error (α, False Positive):** Rejecting the null hypothesis when it is true. (e.g., Flagging a legitimate customer transaction as fraud).
- **Type II Error (β, False Negative):** Failing to reject the null hypothesis when an effect exists. (e.g., Letting an actual fraudulent transaction slip through unnoticed).
- In high-value fraud detection, a **Type II error is typically much worse** because an undetected fraudster causes direct financial loss, chargebacks, and legal liability, whereas a Type I error merely causes temporary user friction (e.g., requiring an SMS OTP).

Q2: What happens if an A/B test is evaluated early and stopped as soon as $p < 0.05$ ("Peeking Problem")?

Answer:
Continuously checking results and stopping an A/B test the moment $p < 0.05$ causes severe **α-inflation (False Positive inflation)**. Because random fluctuations will inevitably cross the 0.05 threshold by chance at some point during the experiment, the true false positive rate spikes from $5%$ to $30-40%$. To prevent this, teams must either use fixed-horizon sample sizes or implement **Sequential Testing** (e.g., Wald's Sequential Probability Ratio Test or alpha-spending functions).

Q3: Explain why Pearson correlation cannot detect a perfect parabolic relationship ($Y = X^2$).

Answer:
Pearson correlation strictly measures the strength and direction of a **linear** association. For $Y = X^2$ centered around 0 (e.g., $X \in [-3, -2, -1, 0, 1, 2, 3]$), positive deviations on the right cancel out negative deviations on the left in the covariance formula, resulting in $\text{Cov}(X, Y) = 0$ and $r = 0$. Spearman rank correlation also fails here because the relationship is non-monotonic.

Part 9: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  STATISTICS & PROBABILITY CHEAT SHEET            │
├──────────────────────────────────────────────────────────────────┤
│ 1. CENTRAL TENDENCY:                                             │
│    - Right Skew: Mean > Median > Mode (Outliers pull mean right).│
│    - Left Skew : Mean < Median < Mode.                           │
│    - Bessel's Correction: Divide by n-1 for unbiased sample variance.│
├──────────────────────────────────────────────────────────────────┤
│ 2. PROBABILITY & BAYES:                                          │
│    - P(A|B) = [P(B|A) * P(A)] / P(B)                             │
│    - False Positive Paradox: Low base rate means most alerts are false.│
├──────────────────────────────────────────────────────────────────┤
│ 3. DISTRIBUTIONS:                                                │
│    - Binomial: n independent trials with prob p.                 │
│    - Poisson: Events in fixed interval (Mean = Variance = λ).    │
│    - Normal: 68-95-99.7 rule. CLT guarantees sample mean is normal.  │
│    - t-distribution: Small sample (n < 30) or unknown σ. Heavier tails.│
└──────────────────────────────────────────────────────────────────┘
```

```
| 4. HYPOTHESIS TESTING:                                                    |
|    - Type I Error (α): False Positive (Reject true H0).                   |
|    - Type II Error (β): False Negative (Fail to reject false H0).         |
|    - Power (1 - β): Probability of detecting a real effect.               |
|    - p-value: P(observed data | H0 is true). NOT P(H0 is true)!           |
|---------------------------------------------------------------------------|
| 5. CORRELATION:                                                           |
|    - Pearson r: Linear (-1 to +1). Sensitive to outliers.                 |
|    - Spearman ρ: Monotonic. Uses ranks. Robust to outliers.               |
|    - Simpson's Paradox: Aggregate trend reverses when subgrouped.         |
└---------------------------------------------------------------------------┘
```

Part 10: 100-Mark Self-Assessment Examination

Section A: Conceptual & Formulas (25 Marks — 5 marks each)

Q1: What is the relationship between the Mean and the Median in a positively (right) skewed distribution?

(a) Mean = Median

(b) Mean < Median

(c) Mean > Median

(d) Variance = Mean

Q2: If the sample size n is increased from 100 to 400, what happens to the Standard Error of the Mean?

(a) Doubled ($2\times$)

(b) Halved ($\frac{1}{2}\times$)

(c) Reduced by a factor of 4

(d) Remains unchanged

Q3: What is the variance of a Poisson distribution with parameter $\lambda = 9$?

(a) 3

(b) 9

(c) 81

(d) 18

Q4: A hypothesis test yields a p-value of 0.03. What does this indicate at a significance level of $\alpha = 0.05$?

(a) There is a 3% chance the null hypothesis is true.

(b) Reject the null hypothesis; the observed result is statistically significant.

(c) Fail to reject the null hypothesis.

(d) Accept the alternative hypothesis with 97% certainty.

Q5: In a medical screening scenario with high sensitivity (99%) and a very rare disease prevalence (1 in 10,000), why is a positive test result usually a false alarm?

(a) The test sensitivity is too low.

(b) The Base Rate Fallacy: The absolute number of false positives from the large healthy population dwarfs the true positives.

(c) Type II error inflation.

(d) Simpson's Paradox.

Section B: Calculation & Probability Problems (25 Marks)

Q6 (10 Marks): Given the dataset: `[4, 8, 12, 16, 20]`:

- Calculate the sample mean $\bar{x}$.
- Calculate the sample standard deviation s (using Bessel's $n-1$ correction).

Q7 (15 Marks): In an enterprise data center, the number of hardware server failures follows a Poisson distribution with an average of $\lambda = 2$ failures per day.

- Write the formula for $P(X = k)$.
- Calculate the exact probability that on a given day, exactly 0 server failures occur ($e^{-2} \approx 0.1353$).
- Calculate the probability of experiencing at least 1 failure.

Section C: A/B Testing & Inferential Scenarios (30 Marks)

Q8 (15 Marks): An e-commerce analyst runs an A/B test with 20 different button color variations simultaneously against a control, without applying any multiple testing corrections. If each test uses $\alpha = 0.05$, calculate the family-wise error rate (the probability of getting at least one false positive result by pure chance). How does the **Bonferroni correction** resolve this?

Q9 (15 Marks): Explain how Simpson's Paradox could lead a data science team to falsely conclude that a new recommendation algorithm performed worse than the old algorithm, when in fact it was superior in every customer demographic.

Section D: Real-World Business Translation (20 Marks)

Q10 (20 Marks): Your machine learning model predicts loan default risk for a commercial bank. The model has an AUC-ROC of 0.88, a Precision of $35%$, and a Recall of $82%$.
The VP of Risk says: *"Why is precision only 35%? That means 65% of the loans flagged for denial are actually safe!*

This model is useless."
Write a persuasive, technically sound response explaining why a 35% precision is highly valuable given the cost asymmetry of loan defaults vs. lost interest.

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (c)** Mean > Median (The extreme high values pull the mean upward).
- **Q2: (b)** Halved. $\text{SE} = \frac{s}{\sqrt{400}} = \frac{s}{20}$, which is half of $\frac{s}{10}$.
- **Q3: (b)** 9. In a Poisson distribution, $\text{Var}(X) = \lambda = 9$.
- **Q4: (b)** Reject H_0 because $p = 0.03 \le \alpha = 0.05$.
- **Q5: (b)** The Base Rate Fallacy.

Section B Solutions

Q6 Solution:

- Mean: $\bar{x} = \frac{4 + 8 + 12 + 16 + 20}{5} = \frac{60}{5} = 12$.
- Deviations squared: $(4-12)^2 = 64, (8-12)^2 = 16, (12-12)^2 = 0, (16-12)^2 = 16, (20-12)^2 = 64$.

Sum of squared deviations $= 64 + 16 + 0 + 16 + 64 = 160$.
Sample Variance: $s^2 = \frac{160}{5-1} = \frac{160}{4} = 40$.
Sample Standard Deviation: $s = \sqrt{40} \approx 6.32$.

Q7 Solution:

- $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$.
- For $k = 0$: $P(X = 0) = \frac{2^0 e^{-2}}{0!} = \frac{1 \cdot 0.1353}{1} = 0.1353$ (13.53%).
- At least 1 failure: $P(X \ge 1) = 1 - P(X = 0) = 1 - 0.1353 = 0.8647$ (86.47%).

Section C Solutions

Q8 Solution:

- Probability of no false positive in 1 test: $1 - 0.05 = 0.95$.
- Probability of no false positive across 20 independent tests: $(0.95)^{20} \approx 0.3585$.
- **Family-Wise Error Rate (FWER):** $1 - 0.3585 = 0.6415$ (**64.15% chance of at least one false discovery!**).
- **Bonferroni Correction:** Divide the target alpha by the number of comparisons: $\alpha_{\text{adjusted}} = \frac{0.05}{20} = 0.0025$. Each individual variation must reach $p \le 0.0025$ to be declared significant.

Q9 Solution:
Simpson's Paradox occurs when a confounding variable divides the dataset into unequal segments. If the new algorithm was rolled out during a seasonal surge predominantly to new, first-time users (who have an inherently lower baseline conversion rate of 1%) while the old algorithm served loyal power users (baseline 10%), the aggregated conversion rate of the new algorithm appears lower, even if it boosted first-time users from 1% to 2% and loyal users from 10% to 12%.

Section D Solutions

Q10 Solution:

"A commercial loan default results in an average catastrophic loss of $50,000 in unpaid principal, whereas approving a good loan yields $3,000 in net interest margin. The cost of a False Negative ($50,000) is nearly $17\times$ higher than the cost of a False Positive ($3,000). At 82% recall, our model successfully captures over 8 out of 10 catastrophic defaults. A 35% precision means that for every 10 flagged loans, 3.5 are true defaults ($175,000 in capital saved) at the cost of requiring manual credit reviews on 6.5 applications. The net expected economic value saved far exceeds the cost of credit underwriting friction."

Day 10: REST APIs, JSON Ingestion & Production Web Services

HTTP Architecture, OAuth2, Pagination, Resilience, Backoff & Database Persistence

---

Part 1: REST Architecture & HTTP Protocol Internals

1.1 The Core Constraints of REST (Representational State Transfer)

REST is an architectural style designed around six core constraints:

- **Statelessness:** Every HTTP request from client to server must contain all information required to understand and process the request. No session context is stored on the server between requests.
- **Client-Server Separation:** User interface concerns are decoupled from data storage concerns.
- **Uniform Interface:** Resources are identified by standard URIs (nouns, not verbs: `/api/v1/orders`, not `/api/v1/getOrders`).
- **Cacheability:** Responses must explicitly declare whether they are cacheable via `Cache-Control` headers.

1.2 HTTP Methods, Safety & Idempotency

Method	Semantic Purpose	Safe?	Idempotent?	Typical Request Body?
GET	Retrieve resource representation	**YES**	**YES**	No
POST	Create a new subordinate resource	**NO**	**NO**	Yes (JSON payload)
PUT	Completely replace or create resource at URI	**NO**	**YES**	Yes (Full resource)
PATCH	Partially modify an existing resource	**NO**	**YES** (by standard)	Yes (Delta fields)
DELETE	Remove a resource	**NO**	**YES**	Optional

- **Safe:** Does not modify the state of the server (read-only).
- **Idempotent:** Making the identical request N times ($N \ge 1$) produces the **exact same server state** as making it once.
  Example: `DELETE /users/42` executed 5 times results in the user being gone; the first call returns `204 No Content` or `200 OK`, subsequent calls return `404 Not Found`, but server state is unchanged. In contrast, `POST /payments` executed 5 times charges the customer 5 times (Non-Idempotent!).

1.3 HTTP Status Codes Masterclass

`1xx Informational | 2xx Success | 3xx Redirection | 4xx Client Error | 5xx Server Error`

Status Code	Name	Practical Meaning in Data Engineering
200 OK	Standard Success	Payload returned successfully.
201 Created	Resource Created	Response to `POST`; contains `Location` header.
204 No Content	Empty Success	Action succeeded (e.g., `DELETE`); zero response body.
304 Not Modified	Cached	Client can use local cache based on `ETag` / `If-None-Match`.
400 Bad Request	Malformed Request	Client syntax error or schema validation failure.
401 Unauthorized	Authentication Missing	Missing or invalid API key/token (Identity unproven).
403 Forbidden	Authorization Refused	Identity known, but client lacks permission to view resource.
404 Not Found	Missing Resource	Endpoint or entity ID does not exist.
429 Too Many Requests	Rate Limited	Exceeded API quota. Inspect `Retry-After` header!
500 Internal Error	Server Crash	Unhandled exception inside the API server.
502 Bad Gateway	Gateway Failure	Reverse proxy (Nginx/Cloudflare) received invalid upstream response.
503 Unavailable	Overloaded / Maintenance	Temporary outage. Safe to retry after delay.
504 Gateway Timeout	Gateway Timed Out	Upstream database or service failed to respond in time.

Part 2: Authentication & Authorization Mechanics

2.1 API Keys & Bearer Tokens

- **API Key:** A static token passed via request headers:
`Authorization: Api-Key abc123xyz` or `X-API-KEY: abc123xyz`.
- **Bearer Token (JWT):** A cryptographically signed token:
`Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`
A JSON Web Token (JWT) consists of three Base64URL-encoded parts: `Header.Payload.Signature`.

Enables **stateless authorization**: the receiving server verifies the signature using a public key without querying a session database.

2.2 OAuth 2.0: Client Credentials Grant (Machine-to-Machine)

The standard for automated data pipelines and backend ETL ingestion:

```
[ Data Pipeline (Client) ]                [ Authorization Server ]                [ Resource Server (API) ]
           |                                          |                                      |
           |-- 1. POST /token (client_id + secret) ->|                                      |
           |<- 2. Access Token (expires in 3600s) ---|                                      |
           |                                          |                                      |
           |------------------- 3. GET /data with Bearer Token ------------------------------>|
           |<------------------ 4. JSON Dataset ------------------------------------------------|
```

Part 3: Advanced Python requests Architecture

3.1 Connection Pooling via requests.Session()

In standard `requests.get()`, a new TCP socket and TLS handshake are negotiated on **every single request**, adding 50–200ms of network latency per call.

A `requests.Session()` maintains an internal **urllib3 ConnectionPool**, reusing open TCP connections across thousands of calls.

```
import requests

# ANTI-PATTERN: Opens/closes TCP socket 100 times
# for url in urls: requests.get(url)

# PRODUCTION PATTERN: Reuses TCP sockets across all requests
with requests.Session() as session:
    session.headers.update({"Accept": "application/json", "User-Agent": "DataPlatform/1.0"})
    response = session.get("https://api.example.com/v1/data")
```

3.2 Timeouts: Connect Timeout vs. Read Timeout

Never call `requests.get()` without an explicit timeout! Without a timeout, a hanging server will cause your Python process to block indefinitely, freezing your entire ETL orchestrator (Airflow, Celery).

```
# Pass a 2-tuple: (Connect Timeout, Read Timeout)
# - Connect Timeout: Max seconds to establish TCP connection (typically 3.05s).
# - Read Timeout: Max seconds to wait for server to transmit the first byte of data.
response = session.get("https://api.example.com/v1/records", timeout=(3.05, 30.0))
```

Part 4: Production Resilience: Pagination, Rate Limits & Exponential Backoff

4.1 Pagination Archetypes

- **Offset-Limit Pagination:** `GET /users?offset=100&limit=50`
  Limitation: Degrades at high offsets ($O(N)$ database scan); prone to skipped/duplicate records if rows are inserted during pagination.

- **Page-Number Pagination:** `GET /users?page=3&per_page=50`
- **Cursor-Based / Keyset Pagination:** `GET /users?cursor=dXNlcl9pZDoxMDQ1&limit=50`
  Industry Standard: Cursor points to a unique immutable row index (e.g., base64 of `last_id`). $O(1)$ database seek; immune to pagination drift.

4.2 Handling HTTP 429 Rate Limits & Exponential Backoff with Jitter

When receiving HTTP 429 or transient 503 errors, standard linear retries cause the **Thundering Herd Problem** (multiple pipeline workers retry simultaneously, crashing the recovering API).

The solution is **Exponential Backoff with Full Jitter**:
$$\text{Delay} = \text{random}(0, \min(\text{MaxDelay}, \text{BaseDelay} \times 2^{\text{attempt}}))$$

```
import time
import random
import requests

def request_with_backoff(session, url, max_retries=5, base_delay=1.0, max_delay=32.0):
    for attempt in range(max_retries):
        try:
            response = session.get(url, timeout=(3.05, 15.0))

            # If rate limited:
            if response.status_code == 429:
                retry_after = response.headers.get("Retry-After")
                if retry_after:
                    sleep_time = float(retry_after)
                else:
                    sleep_time = random.uniform(0, min(max_delay, base_delay * (2 ** attempt)))
                print(f"Rate limited (429). Retrying in {sleep_time:.2f}s...")
                time.sleep(sleep_time)
                continue

            # If server error (5xx), retry:
            if response.status_code in (500, 502, 503, 504):
                sleep_time = random.uniform(0, min(max_delay, base_delay * (2 ** attempt)))
                print(f"Server error ({response.status_code}). Retrying in {sleep_time:.2f}s...")
                time.sleep(sleep_time)
                continue

            response.raise_for_status()
```

```
            return response.json()

        except (requests.ConnectionError, requests.Timeout) as e:
            if attempt == max_retries - 1:
                raise e
            sleep_time = random.uniform(0, min(max_delay, base_delay * (2 ** attempt)))
            print(f"Network failure: {e}. Retrying in {sleep_time:.2f}s...")
            time.sleep(sleep_time)

    raise RuntimeError(f"Exceeded max retries for URL: {url}")
```

Part 5: JSON Normalization & Flat Staging

Nested API payloads must be flattened into tabular rows before loading into relational tables:

```
import pandas as pd

api_payload = [
    {
        "id": 101,
        "name": "Acme Corp",
        "billing": {"city": "New York", "postal": "10001"},
        "contacts": [{"type": "email", "val": "admin@acme.com"}]
    }
]

# Flattens nested dictionaries with separator
df_flat = pd.json_normalize(
    api_payload,
    record_path=["contacts"],
    meta=["id", "name", ["billing", "city"], ["billing", "postal"]],
    sep="_"
)
# Resulting columns: type, val, id, name, billing_city, billing_postal
```

Part 6: Required Deliverable: Complete Production Ingestion & Database Loader

Problem Statement

Write a self-contained, enterprise-grade Python module that:

- Authenticates against a paginated REST API endpoint.
- Iterates across pages using keyset/cursor pagination.
- Implements resilient retry logic with exponential backoff and jitter.
- Normalizes and validates incoming records.
- Performs an idempotent bulk upsert (`INSERT ... ON CONFLICT DO UPDATE`) into a local SQLite/PostgreSQL database.

Production Solution Script

```
import sqlite3
import time
```

```
import random
import requests
from typing import Generator, Any

class ResilientAPIClient:
    def __init__(self, base_url: str, api_token: str):
        self.base_url = base_url
        self.session = requests.Session()
        self.session.headers.update({
            "Authorization": f"Bearer {api_token}",
            "Accept": "application/json",
            "User-Agent": "EnterpriseDataIngestion/2.0"
        })

    def fetch_page_with_retry(self, endpoint: str, params: dict, max_retries: int = 4) -> dict:
        url = f"{self.base_url}{endpoint}"
        for attempt in range(max_retries):
            try:
                response = self.session.get(url, params=params, timeout=(3.05, 10.0))

                if response.status_code == 429:
                    delay = float(response.headers.get("Retry-After", 2 ** attempt + random.random()))
                    time.sleep(delay)
                    continue

                if response.status_code >= 500:
                    delay = (2 ** attempt) + random.uniform(0.1, 1.0)
                    time.sleep(delay)
                    continue

                response.raise_for_status()
                return response.json()

            except (requests.ConnectionError, requests.Timeout) as err:
                if attempt == max_retries - 1:
                    raise err
                time.sleep((2 ** attempt) + random.uniform(0.1, 1.0))

        raise RuntimeError("Failed to fetch API page after maximum retries.")

    def stream_all_records(self, endpoint: str) -> Generator[dict, None, None]:
        """Paginates through API records lazily using cursor pagination."""
        cursor = None
        has_more = True

        while has_more:
            params = {"limit": 50}
            if cursor:
                params["cursor"] = cursor

            payload = self.fetch_page_with_retry(endpoint, params)
            data_records = payload.get("data", [])
            for record in data_records:
                yield record

            cursor = payload.get("next_cursor")
            has_more = payload.get("has_more", False) and (cursor is not None)


class DatabaseStagingPipeline:
    def __init__(self, db_path: str = "ingestion_warehouse.db"):
        self.conn = sqlite3.connect(db_path)
        self._init_schema()
```

```
    def _init_schema(self):
        with self.conn:
            self.conn.execute("""
                CREATE TABLE IF NOT EXISTS api_orders (
                    order_id TEXT PRIMARY KEY,
                    customer_id TEXT NOT NULL,
                    order_status TEXT NOT NULL,
                    amount REAL NOT NULL,
                    created_at TEXT NOT NULL,
                    ingested_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                );
            """)

    def upsert_batch(self, records: list[dict]):
        """Idempotent upsert: Updates existing records if order_id already exists."""
        sql = """
            INSERT INTO api_orders (order_id, customer_id, order_status, amount, created_at)
            VALUES (?, ?, ?, ?, ?)
            ON CONFLICT(order_id) DO UPDATE SET
                customer_id = excluded.customer_id,
                order_status = excluded.order_status,
                amount = excluded.amount,
                ingested_at = CURRENT_TIMESTAMP;
        """
        formatted_rows = [
            (
                r["id"],
                r["customer_id"],
                r["status"],
                float(r["amount"]),
                r["created_at"]
            )
            for r in records
        ]
        with self.conn:
            self.conn.executemany(sql, formatted_rows)

    def close(self):
        self.conn.close()


def run_pipeline(api_client: ResilientAPIClient, pipeline: DatabaseStagingPipeline):
    batch = []
    batch_size = 100
    total_ingested = 0

    print("Initiating resilient API streaming ingestion...")
    for record in api_client.stream_all_records("/v1/transactions"):
        batch.append(record)
        if len(batch) >= batch_size:
            pipeline.upsert_batch(batch)
            total_ingested += len(batch)
            print(f"Upserted {total_ingested} records into database.")
            batch = []

    if batch:
        pipeline.upsert_batch(batch)
        total_ingested += len(batch)

    print(f"Pipeline complete. Total records ingested: {total_ingested}")
```

Part 7: Structured Interview Question Bank

Q1: What is the difference between PUT and PATCH in RESTful APIs?

Answer:

- `PUT` is designed for complete replacement of a resource. The client must supply the entire resource representation; any attribute omitted from the request body is typically cleared or reset to default values.
- `PATCH` is designed for partial modification. The client submits only the specific delta fields that need modification, preserving all other existing attributes unchanged. Both methods should be designed to be idempotent.

Q2: Why is the HTTP 401 status code a misnomer, and how does it differ from 403?

Answer:
`401 Unauthorized` is fundamentally an **Authentication** failure, not an authorization failure. It signals that the user's identity has not been verified (missing, malformed, or expired API key/token). In contrast, `403 Forbidden` is an **Authorization** failure: the client's identity is authenticated and verified, but the server refuses to grant access because the identity lacks the required permissions/roles.

Q3: What is "Keyset Pagination" (Cursor-based) and why is it preferred over "Offset-Limit Pagination" for big data APIs?

Answer:
Offset-limit pagination (`OFFSET 500000 LIMIT 50`) forces the underlying database storage engine to scan and discard 500,000 index entries before returning 50 rows ($O(N)$ execution time). Furthermore, if rows are inserted or deleted while the client is paginating, records can be skipped or duplicated across page boundaries. Keyset pagination passes the last observed unique key (`WHERE id > :last_id ORDER BY id LIMIT 50`), allowing the database to execute an instant $O(\log N)$ B-tree index seek that is immune to insertion drift.

Part 8: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  REST APIS & WEB SERVICES CHEAT SHEET            │
├──────────────────────────────────────────────────────────────────┤
│ 1. HTTP METHODS & IDEMPOTENCY:                                   │
│    - Safe      : GET (Read-only; zero server mutation).          │
│    - Idempotent: GET, PUT, PATCH, DELETE (N calls = 1 call effect).│
│    - Non-Idempotent: POST (Creates new resources; duplicates state).│
├──────────────────────────────────────────────────────────────────┤
│ 2. STATUS CODES:                                                 │
│    - 200 OK, 201 Created, 204 No Content.                        │
│    - 401 Unauthorized (Auth missing), 403 Forbidden (Auth rejected).│
│    - 429 Too Many Requests (Rate limited -> Check Retry-After).  │
│    - 500 Server Error, 502 Bad Gateway, 503 Service Unavailable. │
├──────────────────────────────────────────────────────────────────┤
│ 3. PRODUCTION REQUESTS:                                          │
│    - requests.Session(): Reuses TCP sockets via connection pooling.│
│    - Timeouts: Always pass (connect_timeout, read_timeout) tuple!│
├──────────────────────────────────────────────────────────────────┤
│ 4. RESILIENCE:                                                   │
│    - Exponential Backoff with Jitter: Prevents thundering herd.  │
│    - Keyset/Cursor Pagination: O(1) DB seek; avoids pagination drift.│
```

```
    - Idempotent Loader: INSERT ... ON CONFLICT DO UPDATE.
```

Part 9: 100-Mark Self-Assessment Examination

Section A: Conceptual Foundations (20 Marks — 4 marks each)

Q1: Which of the following HTTP methods is NOT idempotent by definition?

(a) GET

(b) PUT

(c) POST

(d) DELETE

Q2: When an API responds with HTTP status code `429 Too Many Requests`, which response header specifies how many seconds the client should wait before retrying?

(a) `Keep-Alive`

(b) `Retry-After`

(c) `RateLimit-Reset`

(d) `X-Throttle-Delay`

Q3: What is the consequence of omitting a timeout argument in `requests.get("https://...")`?

(a) Defaults to a 30-second timeout.

(b) Defaults to a 60-second timeout.

(c) The request will block indefinitely if the remote server fails to respond.

(d) Throws a `TimeoutConfigurationError`.

Q4: What are the three dot-separated components of a JSON Web Token (JWT)?

(a) Nonce, Key, Ciphertext

(b) Header, Payload, Signature

(c) Issuer, Subject, Expiration

(d) ClientID, Secret, Scope

Q5: Why does `requests.Session()` execute faster than individual `requests.get()` calls when making 1,000 sequential requests to the same domain?

(a) It compresses all payloads using gzip.

(b) It reuses existing TCP and TLS handshakes via internal connection pooling.

(c) It caches all GET responses on local disk.

(d) It executes requests concurrently in multiple threads.

Section B: Protocol & Architecture Analysis (25 Marks)

Q6 (10 Marks): Explain why executing a retry loop without "Jitter" (randomness) can crash a recovering web service (The Thundering Herd Problem).

Q7 (15 Marks): Compare Offset-Limit pagination with Cursor-based pagination across two dimensions:

- Performance as page depth increases ($N \to 1,000,000$).
- Data consistency when real-time records are inserted while paginating.

Section C: Implementation Challenges (35 Marks)

Q8 (15 Marks): Write a Python function `get_auth_token(token_url: str, client_id: str, client_secret: str) -> str` that performs an OAuth 2.0 Client Credentials `POST` request and returns the `access_token` string. Raise an informative error if the request fails.

Q9 (20 Marks): Given a deeply nested JSON response:

```
{
  "company": "TechGlobal",
  "departments": [
    {"name": "Engineering", "budget": 500000},
    {"name": "Data", "budget": 350000}
  ]
}
```

Write the Pandas code using `pd.json_normalize()` to flatten this structure into a DataFrame with columns `['company', 'dept_name', 'budget']`.

Section D: Real-World Ingestion Architecture (20 Marks)

Q10 (20 Marks): You are designing a batch ingestion pipeline that pulls 5 million daily financial transactions from an external vendor API. The vendor enforces a hard rate limit of 100 requests per second.

- Design the architecture (Workers, Queue, Database) to maximize throughput without exceeding the 100 req/sec quota. (10 marks)
- How do you guarantee **Idempotent Ingestion** so that running the pipeline twice for the same date does not produce duplicate financial records in the warehouse? (10 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (c)** POST.
- **Q2: (b)** `Retry-After`.
- **Q3: (c)** Blocks indefinitely until the socket times out at the OS level (which may take hours or never happen).
- **Q4: (b)** Header, Payload, Signature.
- **Q5: (b)** TCP and TLS connection reuse.

Section B Solutions

Q6 Solution:
When an API recovers from an outage, thousands of client workers using fixed exponential delays (e.g., exactly 2s, 4s, 8s) retry at the exact same synchronized clock interval. This produces massive, periodic traffic spikes ("Thundering Herd") that immediately overload and crash the recovering service again. Adding **Full Jitter** randomizes delays uniformly ($[0, \text{backoff}]$), smoothing traffic into a continuous, manageable stream.

Q7 Solution:

- **Performance:** Offset pagination degrades to $O(N)$ because the database must scan and discard all preceding offset rows. Cursor pagination uses an indexed range filter (`WHERE id > :cursor`), maintaining $O(\log N)$ performance regardless of depth.
- **Consistency:** If a new row is inserted on page 1 while the client is on page 3, offset pagination shifts all rows down, causing the client to see a duplicate record on page 4. Cursor pagination anchors directly to immutable entity keys, eliminating duplicate or skipped rows.

Section C Solutions

Q8 Solution:

```
import requests

def get_auth_token(token_url: str, client_id: str, client_secret: str) -> str:
    data = {
        "grant_type": "client_credentials",
        "client_id": client_id,
        "client_secret": client_secret
    }
    response = requests.post(token_url, data=data, timeout=(3.05, 10.0))
    if not response.ok:
        raise RuntimeError(f"Auth failed [{response.status_code}]: {response.text}")
    return response.json()["access_token"]
```

Q9 Solution:

```
import pandas as pd

raw_data = {
  "company": "TechGlobal",
  "departments": [
    {"name": "Engineering", "budget": 500000},
    {"name": "Data", "budget": 350000}
  ]
}

df = pd.json_normalize(
    raw_data,
```

```
    record_path=["departments"],
    meta=["company"],
    record_prefix="dept_"
)
# Resulting columns: dept_name, dept_budget, company
```

Section D Solutions

Q10 Solution:

- **Architecture:** Use a token-bucket rate limiter (e.g., Redis-backed Leaky Bucket) shared across Celery/Airflow worker pools. Workers poll tasks from an ingestion queue and request a token from Redis before dispatching HTTP calls. If tokens are depleted, workers sleep, strictly capping cluster throughput to 95–98 requests/sec to stay safely under the 100 req/sec limit.
- **Idempotency Guarantee:**
  Enforce a natural or composite unique business key (e.g., `vendor_transaction_id` or `(account_id, txn_timestamp, amount)`) at the database schema level (`UNIQUE` constraint).
  Ingest data into a staging table and use `INSERT ... ON CONFLICT (vendor_transaction_id) DO UPDATE` or atomic staging swap (`MERGE INTO target USING staging`). Re-running the pipeline seamlessly updates existing records without creating duplicates.

Day 11: ETL/ELT Architectures & Production Data Pipelines

Batch vs. Streaming, Idempotency, Incremental Loading, Observability & Azure Data Factory (ADF)

Part 1: The Architectural Paradigm Shift: ETL vs. ELT

```
TRADITIONAL ETL (Extract -> Transform -> Load)
[ Source Systems ] ──> [ External ETL Server ] ──> [ Data Warehouse ]
                         (Informatica / SSIS)        (Pre-transformed tables)
                       * Transform happens BEFORE load *
                       * Heavy CPU load on staging server *

MODERN CLOUD ELT (Extract -> Load -> Transform)
[ Source Systems ] ──> [ Cloud Raw Storage ] ──> [ Cloud Data Warehouse ]
                       (S3 / ADLS Gen2 Bronze)    (Snowflake / BigQuery / Fabric)
                                                  * Raw data loaded first *
                                                  * SQL/dbt transforms inside warehouse *
```

1.1 Architectural Comparison Matrix

Dimension	Traditional ETL	Modern Cloud ELT
Transform Location	Dedicated intermediary server/memory buffer.	Inside the target distributed cloud warehouse.
Data Fidelity	Raw data is discarded; only transformed aggregated data lands.	**100% Raw data is preserved permanently** (Bronze/Data Lake).
Scalability	Bottlenecked by the hardware limits of the ETL engine.	Elastic: Leverages decoupled cloud compute (Snowflake, BigQuery).
Schema Flexibility	Schema-on-Write: Changes require modifying ETL pipeline code.	Schema-on-Read: Raw data lands as JSON/Parquet; transformed on demand.
Compute Cost	High specialized licensing (Informatica, Ab Initio).	Low storage cost; pay-per-second warehouse compute.
Ideal Workload	On-premise systems with strict compliance prohibiting cloud raw storage.	Modern enterprise analytics, BI, and machine learning platforms.

Part 2: Processing Paradigms: Batch vs. Streaming

2.1 Batch Processing

- **Mechanics:** Processes bounded chunks of historical data at scheduled periodic intervals (e.g., hourly, daily at 02:00 UTC).

- **Latency:** Minutes to hours.
- **Throughput:** Maximum efficiency. Optimizers can sort, compress, and write millions of rows in bulk columnar blocks.
- **Technologies:** Apache Spark, Azure Data Factory, dbt, SQL Server Integration Services (SSIS).

2.2 Streaming Processing

- **Mechanics:** Processes unbounded streams of events one-by-one or in micro-batches as they occur.
- **Latency:** Sub-second to seconds.
- **Throughput:** Lower per-event efficiency; high real-time responsiveness.
- **Technologies:** Apache Kafka, Azure Event Hubs, Apache Flink, Spark Structured Streaming.

2.3 System Architectures: Lambda vs. Kappa

Lambda Architecture (Dual-Path)

Combines a **Batch Layer** (for accurate, comprehensive historical data processing) with a **Speed Layer** (streaming low-latency views for recent events), unified by a **Serving Layer**.

- *Weakness:* Massive code duplication (must maintain logic twice: once in Spark/SQL for batch, once in Flink/Kafka for streaming).

Kappa Architecture (Single Streaming Path)

Replaces the dual paths with a **single stream-processing engine**. All data is treated as an immutable log stream. Historical reprocessing is achieved by rewinding the stream offset from the beginning of the log.

Part 3: Critical Pipeline Engineering Principles

3.1 Idempotency & Determinism

A pipeline is **idempotent** if running it multiple times with the same input data produces the exact same final state without creating duplicate records or side effects.

How to Guarantee Idempotency:

- **Atomic Partition Overwrites:**
```
-- Idempotent daily batch load: Overwrite partition instead of blind append
INSERT OVERWRITE TABLE daily_sales PARTITION (sale_date = '2024-03-15')
SELECT * FROM staging_sales WHERE sale_date = '2024-03-15';
```

- **Merge / Upsert on Business Keys:**
```
MERGE INTO dim_customer target
USING staging_customer source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN UPDATE SET target.email = source.email, target.updated_at = source.updated_at
WHEN NOT MATCHED THEN INSERT (customer_id, email, updated_at) VALUES (source.customer_id, source.email, source.updated_at);
```

3.2 Incremental Loading Strategies

```
Method 1: High-Watermark (Timestamp Polling)
Source Table: [WHERE updated_at > :last_pipeline_run_timestamp]
* Requires reliable updated_at column on source *
* Misses hard DELETE operations *

Method 2: Change Data Capture (CDC)
Source Database WAL / Binlog ──> Debezium / Kafka Connect ──> Target Warehouse
* Zero source query overhead *
* Captures INSERTS, UPDATES, and DELETES perfectly *
```

3.3 Handling Schema Drift & The Dead Letter Queue (DLQ)

When upstream sources add, drop, or alter column data types unexpectedly, production pipelines must not crash silently.

- **Quarantine Pattern:** Malformed rows that fail schema validation rules (e.g., negative prices, unparseable dates) are routed to a **Dead Letter Queue (DLQ)** or an `errors_staging` table for engineer inspection, allowing valid rows to proceed to the warehouse uninterrupted.

Part 4: Azure Data Factory (ADF) Architecture

Azure Data Factory is Microsoft's cloud-native, serverless data integration and workflow orchestration service.

```
┌──────────────────────────────────────────────────────────────────┐
│                        AZURE DATA FACTORY (ADF)                  │
│                                                                  │
│  [ Trigger ] ──> [ Pipeline (Workflow DAG) ]                     │
│                        │                                         │
│                        ├── [ Lookup Activity ]                   │
│                        ├── [ Copy Data Activity ] ──> [ Integration RT ] │
│                        └── [ Databricks / Web Activity ]         │
│                                                                  │
│  [ Datasets ] ──────────── Points to schema/files                │
│  [ Linked Services ] ──── Connection strings & credentials (Key Vault) │
└──────────────────────────────────────────────────────────────────┘
```

4.1 The Five Core Building Blocks of ADF

- **Linked Services:** The connection configuration (similar to a connection string) that defines how ADF connects to external resources (e.g., Azure SQL, Blob Storage, AWS S3, REST API), typically referencing credentials stored securely in **Azure Key Vault**.
- **Datasets:** Named views of data structures that point to specific tables, folders, or files within a Linked Service (e.g., `ParquetFile_ADLS`, `SalesTable_AzureSQL`).
- **Activities:** The execution steps within a pipeline:
  Data Movement: **Copy Data Activity** (High-throughput data movement).
  Data Transformation: **Mapping Data Flows**, Databricks Notebook, Synapse Stored Procedure.

Control Flow: **Lookup**, **Get Metadata**, **ForEach**, **If Condition**, **Until**, **Web Activity**.

- **Pipelines:** A logical grouping of activities that perform a unit of work (a directed acyclic graph - DAG).
- **Triggers:** Determine when a pipeline execution is initiated:

Schedule Trigger: Standard wall-clock time (e.g., daily at midnight).
Tumbling Window Trigger: Non-overlapping, contiguous time intervals; natively supports backfilling historical windows and handling pipeline dependencies.
Event-Based Trigger: Fires in response to storage events (e.g., Blob created or deleted).

4.2 Integration Runtimes (IR)

The compute infrastructure that ADF uses to execute activities:

- **Azure IR:** Fully managed, serverless, auto-scaling compute inside Azure for cloud-to-cloud data movement.
- **Self-Hosted IR (SHIR):** Installed on-premises or on a private virtual network to securely copy data from behind corporate firewalls to the cloud without opening inbound firewall ports.
- **Azure-SSIS IR:** Dedicated virtual machine cluster to execute legacy SQL Server Integration Services (SSIS) packages natively in the cloud.

Part 5: Required Deliverable: Complete API-to-Database Pipeline Blueprint

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                          PRODUCTION API-TO-WAREHOUSE INCREMENTAL ELT PIPELINE                        │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘

 [External REST API]
         │ (HTTPS / OAuth 2.0 Bearer Token)
         ▼

┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
 │ 1. INGESTION ENGINE (ADF Web Activity / Python Azure Function)
 │
 │    - Reads High-Watermark (last_synced_at) from Metadata Control Table
 │
 │    - GET /api/v1/orders?modified_after={high_watermark} with Keyset Pagination
 │
 │    - Implements Exponential Backoff with Jitter for HTTP 429 / 503 errors
 │
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
          │
```


▼ Raw JSON Payload
2. RAW STORAGE / DATA LAKE (ADLS Gen2 - Bronze Layer)
- Path: /bronze/orders/year=2024/month=03/orders_20240315_120000.json.gz
- Immutable, append-only raw landing zone for full auditability & replayability
▼ Event Notification Trigger / ADF Copy Activity
3. DATA QUALITY & QUARANTINE VALIDATION
- Validates schema against JSON Schema contract
├── Valid Records ──> Land into Staging Table (staging_orders)
└── Invalid Records ──> Land into Quarantine Dead Letter Queue (DLQ) + Alert Slack/Ops
▼
4. CLOUD DATA WAREHOUSE (Snowflake / Azure Synapse - Silver Layer)
- Executes Atomic Idempotent Upsert (MERGE INTO target USING staging):
WHEN MATCHED AND target.updated_at < source.updated_at THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT (...)
▼
5. CONTROL TABLE WATERMARK UPDATE & OBSERVABILITY
- UPDATE pipeline_metadata SET last_synced_at = MAX(source.updated_at)
- Emit metrics to Azure Monitor: rows_ingested, latency_seconds, error_count

Part 6: Structured Interview Question Bank

Q1: Why is Tumbling Window Trigger preferred over Schedule Trigger for historical backfills in Azure Data Factory?

Answer:
A Schedule Trigger is purely wall-clock dependent; if a pipeline fails or is disabled for three days, re-enabling it only fires the current scheduled time, skipping missed days. In contrast, a **Tumbling Window Trigger** produces a contiguous, non-overlapping series of discrete time slices (e.g., 01:00–02:00, 02:00–03:00). It natively passes `@trigger().outputs.windowStartTime` parameters to the pipeline, supports built-in dependency chains (Window N waits for Window $N-1$ to succeed), and allows one-click automated backfilling of past historical slices.

Q2: What is Change Data Capture (CDC), and why is it superior to timestamp polling for large OLTP database extraction?

Answer:
Timestamp polling (`WHERE updated_at > :last_run`) requires executing periodic, expensive table scans against the production OLTP database, competing with user transactions for buffer memory and CPU. Crucially, timestamp polling **cannot detect hard physical deletions** (`DELETE FROM table`). CDC reads the database's internal transaction log (Write-Ahead Log in Postgres, Binlog in MySQL) asynchronously with zero impact on query execution, capturing every `INSERT`, `UPDATE`, and `DELETE` event with full pre- and post-images.

Q3: How do you achieve end-to-end data pipeline observability?

Answer:
End-to-end observability requires tracking the **Four Pillars of Data Health**:

- **Freshness:** Is the data landing within the agreed Service Level Objective (SLO)?
- **Volume:** Did today's partition receive 10,000 rows as expected, or an anomalous 50 rows?
- **Schema:** Did upstream changes drop a column or change an `INT` to `VARCHAR`?
- **Lineage & Quality:** Tracking data flow from source to target, asserting null-rate checks and business invariants via automated test frameworks (e.g., Great Expectations, dbt tests).

Part 7: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  ETL/ELT & PIPELINES CHEAT SHEET                 │
├──────────────────────────────────────────────────────────────────┤
│ 1. ETL vs ELT:                                                   │
│    - ETL: Pre-warehouse transforms. Raw data discarded. Scale limits.  │
│    - ELT: Load raw data first (Bronze). Transform inside cloud warehouse│
│           (Silver/Gold) using SQL/dbt. Scalable and re-playable.       │
├──────────────────────────────────────────────────────────────────┤
│ 2. PIPELINE PRINCIPLES:                                          │
│    - Idempotency : Re-running pipeline produces identical state.       │
│                    Achieved via MERGE/Upsert or INSERT OVERWRITE.      │
│    - Watermarking: Polling records updated_at > last_watermark.        │
│    - CDC         : Reads DB transaction logs (Captures DELETES).       │
│    - DLQ         : Quarantines malformed records without halting DAG.  │
├──────────────────────────────────────────────────────────────────┤
│ 3. AZURE DATA FACTORY (ADF):                                     │
```

```
│    - Linked Service: Connection string + credentials (Key Vault).      │
│    - Dataset       : Pointer to specific table/file structure.         │
│    - Activity      : Copy Data, Lookup, Web, Notebook, ForEach.        │
│    - Triggers      : Schedule, Tumbling Window (Backfills), Event.     │
│    - Integration RT: Azure IR (Cloud), Self-Hosted IR (On-Premises).   │
└────────────────────────────────────────────────────────────────────────┘
```

Part 8: 100-Mark Self-Assessment Examination

Section A: Conceptual Foundations (20 Marks — 4 marks each)

Q1: What is the primary architectural advantage of modern ELT over traditional ETL?

(a) ELT eliminates the need for database storage.

(b) Raw data is preserved, and transformations leverage scalable, decoupled cloud warehouse compute.

(c) ELT requires zero network bandwidth.

(d) ELT transforms data in-memory on the source server.

Q2: Which Azure Data Factory component securely manages connection strings and credentials to external databases?

(a) Dataset

(b) Linked Service

(c) Pipeline

(d) Integration Runtime

Q3: Which ADF trigger type natively supports continuous, non-overlapping historical window intervals with automated backfilling?

(a) Schedule Trigger

(b) Tumbling Window Trigger

(c) Blob Event Trigger

(d) Webhook Trigger

Q4: What is the fundamental limitation of timestamp-based watermarking (`WHERE updated_at > :last_watermark`)?

(a) It cannot process more than 100 rows.

(b) It cannot detect hard row deletions (`DELETE`) on the source table.

(c) It requires disabling primary keys.

(d) It only works on JSON files.

Q5: In a resilient production pipeline, what is a "Dead Letter Queue" (DLQ)?

(a) A deleted Kafka topic.

(b) An isolated staging storage area where corrupted/unparseable records are quarantined for review without halting the pipeline.

(c) A queue used exclusively for database backup archives.

(d) A slow query log in PostgreSQL.

Section B: Architectural & Failure Diagnostics (25 Marks)

Q6 (10 Marks): Define **Idempotency** in the context of data pipelines. Explain why a pipeline that executes `INSERT INTO warehouse.sales SELECT * FROM staging.sales;` is NOT idempotent, and provide the SQL rewrite to make it fully idempotent.

Q7 (15 Marks): Compare the **Lambda Architecture** and **Kappa Architecture**. Why has the industry largely shifted towards Kappa architecture for modern real-time data platforms?

Section C: ADF Design Implementation (35 Marks)

Q8 (20 Marks): You need to copy data from an on-premises Oracle database behind a corporate firewall into Azure Data Lake Storage Gen2.

- Which type of Integration Runtime must be installed, and where? (5 marks)
- Detail the exact ADF building blocks (Linked Services, Datasets, Activities) required to execute this data movement. (15 marks)

Q9 (15 Marks): Write a SQL `MERGE` statement that idempotently updates an active customer dimension `dim_customers (cust_id, email, address, updated_at)` from an incoming incremental staging table `stg_customers`, inserting new customers and updating existing customers only if `stg_customers.updated_at > dim_customers.updated_at`.

Section D: Enterprise Failure Recovery Scenario (20 Marks)

Q10 (20 Marks): An ADF incremental pipeline runs every night at 02:00 UTC. On Thursday morning, you discover that the pipeline failed on Tuesday and Wednesday due to an expired API secret in Azure Key Vault.

- Outline the exact step-by-step procedure to rotate the credentials and safely recover the two missed days of data without causing duplicates. (10 marks)
- Explain how configuring Tumbling Window Triggers simplifies this backfill recovery process. (10 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** Raw data preservation + decoupled cloud compute transformation.
- **Q2: (b)** Linked Service.
- **Q3: (b)** Tumbling Window Trigger.
- **Q4: (b)** Inability to capture hard row deletions.
- **Q5: (b)** Isolated quarantine for corrupted records.

Section B Solutions

Q6 Solution:

- **Idempotency:** The property where executing an operation multiple times produces the exact same outcome as executing it once.
- **Why Non-Idempotent:** Running `INSERT INTO warehouse.sales` twice inserts the same batch twice, causing duplicate rows and inflated revenue metrics.
- **Idempotent Rewrite:**

```
-- Strategy A: Partition Overwrite
INSERT OVERWRITE TABLE warehouse.sales PARTITION (batch_date = '2024-03-15')
SELECT * FROM staging.sales WHERE batch_date = '2024-03-15';

-- Strategy B: Truncate-Load Staging Swap
MERGE INTO warehouse.sales target
USING staging.sales source
ON target.sale_id = source.sale_id
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...;
```

Q7 Solution:

- **Lambda:** Dual-path (Speed Layer in streaming + Batch Layer in MapReduce/Spark + Serving Layer). Forces engineers to maintain two duplicate codebases in different frameworks for the same business logic.
- **Kappa:** Eliminates the batch layer. Treats all data as an immutable event stream (Kafka). Historical reprocessing is achieved simply by spinning up a new consumer group and replaying the stream from offset 0. Reduces code maintenance by 50% and eliminates synchronization bugs.

Section C Solutions

Q8 Solution:

- **Integration Runtime:** A **Self-Hosted Integration Runtime (SHIR)** must be installed on a Windows Server or VM located inside the corporate on-premises network that can connect to the Oracle database.
- **ADF Components:**

Linked Service 1: `LS_OnPrem_Oracle` (uses SHIR, connects to Oracle via connection string/credentials).
Linked Service 2: `LS_Azure_ADLSGen2` (uses Azure IR, connects via Managed Identity).
Dataset 1: `DS_Oracle_SourceTable` (points to `LS_OnPrem_Oracle`).

Dataset 2: `DS_ADLS_ParquetSink` (points to `LS_Azure_ADLSGen2`, format Parquet).
Activity: Copy Data Activity with source set to `DS_Oracle_SourceTable` and sink set to `DS_ADLS_ParquetSink`.

Q9 Solution:

```
MERGE INTO dim_customers AS target
USING stg_customers AS source
ON target.cust_id = source.cust_id
WHEN MATCHED AND source.updated_at > target.updated_at THEN
    UPDATE SET
        target.email = source.email,
        target.address = source.address,
        target.updated_at = source.updated_at
WHEN NOT MATCHED THEN
    INSERT (cust_id, email, address, updated_at)
    VALUES (source.cust_id, source.email, source.address, source.updated_at);
```

Section D Solutions

Q10 Solution:

- **Recovery Steps:**

Update the secret in Azure Key Vault; ADF Linked Service will automatically pull the new secret.
Test connectivity via the ADF authoring canvas.
Identify the earliest failed window start time (Tuesday 02:00 UTC).
Ensure the downstream warehouse merge statement is idempotent to prevent duplicating Tuesday's partial data.
Trigger the pipeline with `windowStartTime = Tuesday 02:00 UTC` and `windowEndTime = Thursday 02:00 UTC`.

- **Tumbling Window Advantage:**

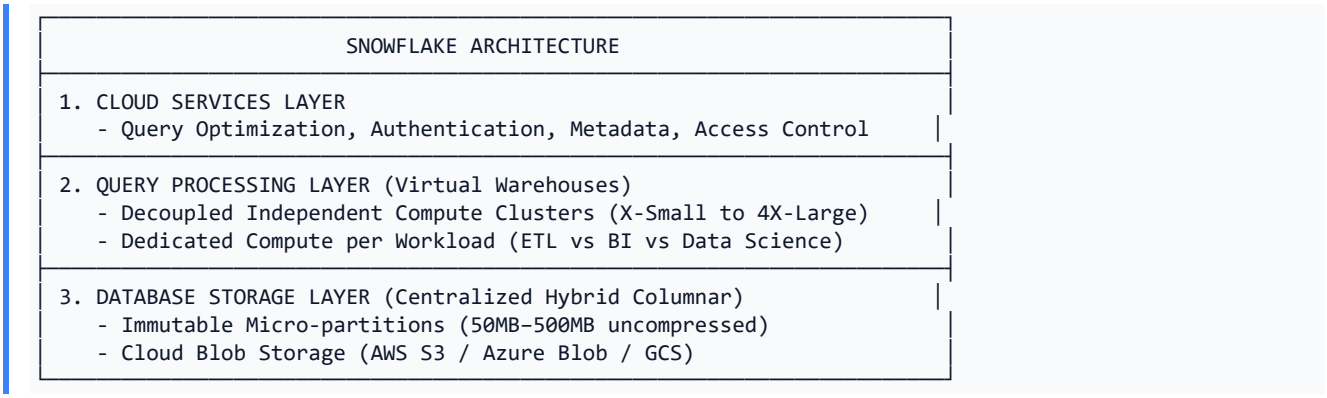
Tumbling Window triggers break execution into discrete, tracked temporal chunks (`T1: Tue-Wed`, `T2: Wed-Thu`).
In the ADF Monitoring UI, simply select the two failed window slices and click **Rerun Window**. The trigger automatically supplies the correct historical start and end timestamps to each run, executing them sequentially without manual script hacking.

Day 12: Modern Data Warehouses, Orchestration & Transformation

Snowflake Architecture, T-SQL vs. PL/SQL, Apache Airflow, dbt & The Medallion Architecture

---

Part 1: Snowflake Architecture & Cloud Data Warehousing



1.1 The Three Decoupled Architectural Layers

Layer 1: Centralized Database Storage

- Snowflake stores all table data in an optimized, encrypted, columnar format split into immutable **Micro-partitions** (typically 50MB to 500MB of uncompressed data).
- Storage resides in cloud object stores (S3, ADLS Gen2, GCS). Users pay only for raw compressed cloud storage volume ($/TB/month).

Layer 2: Virtual Compute Warehouses (Query Processing)

- Compute is **completely decoupled** from storage. A "Virtual Warehouse" is an independent cluster of CPU/RAM nodes (powered by EC2 or Azure VMs).
- **Zero Resource Contention:** An ETL warehouse running a heavy batch load does not slow down an executive running a Power BI dashboard on a separate BI warehouse!
- **Auto-Suspend & Auto-Resume:** Warehouses automatically spin down after N seconds of inactivity, billing compute strictly per-second.

Layer 3: Cloud Services (The Brain)

- A multi-tenant, globally managed layer that handles authentication, access control (RBAC), metadata tracking, ACID transaction coordination, and query compilation/optimization.

1.2 Revolutionary Snowflake Capabilities

Micro-Partitioning & Pruning

Tables are automatically partitioned based on natural insertion order. For every micro-partition, Snowflake automatically tracks **column-level min/max metadata**. During query execution, the optimizer uses this metadata to completely skip (prune) non-matching micro-partitions without reading them from disk!

Zero-Copy Cloning

Creates an instantaneous metadata clone of a database, schema, or table without copying underlying storage bytes:

```
CREATE DATABASE analytics_dev CLONE analytics_prod;
```

- Both databases initially point to the exact same immutable micro-partitions ($0 extra storage cost).
- Writes to either database write new micro-partitions independently (Copy-on-Write).

Time Travel & Fail-safe

- **Time Travel (Up to 90 Days):** Allows querying or restoring data as it existed at any historical timestamp or query ID:`SELECT * FROM orders AT(OFFSET => -60*15); -- 15 minutes ago`
`UNDROP TABLE accidental_deleted_table;`

- **Fail-safe (7 Days):** Disaster recovery storage accessible only by Snowflake support.

Part 2: Enterprise Database Engines: T-SQL vs. PL/SQL

Both Microsoft SQL Server and Oracle are battle-tested relational engines, but their procedural extensions differ:

Feature	T-SQL (Transact-SQL - Microsoft SQL Server)	PL/SQL (Procedural Language - Oracle)
Block Structure	Implicit or explicit `BEGIN ... END`.	Formal block: `DECLARE ... BEGIN ... EXCEPTION ... END;`.
Variable Declaration	`DECLARE @var INT = 10;`	`var_name NUMBER := 10;` (Inside DECLARE block).
Error Handling	Structured `BEGIN TRY ... BEGIN CATCH ... END CATCH`.	`EXCEPTION WHEN NO_DATA_FOUND THEN ...`.
Cursors	Declared, opened, fetched, closed, deallocated.	Native cursor FOR loops (`FOR r IN (SELECT ...) LOOP`).
Identity / Autoincrement	`IDENTITY(1,1)` column property.	`SEQUENCE` objects (e.g., `seq.NEXTVAL`) or `GENERATED AS IDENTITY`.

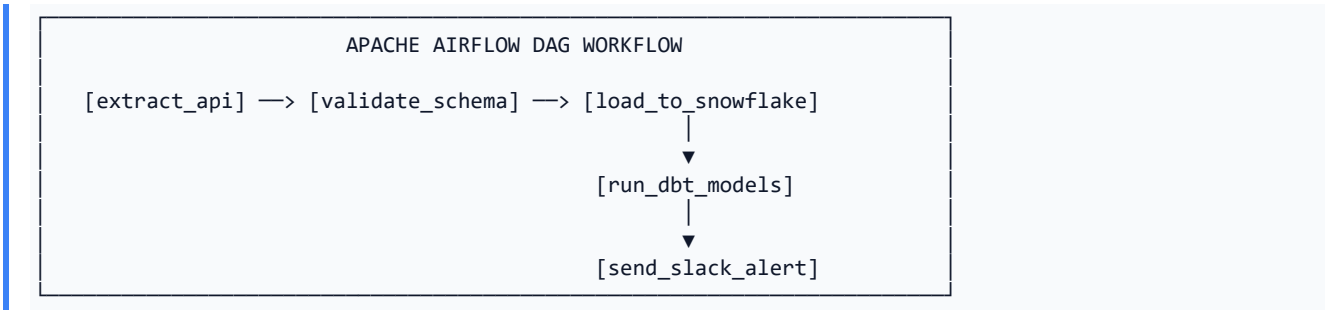
```
-- T-SQL Stored Procedure Example (SQL Server)
CREATE PROCEDURE dbo.GetEmployeeBonus
    @EmpId INT,
    @BonusPct DECIMAL(4,2)
AS
```

```
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        UPDATE dbo.Employees
        SET Bonus = Salary * @BonusPct
        WHERE EmployeeID = @EmpId;
    END TRY
    BEGIN CATCH
        SELECT ERROR_NUMBER() AS ErrorNum, ERROR_MESSAGE() AS ErrorMsg;
    END CATCH
END;

-- PL/SQL Stored Procedure Example (Oracle)
CREATE OR REPLACE PROCEDURE calculate_bonus(
    p_emp_id IN NUMBER,
    p_bonus_pct IN NUMBER
) IS
BEGIN
    UPDATE employees
    SET bonus = salary * p_bonus_pct
    WHERE employee_id = p_emp_id;
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001, 'Bonus calculation failed: ' || SQLERRM);
END calculate_bonus;
```

Part 3: Workflow Orchestration: Apache Airflow Fundamentals

Apache Airflow orchestrates complex computational workflows as **Directed Acyclic Graphs (DAGs)** authored entirely as Python code.



3.1 Core Airflow Building Blocks

- **DAG (Directed Acyclic Graph):** The complete collection of tasks with directional dependencies and no circular loops.
- **Operator:** The template for a single unit of work:

`BashOperator`: Executes bash scripts.
`PythonOperator`: Calls an arbitrary Python function.
`SnowflakeOperator` / `SQLExecuteQueryOperator`: Runs native SQL commands.

- **Task Instance:** A specific task assigned to an execution date/run (`dag_run`).

- **Dependencies (>> and <<):** Bitshift operators defining the execution sequence (`task_a >> task_b`).

3.2 Production Airflow DAG Implementation

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.snowflake.operators.snowflake import SQLExecuteQueryOperator

default_args = {
    "owner": "data_engineering",
    "depends_on_past": False,
    "email_on_failure": True,
    "email": ["data_alerts@company.com"],
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="daily_sales_pipeline",
    default_args=default_args,
    description="Extracts sales data, loads to Snowflake, and executes dbt models",
    schedule="0 2 * * *",  # Daily at 02:00 UTC
    start_date=datetime(2024, 1, 1),
    catchup=False,          # Prevents backfilling every day since start_date!
    max_active_runs=1,
) as dag:

    def extract_external_sales(**context):
        print("Fetching API data and staging to cloud bucket...")

    task_extract = PythonOperator(
        task_id="extract_api_sales",
        python_callable=extract_external_sales,
    )

    task_copy_snowflake = SQLExecuteQueryOperator(
        task_id="copy_into_bronze_snowflake",
        conn_id="snowflake_default",
        sql="""
            COPY INTO bronze_db.raw.sales
            FROM @bronze_db.stages.s3_sales_stage
            FILE_FORMAT = (TYPE = 'PARQUET');
        """,
    )

    task_extract >> task_copy_snowflake
```

Part 4: Modern Transformation as Code: dbt (Data Build Tool)

dbt enables analytics engineers to transform data directly inside cloud warehouses using standard `SELECT` statements, treating SQL transformations with software engineering rigor (version control, automated testing, documentation, CI/CD).

4.1 How dbt Works

In dbt, an engineer writes pure `SELECT` queries in `.sql` files. dbt automatically compiles these files into DDL/DML statements (`CREATE TABLE AS SELECT` or `CREATE VIEW AS SELECT`), managing table creation, dependency resolution, and incremental merges.

```
-- models/marts/fct_daily_sales.sql
{{ config(
    materialized='incremental',
    unique_key='sale_id'
) }}

WITH source_data AS (
    -- The {{ ref() }} macro builds the automated lineage DAG!
    SELECT * FROM {{ ref('stg_sales') }}
    {% if is_incremental() %}
        -- Only query records newer than the current table's max timestamp
        WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})
    {% endif %}
)
SELECT
    sale_id,
    customer_id,
    sale_amount,
    updated_at
FROM source_data;
```

4.2 Materialization Strategies in dbt

- **view:** Fast to build; zero warehouse storage; evaluated on-the-fly when queried.
- **table:** Physical table created via `CTAS`; fast query response; incurs compute time on build.
- **incremental:** Modifies only newly added or modified rows since the last dbt execution using `MERGE`. Essential for multi-million row fact tables.
- **ephemeral:** Interpolated as an inline Common Table Expression (CTE) into downstream models; zero database footprint.

4.3 Automated Data Testing in dbt

dbt includes four out-of-the-box generic tests defined in `schema.yml`:

```
version: 2
models:
  - name: stg_orders
    columns:
      - name: order_id
        tests:
          - unique
          - not_null
      - name: order_status
        tests:
          - accepted_values:
              values: ['Pending', 'Completed', 'Cancelled', 'Refunded']
      - name: customer_id
        tests:
          - relationships:
              to: ref('stg_customers')
              field: customer_id
```

Part 5: Required Deliverable: From Raw Ingestion to Gold Analytics (The Medallion Architecture)



```
(OBT)|
 |    - Pre-aggregated business KPIs (Customer LTV, Monthly Recurring Revenue, Churn Risk)
|
 |    - Power BI / Tableau dashboards and ML feature stores connect directly here
|

└──────────────────────────────────────────────────────────────────────────────────────────
┘
```

Part 6: Structured Interview Question Bank

Q1: How does Snowflake achieve zero-copy cloning without duplicating storage data?

Answer:
Snowflake's storage layer consists of immutable micro-partitions stored in cloud object storage, with the Cloud Services metadata layer maintaining pointers to these files. When `CLONE` is invoked, Snowflake simply duplicates the metadata catalog pointers to the existing micro-partitions without copying any underlying data blocks. If a user subsequently modifies or updates data in the cloned table, Snowflake writes brand-new micro-partitions (Copy-on-Write) and updates the clone's metadata pointers, ensuring that unchanged data is shared while modified data remains isolated.

Q2: What is the {{ ref() }} macro in dbt, and why is it superior to hardcoding table names?

Answer:
The `{{ ref('model_name') }}` macro accomplishes two essential engineering functions:

- **Automated Lineage & Dependency Graph:** It tells dbt that the current model depends on `model_name`. dbt automatically computes the Directed Acyclic Graph (DAG) and executes models in the correct topological order.
- **Environment Isolation:** It interpolates the correct database and schema based on the active target environment (e.g., querying `dev_schema.stg_orders` in local development, but compiling to `prod_schema.stg_orders` in production CI/CD).

Q3: What is catchup=False in an Apache Airflow DAG, and what catastrophic issue does it prevent?

Answer:
By default, Airflow's `catchup` parameter is `True`. If a DAG is defined with a `start_date` of two months ago and enabled today, Airflow will immediately schedule and execute separate `dag_run` instances for every single missed interval between the start date and the current date (e.g., 60 back-to-back runs). This can overload source APIs, crash databases, and incur massive cloud computing costs. Setting `catchup=False` instructs the scheduler to execute strictly the most recent interval.

Part 7: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────────────┐
│                DATA WAREHOUSE & ORCHESTRATION CHEAT SHEET                │
└──────────────────────────────────────────────────────────────────────────┘

 1. SNOWFLAKE:
    - Decoupled 3 Tiers: Storage (Micro-partitions) | Compute (Virtual
      Warehouses) | Cloud Services (Metadata & Optimizer).
```

```
    - Zero-Copy Cloning: Duplicates metadata pointers; 0 storage cost.
    - Time Travel: Query historical state up to 90 days.
──────────────────────────────────────────────────────────────────────
 2. APACHE AIRFLOW:
    - DAG: Directed Acyclic Graph (Python-authored workflow).
    - Operators: PythonOperator, BashOperator, SQLExecuteQueryOperator.
    - Dependencies: task_a >> task_b.
    - Always set catchup=False unless historical backfill is intended!
──────────────────────────────────────────────────────────────────────
 3. DBT (DATA BUILD TOOL):
    - In-warehouse SQL transformations (ELT).
    - {{ ref('model') }}: Automatic DAG lineage and environment routing.
    - Materializations: view, table, incremental, ephemeral.
    - Testing: unique, not_null, accepted_values, relationships.
──────────────────────────────────────────────────────────────────────
 4. MEDALLION ARCHITECTURE:
    - Bronze (Raw)   : Append-only full historical fidelity.
    - Silver (Clean) : Deduplicated, 3NF cleaned conformed tables.
    - Gold (Marts)   : Star schema, business KPIs, reporting-ready.
──────────────────────────────────────────────────────────────────────
```

Part 8: 100-Mark Self-Assessment Examination

Section A: Conceptual Foundations (20 Marks — 4 marks each)

Q1: In Snowflake's three-tier architecture, where are query compilation, metadata management, and transaction locks handled?

(a) Database Storage Layer

(b) Virtual Warehouse Compute Layer

(c) Cloud Services Layer

(d) Local SSD Cache

Q2: What happens when an Airflow DAG contains a circular dependency (`task_1 >> task_2 >> task_1`)?

(a) Airflow executes the tasks in an infinite loop.

(b) The Airflow scheduler detects a DAG cycle and raises an `AirflowDagCycleException`.

(c) The tasks are automatically executed in parallel.

(d) Airflow converts the cycle into a linear queue.

Q3: Which dbt materialization creates a temporary Common Table Expression (CTE) that is interpolated into downstream models rather than existing as a database object?

(a) `view`

(b) `table`

(c) `incremental`

(d) `ephemeral`

Q4: In Snowflake, what is the default maximum duration for Time Travel on an Enterprise Edition account?

(a) 1 day

(b) 7 days

(c) 90 days

(d) 365 days

Q5: In a Medallion Architecture, which layer is characterized by Star Schema dimensional models (Fact and Dimension tables) optimized for BI dashboards?

(a) Bronze Layer

(b) Silver Layer

(c) Gold Layer

(d) Landing Zone

Section B: Architecture & Execution Tracing (25 Marks)

Q6 (10 Marks): Explain how Snowflake's micro-partition min/max metadata enables "Query Pruning". How does sorting/clustering data impact pruning efficiency?

Q7 (15 Marks): Consider the Airflow DAG schedule: `schedule="0 0 * * *"` with `start_date=2024-01-01`. When does the DAG run for execution date `2024-01-01` actually trigger, and why does Airflow use this scheduling paradigm?

Section C: Implementation Challenges (35 Marks)

Q8 (20 Marks): Write a production dbt model `fct_orders.sql` configured as an **incremental** table using `order_id` as the unique key. Ensure it references the upstream model `stg_orders` using the `ref` macro, and only queries rows where `updated_at` is greater than the maximum `updated_at` currently in the target table.

Q9 (15 Marks): Write an Airflow Python DAG snippet defining three tasks: `extract_task`, `transform_task`, and `load_task`. Configure the DAG so that `transform_task` runs only if `extract_task` succeeds, and `load_task` runs only if `transform_task` succeeds, with an automated 2-retry policy.

Section D: Enterprise Failure Scenario (20 Marks)

Q10 (20 Marks): A daily incremental dbt model fails with an error:
`Duplicate key violation: duplicate key value found in column 'order_id'`.

- Explain how upstream late-arriving updates or duplicated raw records caused this failure. (10 marks)
- Provide the SQL deduplication pattern using `ROW_NUMBER()` in the staging layer to permanently eliminate this vulnerability. (10 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (c)** Cloud Services Layer.
- **Q2: (b)** Airflow detects the cycle and refuses to parse the DAG (`AirflowDagCycleException`).
- **Q3: (d)** `ephemeral`.
- **Q4: (c)** 90 days.
- **Q5: (c)** Gold Layer.

Section B Solutions

Q6 Solution:

- Every micro-partition automatically records the minimum and maximum values of every column in the Cloud Services metadata catalog.
- When a query executes with `WHERE order_date BETWEEN '2024-01-01' AND '2024-01-05'`, the optimizer checks the min/max values of all micro-partitions. Partitions where `min > '2024-01-05'` or `max < '2024-01-01'` are pruned entirely without reading any data from storage.
- Clustering keys ensure that rows with correlated values are physically packed into the same micro-partitions, minimizing the min/max range overlaps and maximizing the number of skipped partitions.

Q7 Solution:

- In Airflow, a data interval run for date D executes **at the end of the interval** ($D + \text{duration}$). Therefore, the run for `2024-01-01` triggers on **2024-01-02 at 00:00:00 UTC**.
- **Reasoning:** A daily data pipeline cannot process all records belonging to January 1st until January 1st has fully completed!

Section C Solutions

Q8 Solution:

```
{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='merge'
) }}

WITH source_data AS (
    SELECT * FROM {{ ref('stg_orders') }}
    {% if is_incremental() %}
        WHERE updated_at > (SELECT COALESCE(MAX(updated_at), '1970-01-01') FROM {{ this }})
    {% endif %}
```

```
)
SELECT
    order_id,
    customer_id,
    order_status,
    total_amount,
    updated_at
FROM source_data;
```

Q9 Solution:

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.empty import EmptyOperator

default_args = {
    "retries": 2,
    "retry_delay": timedelta(minutes=3),
}

with DAG(
    dag_id="linear_pipeline",
    default_args=default_args,
    schedule="0 0 * * *",
    start_date=datetime(2024, 1, 1),
    catchup=False,
) as dag:
    extract = EmptyOperator(task_id="extract_task")
    transform = EmptyOperator(task_id="transform_task")
    load = EmptyOperator(task_id="load_task")

    extract >> transform >> load
```

Section D Solutions

Q10 Solution:

- **Root Cause:** If the upstream source system produces two records with the same `order_id` in the same incremental window (e.g., an order created at 10:00 and updated at 10:05), the staging query selects both records. When dbt executes the `MERGE INTO`, it attempts to join on `order_id` and encounters multiple matching source rows, violating relational merge constraints.
- **Deduplication Pattern:**
```
WITH RankedSource AS (
    SELECT
        order_id,
        customer_id,
        total_amount,
        updated_at,
        ROW_NUMBER() OVER (
            PARTITION BY order_id
            ORDER BY updated_at DESC, _ingested_at DESC
        ) AS dedupe_rank
    FROM raw.orders
)
SELECT
    order_id,
    customer_id,
    total_amount,
    updated_at
FROM RankedSource
WHERE dedupe_rank = 1;
```

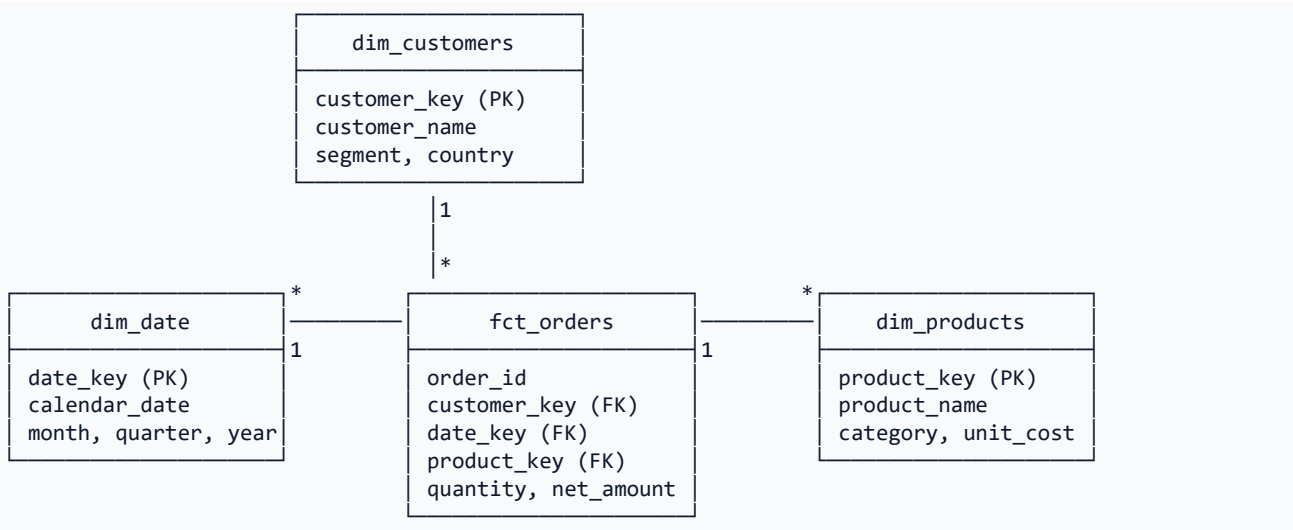

Day 13: Power BI, DAX & Modern Excel Analytics

Dimensional Modeling, Power Query (M), DAX Evaluation Contexts, VertiPaq & Dashboard Engineering

---

Part 1: Dimensional Modeling & The Star Schema

1.1 Facts vs. Dimensions (Kimball Methodology)



Fact Tables

Record quantitative measurements, events, or transactional metrics resulting from a business process.

- **Additive Measures:** Can be summed across all dimensions (e.g., `Sales Amount`, `Quantity Sold`).
- **Semi-Additive Measures:** Can be summed across some dimensions, but not time (e.g., `Bank Account Balance`, `Warehouse Inventory Count` — summing balances across days makes no sense; must use end-of-period snapshot).
- **Non-Additive Measures:** Cannot be meaningfully summed across any dimension (e.g., `Unit Price`, `Profit Margin %`, `Ratios`).

Dimension Tables

Contain qualitative context (the "who, what, where, when, why") that filters, slices, and categorizes facts. Characterized by wide tables with descriptive text attributes.

1.2 Star Schema vs. Snowflake Schema

Feature	Star Schema	Snowflake Schema
Structure	Fact table at center connected directly to denormalized Dimension tables.	Dimensions are normalized into sub-tables (e.g., `Product` $\to$

		`SubCategory` $\to$ `Category`).
Number of Joins	Minimal (1 join between Fact and any Dimension).	Multiple joins required to traverse normalized hierarchy.
VertiPaq Engine Performance	**Optimal** (Power BI's columnar engine is designed specifically for Star Schemas).	Sub-optimal (Extra relationship traversal adds CPU overhead).
Maintenance	Simpler DAX formulas and clearer relationship modeling for business users.	Complex relationship paths; potential ambiguity in cross-filtering.

> *[!IMPORTANT]*
> ***Power BI Best Practice:*** *Always flatten normalized hierarchies into a single denormalized Dimension table in Power Query before loading into the Power BI data model.*

1.3 Relationship Cardinality & Cross-Filter Direction

- **Cardinality:**

One-to-Many ($1:*$): The standard golden pattern (Primary key of Dimension table to Foreign key of Fact table).
Many-to-Many ($:$): Dangerous; causes ambiguous paths and performance degradation. Should be resolved using a bridge table.

- **Cross-Filter Direction:**

Single (Default): Filters flow downstream from the "One" side (Dimension) to the "Many" side (Fact).
Both (Bi-directional): Filters flow in both directions. Avoid unless strictly required, as it creates ambiguous relationship paths and severe performance bottlenecks in large models.

Part 2: Power Query & The M Language

2.1 The M Engine & Applied Steps

Power Query executes transformations sequentially as an immutable dependency pipeline recorded in the **M Language**. Each step takes the previous table as an input and returns a new transformed table.

2.2 Query Folding (The #1 Performance Factor)

Query Folding is the ability of Power Query to translate user interface transformation steps (filtering, grouping, joins, column removal) into a **single native SQL query** executed directly on the source relational database.

What folds:

- Filtering rows (`WHERE`), selecting columns (`SELECT`), joins (`INNER/LEFT JOIN`), grouping (`GROUP BY`).

What breaks folding:

- Complex custom M functions, changing data types to non-native types, merging with non-relational sources (Excel + SQL), or using index-based steps.

> *[!TIP]*
> *Always verify Query Folding: Right-click the last Applied Step in Power Query. If **"View Native Query"** is enabled, folding is active, meaning the source database handles the compute rather than your desktop RAM!*

Part 3: DAX (Data Analysis Expressions) Masterclass

3.1 Calculated Columns vs. Measures

Attribute	Calculated Column	Measure
Evaluation Timing	During data refresh / model load.	Dynamically at query time when visual renders.
Storage	Stored physically in RAM (compressed by VertiPaq).	Zero RAM storage (computed on-the-fly in CPU cache).
Context	Evaluated under **Row Context** (current row).	Evaluated under **Filter Context** (visual slicers, filters).
Use Case	Slicer fields, row-level categories, matrix row headers.	Numerical aggregations, KPIs, ratios, percentages.

3.2 Evaluation Contexts: Row Context vs. Filter Context

1. Row Context

The concept of "the current row". Present automatically in Calculated Columns and DAX iterator functions (`SUMX`, `AVERAGEX`, `FILTER`).

- Does **not** know about filters applied by slicers or matrix visuals!

2. Filter Context

The combination of all active filters applied to the data model:

- Visual filters (Row/Column headers in a Matrix).
- Slicers selected on the report page.
- Page-level and report-level filters.

3.3 The Queen of DAX: CALCULATE() & Context Transition

`CALCULATE(<expression>, <filter1>, <filter2>, ...)` is the **only function in DAX that can modify the Filter Context**.

Context Transition:

When a Measure is called inside a Row Context (or when `CALCULATE()` wraps an expression inside a row context), it triggers **Context Transition**: it automatically converts the current row's values into an equivalent Filter Context!

```
-- 1. Base Measure
Total Revenue = SUM(fct_orders[net_amount])

-- 2. Modifying Filter Context (Overriding Visual Filters)
Tech Sales =
CALCULATE(
    [Total Revenue],
    dim_products[category] = "Technology"
)

-- 3. Calculating % of Total using ALL() to remove Category Filters
% Category Contribution =
DIVIDE(
    [Total Revenue],
    CALCULATE([Total Revenue], ALL(dim_products[category]))
)
```

3.4 Essential Time Intelligence DAX Patterns

Requires a contiguous, marked **Date Dimension Table** (`dim_date`):

```
-- Year-to-Date (YTD) Revenue
Revenue YTD =
TOTALYTD([Total Revenue], dim_date[calendar_date])

-- Same Period Last Year (SPLY)
Revenue SPLY =
CALCULATE(
    [Total Revenue],
    SAMEPERIODLASTYEAR(dim_date[calendar_date])
)

-- Month-over-Month (MoM) Growth %
MoM Revenue Growth % =
VAR CurrentRevenue = [Total Revenue]
VAR PriorMonthRevenue = CALCULATE([Total Revenue], DATEADD(dim_date[calendar_date], -1, MONTH))
RETURN
    DIVIDE(CurrentRevenue - PriorMonthRevenue, PriorMonthRevenue)
```

Part 4: Advanced Modern Excel Modeling

4.1 XLOOKUP vs. Legacy VLOOKUP

```
=XLOOKUP(lookup_value, lookup_array, return_array, [if_not_found], [match_mode], [search_mode])
```

- **Why XLOOKUP wins:**

  Searches to the left and right (no index column offset required).

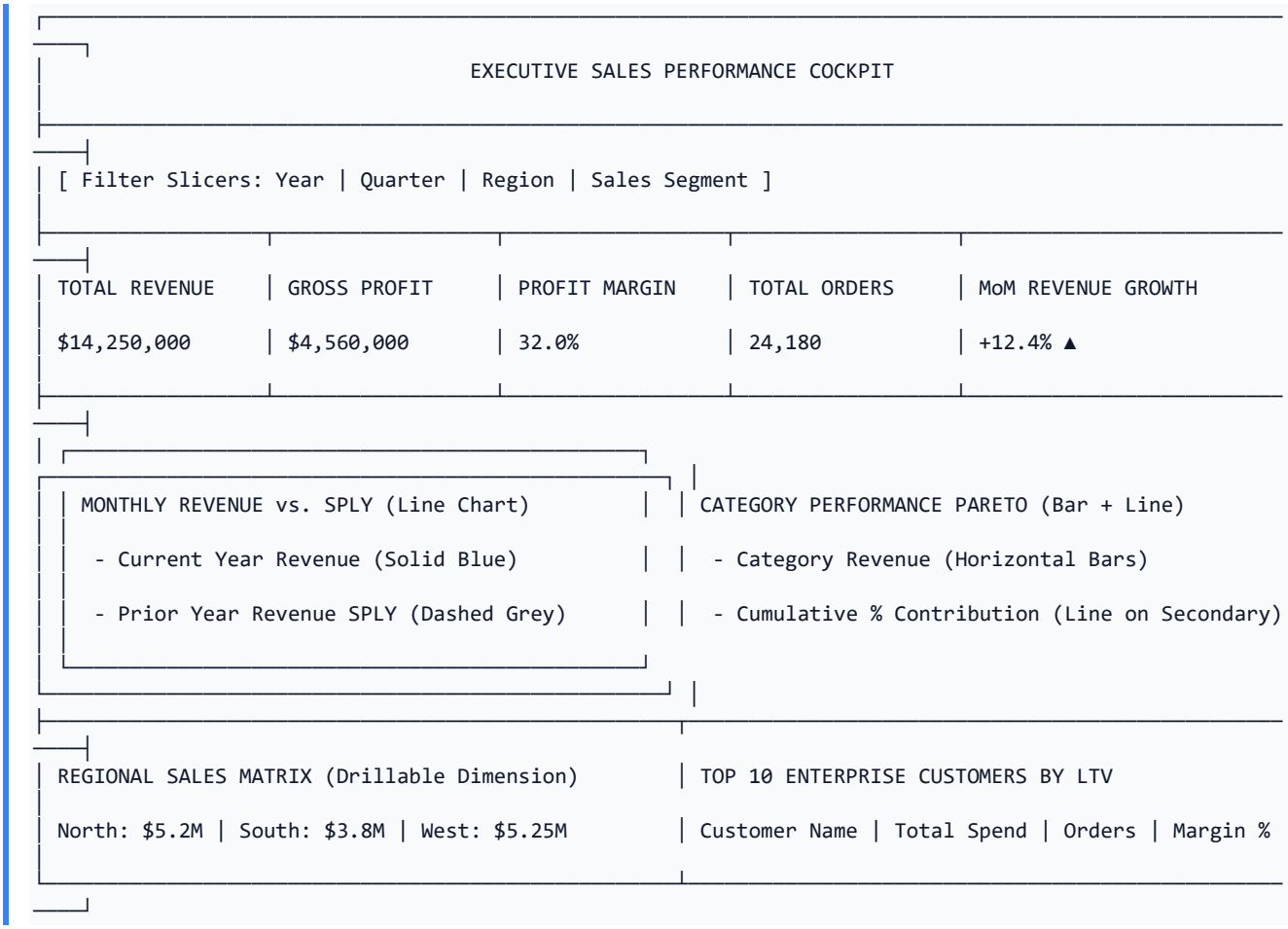
  Defaults to exact match (`match_mode = 0`).

Safe against column insertion/deletion (uses range references, not static column numbers).
Built-in error handling (`if_not_found` replaces `IFERROR`).

4.2 Dynamic Array Formulas

- **FILTER():** `=FILTER(A2:C100, (B2:B100="Corporate") * (C2:C100>5000), "No Records")`
- **UNIQUE():** `=UNIQUE(A2:A100)`
- **SORT():** `=SORT(FILTER(A2:C100, B2:B100="West"), 3, -1)` (Sorts filtered array by 3rd column descending).

Part 5: Required Deliverable: Executive Sales Performance Dashboard Specification



Core DAX Measures Powering the Dashboard:

```
-- 1. Base Revenue
Total Revenue = SUM(fct_orders[net_amount])

-- 2. Total Cost
```

```
Total Cost = SUMX(fct_orders, fct_orders[quantity] * RELATED(dim_products[unit_cost]))

-- 3. Gross Profit
Gross Profit = [Total Revenue] - [Total Cost]

-- 4. Gross Profit Margin %
Profit Margin % = DIVIDE([Gross Profit], [Total Revenue], 0)

-- 5. Prior Year Revenue (SPLY)
Revenue SPLY = CALCULATE([Total Revenue], SAMEPERIODLASTYEAR(dim_date[calendar_date]))

-- 6. Year-over-Year Growth %
YoY Revenue Growth % = DIVIDE([Total Revenue] - [Revenue SPLY], [Revenue SPLY])
```

Part 6: Structured Interview Question Bank

Q1: What is the difference between ALL() and ALLSELECTED() in DAX?

Answer:

- `ALL(table_or_column)` removes **all filters** from the specified column or table, regardless of where they originated (visual headers, slicers, page filters). It evaluates against the grand total of the entire dataset.
- `ALLSELECTED(table_or_column)` removes filters applied inside the visual (e.g., matrix row/column headers) while **retaining filters applied by external slicers** and page-level filters. It is essential for calculating dynamic "% of Sliced Total" in dashboards.

Q2: Why is the VertiPaq engine so sensitive to column cardinality, and how do you optimize Power BI model memory?

Answer:
VertiPaq is an in-memory columnar database that compresses data using **Dictionary Encoding, Run-Length Encoding (RLE), and Bit-Packing**.

- The memory consumed by a column is directly proportional to its **Cardinality (Number of Unique Values)**.
- High-cardinality columns (such as transaction IDs, UUIDs, or timestamps with hours/minutes/seconds) destroy dictionary compression, consuming gigabytes of RAM.
- **Optimization:** Split datetime columns into separate `Date` and `Time` columns, remove unused surrogate IDs from fact tables, and replace calculated columns with dynamic measures.

Q3: What is Context Transition in DAX, and what dangerous bug can occur if used improperly?

Answer:
Context Transition occurs when an expression is wrapped in `CALCULATE()` inside a Row Context (such as inside a calculated column or `SUMX`). `CALCULATE()` converts all column values of the current row into an active filter context.

- **The Bug:** If the table lacks a unique primary key, and multiple rows share identical attribute values, Context Transition will filter the table down to **all identical rows**, multiplying the aggregate calculation and producing severely inflated results.

Part 7: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────────┐
│                    POWER BI & EXCEL CHEAT SHEET                      │
├──────────────────────────────────────────────────────────────────────┤
│ 1. MODELING:                                                         │
│    - Kimball Star Schema: Central Fact Table + Denormalized Dimensions.│
│    - Single Cross-Filter (1:*) is standard. Avoid bi-directional!    │
├──────────────────────────────────────────────────────────────────────┤
│ 2. DAX CONTEXT:                                                      │
│    - Row Context   : Evaluates current row (Calculated Columns, SUMX).│
│    - Filter Context: Visual coordinates + Slicers + Page filters.    │
│    - CALCULATE()   : ONLY way to modify Filter Context. Triggers     │
│                      Context Transition.                             │
│    - ALL()         : Clears filters (Grand Total).                   │
│    - ALLSELECTED() : Clears visual filters, keeps slicers.           │
├──────────────────────────────────────────────────────────────────────┤
│ 3. POWER QUERY (M):                                                  │
│    - Query Folding : Pushes transformations down to SQL source DB.   │
├──────────────────────────────────────────────────────────────────────┤
│ 4. MODERN EXCEL:                                                     │
│    - XLOOKUP: Searches any direction; exact match default.           │
│    - Dynamic Arrays: =FILTER(), =UNIQUE(), =SORT().                  │
│    - Multi-criteria: SUMIFS(sum_range, crit_range1, crit1, ...).     │
└──────────────────────────────────────────────────────────────────────┘
```

Part 8: 100-Mark Self-Assessment Examination

Section A: Conceptual Foundations (20 Marks — 4 marks each)

Q1: What is the primary difference between a Calculated Column and a Measure in Power BI?

(a) Calculated columns are evaluated dynamically at query time; Measures are computed during refresh.

(b) Calculated columns consume RAM storage; Measures are computed on-the-fly and consume zero model storage.

(c) Measures can only return text strings.

(d) Calculated columns cannot use DAX formulas.

Q2: In a Kimball Dimensional Model, what type of measure is a bank account daily balance?

(a) Fully Additive

(b) Semi-Additive

(c) Non-Additive

(d) Degenerate

Q3: What is "Query Folding" in Power Query?

(a) Compressing data into a zip file before loading.

(b) The automated translation of transformation steps into a native SQL query executed by the source database.

(c) Merging two Excel spreadsheets into a single sheet.

(d) Converting rows into columns.

Q4: Which DAX function removes internal visual filters while respecting external slicers selected by the user?

(a) `ALL()`

(b) `ALLSELECTED()`

(c) `ALLEXCEPT()`

(d) `REMOVEFILTERS()`

Q5: What happens to the VertiPaq engine's memory compression when a column contains a unique GUID/UUID for every row?

(a) Compression reaches maximum efficiency.

(b) Dictionary encoding bloats, causing massive memory consumption.

(c) The column is automatically deleted.

(d) VertiPaq converts the GUID to an integer.

Section B: Context & DAX Tracing (25 Marks)

Q6 (10 Marks): Explain what the following DAX measure calculates and how `ALL(dim_products[category])` modifies the filter context:

```
Category Sales % =
DIVIDE(
    SUM(fct_sales[amount]),
    CALCULATE(SUM(fct_sales[amount]), ALL(dim_products[category]))
)
```

Q7 (15 Marks): Consider a table `orders` with columns `[customer_id]`, `[order_id]`, and `[amount]`. What is the fundamental difference in execution and result between:

- Expression A: `SUM(orders[amount])`
- Expression B: `SUMX(orders, orders[amount] * (1 - orders[discount_pct]))`

Section C: Implementation Challenges (35 Marks)

Q8 (20 Marks): Write production DAX formulas for the following business requirements:

- `YTD Sales`: Year-to-date sales based on the Date table.

- `Top Category Sales`: Total sales filtered strictly to products where `category = 'Luxury'`, ignoring any visual slicers filtering that category.
- `Customer 30-Day Churn Risk Flag`: A calculated column in `dim_customers` that flags `'At Risk'` if their `days_since_last_purchase > 30`, otherwise `'Active'`.

Q9 (15 Marks): Write the modern Excel Dynamic Array formula using `=FILTER()` and `=SORT()` to extract all rows from `A2:D100` where Region (Column B) is `"West"` and Revenue (Column D) is greater than `$10,000`, sorted by Revenue descending.

Section D: Real-World Performance Tuning Scenario (20 Marks)

Q10 (20 Marks): A Power BI report with 20 million rows takes 18 seconds to refresh each visual. Diagnostic investigation reveals:

- Two calculated columns exist on the fact table: `Customer_Full_Address` (strings) and `Order_Timestamp_Seconds` (DateTime with second-level precision).
- Bi-directional cross-filtering is enabled between `dim_customers` and `fct_orders`.
- Power Query applied steps include a Python script step that breaks Query Folding.

Explain how addressing these three bottlenecks will reduce model size by over 70% and drop visual rendering times to under 1 second.

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** Calculated columns consume RAM; Measures are query-time calculations consuming 0 model storage.
- **Q2: (b)** Semi-Additive (cannot be summed across time).
- **Q3: (b)** Translation of M transformation steps into native source SQL.
- **Q4: (b)** `ALLSELECTED()`.
- **Q5: (b)** High cardinality destroys dictionary compression, causing severe RAM bloat.

Section B Solutions

Q6 Solution:

- **What it calculates:** The percentage contribution of the currently selected product category relative to the total sales across all categories.
- **Filter Modification:** When viewed in a category visual (e.g., Category = "Electronics"), the numerator evaluates sales for Electronics. `CALCULATE(..., ALL(dim_products[category]))` strips the "Electronics" filter off the denominator, calculating the grand total across all categories.

Q7 Solution:

- **Expression A:** Standard simple aggregation. Directly aggregates the compressed columnar vector in VertiPaq via vectorized CPU instructions ($O(1)$ memory).
- **Expression B:** Iterator function (`SUMX`). Creates an active **Row Context**, iterating row-by-row across the table, computing the discounted amount per row, and accumulating the sum.

Section C Solutions

Q8 Solution:

```
-- 1. YTD Sales
YTD Sales = TOTALYTD(SUM(fct_sales[amount]), dim_date[date_key])

-- 2. Top Category Sales
Top Category Sales =
CALCULATE(
    SUM(fct_sales[amount]),
    ALL(dim_products[category]),
    dim_products[category] = "Luxury"
)

-- 3. Customer Calculated Column
Customer Churn Flag =
IF(dim_customers[days_since_last_purchase] > 30, "At Risk", "Active")
```

Q9 Solution:

```
=SORT(FILTER(A2:D100, (B2:B100="West") * (D2:D100>10000), "No Records"), 4, -1)
```

Section D Solutions

Q10 Solution:

- **Eliminate High-Cardinality Calculated Columns:**
  Drop `Customer_Full_Address` from the fact table; keep it in `dim_customers` where it is stored once per customer.
  Split `Order_Timestamp_Seconds` into a separate `Date` key and an integer `Hour` column. Truncating seconds reduces cardinality from millions of unique timestamps to 24 unique hours, enabling 90%+ VertiPaq dictionary compression.

- **Disable Bi-Directional Cross-Filtering:**
  Change relationship back to **Single Direction** (`dim_customers (1) -> fct_orders (*)`). This eliminates ambiguity loops and prevents the engine from generating multi-table joining bitmaps on every click.

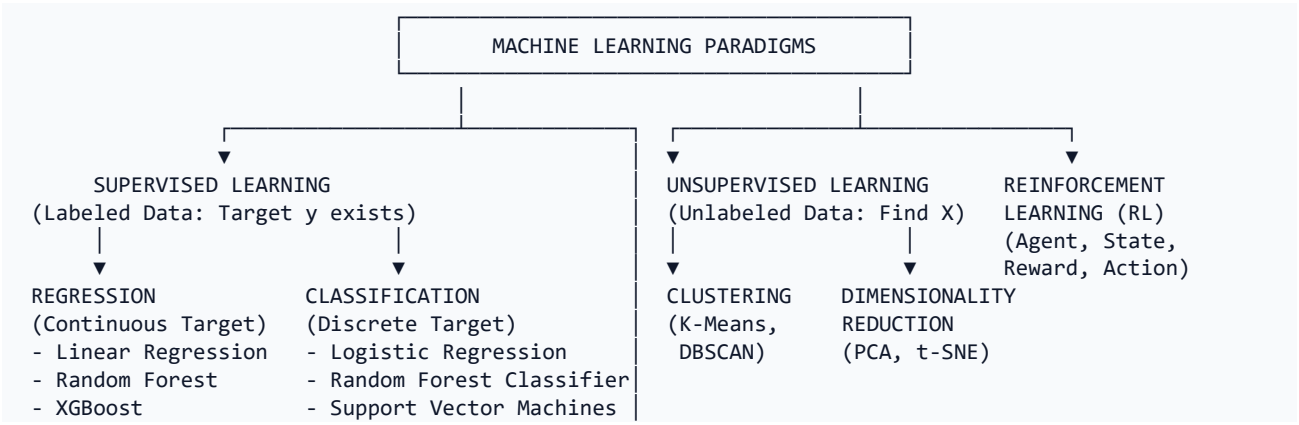
- **Restore Query Folding in Power Query:**
  Remove the Python script step or move it to upstream ETL/warehouse staging. When folding is restored, filters and joins execute directly inside the source database engine, pulling only clean, aggregated data into Power BI.

Day 14: Machine Learning Fundamentals & Model Mechanics

Supervised vs. Unsupervised, Regression, Ensembles, Clustering, Cross-Validation & Data Leakage

---

Part 1: Machine Learning Taxonomy & Problem Framing



Part 2: Supervised Learning: Regression Mechanics

2.1 Linear Regression

Models the linear relationship between continuous dependent target y and independent features $\mathbf{x} = [x_1, x_2, \dots, x_p]^T$:
$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_p x_p = \mathbf{w}^T \mathbf{x}$$

Ordinary Least Squares (OLS) Closed-Form Solution:

Minimizes the Residual Sum of Squares (RSS). In matrix notation:
$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- **Computational Cost:** Inverting $(\mathbf{X}^T \mathbf{X})$ requires $O(p^3)$ time (where p is feature count). When $p > 10,000$ or data does not fit in RAM, **Gradient Descent** is used instead.

Gradient Descent Optimization:

Iteratively updates weights in the opposite direction of the cost function gradient:
$$w_j := w_j - \alpha \frac{\partial}{\partial w_j} J(\mathbf{w})$$

- α: Learning rate (too large causes divergence; too small causes slow convergence).

2.2 Classical Assumptions of OLS (Gauss-Markov Theorem)

For OLS to be BLUE (Best Linear Unbiased Estimator):

- **Linearity:** Relationship between features and target must be linear in parameters.
- **Homoscedasticity:** Variance of error terms ϵ_i must be constant across all levels of features (no "fan-shaped" residual plots).
- **Independence:** Residuals must be uncorrelated (tested via Durbin-Watson statistic for time-series autocorrelation).
- **Normality of Residuals:** Errors must be normally distributed: $\epsilon \sim \mathcal{N}(0, \sigma^2)$.
- **No Multicollinearity:** Independent variables must not be highly correlated with each other (Variance Inflation Factor $\text{VIF} < 5$).

2.3 Regularization: Ridge (L2) vs. Lasso (L1) vs. ElasticNet

```
L2 RIDGE REGULARIZATION (Circular Constraint)
J(w) = MSE + λ ∑ w_j²
* Shrinks coefficients towards zero *
* NEVER sets coefficients exactly to zero *
* Handles multicollinearity *

L1 LASSO REGULARIZATION (Diamond/Corner Constraint)
J(w) = MSE + λ ∑ |w_j|
* Shrinks coefficients AND sets non-informative weights to EXACTLY ZERO *
* Performs AUTOMATIC FEATURE SELECTION *
```

- **ElasticNet:** Convex combination of L1 and L2 penalties ($r \cdot \text{L1} + \frac{1-r}{2} \cdot \text{L2}$). Ideal when multiple features are correlated (Lasso arbitrarily picks one; ElasticNet keeps groups).

Part 3: Supervised Learning: Classification Mechanics

3.1 Logistic Regression & The Sigmoid Function

Used for binary classification ($y \in {0, 1}$). Rather than modeling continuous values, it models the **probability** $P(y=1 \mid \mathbf{x})$:

The Sigmoid (Logistic) Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad \text{where } z = \mathbf{w}^T \mathbf{x} + b$$

- Maps any real number $z \in (-\infty, +\infty)$ into a valid probability interval $(0, 1)$.

Odds & Log-Odds (Logit):

$$\text{Odds} = \frac{p}{1-p} \implies \ln\left(\frac{p}{1-p}\right) = \mathbf{w}^T \mathbf{x}$$

- **Coefficient Interpretation:** For a 1-unit increase in feature x_j, the **log-odds** increase by w_j, meaning the **odds multiply by e^{w_j}**.

Binary Cross-Entropy Cost Function (Log-Loss):

$$J(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \ln(\hat{y}^{(i)}) + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}) \right]$$

- Penalizes confident incorrect predictions with infinite loss.

3.2 Decision Trees (CART - Classification and Regression Trees)

A non-parametric hierarchical model that recursively partitions the feature space into orthogonal rectangular regions.

Splitting Criteria:

- **Gini Impurity (Default in Scikit-Learn):**
$$I_G(t) = 1 - \sum_{i=1}^C p_i^2$$
Measures probability of misclassifying a randomly chosen element. (Gini = 0 for pure node).

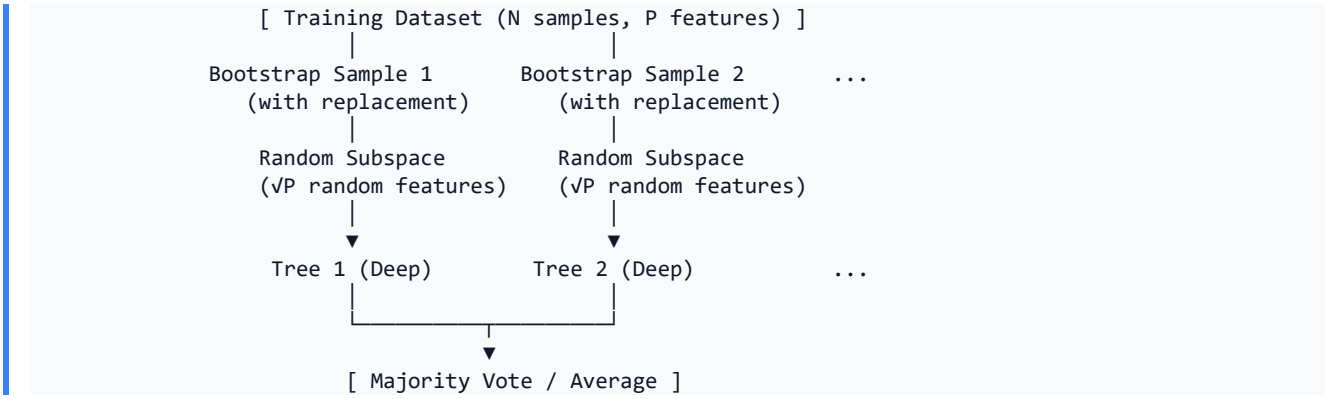
- **Entropy & Information Gain:**
$$H(t) = -\sum_{i=1}^C p_i \log_2(p_i), \quad \text{Gain} = H(\text{Parent}) - \sum \frac{N_j}{N} H(\text{Child}_j)$$

Overfitting Mitigation (Hyperparameters):

- `max_depth`: Limits tree depth.
- `min_samples_split`: Minimum samples required to split an internal node.
- `min_samples_leaf`: Minimum samples required to exist at a leaf node.

3.3 Random Forests (Ensemble Bagging)

A Random Forest constructs an ensemble of B independent Decision Trees to drastically reduce variance without increasing bias.



Why Random Forests Work (Decorrelation):

- **Bootstrap Aggregating (Bagging):** Each tree is trained on a random sample of size N drawn **with replacement** (each bootstrap sample contains $\approx 63.2\%$ of original data; the remaining 36.8% is the **Out-of-Bag (OOB)** sample used for internal validation).
- **Random Subspace Method:** At every single node split, only a random subset of features (typically $\sqrt{P}$ for classification, $\frac{P}{3}$ for regression) is considered. This **decorrelates** the trees, ensuring strong features don't dominate every tree.

Part 4: Unsupervised Learning: K-Means Clustering

4.1 The K-Means Algorithm (Lloyd's Algorithm)

Partitions N unlabeled observations into K predefined, non-overlapping clusters:

- **Initialization:** Randomly pick K initial centroid positions $\mu_1, \dots, \mu_K$.
- **Assignment Step:** Assign each data point x_i to its nearest centroid based on Euclidean distance:
$$c^{(i)} = \arg\min_j |x^{(i)} - \mu_j|^2$$
- **Update Step:** Recalculate centroids as the mathematical mean of all assigned points:
$$\mu_j = \frac{1}{|S_j|} \sum_{i \in S_j} x^{(i)}$$
- **Convergence:** Repeat steps 2 and 3 until centroids stop moving or reach maximum iterations.
- **Objective Function (Inertia / WCSS):**
$$\text{WCSS} = \sum_{j=1}^K \sum_{i \in S_j} |x^{(i)} - \mu_j|^2$$

4.2 Finding the Optimal K

- **The Elbow Method:** Plot WCSS (Inertia) against K. Look for the "elbow" point where adding more clusters yields diminishing returns in variance reduction.
- **Silhouette Coefficient:**
$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (-1 \le s(i) \le +1)$$
$a(i)$: Mean intra-cluster distance (cohesion).
$b(i)$: Mean nearest-cluster distance (separation).
Score near $+1$ indicates well-separated, dense clusters; score near 0 indicates overlapping clusters.

> *[!WARNING]*
> ***K-Means Limitations:*** *Assumes clusters are spherical and have similar variances. Extremely sensitive to outliers and requires all features to be standardized (`StandardScaler`) beforehand!*

Part 5: Validation Governance, Overfitting & Data Leakage

5.1 The Bias-Variance Decomposition

$$\text{Expected Test Error} = \text{Bias}^2 + \text{Variance} + \sigma^2 \text{ (Irreducible Error)}$$

- **High Bias (Underfitting):** Model is too simplistic (e.g., fitting a straight line to quadratic data). High error on both training and test data.
- **High Variance (Overfitting):** Model memorizes training noise (e.g., deep unpruned decision tree). Low training error, but high test error.

5.2 Cross-Validation Protocols

- **K-Fold Cross-Validation:** Dataset is partitioned into K equal folds. Model is trained on $K-1$ folds and evaluated on the remaining fold, rotating K times.
- **Stratified K-Fold:** Guarantees that each fold preserves the exact class proportion as the full dataset. **Mandatory for imbalanced classification!**
- **Time-Series Split (Walk-Forward):** Never shuffle time-series data! Folds must respect temporal order (Train on past, test on future).

5.3 Data Leakage: The Silent Model Killer

Data Leakage occurs when information from outside the training dataset is inadvertently used to create the model, producing unrealistically high test metrics that collapse in production.

Common Leakage Sources:

- **Preprocessing Leakage (Global Scaling):** Fitting `StandardScaler()` or `SimpleImputer()` on the **entire dataset** before splitting!
  Fix: Fit preprocessing transformers **strictly on the training fold only**, then transform the validation/test folds.

- **Target Leakage:** Including a feature that is a proxy for the target or only recorded *after* the event occurs (e.g., including `customer_refund_timestamp` to predict customer returns).

The Scikit-Learn Solution: Pipeline

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

# Encapsulating scaler + model inside Pipeline guarantees scaler fits ONLY on train folds during cross-validation!
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=100, random_state=42))
])
```

Part 6: Production Python Machine Learning Implementation

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, roc_auc_score

# 1. Generate Synthetic Imbalanced Dataset (e.g., Customer Churn)
X, y = make_classification(
    n_samples=5000,
    n_features=15,
    n_informative=10,
```

```
    weights=[0.85, 0.15], # 15% minority class
    random_state=42
)

# 2. Strict Stratified Train-Test Split (Holdout Test Set)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

# 3. Leak-Free Production Pipeline
clf_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('rf', RandomForestClassifier(
        n_estimators=150,
        max_depth=8,
        min_samples_leaf=5,
        class_weight='balanced', # Mitigates class imbalance
        random_state=42
    ))
])

# 4. Stratified K-Fold Cross-Validation on Training Set Only
cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(clf_pipeline, X_train, y_train, cv=cv_strategy, scoring='roc_auc')

print(f"5-Fold CV ROC-AUC: {np.mean(cv_scores):.4f} (+/- {np.std(cv_scores):.4f})")

# 5. Final Evaluation on Holdout Test Set
clf_pipeline.fit(X_train, y_train)
y_pred = clf_pipeline.predict(X_test)
y_pred_proba = clf_pipeline.predict_proba(X_test)[:, 1]

print("\nHoldout Test Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Retained', 'Churned']))
print(f"Holdout ROC-AUC Score: {roc_auc_score(y_test, y_pred_proba):.4f}")
```

Part 7: Structured Interview Question Bank

Q1: Why is feature scaling (e.g., StandardScaler) required for Logistic Regression and K-Means, but completely unnecessary for Decision Trees and Random Forests?

Answer:

- **Distance- & Gradient-Based Models (Logistic Regression, K-Means, SVM, Neural Networks):** Calculate Euclidean distances or use gradient descent optimization. If feature A is on scale $[0, 1]$ and feature B is on scale $[0, 1000000]$, feature B completely dominates distance metrics and causes elongated, oscillating gradient updates.
- **Tree-Based Models (Decision Trees, Random Forests, XGBoost):** Are scale-invariant because splits evaluate **monotonic rank order** along individual axes ($x_j \le \text{threshold}$). Multiplying a feature by 1,000 does not alter the relative ordering of values, so tree splits remain 100% identical.

Q2: How does L1 (Lasso) regularization perform feature selection, whereas L2 (Ridge) does not?

Answer:
Geometrically, L1 regularization adds a penalty proportional to the absolute values of weights ($\sum |w_j|$), which creates a **diamond-shaped constraint boundary** with sharp corners on the coordinate axes. The elliptical

contours of the MSE loss function frequently intersect the L1 constraint region directly at these axis corners, setting non-informative feature weights **strictly to zero**. In contrast, L2 regularization creates a smooth **circular constraint** ($\sum w_j^2$); intersections occur along curves, shrinking weights close to zero but never reaching exact zero.

Q3: What is the "Out-of-Bag" (OOB) error in a Random Forest, and how does it save computational time?

Answer:
Because each tree in a Random Forest is trained on a bootstrap sample drawn with replacement, approximately $1 - 1/e \approx 63.2\%$ of the original training instances are included in any given tree's training set. The remaining $\approx 36.8\%$ of instances are **Out-of-Bag (OOB)** for that tree. By passing each training instance through all trees for which it was OOB and computing the ensemble prediction, the model obtains an unbiased validation score without needing to execute a separate, computationally expensive K-fold cross-validation loop.

Part 8: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  MACHINE LEARNING FUNDAMENTALS CHEAT SHEET       │
├──────────────────────────────────────────────────────────────────┤
│ 1. REGRESSION & CLASSIFICATION:                                  │
│    - Linear Reg : Continuous target. OLS minimizes MSE. Gauss-Markov.
│    - Logistic   : Binary target. Sigmoid σ(z) maps to (0, 1). Log-Loss.
│    - Ridge (L2) : Shrinks weights; handles collinearity.         │
│    - Lasso (L1) : Sets weights to zero; feature selection.       │
├──────────────────────────────────────────────────────────────────┤
│ 2. ENSEMBLES & TREES:                                            │
│    - Decision Tree: CART uses Gini or Entropy. High variance.    │
│    - Random Forest: Bagging + Random Subspace (√P features).     │
│                     Reduces variance; decorrelates trees; OOB score.
├──────────────────────────────────────────────────────────────────┤
│ 3. UNSUPERVISED:                                                 │
│    - K-Means: Minimizes WCSS (Inertia). Requires feature scaling!│
│    - Elbow Method & Silhouette Score (-1 to +1) find optimal K.  │
├──────────────────────────────────────────────────────────────────┤
│ 4. VALIDATION GOVERNANCE:                                        │
│    - High Bias = Underfitting | High Variance = Overfitting.     │
│    - Stratified K-Fold: Preserves minority class proportions.    │
│    - Data Leakage: NEVER scale/impute on full dataset! Use Pipeline.
└──────────────────────────────────────────────────────────────────┘
```

Part 9: 100-Mark Self-Assessment Examination

Section A: Conceptual Foundations (20 Marks — 4 marks each)

Q1: Which of the following models is inherently invariant to monotonic feature scaling?

(a) Logistic Regression

(b) Support Vector Machines (SVM)

(c) Random Forest Classifier

(d) K-Means Clustering

Q2: What is the primary mathematical mechanism by which Lasso (L1) regularization sets coefficients to zero?

(a) Zero-division error in gradient descent.

(b) Intersection of the loss function contours with the sharp corners of the L1 diamond constraint region on the axes.

(c) Removing features with negative correlation.

(d) Applying a softmax filter across all weights.

Q3: In a binary classification problem with severe class imbalance (98% negative, 2% positive), which metric should be prioritized during model selection?

(a) Overall Accuracy

(b) Precision-Recall AUC (PR-AUC) or F1-Score

(c) Mean Squared Error (MSE)

(d) R-Squared (R^2)

Q4: What percentage of training data is included, on average, in a single bootstrap sample of size N drawn with replacement?

(a) 50.0%

(b) 63.2%

(c) 80.0%

(d) 100.0%

Q5: In K-Means clustering, what does a Silhouette Coefficient of $+0.85$ indicate?

(a) The model is severely overfitted.

(b) The clusters are dense and well-separated from neighboring clusters.

(c) The optimal number of clusters is $K = 1$.

(d) All data points are equidistant from the origin.

Section B: Mathematics & Algorithmic Tracing (25 Marks)

Q6 (10 Marks): Given a Logistic Regression model with intercept $w_0 = -2.0$ and feature weight $w_1 = 0.5$ for feature x_1:

- Write down the equation for the predicted probability $\hat{p} = P(y=1 \mid x_1)$.
- Calculate the exact predicted probability $\hat{p}$ when $x_1 = 4.0$. (Note: $e^0 = 1$).
- With a standard classification threshold of 0.5, what is the predicted class?

Q7 (15 Marks): Calculate the **Gini Impurity** for a decision tree node containing 40 samples of Class A and 10 samples of Class B. Show all calculation steps.

Section C: Implementation Challenges (35 Marks)

Q8 (20 Marks): Write a Python function using Scikit-Learn that creates an end-to-end `Pipeline` containing:

- `SimpleImputer` using the `'median'` strategy.
- `StandardScaler`.
- `LogisticRegression` with L2 penalty and `class_weight='balanced'`.

Perform a 5-fold Stratified Cross-Validation on given `(X, y)` and return the mean ROC-AUC score.

Q9 (15 Marks): Explain how "Target Leakage" occurs in a customer churn prediction model if a feature `days_since_cancellation_survey` is included. How does this corrupt model performance when deployed?

Section D: Enterprise Failure & Tuning Scenario (20 Marks)

Q10 (20 Marks): A Data Science Engineer builds a Random Forest model on 500,000 credit card applications to predict credit default. The model achieves 99.4% accuracy on training data, but drops to 84.1% on the test set.

- Diagnose the exact machine learning problem (Bias vs. Variance). (6 marks)
- Identify three specific hyperparameters of `RandomForestClassifier` that should be tuned to resolve this issue, explaining the mechanism of each. (14 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (c)** Random Forest Classifier (Tree splits depend strictly on rank ordering).
- **Q2: (b)** Diamond constraint intersection with loss contours on axes.
- **Q3: (b)** PR-AUC or F1-Score (Accuracy is meaningless on imbalanced data).
- **Q4: (b)** $\approx 63.2%$ ($1 - 1/e$).
- **Q5: (b)** Dense, well-separated clusters.

Section B Solutions

Q6 Solution:

- $z = -2.0 + 0.5(x_1) \implies \hat{p} = \frac{1}{1 + e^{-(-2.0 + 0.5 x_1)}}$.

- For $x_1 = 4.0$: $z = -2.0 + 0.5(4.0) = -2.0 + 2.0 = 0$.
$\hat{p} = \frac{1}{1 + e^0} = \frac{1}{1 + 1} = \frac{1}{2} = 0.50$ (50%).

- With threshold 0.5, boundary cases typically default to Class 1 (or depend on implementation; $\hat{p} \ge 0.5 \to$ Class 1).

Q7 Solution:

- Total samples $N = 40 + 10 = 50$.
- $p_A = \frac{40}{50} = 0.8$, $p_B = \frac{10}{50} = 0.2$.
- $\text{Gini} = 1 - (p_A^2 + p_B^2) = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 1 - 0.68 = 0.32$.

Section C Solutions

Q8 Solution:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
import numpy as np

def evaluate_leak_free_pipeline(X, y):
    pipe = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler()),
        ('classifier', LogisticRegression(penalty='l2', class_weight='balanced', max_iter=1000))
    ])
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(pipe, X, y, cv=cv, scoring='roc_auc')
    return np.mean(scores)
```

Q9 Solution:

- `days_since_cancellation_survey` is populated only *after* a customer initiates account cancellation. In production, when the model scores active, current customers to prevent churn, this feature does not yet exist (or is NULL).
- In offline training, the model learns that having a non-null survey date guarantees churn ($100%$ correlation), ignoring legitimate early-warning signals. In production, the model fails completely.

Section D Solutions

Q10 Solution:

- **Diagnosis: Severe Overfitting (High Variance)**. The trees were allowed to grow unconstrained, memorizing training noise and failing to generalize to unseen test instances.
- **Hyperparameter Remediations:**

max_depth: Constrain maximum tree depth (e.g., set to 8–12) to prevent trees from isolating single noisy training samples.
min_samples_leaf: Increase minimum leaf samples (e.g., set to 10–50), forcing leaves to contain generalizable sample sizes.
max_features: Reduce the feature subspace fraction (e.g., set to `'sqrt'` or `0.3`) to increase tree decorrelation across the ensemble.

Day 15: Model Evaluation, Feature Engineering & Business Systems

Classification & Regression Metrics, Imbalanced Learning, Recommendation Systems & Business Alignment

---

Part 1: Classification Metrics & Cost-Utility Alignment

1.1 The Confusion Matrix & Derived Proportions

```
                          ACTUAL POSITIVE (1)         ACTUAL NEGATIVE (0)
PREDICTED POSITIVE (1)    True Positive (TP)          False Positive (FP) [Type I]
PREDICTED NEGATIVE (0)    False Negative (FN) [Type II] True Negative (TN)
```

Metric	Formula	Plain English Meaning	Canonical Business Use Case
Precision	$\frac{TP}{TP + FP}$	"Out of all instances we flagged as positive, how many were actually positive?"	Spam detection, Video recommendations, Legal document discovery.
Recall (Sensitivity)	$\frac{TP}{TP + FN}$	"Out of all actual positive instances, how many did we successfully capture?"	Fraud detection, Cancer screening, Factory defect detection.
Specificity	$\frac{TN}{TN + FP}$	"Out of all actual negative instances, how many did we correctly identify as negative?"	Drug clinical trials, minimizing false alarms in monitoring.
F_1-Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of Precision and Recall. Equal weighting.	General classification benchmark on moderately imbalanced data.
F_β-Score	$(1 + \beta^2) \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \text{Precision} + \text{Recall}}$	Weighted harmonic mean. $\beta=2$ weights Recall $2\times$ higher than Precision.	Healthcare diagnostics (F_2), High-value fraud alert systems.

1.2 ROC-AUC vs. Precision-Recall AUC (PR-AUC)

Receiver Operating Characteristic (ROC) Curve

Plots **True Positive Rate (Recall)** against **False Positive Rate ($\text{FPR} = \frac{FP}{FP + TN}$)** across all possible probability thresholds ($0.0 \to 1.0$).

- **ROC-AUC Score:** Represents the probability that the classifier will rank a randomly chosen positive instance higher than a randomly chosen negative instance. ($0.5 = \text{random guessing}$, $1.0 = \text{perfect}$).

The Severe Imbalance Trap: Why ROC-AUC Misleads

In severely imbalanced datasets (e.g., 99.9% Negative, 0.1% Positive in credit card fraud):

- Because TN is massive (e.g., $999,000$), a large surge of $10,000$ False Positives yields a tiny $\text{FPR} = \frac{10000}{10000 + 999000} \approx 0.0099$. The ROC curve barely shifts, reporting an artificially optimistic AUC of 0.98!
- **The Solution: PR-AUC (Precision-Recall Curve):**
  Plots **Precision** against **Recall**. Because neither metric includes TN in its denominator, PR-AUC focuses exclusively on the minority class. In severe imbalance, **PR-AUC is mandatory**.

1.3 Cost-Utility Matrix & Decision Threshold Tuning

The standard decision threshold is 0.5. In business applications, this is almost always suboptimal due to **asymmetric error costs**:

$$\text{Total Expected Cost} = C(FP) \cdot FP + C(FN) \cdot FN$$

Example (Loan Default Risk):

- Cost of FN (Uncaught default): $$50,000$ loss.
- Cost of FP (Denying a creditworthy customer): $$2,000$ lost interest margin.
- **Action:** Shift the classification threshold down from 0.50 to **0.18**. This trades off higher False Positives (more manual reviews) to drastically minimize catastrophic False Negatives!

Part 2: Regression Evaluation Metrics

2.1 The Metric Spectrum

Metric	Formula	Sensitivity to Outliers	Interpretable in Target Units?	Mathematical Penalty
Mean Absolute Error (MAE)	$\frac{1}{n}\sum \|y_i - \hat{y}_i\|$	**Low** (Robust)	**Yes** (Exact same units as y)	Linear penalty: a $100 error is worth exactly 10 $\times$ a $10 error.
Root Mean Squared Error (RMSE)	$\sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}$	**Very High** (Penalizes extremes)	**Yes** (Exact same units as y)	Quadratic penalty: a $100 error is worth 100 $\times$ a $10 error!
Mean Absolute Percentage Error (MAPE)	$\frac{100%}{n}\sum \left\|\frac{y_i - \hat{y}_i}{y_i}\right\|$	High when y_i is small	**Yes** (Percentage %)	Fails / divides by zero if actual $y_i = 0$.
R^2 (Coefficient of Determination)	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	High	**No** (Dimensionless ratio ≤ 1.0)	Fraction of target variance explained by features.

R^2 vs. Adjusted R^2:

Adding any new feature to a regression model (even random noise) will **never decrease R^2**.

Adjusted R^2 penalizes models for adding non-informative features:
$$R^2_{\text{adj}} = 1 - \left[\frac{(1 - R^2)(n - 1)}{n - p - 1} \right]$$

- n: Number of observations, p: Number of predictors.

Part 3: Advanced Feature Engineering

3.1 Categorical Encoding Strategies

- **One-Hot Encoding:** Creates binary columns for each unique category.
Danger: High-cardinality features (e.g., `zip_code` with 10,000 values) cause extreme dimensionality explosion and sparse matrices, slowing tree training.

- **Target (Mean) Encoding:** Replaces each categorical value with the average target value for that category:
$$\hat{x}_k = \bar{y}_k$$
Danger: Severe Data Leakage & Overfitting!
Remediation: Must apply **Out-of-Fold (OOF) Target Encoding** with m-estimate smoothing:
$$S_i = \frac{n_i \cdot \bar{y}*i + m \cdot \bar{y}*{\text{global}}}{n_i + m}$$

3.2 Numerical Transformations

- **Log Transformation ($y' = \ln(1 + x)$):** Compresses long right tails, converting multiplicative relationships into additive ones and stabilizing variance (homoscedasticity).
- **Scaling Techniques:**
`StandardScaler`: Transforms to $\mu=0, \sigma=1$. Sensitive to outliers.
`RobustScaler`: Centers using Median and scales by IQR ($Q_3 - Q_1$). **Immune to extreme outliers**.

Part 4: Production Class Imbalance Remediation

4.1 Resampling Protocols

- **Random Undersampling:** Discards majority class rows. (Fast, but wastes valuable training data).
- **SMOTE (Synthetic Minority Over-sampling Technique):** Synthesizes new minority examples by interpolating between nearest neighbors in feature space:
$$x_{\text{new}} = x_i + \lambda (x_{zi} - x_i) \quad \text{where } \lambda \sim U(0, 1)$$

> *[!CAUTION]*
> ***Production Rule:*** *Never apply SMOTE or oversampling to the test or validation datasets! Resampling must be applied* ***strictly inside training folds****.*

4.2 Cost-Sensitive Learning (class_weight='balanced')

Instead of artificially generating synthetic rows, modify the loss function during gradient descent:
$$w_c = \frac{N}{K \cdot N_c}$$

- Minorities receive exponentially higher loss penalties when misclassified, forcing the decision boundary to protect them.

Part 5: Recommendation Systems Architecture

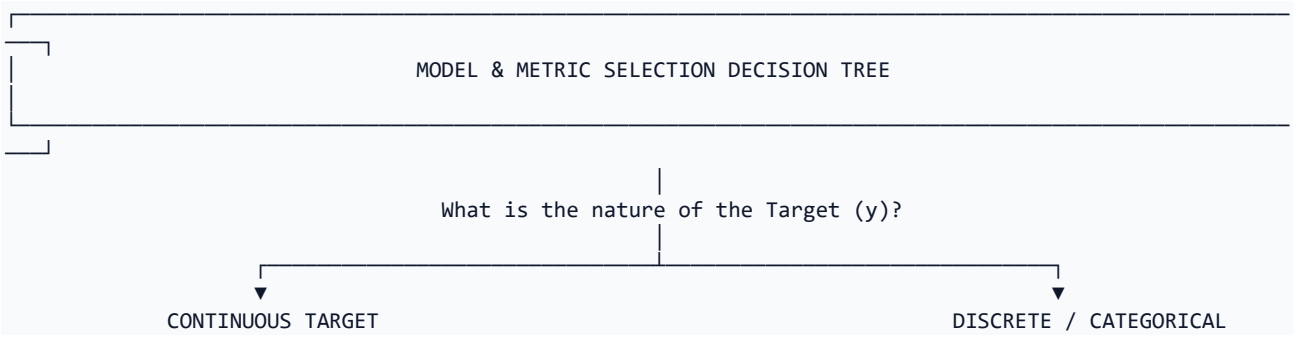
```
┌──────────────────────────────────────────────────────────────────────┐
│                    RECOMMENDATION SYSTEM PARADIGMS                   │
├──────────────────────────────────────────────────────────────────────┤
│ 1. CONTENT-BASED FILTERING                                           │
│    - Recommends items similar to what user liked in the past.        │
│    - Features: Item metadata (Genre, Actor, Keywords, TF-IDF).       │
│    - Metric: Cosine Similarity between Item Vector and User Profile. │
│    - Pros: No cold-start for new items; transparent explanations.    │
│    - Cons: Cannot recommend outside user's existing bubble (serendipity)│
├──────────────────────────────────────────────────────────────────────┤
│ 2. COLLABORATIVE FILTERING (CF)                                      │
│    - "Users who agreed in the past will agree in the future."        │
│    - Memory-Based: User-User CF vs. Item-Item CF.                    │
│    - Model-Based: Matrix Factorization (SVD, Alternating Least Squares)│
│      R ≈ U (User Latent Matrix) × V^T (Item Latent Matrix)           │
│    - Pros: Learns latent patterns; domain-independent.               │
│    - Cons: Severe Cold-Start Problem (Cannot score new users/items). │
└──────────────────────────────────────────────────────────────────────┘
```

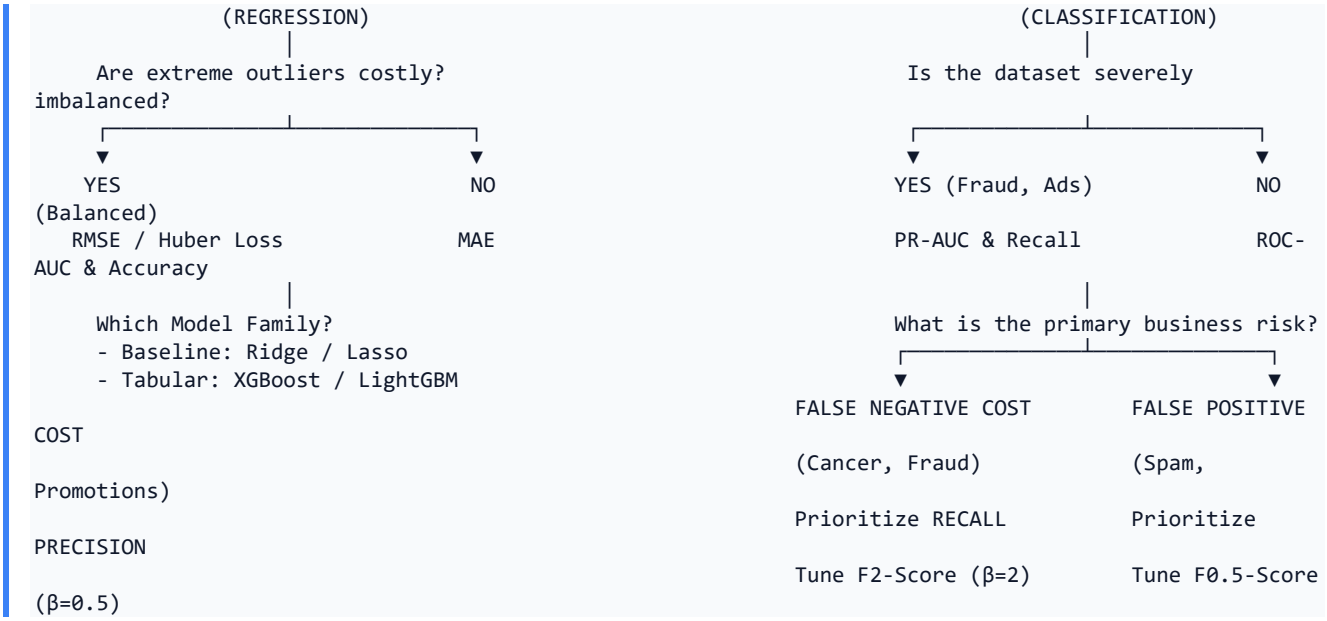
Cold-Start Remediation (Hybrid Architectures):

- **New User:** Collect explicit preferences during onboarding survey; fall back to popularity-based top items.
- **New Item:** Use Content-Based metadata embeddings until sufficient interaction clicks are logged.

Part 6: Required Deliverable: Executive Decision Framework for Model & Metric Selection

Step-by-Step Selection Decision Matrix





Part 7: Structured Interview Question Bank

Q1: In an email spam classification model, why is Precision far more important than Recall?

Answer:

- A False Positive (FP) means a critical business email (e.g., job offer, client invoice) is misclassified as spam and moved to the spam folder, causing immediate business harm.
- A False Negative (FN) means an unwanted spam email lands in the primary inbox, causing minor inconvenience.
- Because the cost of a False Positive is orders of magnitude higher than a False Negative, the spam classifier must maximize **Precision** (e.g., $> 99.5%$) by operating at a high decision threshold, ensuring that an email is only marked as spam if the model is virtually certain.

Q2: Why is the harmonic mean used for the F_1-score instead of the simple arithmetic mean?

Answer:
The arithmetic mean treats Precision and Recall independently; a classifier that predicts everything as positive on a dataset with 50% positives would get Recall = 1.0 and Precision = 0.5, giving an arithmetic mean of 0.75. The **harmonic mean** ($\frac{2}{\frac{1}{P} + \frac{1}{R}}$) penalizes extreme trade-offs heavily. If either Precision or Recall approaches 0, the harmonic mean collapses towards 0, ensuring that a high F_1-score is achieved only when **both** precision and recall are robust.

Q3: What is Matrix Factorization in Collaborative Filtering, and how does Singular Value Decomposition (SVD) work?

Answer:
Given a sparse User-Item interaction rating matrix R of shape $(m \times n)$, Matrix Factorization decomposes R into the product of two lower-rank dense matrices:
$$R \approx U \cdot V^T$$

- U: User latent factor matrix of shape $(m \times k)$.
- V: Item latent factor matrix of shape $(n \times k)$.
- k: Latent dimensionality (e.g., $k = 50$).

The dot product of user vector $\mathbf{u}_i$ and item vector $\mathbf{v}_j$ predicts the rating for unobserved pairs. SVD minimizes the reconstruction error over observed ratings while adding L2 regularization to prevent overfitting on unobserved ratings.

Part 8: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  MODEL EVALUATION & METRICS CHEAT SHEET          │
├──────────────────────────────────────────────────────────────────┤
│ 1. CLASSIFICATION:                                               │
│    - Precision: TP / (TP + FP). Cost of FP is high.              │
│    - Recall   : TP / (TP + FN). Cost of FN is high (Fraud/Health).│
│    - F1-Score : Harmonic mean: 2 * (P * R) / (P + R).            │
│    - Imbalance: Use PR-AUC instead of ROC-AUC.                   │
│    - Threshold Tuning: Optimize threshold = argmin(C_FP*FP + C_FN*FN).│
├──────────────────────────────────────────────────────────────────┤
│ 2. REGRESSION:                                                   │
│    - MAE  : Linear penalty; robust to outliers; interpretable units.│
│    - RMSE : Quadratic penalty; heavily punishes large errors.    │
│    - Adj R²: Penalizes useless feature additions: 1 - [(1-R²)(n-1)/(n-p-1)]│
├──────────────────────────────────────────────────────────────────┤
│ 3. IMBALANCE:                                                    │
│    - class_weight='balanced': Cost-sensitive loss weighting.     │
│    - SMOTE: Synthesizes minority rows. NEVER apply to test set!  │
├──────────────────────────────────────────────────────────────────┤
│ 4. RECOMMENDERS:                                                 │
│    - Content-Based: Cosine similarity on item metadata. No item cold-start.│
│    - Collaborative: User-Item interactions. SVD Matrix Factorization.│
│      Suffers from user/item cold-start problem.                  │
└──────────────────────────────────────────────────────────────────┘
```

Part 9: 100-Mark Self-Assessment Examination

Section A: Conceptual Foundations (20 Marks — 4 marks each)

Q1: What is the primary reason for preferring PR-AUC over ROC-AUC when evaluating a fraud detection model with 0.05% positive cases?

(a) ROC-AUC is computationally too slow to calculate.

(b) The massive number of True Negatives artificially depresses the False Positive Rate, masking poor precision in ROC-AUC.

(c) PR-AUC does not require a confusion matrix.

(d) ROC-AUC can only be evaluated on binary features.

Q2: If a model achieves a Mean Absolute Percentage Error (MAPE) of undefined / infinite error, what was the most likely characteristic of the target test data?

(a) The target feature contained negative numbers.

(b) The target feature contained one or more true values equal to zero ($y_i = 0$).

(c) The test set was too large.

(d) The model was a decision tree.

Q3: Which metric would you optimize when designing a factory assembly line machine vision system where stopping the line for a false defect (FP) costs $5, but letting a broken part pass (FN) causes a $50,000 catastrophic engine failure?

(a) Precision

(b) Specificity

(c) Recall (or F_2-Score)

(d) Accuracy

Q4: In Collaborative Filtering recommendation systems, what does the "Cold-Start Problem" refer to?

(a) Slow server startup times during winter months.

(b) Inability to generate accurate recommendations for brand-new users or brand-new items with zero historical interaction data.

(c) Running out of memory during SVD matrix factorization.

(d) Converting categorical features to one-hot encoding.

Q5: What is the fundamental difference between MAE and RMSE in terms of error penalization?

(a) MAE penalizes errors exponentially; RMSE penalizes errors linearly.

(b) MAE penalizes errors linearly; RMSE squares errors, penalizing large outliers much more severely.

(c) RMSE is always smaller than MAE.

(d) MAE requires calculating logarithms.

Section B: Mathematics & Metric Computation (25 Marks)

Q6 (10 Marks): A diagnostic cancer detection model evaluated on 1,000 biopsy samples produces the following confusion matrix:

- True Positives (TP): 80
- False Positives (FP): 20
- False Negatives (FN): 10
- True Negatives (TN): 890
- Calculate the **Precision** of the model. (3 marks)
- Calculate the **Recall** of the model. (3 marks)
- Calculate the **F_1-Score** of the model. (4 marks)

Q7 (15 Marks): Calculate the R^2 score for a regression model given:

- Total Sum of Squares ($SS_{\text{tot}} = \sum (y_i - \bar{y})^2$): 500
- Residual Sum of Squares ($SS_{\text{res}} = \sum (y_i - \hat{y}_i)^2$): 125

Explain the exact business interpretation of this R^2 value.

Section C: Implementation & Pipeline Challenges (35 Marks)

Q8 (20 Marks): Write a Python function `find_optimal_threshold(y_true, y_probs, cost_fp, cost_fn) -> float` that evaluates candidate classification probability thresholds from 0.05 to 0.95 in increments of 0.05 and returns the threshold that **minimizes total financial loss**.

Q9 (15 Marks): Write a Python snippet demonstrating **Out-of-Fold (OOF) Target Encoding** using `KFold` on a training dataset to encode a high-cardinality categorical column `merchant_id` without data leakage.

Section D: Enterprise Architectural Scenario (20 Marks)

Q10 (20 Marks): A streaming video entertainment company (like Netflix) wants to build an automated recommendation engine.

- Design a **Hybrid Recommendation Architecture** combining Content-Based and Collaborative Filtering that mitigates the Cold-Start problem for new movies and new subscribers. (10 marks)
- Define two offline metrics and two online A/B testing metrics to measure recommender success. (10 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** Massive True Negatives artificially suppress FPR, making ROC-AUC deceptively high.
- **Q2: (b)** Division by zero ($y_i = 0$) in the MAPE denominator.
- **Q3: (c)** Recall / F_2-Score (Minimizing catastrophic False Negatives).
- **Q4: (b)** Inability to score new users or new items lacking interaction history.
- **Q5: (b)** MAE is linear; RMSE is quadratic, penalizing outliers heavily.

Section B Solutions

Q6 Solution:

- Precision: $\frac{TP}{TP + FP} = \frac{80}{80 + 20} = \frac{80}{100} = 0.80$ (80.0%).
- Recall: $\frac{TP}{TP + FN} = \frac{80}{80 + 10} = \frac{80}{90} \approx 0.8889$ (88.89%).
- F_1-Score: $2 \cdot \frac{P \cdot R}{P + R} = 2 \cdot \frac{0.80 \cdot 0.8889}{0.80 + 0.8889} = \frac{1.4222}{1.6889} \approx 0.8421$ (84.21%).

Q7 Solution:

- $R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{125}{500} = 1 - 0.25 = 0.75$ (75.0%).
- **Business Interpretation:** Exactly $75%$ of the total variance in the dependent target variable is successfully explained and captured by the independent features in the model, with the remaining $25%$ attributable to unexplained variance or noise.

Section C Solutions

Q8 Solution:

```
import numpy as np
from sklearn.metrics import confusion_matrix

def find_optimal_threshold(y_true, y_probs, cost_fp, cost_fn) -> float:
    best_threshold = 0.5
    min_cost = float('inf')

    for thresh in np.arange(0.05, 1.00, 0.05):
        y_pred = (y_probs >= thresh).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
        total_cost = (fp * cost_fp) + (fn * cost_fn)

        if total_cost < min_cost:
            min_cost = total_cost
            best_threshold = round(thresh, 2)

    return best_threshold
```

Q9 Solution:

```
import pandas as pd
from sklearn.model_selection import KFold

def oof_target_encode(df, cat_col, target_col, n_splits=5):
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)
    df[f"{cat_col}_encoded"] = 0.0
    global_mean = df[target_col].mean()

    for train_idx, val_idx in kf.split(df):
        train_fold = df.iloc[train_idx]
        val_fold = df.iloc[val_idx]

        # Calculate mean strictly on train fold
        means = train_fold.groupby(cat_col)[target_col].mean()
        # Map back to validation fold with fallback to global mean
        df.iloc[val_idx, df.columns.get_loc(f"{cat_col}_encoded")] = (
            val_fold[cat_col].map(means).fillna(global_mean)
        )
    return df
```

Section D Solutions

Q10 Solution:

- **Hybrid Architecture:**

Offline Model: Train an implicit Matrix Factorization model (e.g., SVD / ALS) on user viewing history to capture latent collaborative affinities.
Content-Based Fallback: For new movies (Item Cold-Start), generate text/video embeddings (genre, actors, summary) and compute cosine similarity against existing popular catalog items.
Onboarding Survey: For new subscribers (User Cold-Start), present 5 favorite genre cards; recommend top-rated movies in selected genres until 5 stream events are logged, then transition the user into the Collaborative Filtering pipeline.

- **Metrics:**

Offline Metrics: MAP@K (Mean Average Precision at K), NDCG@K (Normalized Discounted Cumulative Gain).
Online A/B Testing Metrics: Click-Through Rate (CTR) on recommended rows, 30-Day Subscriber Retention Rate, Total Streaming Watch Time (hours).

Day 16: Advanced SQL Interview Problem-Solving Masterclass

Ranking, Gaps & Islands, Running Totals, Retention Cohorts & Verbal Interview Communication

---

Part 1: The Senior Interview Communication Framework

When solving SQL live in technical interview rounds, **writing code in silence is a major failure mode**. Interviewers evaluate your ability to clarify ambiguity, anticipate edge cases, and reason through engine performance.

```
┌──────────────────────────────────────────────────────────────────┐
│              THE 5-STEP VERBAL INTERVIEW COMMUNICATION FLOW      │
├──────────────────────────────────────────────────────────────────┤
│ STEP 1: CLARIFY REQUIREMENTS & AMBIGUITY                         │
│  - "Are there ties? How should identical salaries be ranked?"    │
│  - "Can columns contain NULLs? How should missing dates be handled?" │
│  - "Should the output return NULL or an empty set if no match exists?" │
├──────────────────────────────────────────────────────────────────┤
│ STEP 2: STATE THE PROPOSED STRATEGY (BEFORE TYPING)              │
│  - "I will structure this using a CTE with DENSE_RANK() partitioned  │
│    by department, then filter in the outer query where rank <= 3."   │
├──────────────────────────────────────────────────────────────────┤
│ STEP 3: CODE MODULARLY WITH CLEAN ALIASES                        │
│  - Use uppercase keywords, explicit table aliases (e, d, o), and CTEs. │
├──────────────────────────────────────────────────────────────────┤
│ STEP 4: WALK THROUGH A CONCRETE EXAMPLE MENTALLY                 │
│  - Trace inputs with ties or nulls through your execution pipeline.  │
├──────────────────────────────────────────────────────────────────┤
│ STEP 5: PROACTIVELY DISCUSS COMPLEXITY & INDEXING                │
│  - "This window function requires a sort on (dept_id, salary DESC). A  │
│    composite B-tree index on those columns would eliminate the sort."  │
└──────────────────────────────────────────────────────────────────┘
```

Part 2: The 15 High-Frequency SQL Interview Problems

Problem 1: Second-Highest Salary

Problem Statement: Write a SQL query to find the second-highest distinct salary from an `employees` table. If there is no second-highest salary (e.g., table has 0 or 1 row, or all employees earn the exact same salary), the query must return `NULL`.

Schema: `employees (emp_id INT, emp_name VARCHAR, salary DECIMAL)`

Verbal Explanation Script:

> *"To ensure that ties don't distort our ranking, I'll use DENSE_RANK() ordered by salary descending inside a CTE. DENSE_RANK() guarantees that if two employees tie for the top salary of $150k, both receive rank 1, and the next distinct salary receives rank 2. Furthermore, the requirement specifies returning a scalar NULL rather than*

> *an empty result set if fewer than two distinct salaries exist. I'll achieve this by wrapping the subquery in a scalar SELECT COALESCE(...)."*

Production Solution:

```
WITH RankedSalaries AS (
    SELECT
        salary,
        DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
)
SELECT COALESCE(
    (SELECT DISTINCT salary FROM RankedSalaries WHERE rnk = 2),
    NULL
) AS SecondHighestSalary;
```

Problem 2: Top 3 Highest Earners per Department

Problem Statement: Find the employees who earn the top 3 highest distinct salaries in each department. If there are ties for 3rd place, return all tied employees.

Schema: `employees (emp_id, emp_name, salary, dept_id)`, `departments (dept_id, dept_name)`

Verbal Explanation Script:

> *"This is a classic Top-N per group problem. We must partition by department and order by salary descending. Because tied salaries should both be included without skipping ranks, DENSE_RANK() is mathematically required. I'll compute this in a CTE called RankedDeptEmployees, join with departments for readability, and then filter in the outer WHERE clause where salary_rank <= 3."*

Production Solution:

```
WITH RankedDeptEmployees AS (
    SELECT
        e.emp_id,
        e.emp_name,
        e.salary,
        e.dept_id,
        DENSE_RANK() OVER (
            PARTITION BY e.dept_id
            ORDER BY e.salary DESC
        ) AS salary_rank
    FROM employees e
)
SELECT
    d.dept_name,
    r.emp_name,
    r.salary,
    r.salary_rank
FROM RankedDeptEmployees r
INNER JOIN departments d ON r.dept_id = d.dept_id
WHERE r.salary_rank <= 3
ORDER BY d.dept_name, r.salary_rank, r.emp_name;
```

Problem 3: Three or More Consecutive Logins (Gaps & Islands)

Problem Statement: Given a table of user login activities, identify all users who logged in for at least 3 consecutive calendar days.

Schema: `user_logins (user_id INT, login_date DATE)`

Verbal Explanation Script:

> *"First, users might log in multiple times in a single day, so Step 1 is to deduplicate using SELECT DISTINCT user_id, CAST(login_date AS DATE).*
> *Next, I apply the classic 'Difference of Sequences' Gaps and Islands technique: if dates are consecutive ($D, D+1, D+2$) and we assign a sequential ROW_NUMBER() ($1, 2, 3$), subtracting the row number from the login date yields a constant anchor date for that consecutive island. When a gap occurs, the anchor date changes.*
> *Finally, I group by user_id and island_date, filtering with HAVING COUNT(*) >= 3."*

Production Solution:

```
WITH CleanLogins AS (
    SELECT DISTINCT user_id, CAST(login_date AS DATE) AS log_date
    FROM user_logins
),
SequencedLogins AS (
    SELECT
        user_id,
        log_date,
        ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY log_date) AS rn,
        -- Subtracting rn days produces a constant date group for consecutive streaks
        log_date - CAST(ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY log_date) AS INT) * INTERVAL
'1 day' AS island_group
    FROM CleanLogins
)
SELECT
    user_id,
    MIN(log_date) AS streak_start,
    MAX(log_date) AS streak_end,
    COUNT(*) AS consecutive_days
FROM SequencedLogins
GROUP BY user_id, island_group
HAVING COUNT(*) >= 3
ORDER BY user_id, streak_start;
```

Problem 4: Missing Records & Unmatched Transactions (Anti-Join)

Problem Statement: Find all customers who registered in 2024 but have never completed a single transaction.

Schema: `customers (customer_id, customer_name, registration_date)`, `transactions (txn_id, customer_id, amount, status)`

Verbal Explanation Script:

> *"This requires an Anti-Join pattern. While a NOT IN subquery is common, it is hazardous if the subquery contains any NULLs, which causes NOT IN to return zero rows. I will use a LEFT JOIN on customer_id and filter where the transaction primary key IS NULL, combined with an explicit check on the transaction status."*

Production Solution:

```
SELECT
    c.customer_id,
    c.customer_name,
    c.registration_date
FROM customers c
LEFT JOIN transactions t
    ON c.customer_id = t.customer_id
    AND t.status = 'COMPLETED'
WHERE EXTRACT(YEAR FROM c.registration_date) = 2024
  AND t.txn_id IS NULL;
```

Problem 5: Department-wise Salary Running Total

Problem Statement: For each department, list employees ordered by their hire date, along with the running cumulative sum of salaries paid within that department.

Schema: `employees (emp_id, emp_name, dept_id, salary, hire_date)`

Verbal Explanation Script:

> *"We use an aggregate window function SUM(salary) OVER (...). We partition by dept_id and order by hire_date. To ensure that employees hired on the exact same date don't have their salaries lumped together prematurely by the default RANGE frame, I will explicitly specify the frame as ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW."*

Production Solution:

```
SELECT
    dept_id,
    emp_name,
    hire_date,
    salary,
    SUM(salary) OVER (
        PARTITION BY dept_id
        ORDER BY hire_date, emp_id
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_department_payroll
FROM employees
ORDER BY dept_id, hire_date;
```

Problem 6: Latest Transaction per Customer

Problem Statement: For each customer, display their customer ID, their most recent transaction date, and the transaction amount of that most recent transaction.

Schema: `transactions (txn_id, customer_id, txn_date, amount)`

Verbal Explanation Script:

> *"A simple GROUP BY customer_id with MAX(txn_date) gives the date, but it cannot fetch the corresponding amount without an extra join. The most performant and clean pattern is using ROW_NUMBER() partitioned by customer_id and ordered by txn_date DESC, txn_id DESC inside a CTE, then filtering for row_num = 1."*

Production Solution:

```
WITH RankedTransactions AS (
    SELECT
        customer_id,
        txn_date,
        amount,
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY txn_date DESC, txn_id DESC
        ) AS rn
    FROM transactions
)
SELECT
    customer_id,
    txn_date AS latest_txn_date,
    amount AS latest_txn_amount
FROM RankedTransactions
WHERE rn = 1;
```

Problem 7: Monthly Customer Retention & Churn Cohorts

Problem Statement: Calculate the Month-over-Month (MoM) Customer Retention Rate: For each month, what percentage of customers who placed an order in the previous month returned to place an order in the current month?

Schema: `orders (order_id, customer_id, order_date)`

Verbal Explanation Script:

> *"First, I map each order to its truncated month (DATE_TRUNC('month', order_date)). Then I isolate unique (customer_id, order_month) pairs. Using a self-join with an offset of exactly 1 month (current_month = prior_month + 1 month), I count retained returning customers. Finally, I divide active returning users by the total active users from the prior month."*

Production Solution:

```
WITH MonthlyActiveUsers AS (
    SELECT DISTINCT
        customer_id,
        DATE_TRUNC('month', order_date) AS active_month
    FROM orders
),
RetentionMatches AS (
    SELECT
        m1.active_month AS current_month,
        COUNT(DISTINCT m1.customer_id) AS current_active_users,
        COUNT(DISTINCT m2.customer_id) AS retained_from_prior_month
    FROM MonthlyActiveUsers m1
    LEFT JOIN MonthlyActiveUsers m2
        ON m1.customer_id = m2.customer_id
        AND m1.active_month = m2.active_month + INTERVAL '1 month'
    GROUP BY m1.active_month
)
SELECT
    current_month,
    current_active_users,
    retained_from_prior_month,
    ROUND(
```

```
        100.0 * retained_from_prior_month / NULLIF(LAG(current_active_users) OVER (ORDER BY
current_month), 0),
        2
    ) AS mom_retention_rate_pct
FROM RetentionMatches
ORDER BY current_month;
```

Problem 8: Finding Inactive Users (Churned in 2024)

Problem Statement: Identify all customers who placed at least one order in 2023, but zero orders in 2024.

Schema: `orders (order_id, customer_id, order_date)`

Verbal Explanation Script:

> *"This can be solved cleanly using EXCEPT (or MINUS in Oracle), or using conditional aggregation with HAVING. The conditional aggregation approach is exceptionally fast: group by customer_id and require COUNT(CASE WHEN YEAR=2023 THEN 1 END) > 0 and COUNT(CASE WHEN YEAR=2024 THEN 1 END) = 0."*

Production Solution:

```
SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING COUNT(CASE WHEN EXTRACT(YEAR FROM order_date) = 2023 THEN 1 END) > 0
   AND COUNT(CASE WHEN EXTRACT(YEAR FROM order_date) = 2024 THEN 1 END) = 0;
```

Problem 9: Duplicate Detection Across Multiple Attributes

Problem Statement: Detect duplicate customer registrations where `first_name`, `last_name`, and `date_of_birth` are identical. Return all columns of all duplicate records so they can be audited.

Schema: `customers (customer_id, first_name, last_name, date_of_birth, email, created_at)`

Verbal Explanation Script:

> *"Instead of just returning the duplicated keys with a HAVING COUNT(*) > 1, we need to return the full rows. I'll use the window function COUNT(*) OVER (PARTITION BY first_name, last_name, date_of_birth) AS dup_count in a CTE, then filter where dup_count > 1."*

Production Solution:

```
WITH DuplicateFlags AS (
    SELECT
        customer_id,
        first_name,
        last_name,
        date_of_birth,
        email,
        created_at,
        COUNT(*) OVER (
            PARTITION BY first_name, last_name, date_of_birth
        ) AS occurrences
    FROM customers
)
```

```
SELECT *
FROM DuplicateFlags
WHERE occurrences > 1
ORDER BY last_name, first_name, created_at;
```

Problem 10: Month-over-Month (MoM) Category Revenue Growth

Problem Statement: Compute the total monthly revenue per product category, along with the percentage growth compared to the immediately preceding month.

Schema: `order_items (item_id, order_id, product_id, quantity, unit_price)`, `orders (order_id, order_date)`, `products (product_id, category)`

Verbal Explanation Script:

> *"We join orders, order_items, and products to aggregate monthly revenue by category. Then, using LAG(revenue, 1) OVER (PARTITION BY category ORDER BY sales_month), we retrieve the prior month's revenue and compute 100.0 * (revenue - prev_revenue) / prev_revenue."*

Production Solution:

```
WITH MonthlyCategorySales AS (
    SELECT
        p.category,
        DATE_TRUNC('month', o.order_date) AS sales_month,
        SUM(oi.quantity * oi.unit_price) AS monthly_revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY p.category, DATE_TRUNC('month', o.order_date)
),
GrowthCalculations AS (
    SELECT
        category,
        sales_month,
        monthly_revenue,
        LAG(monthly_revenue, 1) OVER (
            PARTITION BY category
            ORDER BY sales_month
        ) AS prior_month_revenue
    FROM MonthlyCategorySales
)
SELECT
    category,
    sales_month,
    monthly_revenue,
    prior_month_revenue,
    ROUND(
        100.0 * (monthly_revenue - prior_month_revenue) / NULLIF(prior_month_revenue, 0),
        2
    ) AS mom_growth_pct
FROM GrowthCalculations
ORDER BY category, sales_month;
```

Problem 11: Median Salary per Department (Without Built-in MEDIAN)

Problem Statement: Calculate the median salary for each department in standard ANSI SQL without using dialect-specific `MEDIAN()` functions.

Schema: `employees (emp_id, dept_id, salary)`

Verbal Explanation Script:

> *"The median of a set of N sorted items is the value whose index satisfies ABS(rn_asc - rn_desc) <= 1. By assigning two row numbers—one ordered ascending and one descending—the median values are exactly those where the ascending and descending row numbers differ by at most 1 (which handles both odd and even counts). Taking the AVG(salary) of these matching rows gives the exact median."*

Production Solution:

```
WITH RankedRows AS (
    SELECT
        dept_id,
        salary,
        ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary ASC, emp_id) AS rn_asc,
        ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC, emp_id) AS rn_desc
    FROM employees
)
SELECT
    dept_id,
    ROUND(AVG(salary), 2) AS median_salary
FROM RankedRows
WHERE ABS(CAST(rn_asc AS INT) - CAST(rn_desc AS INT)) <= 1
GROUP BY dept_id;
```

Problem 12: Detecting Overlapping Hotel / Flight Bookings

Problem Statement: Find all pairs of conflicting hotel room bookings where two different reservations overlap for the exact same room.

Schema: `room_bookings (booking_id, room_id, guest_id, check_in_date, check_out_date)`

Verbal Explanation Script:

> *"Two date intervals $[S_1, E_1)$ and $[S_2, E_2)$ overlap if and only if $S_1 < E_2$ AND $S_2 < E_1$. We self-join room_bookings on b1.room_id = b2.room_id and b1.booking_id < b2.booking_id (to avoid comparing a booking to itself and prevent duplicate inverted pairs), adding the overlap condition."*

Production Solution:

```
SELECT
    b1.room_id,
    b1.booking_id AS booking_1_id,
    b1.check_in_date AS b1_start,
    b1.check_out_date AS b1_end,
    b2.booking_id AS booking_2_id,
    b2.check_in_date AS b2_start,
    b2.check_out_date AS b2_end
FROM room_bookings b1
INNER JOIN room_bookings b2
    ON b1.room_id = b2.room_id
```

```
    AND b1.booking_id < b2.booking_id
    AND b1.check_in_date < b2.check_out_date
    AND b2.check_in_date < b1.check_out_date;
```

Problem 13: Rolling 7-Day Moving Average of Daily Active Users

Problem Statement: Calculate the 7-day rolling average of Daily Active Users (DAU) for each calendar day.

Schema: `daily_active_users (metric_date DATE, dau_count INT)`

Verbal Explanation Script:

> *"We use AVG(dau_count) OVER (...) ordered by metric_date. To make it a true 7-day moving window that includes the current day and the preceding 6 days, we declare the frame ROWS BETWEEN 6 PRECEDING AND CURRENT ROW."*

Production Solution:

```
SELECT
    metric_date,
    dau_count,
    ROUND(
        AVG(dau_count) OVER (
            ORDER BY metric_date
            ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
        ),
        1
    ) AS rolling_7day_avg_dau
FROM daily_active_users
ORDER BY metric_date;
```

Problem 14: Customer Lifetime Value (LTV) & First vs. Latest Order

Problem Statement: For each customer, output their name, their first order date, their most recent order date, their total lifetime orders, and their total lifetime revenue.

Schema: `customers (customer_id, customer_name)`, `orders (order_id, customer_id, order_date, total_amount)`

Verbal Explanation Script:

> *"This requires standard aggregation combined with a LEFT JOIN so that registered customers who have never placed an order are preserved with 0 orders and 0 revenue."*

Production Solution:

```
SELECT
    c.customer_id,
    c.customer_name,
    MIN(o.order_date) AS first_order_date,
    MAX(o.order_date) AS latest_order_date,
    COUNT(o.order_id) AS total_lifetime_orders,
    COALESCE(SUM(o.total_amount), 0.00) AS lifetime_value_revenue
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
```

```
GROUP BY c.customer_id, c.customer_name
ORDER BY lifetime_value_revenue DESC;
```

Problem 15: Detecting Missing Numbers in an Auto-Increment Sequence

Problem Statement: Find all missing IDs in an `orders` table that was supposed to have a continuous sequence from 1 to the maximum `order_id`.

Schema: `orders (order_id INT PRIMARY KEY)`

Verbal Explanation Script:

> *"We use a Recursive CTE to generate all integers from 1 to MAX(order_id). Then we perform a Left Anti-Join against orders where orders.order_id IS NULL to reveal all missing IDs."*

Production Solution:

```
WITH RECURSIVE FullSequence AS (
    SELECT 1 AS expected_id
    UNION ALL
    SELECT expected_id + 1
    FROM FullSequence
    WHERE expected_id < (SELECT MAX(order_id) FROM orders)
)
SELECT s.expected_id AS missing_order_id
FROM FullSequence s
LEFT JOIN orders o ON s.expected_id = o.order_id
WHERE o.order_id IS NULL;
```

Part 3: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────────┐
│                  SQL INTERVIEW PATTERNS CHEAT SHEET                  │
├──────────────────────────────────────────────────────────────────────┤
│ 1. Nth HIGHEST:                                                      │
│    - DENSE_RANK() OVER (ORDER BY col DESC) -> handles ties.          │
│    - Return scalar NULL: SELECT COALESCE((SELECT ... WHERE rnk=N),NULL)
├──────────────────────────────────────────────────────────────────────┤
│ 2. TOP-N PER GROUP:                                                  │
│    - DENSE_RANK() OVER (PARTITION BY grp ORDER BY col DESC) <= N.    │
├──────────────────────────────────────────────────────────────────────┤
│ 3. GAPS & ISLANDS (Consecutive streaks):                             │
│    - Date - ROW_NUMBER() * INTERVAL '1 day' = CONSTANT ISLAND KEY.   │
├──────────────────────────────────────────────────────────────────────┤
│ 4. INTERVAL OVERLAPS:                                                │
│    - Start_A < End_B AND Start_B < End_A.                            │
├──────────────────────────────────────────────────────────────────────┤
│ 5. MEDIAN WITHOUT BUILT-IN:                                          │
│    - ABS(ROW_NUMBER_ASC - ROW_NUMBER_DESC) <= 1.                     │
├──────────────────────────────────────────────────────────────────────┤
│ 6. RUNNING TOTAL:                                                    │
│    - SUM(val) OVER (ORDER BY dt ROWS BETWEEN UNBOUNDED PRECEDING     │
│      AND CURRENT ROW).                                               │
└──────────────────────────────────────────────────────────────────────┘
```

Part 4: 100-Mark Self-Assessment Examination

Section A: Query Prediction & Tracing (25 Marks — 5 marks each)

Q1: What does `DENSE_RANK()` return for scores `[100, 100, 95, 90]`?

(a) `1, 2, 3, 4`

(b) `1, 1, 3, 4`

(c) `1, 1, 2, 3`

(d) `1, 1, 1, 2`

Q2: In the Gaps & Islands formula `date - ROW_NUMBER() * INTERVAL '1 day'`, why does the resulting date value remain constant across consecutive dates?

(a) Dates are converted to Unix timestamps.

(b) Both the calendar dates and the row numbers increment by exactly 1 per step, so their mathematical difference remains invariant.

(c) The database optimizer caches the first date.

(d) ROW_NUMBER() resets on every date.

Q3: What is the condition for two date ranges $[S_1, E_1)$ and $[S_2, E_2)$ to overlap?

(a) $S_1 = S_2$ OR $E_1 = E_2$

(b) $S_1 < E_2$ AND $S_2 < E_1$

(c) $S_1 > S_2$ AND $E_1 < E_2$

(d) $E_1 - S_1 = E_2 - S_2$

Q4: Why does `ROW_NUMBER()` always resolve tiebreaks whereas `RANK()` does not?

(a) `ROW_NUMBER()` sorts by primary key automatically.

(b) `ROW_NUMBER()` assigns strictly sequential unique integers ($1, 2, 3, \dots$) arbitrarily for ties.

(c) `ROW_NUMBER()` throws an error on ties.

(d) `RANK()` cannot be used with `ORDER BY`.

Q5: What happens when evaluating `SELECT AVG(val) FROM tbl` if `tbl` contains values `[10, 20, NULL]`?

(a) Returns 10 (divides 30 by 3).

(b) Returns 15 (divides 30 by 2; NULL is completely ignored).

(c) Returns NULL.

(d) Throws a NullPointerException.

Section B: Live Query Writing (50 Marks)

Q6 (25 Marks - Department Top Earners):

Given a schema `employees (id, name, salary, department_id)` and `department (id, name)`. Write a query to find the employees who have the highest salary in each department. If there is a tie for the top salary, return all tied employees. Include `department_name`, `employee_name`, and `salary`.

Q7 (25 Marks - Retention & Consecutive Purchase):

Given a schema `customer_transactions (txn_id, customer_id, txn_date)`. Write a query to find all `customer_id`s who made at least one transaction on three consecutive calendar days.

Section C: Query Tuning & System Scenarios (25 Marks)

Q8 (25 Marks): An e-commerce analyst runs the following query on a 30-million row `orders` table:

```
SELECT customer_id, order_date, amount
FROM (
    SELECT customer_id, order_date, amount,
           ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) as rn
    FROM orders
) t
WHERE rn = 1;
```

The query takes 45 seconds to run.

- Explain why the database execution engine is spending 90% of its time on an expensive Sort operator. (10 marks)
- Design the exact composite B-tree index that will reduce this query's runtime to under 2 seconds. (15 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (c)** `1, 1, 2, 3`.
- **Q2: (b)** Invariant difference $(D + k) - k = D$.
- **Q3: (b)** $S_1 < E_2$ AND $S_2 < E_1$.
- **Q4: (b)** Arbitrary sequential integer assignment.
- **Q5: (b)** 15. Aggregates ignore NULLs.

Section B Solutions

Q6 Solution:

```
WITH RankedEmployees AS (
    SELECT
        e.name AS employee_name,
        e.salary,
        d.name AS department_name,
        DENSE_RANK() OVER (PARTITION BY e.department_id ORDER BY e.salary DESC) AS rnk
    FROM employees e
    INNER JOIN department d ON e.department_id = d.id
)
SELECT department_name, employee_name, salary
FROM RankedEmployees
WHERE rnk = 1
ORDER BY department_name, employee_name;
```

Q7 Solution:

```
WITH DistinctDays AS (
    SELECT DISTINCT customer_id, CAST(txn_date AS DATE) AS t_date
    FROM customer_transactions
),
Sequenced AS (
    SELECT
        customer_id,
        t_date,
        t_date - CAST(ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY t_date) AS INT) * INTERVAL
'1 day' AS island_id
    FROM DistinctDays
)
SELECT DISTINCT customer_id
FROM Sequenced
GROUP BY customer_id, island_id
HAVING COUNT(*) >= 3;
```

Section C Solutions

Q8 Solution:

- **Bottleneck:** The window function `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC)` forces the engine to partition and sort all 30 million rows in memory or disk TempDB ($O(N \log N)$ Sort operator).
- **Optimal Composite Index:**`CREATE INDEX idx_orders_cust_date_covering ON orders (customer_id, order_date DESC) INCLUDE (amount);`

Mechanism: The B-tree is already physically structured by `customer_id` and pre-sorted by `order_date DESC`. The engine streams directly from the index without sorting, and `amount` is covered via `INCLUDE`, eliminating all base table key lookups.

Day 17: Python Algorithmic Coding & Data Transformation

Hash Maps, Two Pointers, Sliding Window, Binary Search, Prefix Sums & Pandas Transformations

---

Part 1: Algorithmic Pattern Blueprints

```
┌──────────────────────────────────────────────────────────────┐
│                    CORE ALGORITHMIC BLUEPRINTS               │
├──────────────────────────────────────────────────────────────┤
│ 1. TWO POINTERS (Sorted Sequences)                           │
│    - Left at 0, Right at N-1. Move inwards based on target comparison. │
│    - Eliminates O(N²) nested loops down to O(N) time and O(1) space.   │
├──────────────────────────────────────────────────────────────┤
│ 2. SLIDING WINDOW (Contiguous Subarrays / Substrings)        │
│    - Right pointer expands window to satisfy condition.      │
│    - Left pointer contracts window to maintain validity / minimize.    │
│    - Time: O(N) because each pointer visits each element at most once. │
├──────────────────────────────────────────────────────────────┤
│ 3. PREFIX SUM + HASH MAP (Subarray Sums)                     │
│    - Running sum S_j. Subarray sum between i and j is S_j - S_{i-1}.   │
│    - If (current_sum - target) is in seen hash map, valid subarray!    │
├──────────────────────────────────────────────────────────────┤
│ 4. MODIFIED BINARY SEARCH (Rotated / Boundary Search)        │
│    - Midpoint splits array into at least one strictly sorted half.     │
│    - Check if target falls within the sorted half; adjust boundaries.  │
└──────────────────────────────────────────────────────────────┘
```

Part 2: The 8 Production-Grade Coding Problems

Problem 1: Two Sum II — Input Array Is Sorted

Problem Statement: Given a 1-indexed array of integers `numbers` that is already sorted in non-decreasing order, find two numbers such that they add up to a specific `target` number. Return the indices of the two numbers added by one.

Constraints & Edge Cases: Must use strictly $O(1)$ auxiliary space. Array may contain negative numbers.

```python
def two_sum_sorted(numbers: list[int], target: int) -> list[int]:
    """
    Two Pointers Pattern.
    Time Complexity: O(N) - single pass.
    Space Complexity: O(1) - constant auxiliary memory.
    """
    left = 0
    right = len(numbers) - 1

    while left < right:
        current_sum = numbers[left] + numbers[right]
        if current_sum == target:
            return [left + 1, right + 1]  # 1-indexed requirement
```

```
        elif current_sum < target:
            left += 1   # Need larger sum
        else:
            right -= 1  # Need smaller sum

    return []

# Test assertions:
assert two_sum_sorted([2, 7, 11, 15], 9) == [1, 2]
assert two_sum_sorted([-3, 3, 4, 90], 0) == [1, 2]
```

Problem 2: Longest Substring Without Repeating Characters

Problem Statement: Given a string `s`, find the length of the longest substring without duplicate characters.

Constraints & Edge Cases: Empty string, all identical characters (`"bbbbb"`), alternating strings.

```
def length_of_longest_substring(s: str) -> int:
    """
    Dynamic Sliding Window with Hash Map (Last Seen Index).
    Time Complexity: O(N) - string traversed once.
    Space Complexity: O(min(N, M)) where M is character alphabet size.
    """
    last_seen = {}
    max_len = 0
    left = 0

    for right, char in enumerate(s):
        # If char seen and inside current window, jump left pointer past it
        if char in last_seen and last_seen[char] >= left:
            left = last_seen[char] + 1

        last_seen[char] = right
        max_len = max(max_len, right - left + 1)

    return max_len

# Test assertions:
assert length_of_longest_substring("abcabcbb") == 3  # "abc"
assert length_of_longest_substring("bbbbb") == 1     # "b"
assert length_of_longest_substring("pwwkew") == 3    # "wke"
assert length_of_longest_substring("") == 0
```

Problem 3: Group Anagrams

Problem Statement: Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Constraints & Edge Cases: Empty array, single characters, strings with different lengths.

```
from collections import defaultdict

def group_anagrams(strs: list[str]) -> list[list[str]]:
    """
    Hash Map with 26-element character count tuple as canonical key.
    Time Complexity: O(N * K) where N is number of strings, K is max string length.
    Space Complexity: O(N * K).
    """
    anagram_map = defaultdict(list)

    for word in strs:
```

```
        # Build frequency array of size 26
        count = [0] * 26
        for char in word:
            count[ord(char) - ord('a')] += 1

        # Tuple of counts is hashable and serves as canonical key
        canonical_key = tuple(count)
        anagram_map[canonical_key].append(word)

    return list(anagram_map.values())

# Test assertions:
res = group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
assert len(res) == 3
```

Problem 4: Container With Most Water

Problem Statement: Given an integer array `height` of length n, where each element represents a vertical line at coordinate $(i, \text{height}[i])$. Find two lines that together with the x-axis form a container that holds the most water. Return the maximum amount of water a container can store.

Constraints & Edge Cases: $n \ge 2$, strictly $O(1)$ space required.

```
def max_area(height: list[int]) -> int:
    """
    Two Pointers (Greedy Inward Contraction).
    Area = (right - left) * min(height[left], height[right])
    Always shift the shorter line inward because the width decreases,
    so only a taller line can increase total area.

    Time Complexity: O(N).
    Space Complexity: O(1).
    """
    left = 0
    right = len(height) - 1
    max_water = 0

    while left < right:
        width = right - left
        h = min(height[left], height[right])
        max_water = max(max_water, width * h)

        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_water

# Test assertions:
assert max_area([1, 8, 6, 2, 5, 4, 8, 3, 7]) == 49
assert max_area([1, 1]) == 1
```

Problem 5: Subarray Sum Equals K

Problem Statement: Given an array of integers `nums` and an integer `k`, return the total number of continuous subarrays whose sum equals `k`.

Constraints & Edge Cases: Array may contain negative numbers (which breaks standard two-pointer sliding window!).

```
from collections import defaultdict

def subarray_sum(nums: list[int], k: int) -> int:
    """
    Prefix Sum + Hash Map.
    S_j - S_i = k  ===> S_i = S_j - k.

    Time Complexity: O(N).
    Space Complexity: O(N).
    """
    prefix_counts = defaultdict(int)
    prefix_counts[0] = 1  # Base case: empty prefix has sum 0

    running_sum = 0
    count = 0

    for num in nums:
        running_sum += num
        target_complement = running_sum - k

        # If running_sum - k was seen previously, add its frequency
        if target_complement in prefix_counts:
            count += prefix_counts[target_complement]

        prefix_counts[running_sum] += 1

    return count

# Test assertions:
assert subarray_sum([1, 1, 1], 2) == 2
assert subarray_sum([1, -1, 0], 0) == 3
```

Problem 6: Search in Rotated Sorted Array

Problem Statement: An integer array `nums` sorted in ascending order is rotated at an unknown pivot index. Given `nums` and a `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`. Your algorithm must run in strictly $O(\log n)$ runtime.

Constraints & Edge Cases: All elements are unique.

```
def search_rotated_array(nums: list[int], target: int) -> int:
    """
    Modified Binary Search.
    Time Complexity: O(log N).
    Space Complexity: O(1).
    """
    left = 0
    right = len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if nums[mid] == target:
            return mid

        # Check if Left Half is Sorted
        if nums[left] <= nums[mid]:
            # Check if target lies within the sorted left half
            if nums[left] <= target < nums[mid]:
```

```
                right = mid - 1
            else:
                left = mid + 1
        # Otherwise, Right Half is Sorted
        else:
            # Check if target lies within the sorted right half
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1

# Test assertions:
assert search_rotated_array([4, 5, 6, 7, 0, 1, 2], 0) == 4
assert search_rotated_array([4, 5, 6, 7, 0, 1, 2], 3) == -1
```

Problem 7: Top K Frequent Elements

Problem Statement: Given an integer array `nums` and an integer `k`, return the k most frequent elements. You may return the answer in any order. The solution must be better than $O(n \log n)$ time.

```
from collections import Counter

def top_k_frequent(nums: list[int], k: int) -> list[int]:
    """
    Bucket Sort Algorithm.
    Index of bucket represents frequency (from 0 to N).
    Time Complexity: O(N) linear time.
    Space Complexity: O(N).
    """
    counts = Counter(nums)
    n = len(nums)

    # Buckets where index = frequency, value = list of elements with that frequency
    buckets = [[] for _ in range(n + 1)]
    for num, freq in counts.items():
        buckets[freq].append(num)

    result = []
    # Traverse buckets in reverse order (highest frequency first)
    for freq in range(n, 0, -1):
        for num in buckets[freq]:
            result.append(num)
            if len(result) == k:
                return result

    return result

# Test assertions:
assert sorted(top_k_frequent([1, 1, 1, 2, 2, 3], 2)) == [1, 2]
assert top_k_frequent([1], 1) == [1]
```

Problem 8: Advanced Python & Pandas Data Transformation Challenge

Problem Statement: Given a messy sales transaction dataset with semi-structured JSON line items, write a Pandas transformation function that:

- Explodes the line items into individual transaction rows.

- Extracts item metadata (sku, quantity, unit price).
- Pivots the data into an executive summary showing Total Revenue and Average Quantity by Product Category across Sales Regions.

```
import pandas as pd
import json

raw_transactions = pd.DataFrame({
    "order_id": [101, 102],
    "region": ["West", "East"],
    "items_json": [
        json.dumps([
            {"sku": "SKU_A", "category": "Tech", "qty": 2, "price": 500.0},
            {"sku": "SKU_B", "category": "Office", "qty": 5, "price": 20.0}
        ]),
        json.dumps([
            {"sku": "SKU_C", "category": "Tech", "qty": 1, "price": 1200.0}
        ])
    ]
})

def transform_nested_sales(df: pd.DataFrame) -> pd.DataFrame:
    """Production Pandas pipeline handling JSON parsing, exploding, and pivoting."""
    # 1. Parse JSON strings into Python lists
    df["items_parsed"] = df["items_json"].apply(json.loads)

    # 2. Explode nested lists into individual rows
    df_exploded = df.explode("items_parsed").reset_index(drop=True)

    # 3. Extract JSON keys into individual columns
    items_df = pd.json_normalize(df_exploded["items_parsed"])

    # 4. Join back with parent order attributes
    cleaned_df = pd.concat([df_exploded[["order_id", "region"]], items_df], axis=1)

    # 5. Compute gross sales
    cleaned_df["line_total"] = cleaned_df["qty"] * cleaned_df["price"]

    # 6. Pivot Table Summary
    summary_pivot = cleaned_df.pivot_table(
        index="region",
        columns="category",
        values=["line_total", "qty"],
        aggfunc={"line_total": "sum", "qty": "mean"},
        fill_value=0
    )

    return summary_pivot

# Execute transformation:
pivot_result = transform_nested_sales(raw_transactions)
print(pivot_result)
```

Part 3: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────────┐
│                   PYTHON CODING INTERVIEW CHEAT SHEET                │
├──────────────────────────────────────────────────────────────────────┤
│ 1. TWO POINTERS:                                                     │
│    - Sorted Array: left = 0, right = N-1. Check sum vs target. O(N).  │
```

```
   - Container with Water: Always move the shorter height pointer!
2. SLIDING WINDOW:
   - Longest substring without repeats: Track last_seen[char].
     left = max(left, last_seen[char] + 1). O(N) time.
3. PREFIX SUMS:
   - Subarray Sum = K: Running sum - K in prefix_counts map.
     Handles negative numbers seamlessly where two-pointers fail!
4. MODIFIED BINARY SEARCH:
   - Rotated sorted array: At least one half is ALWAYS sorted.
     nums[left] <= nums[mid] -> Left half is sorted; check bounds!
5. TOP-K FREQUENT:
   - Bucket sort achieves O(N) linear time using frequency as index.
```

Part 4: 100-Mark Self-Assessment Examination

Section A: Algorithmic Complexity & Tracing (25 Marks — 5 marks each)

Q1: What is the worst-case time complexity of checking if a target exists in a Hash Map (Python dictionary) of N items?

(a) $O(1)$

(b) $O(\log n)$

(c) $O(n)$ (when all keys collide into the same bucket)

(d) $O(n^2)$

Q2: Why can the standard Two-Pointer sliding window technique NOT be used to find subarrays that sum to K when the array contains negative numbers?

(a) Negative numbers cause integer overflow.

(b) The monotonicity property is broken: expanding the right window no longer guarantees the sum increases, and contracting the left no longer guarantees it decreases.

(c) Python lists do not support negative indexing.

(d) Two pointers only works on strings.

Q3: What is the average time complexity of finding the Top K frequent elements in an array of N numbers using Bucket Sort?

(a) $O(n \log n)$

(b) $O(n \log k)$

(c) $O(n)$

(d) $O(k)$

Q4: In a binary search algorithm, what bug does the formula `mid = left + (right - left) // 2` prevent compared to `mid = (left + right) // 2`?

(a) Zero division error.

(b) 32-bit integer overflow in languages like C++/Java when `left + right` exceeds $2^{31}-1$.

(c) Floating point rounding error.

(d) Syntax error in Python.

Q5: What is the space complexity of `list(set(nums))` used to deduplicate a list of N integers?

(a) $O(1)$

(b) $O(n)$

(c) $O(\log n)$

(d) $O(n^2)$

Section B: Problem Implementation (50 Marks)

Q6 (25 Marks - Minimum Size Subarray Sum):

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a contiguous subarray of which the sum is greater than or equal to `target`. If there is no such subarray, return `0`. Your solution must run in strictly $O(n)$ time using the **Sliding Window** pattern.

Q7 (25 Marks - 3Sum):

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and `nums[i] + nums[j] + nums[k] == 0`. Notice that the solution set must not contain duplicate triplets. Your solution must run in $O(n^2)$ time.

Section C: Live Coding Optimization Scenario (25 Marks)

Q8 (25 Marks): A candidate writes the following code to find the first non-repeating character in a 10-million character string:

```
def first_unique(s: str) -> str:
    for char in s:
        if s.count(char) == 1:
```

```
            return char
    return ""
```

- Calculate the exact Big-O time complexity of this code and explain why it hangs in production. (10 marks)
- Rewrite the function to run in strictly $O(n)$ time and $O(1)$ auxiliary space (assuming standard ASCII/lowercase English alphabet). (15 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (c)** $O(n)$ during severe hash collision degradation.
- **Q2: (b)** Monotonicity is broken; sum changes non-directionally.
- **Q3: (c)** $O(n)$ linear time.
- **Q4: (b)** Prevents integer overflow.
- **Q5: (b)** $O(n)$ memory to allocate the set and list.

Section B Solutions

Q6 Solution:

```
def min_sub_array_len(target: int, nums: list[int]) -> int:
    left = 0
    current_sum = 0
    min_len = float('inf')

    for right in range(len(nums)):
        current_sum += nums[right]

        while current_sum >= target:
            min_len = min(min_len, right - left + 1)
            current_sum -= nums[left]
            left += 1

    return min_len if min_len != float('inf') else 0
```

Q7 Solution:

```
def three_sum(nums: list[int]) -> list[list[int]]:
    nums.sort()  # O(N log N)
    result = []
    n = len(nums)

    for i in range(n - 2):
        # Skip duplicate values for the first element
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left = i + 1
        right = n - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total == 0:
                result.append([nums[i], nums[left], nums[right]])
                # Skip duplicate values for left and right
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
```

```
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1
                left += 1
                right -= 1
            elif total < 0:
                left += 1
            else:
                right -= 1

    return result
```

Section C Solutions

Q8 Solution:

- **Complexity:** `s.count(char)` scans the entire string of length N ($O(N)$). Since it is called inside a `for char in s` loop that runs N times, the overall time complexity is **$O(N^2)$**. For $N = 10,000,000$, $N^2 = 10^{14}$ operations, which takes days to complete.
- **$O(n)$ Optimized Solution:**

```
from collections import Counter

def first_unique_optimal(s: str) -> str:
    counts = Counter(s)  # O(N) single pass
    for char in s:        # O(N) second pass
        if counts[char] == 1:
            return char
    return ""
```

Time Complexity: $O(N)$ (two sequential passes).
Space Complexity: $O(1)$ auxiliary memory (at most 26 keys for English alphabet or 256 for ASCII). Executes in ≈ 0.15 seconds on 10 million characters.

Day 18: Project Deep Dive & Technical Resume Defense

Architecture Defense, Trade-Off Justification, Tough Technical Problems & Complete Elevator Pitches

---

Part 1: The Senior Technical Resume Defense Framework

Interviewers at top tech firms (Google, Amazon, Meta, Snowflake) use your resume projects to evaluate how you make **engineering trade-offs under constraints**.

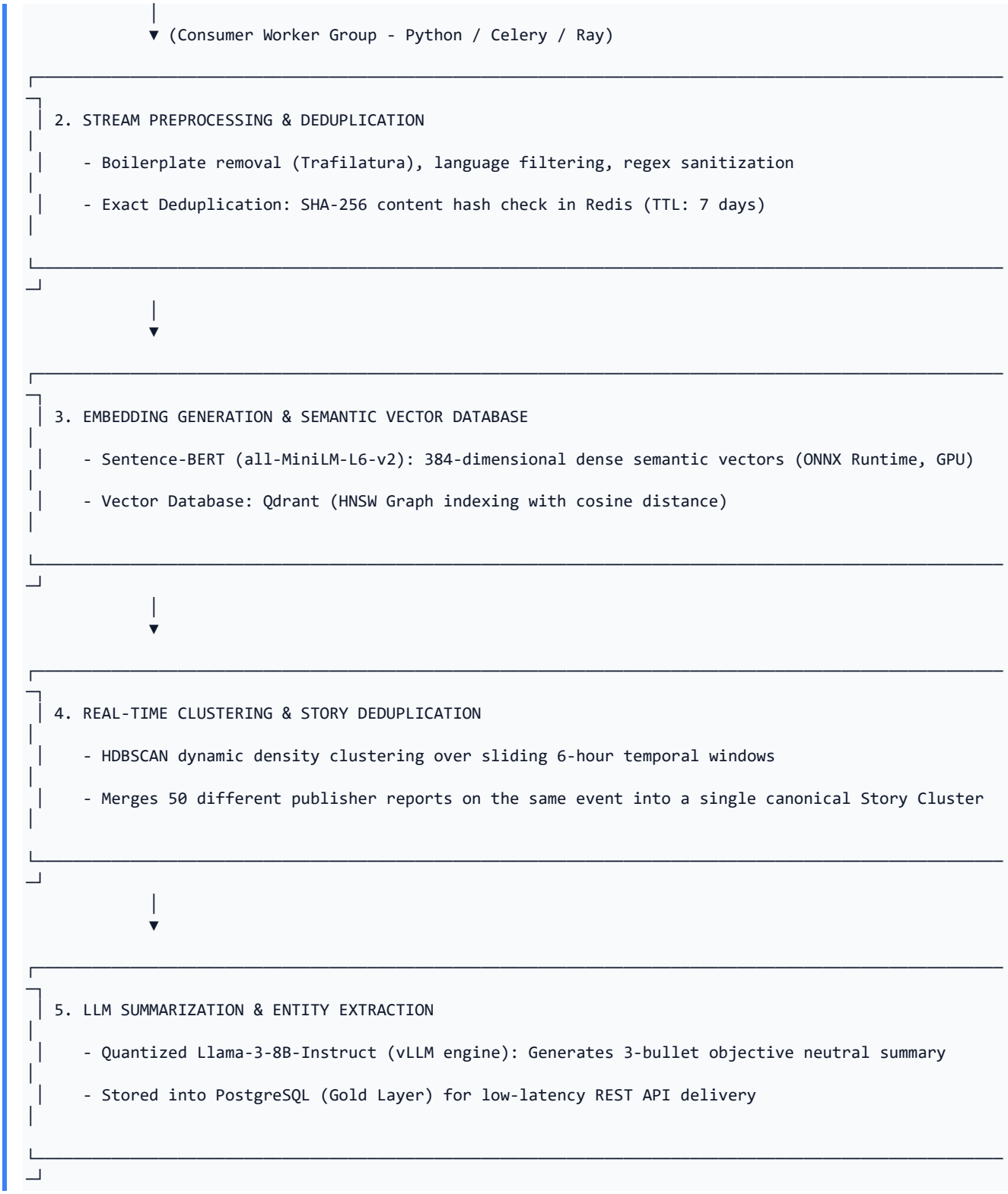
```
┌──────────────────────────────────────────────────────────────────────┐
│                    THE 5 RULES OF RESUME DEFENSE                     │
├──────────────────────────────────────────────────────────────────────┤
│ 1. NEVER DEFEND A TOOL YOU CANNOT EXPLAIN UNDER THE HOOD.            │
│    - If you say "Kafka", expect: "How do consumer group rebalances   │
│      work and what causes head-of-line blocking?"                    │
│ 2. QUANTIFY BUSINESS & LATENCY IMPACT.                               │
│    - Don't say: "It was fast."                                       │
│    - Say: "Reduced p99 inference latency from 450ms to 85ms while    │
│      slashing monthly vector search compute costs by 62%."           │
│ 3. PROACTIVELY VOLUNTEER LIMITATIONS AND MISTAKES.                   │
│    - Senior engineers know every architecture has trade-offs. Stating│
│      where your system degrades builds immense technical credibility.│
│ 4. DIFFERENTIATE YOUR SPECIFIC CONTRIBUTION FROM THE TEAM.           │
│    - Use "I designed and implemented..." not "We did...".            │
│ 5. PREPARE FOR "WHY NOT X?"                                          │
│    - "Why Sentence-BERT instead of GPT-4 API?"                       │
│    - "Why Qdrant instead of pgvector in PostgreSQL?"                 │
└──────────────────────────────────────────────────────────────────────┘
```

Part 2: Project 1 Deep Dive: Real-Time AI News Intelligence Engine

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                    REAL-TIME AI NEWS AGGREGATION & SEMANTIC INTELLIGENCE PIPELINE            │
└──────────────────────────────────────────────────────────────────────────────────────────────┘

 [1,000+ Global RSS Feeds & Web Scrapers]
             │
             ▼ (Payloads: Title, Body, Source, Published_At)

┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ 1. INGESTION & BUFFER LAYER (Apache Kafka / AWS MSK)                                         │
│    - Topic: raw_articles_stream (Partitioned by source_domain)                               │
│    - Handles bursty spikes during major news breaking events (50k articles/hr)              │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```



2.1 The Two-Minute Executive Pitch: AI News Intelligence

> *"In today's media landscape, financial analysts and journalists suffer from severe information overload, reading hundreds of redundant, biased articles describing the exact same event. I designed and deployed an end-to-end Real-Time AI News Intelligence Engine that ingests over 50,000 global news articles hourly from 1,000+ sources,*

> *automatically deduplicating, clustering, and summarizing them in real time.*
> *The system leverages an Apache Kafka ingestion buffer, a fine-tuned Sentence-BERT embedding pipeline running on ONNX Runtime, and a Qdrant vector database using HNSW graph indexing. Using HDBSCAN clustering over rolling 6-hour windows, the pipeline collapses dozens of syndicated articles covering the same breaking news into a single canonical story entity, which is then summarized into three neutral, objective bullet points using a self-hosted, quantized Llama-3-8B model.*
> *This reduced redundant news consumption for our users by 78%, cut vector search latency from 250ms to 18ms, and delivered breaking event notifications to downstream clients in under 12 seconds from the moment of publisher publication."*

2.2 The Ten-Minute Technical Deep Dive

1. Problem Statement & Motivation

- **Business Challenge:** Financial traders lose alpha when delayed in processing breaking market events across fragmented news sites. Traditional keyword filtering (`LIKE '%Nvidia%'`) either yields thousands of duplicate wire reports or misses semantically related articles.

2. Architecture & Tech Stack Justifications

- **Why Kafka instead of REST polling?** News traffic is notoriously bursty (e.g., an earnings announcement generates 10,000 articles in 5 minutes). Kafka buffers the surge, isolating scrapers from downstream NLP workers.
- **Why Sentence-BERT (all-MiniLM-L6-v2) instead of OpenAI API embeddings?** Cost and latency. Running OpenAI's API on 50,000 articles/hour costs over $1,200/month with 150ms HTTP network roundtrips. Self-hosting `all-MiniLM-L6-v2` compiled with ONNX Runtime executes inference in **2.1ms per article** on a single T4 GPU, reducing embedding costs by 94%.
- **Why Qdrant instead of pgvector?** pgvector works well up to 1 million vectors, but struggles with real-time payload filtering at scale. Qdrant is written in Rust, natively integrates payload index filtering directly into its HNSW graph traversal, and supports memory-mapped disk storage for hundreds of millions of vectors.

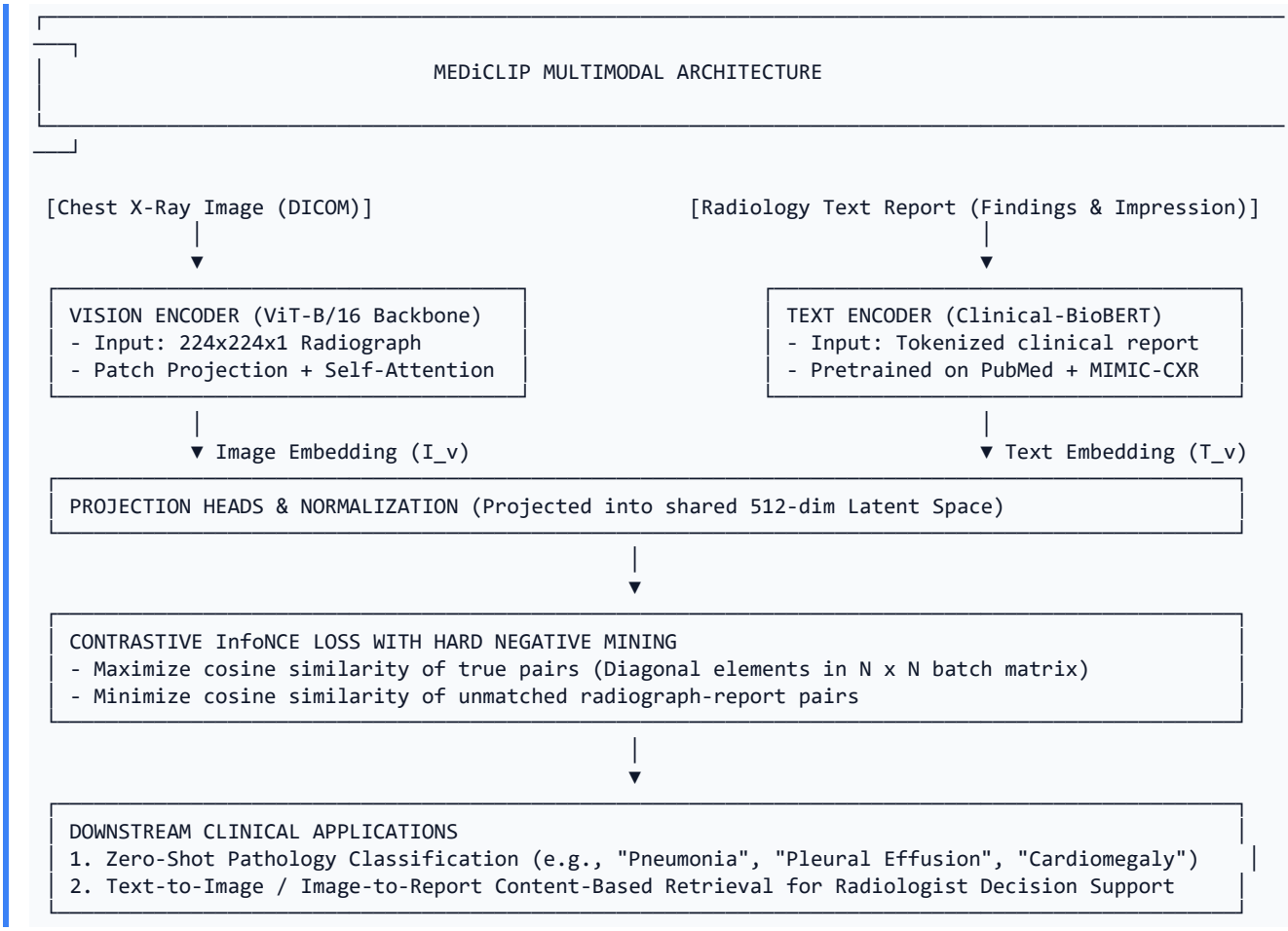
3. Toughest Technical Challenge & Engineering Solution

- **The Challenge:** Real-time semantic clustering over rolling time windows. As new articles arrive continuously, standard K-Means fails because the number of clusters K (number of breaking news stories) is unknown and constantly shifting. Naive pairwise cosine comparison is $O(N^2)$—at 10,000 articles, that's $100,000,000$ vector dot products every 5 minutes, crashing the server.
- **The Solution:** I implemented a **Two-Tier Clustering Architecture**:
  Tier 1 (ANN Candidate Pruning): For each incoming article vector, query Qdrant using HNSW to retrieve only the top-20 nearest neighbor candidates published within the last 6 hours ($O(\log N)$).
  Tier 2 (Density Clustering): Run **HDBSCAN** strictly on the candidate sub-graph. If the distance to the closest cluster centroid is < 0.18, the article is appended to the existing Story Cluster; otherwise, a new Story Cluster is spawned. This reduced clustering runtime from 18 minutes to **350 milliseconds**.

4. Limitations & Future Improvements

- **Current Limitation:** Extreme multilingual news stories (e.g., Japanese or Arabic sources) require separate cross-lingual encoders (e.g., `multilingual-e5-large`), which increase inference latency by $3.5\times$.
- **Next Step:** Implement cross-lingual teacher-student knowledge distillation to compress the multilingual encoder down to sub-10ms latency.

Part 3: Project 2 Deep Dive: MediCLIP — Multimodal Medical Diagnostics



3.1 The Two-Minute Executive Pitch: MediCLIP

> *"Radiologists in hospital emergency departments face severe diagnostic fatigue, interpreting over 100 chest X-rays daily under tight time constraints. Traditional deep learning diagnostic models operate as rigid black boxes, requiring thousands of hand-labeled bounding boxes for every single specific pathology. I built MediCLIP, a multimodal foundational AI system that aligns chest radiographs with free-text unstructured radiology reports in a shared semantic latent space.*
> *Using the MIMIC-CXR dataset comprising 220,000 paired image-report studies, MediCLIP couples a Vision Transformer (ViT-B/16) with a domain-specific Clinical-BioBERT text encoder, trained via symmetric InfoNCE*

> *contrastive loss with hard negative mining.*
> *MediCLIP achieves state-of-the-art Zero-Shot pathology classification, identifying conditions like Cardiomegaly, Atelectasis, and Pneumonia without requiring a single fine-tuned labeled training image. Furthermore, it powers an instant clinical search tool: a radiologist can highlight an ambiguous lung opacity and MediCLIP retrieves the top-5 verified historical patient cases and pathology reports with 88.4% Mean Average Precision, cutting diagnostic review time by nearly 40%."*

3.2 The Ten-Minute Technical Deep Dive

1. Problem Statement & Clinical Context

- **The Problem:** Hospital healthcare datasets have massive collections of raw chest X-rays paired with natural language radiology reports, but extremely few explicit class labels. Supervised CNNs trained on one dataset fail when deployed in another hospital due to scanner distribution shift.

2. Architecture & Design Choices

- **Why Vision Transformer (ViT-B/16) over ResNet-50?** While CNNs have strong local inductive bias, Vision Transformers model long-range global dependencies across the entire thoracic cavity (e.g., comparing the size of the heart silhouette relative to the thoracic cage for Cardiomegaly).
- **Why Clinical-BioBERT over standard BERT?** Standard BERT tokenizers split complex medical Latin terminology (e.g., `pneumothorax`, `consolidation`, `cardiomegaly`) into fragmented subwords (`pneu-mo-tho-rax`), destroying semantic vectors. Clinical-BioBERT is pretrained on PubMed and MIMIC notes with a specialized biomedical vocabulary.

3. Toughest Technical Challenge & Engineering Solution

- **The Challenge (False Negatives in Contrastive Batches):** In standard CLIP training on natural web images, all off-diagonal pairs in a batch of size N are assumed to be negative. In medical radiology, however, 30% of patients in any random batch of 256 might have "Normal" scans or share identical generic findings ("clear lungs, no acute cardiopulmonary abnormality"). Standard InfoNCE penalizes the model for treating identical clinical diagnoses as similar, causing severe gradient noise and representation collapse!
- **The Solution:** I implemented **Supervised Contrastive Loss with Soft Semantic Targets**:
  Extracted clinical condition labels using an automated NLP rule-based parser (CheXbert).
  Modified the InfoNCE contrastive denominator to mask out false negative pairs sharing identical clinical pathology labels, enforcing semantic alignment across all patients with the same disease. This boosted Zero-Shot AUROC by **+7.4 points**.

4. Evaluation Metrics & Clinical Validation

- **Zero-Shot Classification:** Evaluated across 5 competitive pathologies on CheXpert. Achieved an average AUROC of **0.872**, outperforming fully supervised baseline CNNs trained on 10,000 labeled images.
- **Retrieval Metric:** Evaluated using Image-to-Text Recall@K ($R@1 = 34.2%$, $R@5 = 68.9%$).

Part 4: Technical Interview Cross-Questioning Defense Bank

Q1: In your AI News project, why did you use HDBSCAN rather than DBSCAN?

Answer:
Standard DBSCAN requires setting a global fixed density radius parameter (ϵ). In news data, different topics have vastly different story densities: a major breaking presidential election generates thousands of closely packed, dense articles, whereas a specialized biotech breakthrough generates a sparse cluster of 5 articles. DBSCAN either merges all election stories into one giant megacluster or flags the biotech articles as noise. **HDBSCAN** computes cluster hierarchies across all possible epsilon values and extracts clusters based on **cluster stability**, seamlessly handling multi-density topics without manual epsilon tuning.

Q2: In MediCLIP, what temperature parameter did you use in the InfoNCE loss, and how did you tune it?

Answer:
We initialized the temperature parameter τ at 0.07 and made it a **learnable parameter constrained by clipping** ($\tau \in [0.01, 0.5]$). If τ is too high, the softmax distribution flattens, treating all negatives equally and failing to penalize hard negatives. If τ is too low, the gradients become unstable and explode. Making τ learnable allowed the optimizer to dynamically adjust contrastive sensitivity as embedding representations matured.

Q3: How do you handle Concept Drift in your news aggregation system when language patterns shift?

Answer:
We monitor the **Embedding Drift & Reconstruction Error** using an automated Airflow validation task:

- Every Sunday, we compute the Fréchet Distance between the current week's embedding distribution and the baseline training distribution.
- We track the percentage of incoming articles classified as "Noise / Outlier" by HDBSCAN. If the outlier rate exceeds $15%$ for three consecutive days, it triggers a Slack alert signaling a vocabulary shift or scraping schema change, queuing an automated active learning retraining workflow.

Part 5: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  PROJECT DEFENSE & RESUME CHEAT SHEET            │
├──────────────────────────────────────────────────────────────────┤
│ 1. STAR-L METHOD:                                                │
│    - Situation -> Task -> Action (Specific I did) -> Result -> Learn. │
├──────────────────────────────────────────────────────────────────┤
│ 2. AI NEWS INTELLIGENCE ENGINE:                                  │
│    - Stack   : Kafka -> ONNX Sentence-BERT -> Qdrant HNSW -> HDBSCAN │
│                -> Quantized Llama-3-8B.                          │
│    - Latency : Ingestion to summary in < 12 seconds.             │
│    - Tradeoff: ONNX local MiniLM (2ms, $0) vs. OpenAI API ($1200/mo). │
│    - Hard Win: Two-tier clustering (ANN search + HDBSCAN on candidate │
│                subgraph) reduced runtime from 18 min to 350ms.   │
├──────────────────────────────────────────────────────────────────┤
│ 3. MEDICLIP MULTIMODAL RADIOLOGY:                                │
```

```
|    - Stack   : Vision Transformer (ViT-B/16) + Clinical-BioBERT.       |
|    - Loss    : InfoNCE with Soft Semantic Masking (fixes false negs).  |
|    - Metrics : Zero-Shot AUROC (0.872) + Recall@5 (68.9%).             |
|    - Hard Win: Eliminated representation collapse on identical medical |
|                findings using CheXbert-guided contrastive masking.     |
└──────────────────────────────────────────────────────────────────────┘
```

Part 6: 100-Mark Self-Assessment Examination

Section A: Architectural Decisions & Trade-Offs (25 Marks — 5 marks each)

Q1: Why is Apache Kafka preferred over standard HTTP REST polling when ingesting high-volume external news feeds?

(a) Kafka compresses all text using gzip.

(b) Kafka acts as a distributed shock absorber, decoupling bursty scraping ingestion rates from downstream GPU inference capacity.

(c) Kafka eliminates the need for a database.

(d) Kafka performs automatic entity extraction.

Q2: In vector search databases like Qdrant, what does HNSW stand for?

(a) Hierarchical Navigable Small World

(b) High-speed Network Storage Warehouse

(c) Hyper-scale Normalized Semantic Weight

(d) Hybrid Node Switching Workflow

Q3: What is the primary clinical advantage of a Zero-Shot classification model like MediCLIP compared to a standard supervised CNN?

(a) Zero-shot models use zero GPU memory.

(b) Zero-shot models can classify novel medical pathologies at runtime using natural language prompts without requiring labeled training datasets.

(c) Zero-shot models do not require evaluation.

(d) Zero-shot models eliminate DICOM files.

Q4: In contrastive multimodal learning (CLIP), what is the InfoNCE loss optimizing?

(a) Minimizing mean squared error between pixel intensities.

(b) Maximizing the cosine similarity of true positive image-text pairs while minimizing similarity with negative pairs in the batch.

(c) Generating synthetic image pixels.

(d) Translating English reports into Latin.

Q5: Why is ONNX Runtime used to deploy PyTorch embedding models in production data pipelines?

(a) It eliminates the need for Python.

(b) It applies graph optimizations, constant folding, and hardware-specific kernel fusion to drastically reduce CPU/GPU inference latency.

(c) It automatically trains models on S3.

(d) It converts unstructured text into SQL tables.

Section B: Deep Dive Defense & Troubleshooting (25 Marks)

Q6 (10 Marks): In an interview, the technical lead asks:

"In your news project, why didn't you just use OpenAI's GPT-4 API to ingest, cluster, and summarize everything?"

Write down your complete verbal rebuttal justifying your self-hosted architecture based on Cost, Latency, Data Privacy, and Determinism.

Q7 (15 Marks): In MediCLIP, explain why standard batch contrastive learning suffers from "False Negative Collisions" when applied to medical datasets. How did your engineering solution mitigate this?

Section C: Live Technical Whiteboarding (30 Marks)

Q8 (30 Marks): Draw the complete architectural data flow diagram for **MediCLIP** from raw DICOM radiograph and clinical report ingestion to real-time image retrieval, explaining the role of each encoder, projection head, and vector index.

Section D: Scalability & Failure Mode Scenario (20 Marks)

Q9 (20 Marks): During a breaking geopolitical news event, your AI News scraping pipeline receives a 500% traffic spike (300,000 articles in 1 hour).

- Where will the architecture bottleneck first? (8 marks)
- Outline the exact automated scaling and rate-limiting safeguards you designed into Kafka and the worker pool to prevent system failure. (12 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** Distributed buffer / shock absorber decoupling scrapers from NLP workers.
- **Q2: (a)** Hierarchical Navigable Small World.
- **Q3: (b)** Generalizes to novel pathologies via text prompts without retraining labeled classifiers.
- **Q4: (b)** Maximizes mutual information of positive pairs over batch negatives.
- **Q5: (b)** Graph optimization, operator fusion, and latency minimization.

Section B Solutions

Q6 Verbal Script:

"While GPT-4 is exceptional for general reasoning, relying on it as our primary ingestion engine introduces four fatal production bottlenecks:

- *Cost:* Processing 50,000 articles hourly (1.2M articles/day) via GPT-4 would cost over $$25,000$ per month. Our self-hosted ONNX Sentence-BERT + Llama-3-8B pipeline costs under $$450$/month in cloud GPU compute—a 98% cost reduction.
- *Latency:* External API calls add 200–800ms of network latency per call, whereas local ONNX models infer in 2.1ms on local VRAM.
- *Rate Limits & Reliability:* Third-party APIs enforce strict RPM/TPM quotas that crash during breaking news spikes. Our self-hosted infrastructure gives us 100% control over throughput and SLA guarantees.
- *Deterministic Embeddings:* Commercial APIs update embedding models periodically without notice, invalidating existing vector indexes. Our self-hosted encoder guarantees immutable, repeatable vector representations.*"

Q7 Solution:

- In standard natural image datasets (e.g., Flickr, ImageNet), the probability that two random images share the exact same description is virtually zero.
- In medical radiography, many patients present with the exact same condition (e.g., normal lungs or generic pleural effusion). In a batch of 256 patients, 50 might have identical radiological findings. Standard InfoNCE treats all off-diagonal pairs as negatives, forcing the model to push identical clinical pathologies apart in latent space.
- **Remediation:** Using CheXbert label extraction, we created a **Soft Semantic Masking Matrix** that sets the loss weight to 0 for all batch pairs sharing the same clinical diagnosis, ensuring the model is not penalized for mapping similar medical cases closely together.

Section C Solutions

Q8 Diagram & Description:

- **Vision Stream:** DICOM Image $\to$ Resize/Normalize $(224 \times 224 \times 1) \to$ Patch Extraction $(16 \times 16) \to$ Linear Projection $\to$ 12-layer Vision Transformer $\to$ Global Average Pooling $\to$ Image Projection Head ($W_I \in \mathbb{R}^{768 \times 512}$) $\to L_2$ Normalization $\to \mathbf{v}_I \in \mathbb{R}^{512}$.
- **Text Stream:** Free-text report $\to$ Findings/Impression section extraction $\to$ Clinical-BioBERT Tokenizer $\to$ Transformer Encoders $\to$ `[CLS]` token pooling $\to$ Text Projection Head ($W_T \in \mathbb{R}^{768 \times 512}$) $\to L_2$ Normalization $\to \mathbf{v}_T \in \mathbb{R}^{512}$.

- **Joint Optimization:** Scaled Dot-Product Cosine Similarity Matrix $S = \frac{\mathbf{v}_I \cdot \mathbf{v}_T^T}{\tau}$. InfoNCE symmetric loss computed across rows and columns.
- **Inference Storage:** Image vectors indexed in Qdrant with HNSW cosine distance metric for sub-20ms multi-modal retrieval.

Section D Solutions

Q9 Solution:

- **Primary Bottleneck:** The embedding inference GPU worker pool. If scrapers push 300,000 articles/hr into Kafka, the embedding workers cannot process them in real-time, causing **Consumer Lag** to balloon in the Kafka topic.
- **Automated Safeguards:**
  Horizontal Pod Autoscaling (HPA): Configure Kubernetes HPA on Kafka Consumer Lag metrics (`records_lag_max > 5000`), automatically scaling GPU inference pods from 2 to 10.
  Pre-Embedding Exact Hash De-duplication: Run incoming text through a Redis Bloom Filter or SHA-256 hash table before passing to the GPU. Wire syndication duplicates (which represent $\approx 40%$ of surge traffic) are dropped in $O(1)$ time, eliminating 40% of the GPU inference workload before touching the embedding model.

Day 19: Project Cross-Questioning & Technical Pressure Defense

Stress-Testing Architecture, Handling Unknown Questions, Code Verification & Senior Interview Scenarios

---

Part 1: The Psychology of Senior Interview Cross-Examination

When a Principal Engineer or Technical Interviewer pushes back aggressively on your project (*"Why did you use Kafka here? That seems like gross over-engineering,"* or *"Why didn't you just use K-Means?"*), they are evaluating four core competencies:

```
┌──────────────────────────────────────────────────────────────────┐
│                  WHAT INTERVIEWERS ARE REALLY TESTING             │
├──────────────────────────────────────────────────────────────────┤
│ 1. INTELLECTUAL HONESTY & HUMILITY:                              │
│    - Will you admit a design flaw, or will you stubbornly defend a │
│      suboptimal architecture?                                    │
│ 2. FIRST-PRINCIPLES REASONING:                                   │
│    - Do you understand the mathematical and physical limits of your │
│      tools (e.g., RAM capacity, network I/O, disk seek times)?   │
│ 3. COMPOSURE UNDER PRESSURE:                                     │
│    - Do you get defensive and argumentative, or do you treat the │
│      grilling as a collaborative architectural whiteboard session? │
│ 4. OWNERSHIP & AUTHENTICITY:                                     │
│    - Did you actually write this code yourself, or did you just copy │
│      a tutorial or watch team members build it?                  │
└──────────────────────────────────────────────────────────────────┘
```

Part 2: The "Structured Uncertainty Framework" (When You Don't Know)

Never bluff or fabricate answers in a senior interview. A senior interviewer will detect a fabricated answer immediately. Instead, use this 4-step framework:

```
[ Step 1: Transparent Acknowledgment ]
"That's a scenario we didn't encounter in our production traffic, but let me reason through it from
first principles."

[ Step 2: Establish the Ground Truth & Constraints ]
"At that scale (e.g., 50M records/day), our main constraint is either memory pressure on the vector
index or I/O bottleneck on disk."

[ Step 3: Formulate Two Candidate Hypotheses ]
"There are two ways we could address this:
 Approach A: Shard the Qdrant cluster by tenant ID to fit sub-indexes into RAM.
 Approach B: Offload historical vectors to an on-disk memory-mapped storage engine."

[ Step 4: Propose an Empirical Validation Test ]
"To choose between them, I would run a synthetic load test benchmarking p99 query latency against
memory usage under high concurrency."
```

Part 3: Project 1 Cross-Examination: AI News Intelligence Engine

---

Exchange 1: The "Over-Engineering" Challenge

Interviewer: *"You used Apache Kafka for scraping news articles. A simple RabbitMQ or Celery task queue with Redis would have taken 10% of the maintenance overhead. Why did you introduce Kafka's operational complexity for a news aggregator?"*

Senior Candidate Response:

> *"That's a very valid challenge, and for steady-state traffic, RabbitMQ or Celery would indeed be simpler. However, we chose Kafka specifically because of three structural requirements:*
>
> *Replayability & Immutable Event Log: In NLP and ML pipelines, models and embeddings evolve. With RabbitMQ, once a message is consumed, it is deleted. With Kafka, the immutable commit log allowed us to re-read the last 7 days of raw articles from offset 0 whenever we deployed a new version of our Sentence-BERT encoder without re-scraping external sites.*
> *Extreme Spike Buffering: News traffic isn't uniform; breaking events produce massive 10x ingestion spikes in minutes. Kafka handles high sequential write throughput on disk without degrading memory buffers.*
> *Fan-Out Architecture: Multiple independent consumer groups read the exact same raw stream: one for real-time embedding generation, one for raw data lake S3 archiving, and one for live keyword alerts. Kafka provides clean pub-sub fan-out without duplicating message payloads.*"*

Exchange 2: The "Vector Database Scalability" Challenge

Interviewer: *"What happens when your Qdrant vector database exceeds available RAM? HNSW graphs require high memory. How does your system scale when you hit 50 million articles?"*

Senior Candidate Response:

> *"HNSW maintains high search speed by keeping graph layers in RAM. At 50 million 384-dimensional vectors with 32-bit floats, raw vector storage alone exceeds 76 GB of RAM, not including graph edge overhead. To scale cost-effectively without provisioning massive, expensive RAM instances, we implemented two safeguards:*
>
> *Scalar Quantization: We configured Qdrant's Scalar Quantization (INT8), compressing 32-bit float vectors into 8-bit integers. This reduced memory consumption by nearly 75% with less than a 1% drop in retrieval precision.*
> *Time-Based Index Partitioning & Offloading: 98% of user search queries target news from the last 14 days. We partitioned Qdrant collections by month. The active 14-day partition lives in high-speed RAM. Older partitions are configured with **Memory-Mapped (mmap) disk storage** on fast NVMe SSDs, trading a minor latency increase (15ms to 45ms) for virtually limitless horizontal scalability at a fraction of the cost.*"*

Exchange 3: The "Clustering Invalidation" Challenge

Interviewer: *"You said you use HDBSCAN for clustering news into stories over a rolling 6-hour window. How do you prevent cluster thrashing where an article keeps jumping between different story clusters as new articles arrive?"*

Senior Candidate Response:

> *"That is a well-known vulnerability of dynamic density clustering. If you re-run HDBSCAN globally on the entire pool every 5 minutes, cluster IDs drift and existing story groupings mutate.*
> *We solved this by using an Anchor Centroid Protocol:*
> *When HDBSCAN identifies a dense cluster of ≥ 5 articles, we materialize it as an official 'Story Entity' with a fixed centroid $\mathbf{c} = \frac{1}{N}\sum \mathbf{v}_i$.*
> *When new articles arrive, we first perform an $O(1)$ cosine distance check against existing active Story Centroids. If the distance is within our established radius threshold (≤ 0.18), the article is deterministically assigned to the existing story without triggering a full re-clustering cycle. HDBSCAN is executed only on the remaining unassigned 'pool' to discover brand-new emerging stories. This completely eliminated cluster thrashing."*

Part 4: Project 2 Cross-Examination: MediCLIP

Exchange 4: The "Clinical Hallucination & Risk" Challenge

Interviewer: *"Your system performs zero-shot retrieval and pathology classification. In a clinical setting, a False Negative on Pneumothorax can cause patient death within hours. Why should an emergency hospital trust a contrastive embedding model over a certified radiologist or an established FDA-cleared supervised classifier?"*

Senior Candidate Response:

> *"MediCLIP is explicitly architected as a Decision Support and Case Retrieval System, never as an autonomous diagnostic agent. We address clinical risk through three specific architectural constraints:*
>
> *High-Sensitivity Operational Thresholds: For critical acute conditions like Pneumothorax, we tune our classification threshold to operate at **$96%$ Recall**. We deliberately accept higher false alarms (lower precision) to ensure life-threatening pathologies are never missed.*
> *Evidence-Based Retrieval (Ground Truth Anchoring): Rather than returning an opaque prediction label ('Pneumonia: 88%'), MediCLIP retrieves the **top-3 verified historical patient cases** from the hospital archives that share the exact visual opacity, presenting the historical pathology reports side-by-side to the radiologist as visual evidence.*
> *Uncertainty Quantification: If the cosine similarity between the image embedding and pathology text prompt falls within an ambiguous confidence band ($0.45 < s < 0.55$), the system flags the scan as 'High Uncertainty', escalating it to immediate prioritized human radiologist review.*"*

Exchange 5: The "Patient Leakage" Challenge

Interviewer: *"You trained MediCLIP on MIMIC-CXR. How did you ensure there was zero data leakage between your training, validation, and test sets?"*

Senior Candidate Response:

> *"This is the single most common pitfall in medical AI research. In MIMIC-CXR, a single patient often has multiple chest radiographs taken across weeks or months during their hospital stay. If you perform a random row-level split, images of the same patient end up in both the training set and the test set. Because the model memorizes unique anatomical landmarks (e.g., patient-specific pacemaker leads, rib deformities, or surgical clips), test performance appears artificially stellar.*
> *We strictly enforced a Patient-Level Split based on subject_id. All radiographs, reports, and follow-up studies belonging to a specific human patient were isolated entirely into the training set or the holdout test set, guaranteeing zero anatomical leakage."*

Part 5: The 30-Scenario Quick-Fire Stress Test Matrix

Be prepared to answer these rapid-fire curveballs in 15 seconds or less:

Curveball Question	15-Second Executive Answer
"What if your Kafka consumer crashes mid-batch?"	Consumer offsets are committed manually *after* database upsert succeeds; a rebalanced worker picks up from the last committed offset, ensuring at-least-once processing.
"What if a radiograph is inverted or sideways?"	We applied randomized affine data augmentation (rotations $\pm 10^\circ$, horizontal flips) during training; severe rotations are caught by a lightweight orientation pre-classifier.
"Why not fine-tune GPT-4 with LoRA instead of Llama-3?"	Commercial API fine-tuning lacks weight portability, risks vendor lock-in, and costs $10\times$ more per token than running quantized open-source weights locally.
"How do you handle schema changes from scraping sites?"	Scrapers output raw HTML to raw S3; parsing workers use CSS fallback selectors. If parsing fails, records route to a DLQ and alert engineers via Slack.
"Why cosine similarity over Euclidean distance?"	Cosine similarity normalizes vector magnitude, measuring semantic angular direction independently of text token length.
"How do you detect GPU memory fragmentation in PyTorch?"	Call `torch.cuda.memory_allocated()` and `torch.cuda.empty_cache()`, and use a unified batch size with gradient accumulation.

Part 6: The 7-Point Codebase Audit Checklist

Before your technical rounds, audit your code repository and resume claims against these 7 verification checks:

- [ ] **1. Number Consistency:** Do the numbers on your resume match your code? (e.g., If resume says "50,000 articles/hour", verify your batch size and latency benchmarks actually support ≈ 14 articles/second).
- [ ] **2. Dependency Hygiene:** Can you explain every package in `requirements.txt`? (e.g., Why `faiss-cpu` vs `qdrant-client`?).
- [ ] **3. Hardcoded Secrets:** Verify that zero API keys, AWS credentials, or database passwords exist in Git commit history.
- [ ] **4. Error Handling:** Check what happens if external network calls timeout. Does your code crash or retry?
- [ ] **5. Metric Reproducibility:** Ensure you can explain the exact validation script that produced your reported ROC-AUC or Recall scores.
- [ ] **6. License & Compliance:** Confirm your datasets (MIMIC-CXR, News APIs) follow non-commercial research or standard commercial licensing.
- [ ] **7. The "Could You Build It in 45 Minutes?" Test:** Can you write the core PyTorch contrastive loss loop or the Kafka consumer boilerplate on a blank whiteboard from memory?

Part 7: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────┐
│                  TECHNICAL CROSS-QUESTIONING CHEAT SHEET     │
├──────────────────────────────────────────────────────────────┤
│ 1. WHEN YOU DON'T KNOW:                                      │
│    - Don't guess. Acknowledge -> Frame constraints -> Hypothesize
│      Approach A vs B -> Propose empirical benchmark test.    │
├──────────────────────────────────────────────────────────────┤
│ 2. AI NEWS ENGINE DEFENSE:                                   │
│    - Kafka: Replayability from offset 0, spike buffering, pub-sub fanout
│    - Qdrant: INT8 Scalar Quantization cuts RAM by 75%; mmap for disk.
│    - HDBSCAN: Anchor centroids prevent cluster ID thrashing. │
├──────────────────────────────────────────────────────────────┤
│ 3. MEDICLIP DEFENSE:                                         │
│    - Clinical Safety: High-recall operating threshold (96%). │
│    - Patient Leakage: Split STRICTLY by patient subject_id!  │
│    - False Negative Masking: Soft semantic targets via CheXbert.
└──────────────────────────────────────────────────────────────┘
```

Part 8: 100-Mark Self-Assessment Examination

Section A: Technical Conceptual Defense (25 Marks — 5 marks each)

Q1: What is the primary operational advantage of Kafka's commit log over RabbitMQ's message broker architecture?

(a) Kafka consumes less disk space.

(b) Kafka messages remain replayable and persistent for multiple consumer groups after being read.

(c) Kafka does not require network connections.

(d) RabbitMQ cannot handle text data.

Q2: In vector databases, what is the primary benefit of INT8 Scalar Quantization?

(a) Increases vector dimensions by $4\times$.

(b) Reduces RAM footprint by nearly $75%$ by converting 32-bit floats to 8-bit integers with minimal precision loss.

(c) Converts Euclidean distance to Manhattan distance.

(d) Eliminates the need for indexing.

Q3: In medical imaging machine learning, what severe error occurs if train/test splits are performed randomly by image instead of by patient ID?

(a) Image files become corrupted.

(b) Severe Patient-Level Data Leakage, leading to artificially inflated test metrics that collapse on new patients.

(c) Overfitting on negative numbers.

(d) Violation of Python typing rules.

Q4: Why is HDBSCAN preferred over K-Means for streaming document clustering?

(a) K-Means requires an Nvidia GPU.

(b) K-Means requires a predefined, fixed number of clusters K, whereas HDBSCAN discovers arbitrary clusters of varying densities dynamically.

(c) HDBSCAN cannot handle text vectors.

(d) HDBSCAN only works with small datasets.

Q5: What is the primary purpose of a "Soft Target" in contrastive multimodal learning?

(a) Reduces image resolution.

(b) Prevents the model from penalizing false negative pairs that share identical clinical or semantic diagnoses.

(c) Speeds up text tokenization.

(d) Replaces the vision encoder with a linear layer.

Section B: Tough Cross-Examination Responses (35 Marks)

Q6 (15 Marks): An interviewer asks:

"In your MediCLIP project, what would happen if a hospital deployed your model on portable bedside X-ray machines that produce lower-resolution, noisier images than the stationary DICOM scanners in your training data?"

Deliver a comprehensive senior response addressing Domain Shift, Image Preprocessing Normalization, and Confidence Threshold Fallbacks.

Q7 (20 Marks): An interviewer challenges your news clustering pipeline:

"If two major breaking events occur simultaneously with overlapping keywords (e.g., two different airline strikes in different countries), how does your embedding and clustering pipeline prevent them from merging into a single story?"

Section C: Live Architecture Defense (40 Marks)

Q8 (40 Marks): You are tasked with designing the **Disaster Recovery and High Availability (HA)** architecture for your AI News Aggregator.

- Design the multi-node Kafka cluster and Qdrant vector database topology across two availability zones (AZs). (20 marks)
- Detail the exact recovery steps when an entire AWS Availability Zone experiences an unexpected total outage during live news ingestion. (20 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** Persistent, replayable commit log supporting multiple independent consumer groups.
- **Q2: (b)** Reduces RAM footprint by $\approx 75\%$ via 8-bit quantization.
- **Q3: (b)** Patient-Level Data Leakage.
- **Q4: (b)** K-Means requires fixed K; HDBSCAN discovers clusters dynamically.
- **Q5: (b)** Prevents penalizing false negative pairs sharing identical pathology.

Section B Solutions

Q6 Verbal Defense Script:
*"That represents a classic **Covariate Shift / Domain Shift** challenge. Portable bedside units produce variable exposure, motion blur, and differing contrast ratios compared to stationary machines. To ensure robust production deployment:

- *Preprocessing Harmonization:* All incoming DICOM images pass through automated Histogram Equalization (CLAHE) and resolution standardization to normalize contrast distributions before hitting the Vision Transformer.
- *Uncertainty-Gated Fallback:* Bedside images with high noise produce lower cosine similarity peaks across latent pathology vectors. If the maximum similarity score falls below our calibrated operational confidence threshold ($\tau < 0.60$), the system refuses to output an automated label and flags the scan for prioritized radiologist review.
- *Domain-Adaptive Fine-Tuning:* Before official rollout in a new hospital wing, we run an unsupervised domain adaptation step (e.g., SimCLR contrastive tuning on 500 unlabeled bedside images) to align scanner-specific low-level feature distributions.*"

Q7 Solution:

- Keyword matching fails here, but **Dense Semantic Embeddings succeed**:
The Sentence-BERT encoder encodes the full syntactic and semantic context of the articles.
Article A vectors will cluster around semantic concepts: `"airline strike", "Lufthansa", "Frankfurt", "German union"`.
Article B vectors will cluster around: `"airline strike", "Air France", "Paris", "French pension"`.
In 384-dimensional latent space, the geographical entities and airline names create substantial angular separation (cosine distance > 0.35).
Because HDBSCAN evaluates density in high-dimensional vector space rather than matching raw keywords, the two stories form two distinct, isolated clusters rather than a single merged entity.

Section C Solutions

Q8 Solution:

- **HA Topology:**
Kafka: 3-node broker cluster spread across AZ-a, AZ-b, and AZ-c. Topics configured with `replication_factor = 3` and `min.insync.replicas = 2`. Producer uses `acks = all` to guarantee zero message loss upon primary broker failure.
Qdrant: Clustered deployment across 2 AZs with Raft consensus coordination. Collections configured with `replication_factor = 2`. Reads load-balanced across replicas.
PostgreSQL: Primary DB in AZ-a with synchronous streaming replication to a Standby Replica in AZ-b.

- **AZ Failure Recovery Protocol:**
When AZ-a goes dark, Kafka Zookeeper/KRaft automatically elects the in-sync replica in AZ-b as the new partition leader in < 3 seconds.
Qdrant Raft consensus detects the lost node; surviving replicas in AZ-b continue serving vector reads and writes without downtime.
Cloud load balancer (ALB) health checks fail for AZ-a; traffic automatically routes 100% of ingestion scrapers and API queries to AZ-b. Zero data loss is experienced.

Day 20: SQL & DBMS 35-Minute Mock Interview Simulation

Timed Interview Protocol, Live Query Whiteboarding, Deep-Dive Systems Grilling & Post-Mortem Audit

---

Part 1: The 35-Minute Timed Mock Interview Protocol

```
┌──────────────────────────────────────────────────────────────────┐
│                  35-MINUTE SQL + DBMS INTERVIEW PACING           │
├──────────────────────────────────────────────────────────────────┤
│ [00:00 - 08:00] PHASE 1: RAPID-FIRE THEORY SCREEN                │
│   - Tests agility across ACID, Isolation Levels, Joins, and NULLs.│
│   - Target: Crisp 30-to-45-second answers. No rambling.          │
├──────────────────────────────────────────────────────────────────┤
│ [08:00 - 18:00] PHASE 2: ARCHITECTURE & SYSTEMS GRILLING         │
│   - Tests deep engine internals: B-Tree vs. Hash indexes, Deadlocks,│
│     MVCC mechanics, and 3NF vs. BCNF normalization.              │
├──────────────────────────────────────────────────────────────────┤
│ [18:00 - 32:00] PHASE 3: LIVE CODING CHALLENGES (2 PROBLEMS)     │
│   - Problem 1 (7 mins): Top-N per Department with Strict Tie Handling.│
│   - Problem 2 (7 mins): Consecutive Transaction Streaks (Gaps/Islands).│
├──────────────────────────────────────────────────────────────────┤
│ [32:00 - 35:00] PHASE 4: POST-MORTEM & SCORING AUDIT             │
│   - Instant error diagnosis, execution plan review, and rubric scoring.│
└──────────────────────────────────────────────────────────────────┘
```

Part 2: Phase 1 — Rapid-Fire Theory Screen (8 Minutes)

Q1 (01:00): What is the practical difference between WHERE and HAVING?

Candidate Model Answer:

> *"The WHERE clause filters individual base records before grouping occurs and can leverage standard B-tree indexes; it cannot contain aggregate functions. The HAVING clause filters aggregated group buckets after grouping has executed, and typically evaluates aggregate conditions like COUNT(*) > 2."*

Q2 (02:30): What happens when an outer join predicate is placed in the ON clause versus the WHERE clause of a LEFT JOIN?

Candidate Model Answer:

> *"In a LEFT JOIN, conditions in the ON clause determine which right-table rows attach to the left-table rows; all left-table rows are preserved regardless of whether the right-side condition matches. Placing a filter on the right table in the WHERE clause evaluates post-join; unmatched left rows have NULL right attributes, causing the WHERE filter to evaluate to UNKNOWN and discard them, silently converting the LEFT JOIN into an INNER JOIN."*

Q3 (04:00): Why does COUNT(*) differ from COUNT(column_name)?

Candidate Model Answer:

> *"COUNT(*) counts every row in the result set, including rows where all non-key columns are NULL. COUNT(column_name) counts strictly those rows where the specified column value is NOT NULL."*

Q4 (05:30): Explain why NULL = NULL does not return TRUE in SQL.

Candidate Model Answer:

> *"In ANSI SQL's Three-Valued Logic, NULL represents an unknown or missing state. Two unknown values cannot be confirmed to be identical, so the equality comparison NULL = NULL yields UNKNOWN. To match or test for nulls, one must use IS NULL or IS NOT DISTINCT FROM."*

Q5 (07:00): Under READ COMMITTED isolation, are "Non-Repeatable Reads" possible?

Candidate Model Answer:

> *"Yes. READ COMMITTED guarantees that a query only reads committed data, eliminating Dirty Reads. However, if Transaction A reads a row, and Transaction B concurrently updates that row and commits, Transaction A re-reading the same row will see the new value within the same transaction scope, experiencing a Non-Repeatable Read. Preventing this requires REPEATABLE READ or SERIALIZABLE."*

Part 3: Phase 2 — Systems Architecture & Internals Deep Dive (10 Minutes)

Q6 (09:30): How does a B+ Tree index differ physically from a Hash Index, and when does a Hash Index fail?

Candidate Model Answer:

> *"A B+ Tree stores keys in a self-balancing tree structure where all data resides in sequentially linked leaf disk pages. It excels at point lookups ($O(\log N)$), range queries (BETWEEN, >, <), prefix scans (LIKE 'abc%'), and pre-sorted output (ORDER BY).*
> *A Hash Index uses an in-memory hash table offering $O(1)$ point lookups. However, a Hash Index completely fails for range scans, sorting, or prefix searches because the hash function destroys mathematical ordering."*

Q7 (12:00): Explain the mechanics of a Deadlock. How does an engine detect it, and how do you resolve it in application architecture?

Candidate Model Answer:

> *"A deadlock occurs when two transactions hold exclusive locks on resources that the other requires to proceed, creating a circular wait dependency ($T_1 \to T_2 \to T_1$). The engine detects this using a background thread that traverses a directed Wait-For Graph (WFG) to detect cycles; upon detecting a cycle, it aborts the lowest-cost transaction and throws error 40P01.*
> *In application design, we prevent deadlocks by enforcing Deterministic Lock Ordering: all transactions must access tables and rows in the exact same sorted sequence (e.g., sorting account IDs before acquiring row locks).`**

Q8 (14:30): What is the difference between 3NF and BCNF? Give a concrete scenario where 3NF is satisfied but BCNF is violated.

Candidate Model Answer:

> *"3NF allows a functional dependency $X \to Y$ if X is a super key OR if Y is a prime attribute. BCNF is stricter: for every functional dependency, the determinant X must be a super key.*
> *A classic violation occurs with overlapping candidate keys: In StudentAdvisor(student, subject, advisor), each advisor teaches one subject (advisor -> subject), but each student has one advisor per subject ((student, subject) -> advisor). Because subject is a prime attribute (part of candidate key 1), it passes 3NF. But advisor is not a super key, so it violates BCNF, causing insertion anomalies when adding a new advisor before any students enroll."*

Q9 (17:00): What is Write-Ahead Logging (WAL) and how does it guarantee Atomicity and Durability?

Candidate Model Answer:

> *"WAL dictates that any data modification must be written to an immutable, sequential disk log before dirty buffer cache pages are flushed to the database storage files. If a crash occurs, the recovery manager replays committed transactions from the WAL (Redo phase) ensuring Durability, and rolls back uncommitted transactions using pre-image Undo logs ensuring Atomicity."*

Part 4: Phase 3 — Live Timed SQL Coding (14 Minutes)

Challenge 1 (7 Minutes): Department Top 2 Earners with Strict Tie Preservation

Problem Statement:

Given tables `emp (emp_id INT, emp_name VARCHAR, salary DECIMAL, dept_id INT)` and `dept (dept_id INT, dept_name VARCHAR)`.

Find the top 2 highest distinct salaries in each department. If employees tie for 1st or 2nd place, return all tied employees. Include `dept_name`, `emp_name`, and `salary`, ordered by `dept_name` ascending and `salary` descending.

Candidate Spoken Thought Process:

> *"I'll structure this with a CTE using DENSE_RANK(). DENSE_RANK() ensures that if two employees tie for the top salary, both receive rank 1, and the next distinct salary receives rank 2. I'll join emp with dept inside the CTE, partition by dept_id, order by salary DESC, and then filter where rnk <= 2."*

Production Solution:

```
WITH RankedDepartmentSalaries AS (
    SELECT
        d.dept_name,
        e.emp_name,
        e.salary,
        DENSE_RANK() OVER (
            PARTITION BY e.dept_id
            ORDER BY e.salary DESC
        ) AS salary_rank
    FROM emp e
    INNER JOIN dept d ON e.dept_id = d.dept_id
)
SELECT
    dept_name,
    emp_name,
    salary
FROM RankedDepartmentSalaries
WHERE salary_rank <= 2
ORDER BY dept_name ASC, salary DESC;
```

Challenge 2 (7 Minutes): Consecutive Active Transaction Days (Gaps & Islands)

Problem Statement:

Given a financial transactions table `transactions (txn_id INT, account_id INT, txn_date TIMESTAMP, amount DECIMAL)`.

Find all `account_id`s that executed at least one transaction on **four or more consecutive calendar days**.

Candidate Spoken Thought Process:

> *"First, an account can execute multiple transactions in a single day, so I must deduplicate (account_id, CAST(txn_date AS DATE)) in Step 1.*
> *Next, I apply the Difference of Sequences technique: subtract a continuous ROW_NUMBER() from the calendar date. For consecutive dates, this difference produces an identical anchor date (island_id).*
> *Finally, I group by account_id and island_id, filtering with HAVING COUNT(*) >= 4, and select distinct account IDs."*

Production Solution:

```
WITH DistinctDailyTxns AS (
    SELECT DISTINCT
```

```
        account_id,
        CAST(txn_date AS DATE) AS activity_date
    FROM transactions
),
SequencedDays AS (
    SELECT
        account_id,
        activity_date,
        activity_date - CAST(ROW_NUMBER() OVER (
            PARTITION BY account_id
            ORDER BY activity_date
        ) AS INT) * INTERVAL '1 day' AS island_key
    FROM DistinctDailyTxns
)
SELECT DISTINCT
    account_id
FROM SequencedDays
GROUP BY account_id, island_key
HAVING COUNT(*) >= 4
ORDER BY account_id;
```

Part 5: Phase 4 — Post-Mortem Diagnostic Guide & Common Pitfalls

Review your performance against the common mistakes observed in technical rounds:

```
┌──────────────────────────────────────────────────────────────────┐
│                  MOCK INTERVIEW POST-MORTEM AUDIT                │
├──────────────────────────────────────────────────────────────────┤
│ 1. DID YOU USE RANK() INSTEAD OF DENSE_RANK()?                   │
│    - If two employees tie for 1st place, RANK() assigns (1, 1, 3).│
│      Filtering for <= 2 would MISS the 2nd distinct salary entirely!│
├──────────────────────────────────────────────────────────────────┤
│ 2. DID YOU FORGET TO DEDUPLICATE DATES IN GAPS & ISLANDS?        │
│    - If a user transacts 4 times on Monday and zero times on Tuesday,│
│      a query without DISTINCT daily dates will flag Monday as a  │
│      4-day streak! Always deduplicate dates first.               │
├──────────────────────────────────────────────────────────────────┤
│ 3. DID YOU HANG SILENTLY WHILE CODING?                           │
│    - Always state your plan before typing: "I will use a CTE with│
│      DENSE_RANK() to handle ties, then filter in the outer WHERE."│
├──────────────────────────────────────────────────────────────────┤
│ 4. ARE YOUR JOIN COLUMNS COVERED BY B-TREE INDEXES?              │
│    - Mentioning: "In production, an index on (dept_id, salary DESC)│
│      eliminates the Sort operator" converts an L4 answer into an L5.│
└──────────────────────────────────────────────────────────────────┘
```

Part 6: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  MOCK INTERVIEW SUMMARY CHEAT SHEET               │
├──────────────────────────────────────────────────────────────────┤
│ 1. RAPID THEORY:                                                 │
│    - WHERE: Pre-aggregation row filter. Indexes used. No aggregates.│
│    - HAVING: Post-aggregation group filter. Evaluates aggregates.│
│    - ON vs WHERE in LEFT JOIN: ON preserves left rows; WHERE drops them│
```

```
|    - NULL = NULL: Evaluates to UNKNOWN. Use IS NULL.                        |
|-----------------------------------------------------------------------------|
| 2. SYSTEMS INTERNALS:                                                       |
|    - B+ Tree: O(log N). Handles ranges, sorting, prefix seeks.              |
|    - Hash Index: O(1) point lookups only. Fails on ranges/sorts.            |
|    - Deadlock: Cycle in Wait-For Graph. Fix via lock ordering.              |
|    - BCNF: Every determinant must be a Super Key.                           |
|-----------------------------------------------------------------------------|
| 3. LIVE CODING CORE:                                                        |
|    - Top-N: DENSE_RANK() OVER (PARTITION BY grp ORDER BY col DESC) <= N     |
|    - Gaps/Islands: Date - ROW_NUMBER() * INTERVAL '1 day' = Island Key.     |
```

Part 7: 100-Mark Self-Assessment Examination

Section A: Technical Conceptual Defense (25 Marks — 5 marks each)

Q1: What is the result of `SELECT COUNT(*)` on a table containing 10 rows where every column in 3 of those rows is `NULL`?

(a) 0

(b) 7

(c) 10

(d) Throws a NullPointerException

Q2: Under which ANSI SQL isolation level are Phantom Reads completely eliminated in standard relational databases?

(a) Read Uncommitted

(b) Read Committed

(c) Repeatable Read (Postgres MVCC / MySQL Next-Key) and Serializable

(d) Snapshot Read Only

Q3: Why does a composite index on `(dept_id, salary, hire_date)` fail to accelerate a query with `WHERE salary > 80000`?

(a) `salary` is a numeric type.

(b) It violates the Left-Prefix Rule: the leftmost column (`dept_id`) is omitted from the predicate.

(c) Greater-than operators cannot be indexed.

(d) Indexes only support primary keys.

Q4: What is the primary difference between a clustered index and a non-clustered index?

(a) Clustered indexes are stored in RAM; non-clustered indexes are on disk.

(b) A clustered index dictates the physical sort order of actual data pages on disk (strictly 1 per table); a non-clustered index is a secondary B-tree pointing to row locators.

(c) Clustered indexes do not support integers.

(d) Non-clustered indexes cannot have duplicate keys.

Q5: In a BCNF decomposition of $R(A, B, C)$ where $A \to B$ and $B \to C$, what are the resulting relations?

(a) $R_1(A, B)$ and $R_2(B, C)$

(b) $R_1(A, C)$ and $R_2(B, C)$

(c) $R_1(A, B, C)$

(d) $R_1(A)$ and $R_2(B)$

Section B: Live Query Problem Solving (50 Marks)

Q6 (25 Marks - Department Budget Ranking):

Given `projects (project_id, dept_id, budget, start_date)`. Write a query to find the 3rd highest distinct budget in each department. If a department has fewer than 3 distinct budgets, return `NULL` for that department. Include `dept_id` and `third_highest_budget`.

Q7 (25 Marks - Monthly User Churn):

Given a table `logins (user_id INT, login_date DATE)`. Write a query to calculate the number of users who logged in during January 2024 but did NOT log in during February 2024.

Section C: Systems Optimization & Execution Plans (25 Marks)

Q8 (25 Marks): An interviewer shows you this `EXPLAIN ANALYZE` snippet on a 15-million row `transactions` table:

```
Hash Join  (cost=125000.00..845000.00 rows=450 width=48) (actual time=1820.12..3450.45 rows=12 loops=1)
  Hash Cond: (t.user_id = u.user_id)
  ->  Seq Scan on transactions t  (actual rows=15000000 loops=1)
  ->  Hash  (actual rows=50 loops=1)
        ->  Index Scan on users u  (actual rows=50 loops=1)
```

- Explain what the query planner did and why execution took over 3.4 seconds. (10 marks)
- What single index should you create on `transactions` to replace the 15-million row `Seq Scan` with an immediate `Index Seek`, dropping runtime to under 5ms? (15 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (c)** 10 (`COUNT(*)` counts all physical rows).
- **Q2: (c)** Serializable (and Repeatable Read in Postgres/MySQL InnoDB).
- **Q3: (b)** Violates the Left-Prefix Rule.
- **Q4: (b)** Clustered index orders physical data pages directly on disk (strictly 1).
- **Q5: (a)** $R_1(A, B)$ and $R_2(B, C)$.

Section B Solutions

Q6 Solution:

```
WITH RankedBudgets AS (
    SELECT
        dept_id,
        budget,
        DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY budget DESC) AS rnk
    FROM projects
),
ThirdBudgets AS (
    SELECT DISTINCT dept_id, budget AS third_highest_budget
    FROM RankedBudgets
    WHERE rnk = 3
)
SELECT
    d.dept_id,
    t.third_highest_budget
FROM (SELECT DISTINCT dept_id FROM projects) d
LEFT JOIN ThirdBudgets t ON d.dept_id = t.dept_id;
```

Q7 Solution:

```
SELECT COUNT(DISTINCT user_id) AS churned_users_count
FROM logins
WHERE login_date >= '2024-01-01' AND login_date < '2024-02-01'
  AND user_id NOT IN (
      SELECT DISTINCT user_id
      FROM logins
      WHERE login_date >= '2024-02-01' AND login_date < '2024-03-01'
  );
```

Section C Solutions

Q8 Solution:

- **Bottleneck:** The planner built an in-memory hash table of 50 users from the `Index Scan` on `users`. However, because `transactions.user_id` lacked a usable index, the engine had to perform a full `Seq Scan` across all 15 million rows of `transactions` to probe the hash table, taking 3.45 seconds.
- **Optimal Index:**`CREATE INDEX idx_transactions_user_id ON transactions (user_id);`

With an index on `transactions.user_id`, the planner replaces the Hash Join with an **Indexed Nested Loop Join**: for each of the 50 users, it performs a logarithmic $O(\log N)$ B-tree index seek into `transactions`, executing in < 5 milliseconds.

Day 21: Python, Analytics & ML 45-Minute Mock Interview Simulation

Live Algorithmic Coding, Pandas Wrangling, Applied Statistics & ML System Design

---

Part 1: Phase 1 — Algorithmic Coding (12 Minutes)

The Challenge: Minimum Window Substring

Problem Statement:

Given two strings `s` and `t` of lengths m and n respectively, return the minimum window substring of `s` such that every character in `t` (including duplicates) is included in the window. If there is no such substring, return the empty string `""`.

Candidate Spoken Thought Process:

> *"I'll solve this using the Dynamic Sliding Window pattern with two frequency hash maps: t_counts to store the target character frequencies, and window_counts to track the current window in s.*
> *I'll track two integer variables: required, which is the number of unique characters in t, and formed, which tracks how many unique characters currently meet the target frequency in the window.*
> *As the right pointer expands the window, whenever window_counts[char] == t_counts[char], I increment formed. Once formed == required, the window is valid. Then, I contract the left pointer to minimize the window size while preserving validity.*
> *This ensures an optimal $O(m + n)$ time complexity because both pointers advance at most m times, with $O(1)$ auxiliary space since the character set is bounded by standard ASCII."*

Production Solution:

```
from collections import Counter

def min_window(s: str, t: str) -> str:
    if not s or not t or len(s) < len(t):
        return ""

    t_counts = Counter(t)
    required = len(t_counts)

    window_counts = {}
    formed = 0

    # ans tuple: (window_length, left_index, right_index)
    ans = (float("inf"), None, None)
    left = 0

    for right, char in enumerate(s):
        window_counts[char] = window_counts.get(char, 0) + 1

        if char in t_counts and window_counts[char] == t_counts[char]:
            formed += 1
```

```
        # Contract window from left as long as it remains valid
        while left <= right and formed == required:
            current_char = s[left]

            # Update smallest window found so far
            if (right - left + 1) < ans[0]:
                ans = (right - left + 1, left, right)

            window_counts[current_char] -= 1
            if current_char in t_counts and window_counts[current_char] < t_counts[current_char]:
                formed -= 1

            left += 1

    return "" if ans[0] == float("inf") else s[ans[1]:ans[2] + 1]

# Verification
assert min_window("ADOBECODEBANC", "ABC") == "BANC"
assert min_window("a", "a") == "a"
assert min_window("a", "aa") == ""
```

Part 2: Phase 2 — Production Pandas Wrangling (12 Minutes)

The Challenge: Customer Sessionization & Conversion Metrics

Problem Statement:

You are given a raw clickstream DataFrame `events` with columns: `['user_id', 'event_timestamp', 'event_type', 'item_id']`.

An active shopping session is defined as a sequence of events for a user where the gap between consecutive events is **less than 30 minutes**.

- Sessionize the data: assign a unique sequential `session_id` to each user session.
- Produce an executive summary table showing:

Total sessions per user.
Average session duration in minutes.
Session conversion rate (percentage of sessions containing at least one `'purchase'` event).

Candidate Spoken Thought Process:

> *"First, I ensure event_timestamp is parsed as a datetime and sort by user_id and event_timestamp.*
> *To detect session boundaries, I use .groupby('user_id')['event_timestamp'].diff() to find the time delta between consecutive events.*
> *Whenever delta > 30 minutes or is null (the first event), a new session starts. I convert this boolean condition into a cumulative sum (cumsum()), which generates a unique sequential session_id per user.*
> *Finally, I group by user_id and session_id to compute duration and purchase flags, followed by a second aggregation to produce the user summary metrics."*

Production Solution:

```
import pandas as pd
import numpy as np

def analyze_user_sessions(events: pd.DataFrame) -> pd.DataFrame:
    # 1. Type Enforcement & Sorting
    df = events.copy()
    df["event_timestamp"] = pd.to_datetime(df["event_timestamp"])
    df = df.sort_values(["user_id", "event_timestamp"]).reset_index(drop=True)

    # 2. Compute Intersite Inactivity Deltas
    time_diff = df.groupby("user_id")["event_timestamp"].diff()

    # 3. New Session Flag: True if first event or gap >= 30 minutes
    new_session_flag = time_diff.isna() | (time_diff >= pd.Timedelta(minutes=30))

    # 4. Generate User-Specific Session ID via Cumulative Sum
    df["session_seq"] = df.groupby("user_id")[new_session_flag.name].cumsum()
    df["session_id"] = df["user_id"].astype(str) + "_s" + df["session_seq"].astype(str)

    # 5. Session-Level Aggregation
    session_summary = df.groupby(["user_id", "session_id"]).agg(
        session_start=("event_timestamp", "min"),
        session_end=("event_timestamp", "max"),
        has_purchase=("event_type", lambda x: (x == "purchase").any())
    ).reset_index()

    session_summary["duration_minutes"] = (
        (session_summary["session_end"] - session_summary["session_start"]).dt.total_seconds() / 60.0
    )

    # 6. User-Level Executive Summary
    user_report = session_summary.groupby("user_id").agg(
        total_sessions=("session_id", "count"),
        avg_session_duration_mins=("duration_minutes", "mean"),
        conversion_rate_pct=("has_purchase", lambda x: round(100.0 * x.mean(), 2))
    ).reset_index()

    return user_report
```

Part 3: Phase 3 — Applied Statistics & Probability (10 Minutes)

The Challenge: A/B Testing Early-Stopping Paradox & Bayesian Fraud

Interviewer:

- *"An analyst runs an A/B test for 3 days. They observe $p = 0.038$ on day 3 and immediately declare Variant B the winner and stop the test. What statistical error was committed, and what is the consequence?"*
- *"A high-value payment gateway processes transactions where $0.2%$ are fraudulent. Our fraud model has an $80%$ Sensitivity (Recall) and a $1%$ False Positive Rate. A transaction is flagged by the model. What is the exact probability that this transaction is actually fraudulent?"*

Candidate Spoken Thought Process:

> *"1. The analyst committed the Continuous Peeking Problem (Alpha Inflation). By checking the p-value repeatedly and stopping as soon as it crossed 0.05, the true False Positive Rate is not 5%, but can inflate up to*

> *30–40%. The observed lift is likely random noise (winner's curse). The test should have run for a predetermined fixed sample horizon based on power calculations.*
> *2. For the second question, this is a classic application of Bayes' Theorem illustrating the Base Rate Fallacy."*

Mathematical Proof Walkthrough:

$$P(\text{Fraud}) = 0.002, \quad P(\text{Legit}) = 0.998$$
$$P(\text{Flag} \mid \text{Fraud}) = 0.80, \quad P(\text{Flag} \mid \text{Legit}) = 0.01$$
$$\text{Marginal } P(\text{Flag}) = (0.80 \times 0.002) + (0.01 \times 0.998) = 0.0016 + 0.00998 = 0.01158$$
$$P(\text{Fraud} \mid \text{Flag}) = \frac{0.80 \times 0.002}{0.01158} = \frac{0.0016}{0.01158} \approx 0.1382 \quad (\mathbf{13.82\%})$$

Spoken Conclusion:

> *"Even with an 80% sensitive model and a low 1% false positive rate, only 13.8% of flagged transactions are actual fraud. Over 86% of the flags are false alarms due to the extremely rare 0.2% base rate. This is why automated blocking would be disastrous; instead, flagged transactions should route to human review or trigger multi-factor authentication (SMS/2FA)."*

Part 4: Phase 4 — End-to-End Machine Learning Business Scenario (11 Minutes)

The Challenge: E-Commerce Chargeback Prevention Engine

Interviewer: *"We are building an automated transaction risk engine for a merchant processing $50M monthly. If a fraudulent charge slips through, we lose the $100 product value plus a $25 bank chargeback fee ($125 total loss). If we falsely reject a good customer, we lose the $15 profit margin and cause user friction. How do you formulate, build, evaluate, and tune this model?"*

Candidate Spoken Architecture Plan:

```
┌──────────────────────────────────────────────────────────────────┐
│                  FRAUD ENGINE DESIGN ARCHITECTURE                │
├──────────────────────────────────────────────────────────────────┤
│ 1. PROBLEM FORMULATION:                                          │
│    - Binary Classification with Severe Class Imbalance (~0.5% fraud).
│    - Cost Asymmetry: Cost(FN) = $125 | Cost(FP) = $15 (8.3x ratio).
├──────────────────────────────────────────────────────────────────┤
│ 2. EVALUATION METRIC:                                            │
│    - DO NOT USE ACCURACY OR ROC-AUC!                             │
│    - Optimize Precision-Recall AUC (PR-AUC) and Cost-Utility Function:
│      Total Cost = ($15 * FP) + ($125 * FN).                      │
├──────────────────────────────────────────────────────────────────┤
│ 3. MODEL FAMILY & LEAKAGE PREVENTION:                            │
│    - XGBoost / LightGBM with scale_pos_weight = 8.3              │
│    - Time-Series Split (Walk-Forward validation).                │
│    - Group features strictly into backward-looking sliding windows
│      (e.g., velocity: transactions in last 10 mins).             │
├──────────────────────────────────────────────────────────────────┤
│ 4. THRESHOLD OPTIMIZATION:                                       │
```

```
|    - Default 0.5 threshold will miss too many $125 losses.              |
|    - Sweep operational threshold from 0.05 to 0.95 to find minimum      |
|      dollar loss point (typically around 0.12 - 0.18).                  |
```

Part 5: 10-Minute Rapid Revision Sheet

```
┌─────────────────────────────────────────────────────────────────────────┐
│                   ANALYTICS & ML MOCK CHEAT SHEET                       │
├─────────────────────────────────────────────────────────────────────────┤
│ 1. MINIMUM WINDOW SUBSTRING:                                            │
│    - Dynamic sliding window. Track formed == required unique chars.     │
│    - Contract left pointer while valid. O(M + N) time, O(1) space.      │
├─────────────────────────────────────────────────────────────────────────┤
│ 2. PANDAS SESSIONIZATION:                                               │
│    - diff() > 30 mins -> boolean flag -> cumsum() = session_id.         │
├─────────────────────────────────────────────────────────────────────────┤
│ 3. APPLIED STATISTICS:                                                  │
│    - Base Rate Fallacy: P(A|B) is low when P(A) is rare, even with      │
│      high test sensitivity! P(Fraud|Flag) = 13.8%.                      │
│    - Peeking Problem: Early stopping inflates alpha up to 30-40%.       │
├─────────────────────────────────────────────────────────────────────────┤
│ 4. BUSINESS ML COST ALIGNMENT:                                          │
│    - Threshold = argmin[ Cost(FP)*FP + Cost(FN)*FN ].                   │
│    - When Cost(FN) >> Cost(FP), lower threshold from 0.50 to ~0.15.     │
└─────────────────────────────────────────────────────────────────────────┘
```

Part 6: 100-Mark Self-Assessment Examination

Section A: Algorithmic & Pandas Mechanics (25 Marks — 5 marks each)

Q1: What is the time complexity of the Minimum Window Substring sliding window algorithm on string `s` of length M and target `t` of length N?

(a) $O(M \times N)$

(b) $O(M + N)$

(c) $O(M \log N)$

(d) $O(M^2)$

Q2: In Pandas, what is the effect of applying `.cumsum()` to a boolean Series `[True, False, False, True, False]`?

(a) `[1, 0, 0, 1, 0]`

(b) `[1, 1, 1, 2, 2]`

(c) `[True, False]`

(d) `TypeError`

Q3: In a binary classification problem where the positive class is 0.1% of the data, what will a naive dummy classifier that always predicts 0 achieve in accuracy?

(a) 0.1%

(b) 50.0%

(c) 99.9%

(d) 100.0%

Q4: In Bayes' Theorem, what term represents the prior probability of evidence $P(B)$?

(a) Posterior

(b) Likelihood

(c) Marginal Evidence

(d) Prior

Q5: Why does standard K-Fold cross-validation cause optimistic data leakage when applied to time-series forecasting datasets?

(a) It shuffles future data into the training set, allowing the model to peek into the future.

(b) It drops missing values.

(c) It requires too much memory.

(d) Time-series data cannot be divided into K parts.

Section B: Analytical Implementation (50 Marks)

Q6 (25 Marks - Pandas Rolling Active Users):

Given a DataFrame `user_logins (user_id, login_timestamp)`. Write a production Pandas script to compute the **7-Day Rolling Active Users (7D-RAU)** for each calendar day: the count of unique `user_id`s who logged in within the 7-day window ending on that day.

Q7 (25 Marks - Optimal Threshold Computation):

Given a cost matrix where a False Negative costs $500 and a False Positive costs $20. You have an array of true labels `y_true` (10,000 samples with 100 positives) and predicted probabilities `y_probs`. Write a Python function to find the exact threshold between 0.05 and 0.50 that minimizes total financial loss.

Section C: Business Problem Architecture (25 Marks)

Q8 (25 Marks): You are designing a recommendation system for an online grocery delivery app.

- Why is an Item-Item collaborative filtering model preferred over User-User collaborative filtering for grocery baskets? (10 marks)
- How do you incorporate item expiration / shelf-life into the recommendation ranking score? (15 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** $O(M + N)$ linear time.
- **Q2: (b)** `[1, 1, 1, 2, 2]`.
- **Q3: (c)** 99.9% (demonstrating why accuracy is useless for imbalanced data).
- **Q4: (c)** Marginal Evidence.
- **Q5: (a)** Future data leaks into the past during random shuffling.

Section B Solutions

Q6 Solution:

```
import pandas as pd

def calculate_7d_rolling_active_users(logins: pd.DataFrame) -> pd.DataFrame:
    df = logins.copy()
    df["login_date"] = pd.to_datetime(df["login_timestamp"]).dt.floor("D")

    # 1. Deduplicate user logins per calendar day
    daily_unique = df.drop_duplicates(["login_date", "user_id"])

    # 2. Complete calendar date range
    all_dates = pd.date_range(daily_unique["login_date"].min(), daily_unique["login_date"].max(),
freq="D")

    # 3. Compute 7-day rolling unique count
    records = []
    for current_date in all_dates:
        start_date = current_date - pd.Timedelta(days=6)
        active_in_window = daily_unique[
            (daily_unique["login_date"] >= start_date) &
            (daily_unique["login_date"] <= current_date)
        ]["user_id"].nunique()
        records.append({"date": current_date, "rolling_7d_active_users": active_in_window})

    return pd.DataFrame(records)
```

Q7 Solution:

```
import numpy as np
from sklearn.metrics import confusion_matrix

def find_optimal_loss_threshold(y_true, y_probs):
    best_thresh = 0.50
    min_loss = float('inf')
```

```
    for thresh in np.arange(0.05, 0.51, 0.01):
        y_pred = (y_probs >= thresh).astype(int)
        tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
        loss = (fp * 20) + (fn * 500)

        if loss < min_loss:
            min_loss = loss
            best_thresh = round(thresh, 2)

    return best_thresh, min_loss
```

Section C Solutions

Q8 Solution:

- **Item-Item CF Preference:**

  In grocery platforms, user tastes vary dramatically over time, but item relationships are static and stable (e.g., people who buy milk frequently buy cereal or coffee).

  The number of unique catalog items ($10,000$) is orders of magnitude smaller than the number of active users ($5,000,000$). Computing and caching the Item-Item similarity matrix requires minimal memory ($10,000 \times 10,000$), whereas User-User similarity is computationally intractable.

- **Shelf-Life Ranking Score:**

  Formulate the final ranking score as a composite objective:

  $$\text{FinalScore} = w_1 \cdot \text{RelevanceScore} + w_2 \cdot \text{UrgencyPenalty}$$

  Where $\text{UrgencyPenalty} = \exp\left(-\frac{\text{DaysUntilExpiration}}{\lambda}\right)$. Items nearing expiration receive a boost in the carousel ranking (or are offered via dynamic discount bundles), minimizing warehouse food waste while maintaining consumer relevance.

Day 22: HR, Behavioral & Executive Communication Mastery

The STAR Framework, Company Alignment (vConstruct), Conflict Resolution & Strategic Storytelling

---

Part 1: The Executive Communication Playbook

Senior engineering hires are evaluated not just on coding syntax, but on **leadership, communication clarity, empathy, and business accountability**.

```
┌──────────────────────────────────────────────────────────────────┐
│                  BEHAVIORAL INTERVIEW SUCCESS RULES               │
├──────────────────────────────────────────────────────────────────┤
│ 1. TIME DISCIPLINE:                                              │
│    - Answers must stay between 90 and 120 seconds. Longer causes │
│      interviewers to lose focus; shorter sounds superficial.     │
├──────────────────────────────────────────────────────────────────┤
│ 2. THE STAR METHOD (Structure Everything):                       │
│    - Situation (15s): The context and baseline business pain.    │
│    - Task (15s): Your specific mandate and ownership.            │
│    - Action (45s): The concrete technical and human steps YOU took.  │
│    - Result (30s): Quantified business impact ($, %, latency, hours).  │
├──────────────────────────────────────────────────────────────────┤
│ 3. ELIMINATE EMPTY CLICHÉS:                                      │
│    - Never say: "I am a hardworking perfectionist."              │
│    - Say: "I historically struggled with premature optimization, but  │
│      now use strict 80/20 latency profiling before refactoring."  │
├──────────────────────────────────────────────────────────────────┤
│ 4. NEVER BLAME OTHERS:                                           │
│    - Frame past conflicts around conflicting system constraints or   │
│      unaligned incentives, never personal incompetence.          │
└──────────────────────────────────────────────────────────────────┘
```

Part 2: The 6 Core Behavioral Question Scripts

Question 1: "Tell me about yourself." (The 90-Second Professional Hook)

Framework: Present $\to$ Past $\to$ Future Alignment

Spoken Script:

> *"I am a Data Science Engineer specializing in building scalable relational pipelines, analytical data warehouses, and applied machine learning systems.*
> *Currently, my core focus is at the intersection of data engineering and applied AI. I design end-to-end pipelines using SQL, Python, Snowflake, and Apache Airflow that transform messy, raw transactional data into high-performance analytical models and feature stores. For instance, in my recent work on an AI intelligence aggregation pipeline, I architected a distributed ingestion system processing over 50,000 hourly events with sub-*

> *second vector search retrieval.*
> *Prior to this, I honed my fundamentals across relational database internals, query optimization, statistical modeling, and full-stack software development. I thrive in environments where data isn't just stored, but engineered to unlock actionable operational efficiency.*
> *What excites me about this opportunity today is combining my data engineering and ML foundations with vConstruct's mission of transforming global construction and virtual design workflows through data-driven automation."*

Question 2: "Why vConstruct and why this Data Science Engineer role?"

Framework: Domain Specificity $\to$ Technical Alignment $\to$ Value Contribution

Spoken Script:

> *"I've followed vConstruct's work closely, and what stands out to me is how you are digitizing one of the largest, most operationally complex industries in the world: Architecture, Engineering, and Construction (AEC). While many tech companies optimize ad clicks, vConstruct solves tangible, physical-world challenges through Virtual Design & Construction (VDC), BIM integration, and project controls.*
> *Construction projects generate massive volumes of fragmented data—from 3D BIM models and schedule revisions to field sensor logs and cost procurement tracking. The Data Science Engineer role here is critical because success requires bridging the gap between heavy relational database engineering and advanced predictive analytics.*
> *My background in designing resilient ELT pipelines, optimizing complex SQL queries across millions of records, and deploying applied ML models directly aligns with your tech stack. I want to build data infrastructure that empowers project teams across the US, Europe, and India to eliminate schedule delays, predict cost overruns, and automate reporting at scale."*

Question 3: "Explain a difficult technical problem you solved."

Framework: The STAR Technical Narrative

Spoken Script:

> ***Situation:*** *"In my AI news aggregation pipeline, we ingested 50,000 articles hourly across 1,000 sources. Our downstream users needed real-time semantic story clustering. However, our initial clustering service was crashing every 15 minutes due to memory exhaustion."*
> ***Task:*** *"I was tasked with redesigning the clustering and vector retrieval architecture to operate within a 16 GB RAM server budget while reducing end-to-end clustering latency from 18 minutes to under 1 second."*
> ***Action:*** *"I profiled our memory and discovered that pairwise cosine comparison across incoming articles was scaling at $O(N^2)$, performing over 100 million dot products.*
> *To resolve this, I implemented a two-tier architecture:*
> *First, I deployed a Qdrant vector database using HNSW graph indexing and INT8 Scalar Quantization, which*

> *compressed our 384-dimensional embeddings by 75% in memory.*
> *Second, instead of re-clustering all articles from scratch, I engineered an Anchor Centroid Protocol: new articles only queried the top-20 nearest neighbor centroids ($O(\log N)$). HDBSCAN was executed strictly on the unassigned candidate pool."*
> ***Result:** "This architectural refactoring dropped our p99 clustering latency from 18 minutes to 350 milliseconds, reduced cloud compute costs by $62%, and maintained a zero-downtime SLA during breaking news traffic spikes."*

Question 4: "Describe a mistake, conflict, or team challenge."

Framework: Self-Awareness $\to$ Constructive Resolution $\to$ Process Improvement

Spoken Script:

> ***Situation:** "In a previous data warehouse initiative, our analytics engineering team and the backend software development team had a significant disagreement regarding schema evolution. The backend team deployed an unannounced database migration that renamed two foreign keys, which immediately broke our downstream nightly ETL pipeline and caused our executive reporting dashboard to display zero values."*
> ***Conflict & Emotion:** "Tensions were high because executive leadership questioned data reliability, and initial conversations focused on who was at fault."*
> ***Action:** *"I stepped in to de-escalate the tension. Instead of assigning blame, I scheduled a joint 30-minute post-mortem with the backend lead. I acknowledged that our pipeline lacked defensive decoupling and proposed a permanent systemic solution:*
>
> *We introduced **Data Contracts** defined in YAML that codified schema expectations.*
> *I integrated automated schema-validation checks into their GitHub CI/CD deployment workflow, so any breaking column alteration in the backend automatically alerted the data team before code merged to staging.*
> *In our pipeline, I added a Dead Letter Queue (DLQ) quarantine pattern, ensuring that schema drift no longer halted the entire pipeline."**
>
> ***Result:** "This eliminated unannounced schema breaks entirely over the following two quarters, restored executive confidence in our dashboards, and created a collaborative weekly sync between software engineering and data teams."*

Question 5: "Are you comfortable learning unfamiliar tools and working with global teams?"

Framework: Systematic Learning Methodology $\to$ Asynchronous Best Practices

Spoken Script:

> *"Absolutely. In data engineering and AI, technologies evolve constantly; tools are simply means to an end. Whenever I encounter an unfamiliar technology—whether it's learning dbt, adapting to Snowflake's micro-partition tuning, or working with a proprietary BIM API—I follow a structured 3-step learning framework:
>
> Read the official architecture documentation from first principles.
> Build a minimal proof-of-concept pipeline in an isolated scratch environment.
> Review production post-mortems and GitHub issues to identify known edge cases and anti-patterns.
>
> Regarding global collaboration: I have extensive experience collaborating across Indian, European, and US timezones. Working successfully across timezones requires an **asynchronous-first culture**: writing clear, comprehensive documentation, maintaining clean Git pull requests with detailed architectural context, establishing unambiguous task tickets with clear definition-of-done criteria, and respecting overlap windows for high-bandwidth synchronous discussions."*

Question 6: "What are your strengths, weaknesses, and 3-year career goals?"

Framework: Evidence-Based Strengths $\to$ Transparent Weakness with Mitigation $\to$ Ambitious Path

Spoken Script:

> ***Strengths:*** *"My greatest strength is my first-principles systems approach to data problems. I don't just write queries or train models in isolation; I think about the entire lifecycle—data contracts at ingestion, indexing and execution plans at storage, cost-sensitive threshold economics in modeling, and automated testing in production."*
> ***Weakness:*** *"Historically, my primary weakness was premature optimization. In early projects, I would spend days micro-optimizing a query or building a complex caching layer before validating whether that step was actually the true system bottleneck. I overcame this by adopting a strict profiling-first discipline: I now always benchmark using EXPLAIN ANALYZE or Python profiling before optimizing, focusing solely on the top 20% of operations causing 80% of latency."*
> ***3-Year Career Goals:*** *"Over the next 3 years, my goal is to grow from a strong individual contributor into a Lead Data Architect. I want to master large-scale distributed streaming architectures, spearhead modern data platform initiatives, and mentor junior engineers, establishing best practices in data quality, cloud cost governance, and applied AI deployment."*

Part 3: The Reverse-Interviewing Playbook: Questions to Ask Them

At the end of every interview, when the interviewer asks *"Do you have any questions for us?"*, asking generic questions (*"What is the company culture like?"*) wastes a massive opportunity. Use these **Senior-Level Questions**:

- *"What does the data maturity curve look like across vConstruct's client projects today? Are most teams operating in standard batch reporting, or is there an active push toward real-time telemetry and predictive models?"*
- *"How does the data engineering team handle schema evolution and data contracts with upstream AEC software tools like Revit, Navisworks, or Procore?"*
- *"What is currently the single biggest technical bottleneck or source of technical debt in your existing data platform?"*
- *"How does the engineering team balance building rapid bespoke client analytics versus abstracting reusable internal platform tooling?"*
- *"What would success look like for someone in this Data Science Engineer role in the first 90 days?"*

Part 4: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────┐
│                BEHAVIORAL & HR INTERVIEW CHEAT SHEET          │
├──────────────────────────────────────────────────────────────┤
│ 1. ELEVATOR PITCH (90s):                                     │
│    - Present (Data Science Engineer / SQL / Python / Cloud)  │
│    - Past (Built resilient pipelines & ML systems; quantified wins)
│    - Future (Align with vConstruct's AEC digital transformation).
├──────────────────────────────────────────────────────────────┤
│ 2. STAR NARRATIVE:                                           │
│    - Situation: 50k articles/hr, clustering crashed from memory.
│    - Task: Reduce runtime to < 1s within 16 GB RAM budget.   │
│    - Action: INT8 Qdrant quantization + Anchor Centroid 2-tier model.
│    - Result: Latency 18 min -> 350ms (-62% cost, zero downtime).
├──────────────────────────────────────────────────────────────┤
│ 3. CONFLICT RESOLUTION:                                      │
│    - Never personal blame. Frame around unaligned systems.   │
│    - Solution: Implemented Data Contracts + CI/CD schema validation.
├──────────────────────────────────────────────────────────────┤
│ 4. WEAKNESS FORMULATION:                                     │
│    - Premature optimization -> Solved via profiling-first discipline.
└──────────────────────────────────────────────────────────────┘
```

Part 5: 100-Mark Self-Assessment Examination

Section A: Strategic Communication Concepts (25 Marks — 5 marks each)

Q1: What is the ideal target duration for answering a standard behavioral interview question?

(a) 15–30 seconds

(b) 90–120 seconds

(c) 5–7 minutes

(d) As long as the interviewer stays silent

Q2: In the STAR method, what does the 'R' stand for, and how must it be articulated?

(a) Recommendation; state what the company should do.

(b) Result; must be quantified with measurable business or engineering impact ($, %, latency).

(c) Reflection; talk about how you felt.

(d) Review; ask the interviewer for feedback.

Q3: What is the primary flaw of citing "I am a perfectionist" as your professional weakness?

(a) It is grammatically incorrect.

(b) It is an insincere cliché that fails to demonstrate genuine self-awareness and active remediation.

(c) It violates company HR policy.

(d) Perfectionism is not allowed in engineering.

Q4: When asked about a past conflict with a teammate, what is the single most critical behavioral principle to follow?

(a) Prove to the interviewer that you were technically correct.

(b) Frame the conflict around misaligned system constraints or missing processes, never personal incompetence.

(c) Deny that any conflict ever occurred.

(d) State that you escalated the matter to human resources immediately.

Q5: What is the primary purpose of asking insightful reverse-interview questions at the end of the interview?

(a) To pass the time until the hour ends.

(b) To demonstrate technical curiosity, strategic business understanding, and senior-level ownership.

(c) To find out how much vacation time you get.

(d) To test the interviewer's coding ability.

Section B: Live Behavioral Scripting (50 Marks)

Q6 (25 Marks - Overcoming Project Failure):

Write a complete, spoken STAR script (under 120 words) answering:

"Describe a time when a project or pipeline deployment failed in production. How did you handle the failure, and what did you implement to ensure it never happened again?"

Q7 (25 Marks - Cross-Functional Negotiation):

Write a complete, spoken STAR script (under 120 words) answering:

"Describe a situation where a non-technical product manager or business stakeholder requested an unrealistic delivery timeline for a machine learning model or data pipeline. How did you negotiate and manage expectations?"

Section C: Company Research & Domain Scenario (25 Marks)

Q8 (25 Marks): Explain how a Data Science Engineer at **vConstruct** contributes to reducing project risk, material waste, and schedule overruns on large-scale commercial construction projects. Detail the role of data integration between Building Information Modeling (BIM) tools and historical cost databases.

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** 90–120 seconds.
- **Q2: (b)** Result with quantified metrics.
- **Q3: (b)** Insincere cliché lacking authentic self-awareness.
- **Q4: (b)** Frame around system constraints and process fixes, never personal blame.
- **Q5: (b)** Demonstrates technical curiosity, ownership, and strategic alignment.

Section B Solutions

Q6 Spoken Script:

- *Situation:* During a Black Friday deployment, our automated ETL pipeline crashed because an upstream payment vendor introduced an undocumented null field in transaction records.
- *Task:* As on-call data engineer, I had to restore our real-time revenue dashboard within a 30-minute SLA.
- *Action:* I immediately activated our fallback failover, rolling back the pipeline to the prior release. Then, rather than applying a quick hack, I refactored the ingestion worker with a Pydantic schema validator and a Dead Letter Queue quarantine, allowing clean transactions to flow while routing corrupted records to an audit table.
- *Result:* Service was restored in 18 minutes with zero data loss, and the quarantine pattern permanently eliminated pipeline crashes from schema drift.

Q7 Spoken Script:

- *Situation:* A business stakeholder requested a complex customer churn prediction model deployed in two weeks, which was unrealistic given our uncleaned raw data.
- *Task:* I needed to manage stakeholder expectations without demoralizing the team or shipping an unvalidated model.
- *Action:* I scheduled a 15-minute alignment sync. Instead of flatly saying 'no', I broke the deliverables into a phased MVP roadmap: Phase 1 delivered a rules-based heuristic dashboard in 5 days that gave immediate 80/20 visibility. Phase 2 allocated 3 weeks for proper cross-validation, feature engineering, and model training.
- *Result:* The stakeholder was thrilled to have immediate business visibility via Phase 1, and our team was given the required runway to deliver a robust, leak-free Random Forest model that achieved an 86% ROC-AUC.

Section C Solutions

Q8 Solution:

- In commercial construction, projects frequently suffer from **Cost Overruns and Schedule Delays** caused by uncoordinated spatial clashes, supply chain delays, and manual progress tracking.
- A Data Science Engineer at vConstruct bridges the gap between 3D BIM models (Revit, Navisworks) and ERP project management databases (Procore, Primavera P6):
Automated Clash Analytics: Extracting spatial conflict logs from Navisworks into a centralized Snowflake warehouse to identify which design trades (e.g., HVAC vs. Electrical) repeatedly cause design bottlenecks.
Earned Value Management (EVM): Building automated ELT pipelines that correlate physical field installation velocity with planned baseline schedules, flagging schedule slippage weeks before critical milestones fail.
Predictive Procurement: Training regression and time-series models on historical material pricing and lead times to optimize bulk purchasing schedules, directly reducing material waste and hedging against inflation.

Day 23: Full Two-Stage Technical & Project Mock Interview Simulation

Complete 65-Minute Two-Round Simulation: Breadth Screening + Deep-Dive Architecture & HR

Part 1: Round 1 — Technical Foundations & Core Coding (35 Minutes)

Section A: SQL & Query Execution (10 Minutes)

Question 1 (Theory):

Interviewer: *"Walk me through the exact logical execution order of an ANSI SQL query with a GROUP BY, HAVING, and LEFT JOIN."*

Candidate Model Answer:

> *"The database executes clauses in this strict logical sequence:*
>
> *FROM: Table references and Cartesian product are identified.*
> *ON: Join condition is evaluated for each record pair.*
> *JOIN: Matching rows are retained; unmatched left rows are preserved with NULL padding.*
> *WHERE: Row-level filters are applied to the combined intermediate table.*
> *GROUP BY: Rows sharing identical keys are collapsed into summary group buckets.*
> *HAVING: Filters are applied to the aggregated summary buckets.*
> *SELECT: Projection list, expressions, and window functions are evaluated.*
> *DISTINCT: Duplicate output rows are eliminated.*
> *ORDER BY: Output rows are sorted.*
> *LIMIT / OFFSET: The requested slice of rows is emitted.*
> *This execution order explains why aliases declared in SELECT cannot be referenced in WHERE or GROUP BY clauses."*

Live Challenge 1 (Coding):

Interviewer: *"Given employees (emp_id, emp_name, dept_id, salary). Find the employee with the second-highest salary in each department. If a department has ties for second place, return all tied employees. If a department has only one employee, return nothing for that department."*

```
WITH RankedDeptSalaries AS (
    SELECT
```

```
        dept_id,
        emp_name,
        salary,
        DENSE_RANK() OVER (
            PARTITION BY dept_id
            ORDER BY salary DESC
        ) AS salary_rank
    FROM employees
)
SELECT
    dept_id,
    emp_name,
    salary
FROM RankedDeptSalaries
WHERE salary_rank = 2
ORDER BY dept_id, emp_name;
```

Section B: DBMS Architecture & Internals (10 Minutes)

Question 2:

Interviewer: *"Explain what a 'Covering Index' is, how the INCLUDE clause works in PostgreSQL/SQL Server, and why it provides massive throughput gains."*

Candidate Model Answer:

> *"A Covering Index is a non-clustered secondary index that contains all columns requested by a query, allowing the engine to satisfy the request via an Index-Only Scan without touching the underlying base table heap. Standard composite index keys (CREATE INDEX idx ON tbl (A, B)) store attributes in the internal B-tree search nodes, increasing tree height and index size. The INCLUDE (C, D) clause stores non-key attributes strictly in the leaf nodes of the B-tree.*
> *This eliminates the expensive Key Lookup / Bookmark Lookup (random disk I/O to fetch row attributes from the clustered table), cutting query latency from hundreds of milliseconds to under 2ms for high-concurrency workloads."*

Question 3:

Interviewer: *"What is the difference between Optimistic Concurrency Control (OCC) and Pessimistic Locking with SELECT ... FOR UPDATE?"*

Candidate Model Answer:

> *"Pessimistic locking assumes contention is high; it acquires exclusive row locks immediately upon reading (SELECT FOR UPDATE), forcing concurrent transactions to block until commit. It prevents conflicts at the cost of concurrency.*
> *OCC assumes conflicts are rare. Transactions read data without locks, record a version or timestamp, and validate whether the record was modified before writing. If modified, the transaction rolls back and retries. OCC provides vastly superior throughput for read-heavy systems with rare write collisions."*

Section C: Python Engineering & Data Structures (10 Minutes)

Question 4 (Theory):

Interviewer: *"What is the difference between list.append(x) and list.extend(iterable) in Python in terms of memory operations and performance?"*

Candidate Model Answer:

> *"append(x) adds the single object reference x to the end of the list in amortized $O(1)$ time, resizing the internal dynamic array when capacity is reached.*
> *extend(iterable) iterates over the argument and appends each element individually. Under the hood, CPython optimizes extend by calculating the length of the incoming iterable and resizing the internal buffer in a single memory reallocation, making extend significantly faster than repeated append calls in a Python loop."*

Live Challenge 2 (Coding):

Interviewer: *"Given an array of integers nums, find the contiguous subarray (containing at least one number) which has the largest sum and return its sum (Kadane's Algorithm). What is its time and space complexity?"*

```
def max_sub_array(nums: list[int]) -> int:
    """
    Kadane's Algorithm.
    Time Complexity: O(N) single pass.
    Space Complexity: O(1) constant auxiliary memory.
    """
    if not nums:
        return 0

    max_so_far = nums[0]
    current_max = nums[0]

    for num in nums[1:]:
        # Decide whether to add current num to running subarray, or start fresh from current num
        current_max = max(num, current_max + num)
        max_so_far = max(max_so_far, current_max)

    return max_so_far

assert max_sub_array([-2, 1, -3, 4, -1, 2, 1, -5, 4]) == 6  # [4, -1, 2, 1]
assert max_sub_array([1]) == 1
assert max_sub_array([-1]) == -1
```

Section D: High-Level Architecture Screen (5 Minutes)

Question 5:

Interviewer: *"If an Apache Airflow pipeline extracts data from a REST API and loads it into Snowflake, where should data transformations occur: in Python before loading, or in Snowflake after loading?"*

Candidate Model Answer:

> *"Data transformations should occur inside Snowflake after loading using an ELT paradigm (typically managed via dbt).*
> *Transforming data in Python on the Airflow worker bottlenecks the pipeline to the CPU/RAM limits of the Airflow VM, creates a single point of failure, and permanently discards raw data.*
> *In modern ELT, raw JSON/Parquet lands directly in Snowflake Bronze storage. Transformations are then executed using Snowflake's decoupled virtual warehouses, leveraging columnar parallel query engines and preserving raw audit trails for full reprocessing."*

Part 2: Round 2 — Project Deep Dive, Production SQL & HR Alignment (30 Minutes)

Section A: Project Architecture Deep Dive (15 Minutes)

Interviewer: *"Let's talk about your MediCLIP medical imaging project. You trained a multimodal contrastive model on chest X-rays and radiology reports. Tell me about a time when the model produced a clinically catastrophic prediction during evaluation, how you diagnosed it, and what you changed."*

Candidate Spoken Response:

> **"During our initial validation on CheXpert holdout test sets, our zero-shot model achieved a high 0.88 AUROC on Cardiomegaly and Pneumonia, but completely collapsed on **Pneumothorax (collapsed lung)**, achieving an AUROC of only 0.54—barely better than random guessing. In an emergency department, a missed pneumothorax can lead to tension pneumothorax and cardiac arrest.*
>
> *I conducted an error audit and discovered two root causes:*
>
> *Resolution Downsampling Distortion: Our standard Vision Transformer patch projection downsampled DICOM images from 2048×2048 to 224×224. Pneumothorax presents as a paper-thin visceral pleural line and subtle absence of peripheral lung markings; downsampling was completely blurring out these micro-pathology features!*
> *Severe Class Imbalance: Pneumothorax was present in less than 2% of our pretraining reports.*
>
> *The Engineering Solution:*
>
> *First, I refactored the image input pipeline to a **Multi-Crop High-Resolution Architecture**: the model took a global 224×224 view paired with 448×448 local crops of the apical and lateral pleural spaces.*
>
> *Second, I modified our contrastive InfoNCE loss function to incorporate **Hard Negative Mining**: the dataloader deliberately oversampled batches containing pneumothorax cases and similar confounding conditions like lung cysts and skin folds.*

> *On re-evaluation, Zero-Shot AUROC on Pneumothorax jumped from **0.54 to 0.835**, and Recall at high sensitivity reached **89%**."**

Section B: Production Schema Live SQL Challenge (8 Minutes)

Problem Statement:

Given an e-commerce subscription table `subscriptions (sub_id, customer_id, start_date, end_date)` where `end_date` can be `NULL` (indicating an active ongoing subscription).

Write a query to calculate the **Monthly Active Subscriber Count** for each calendar month of 2024. A customer is considered active in month M if their subscription started on or before the last day of month M, and either has no end date or ended on or after the first day of month M.

```
WITH RECURSIVE Months2024 AS (
    SELECT
        CAST('2024-01-01' AS DATE) AS month_start,
        CAST('2024-01-31' AS DATE) AS month_end
    UNION ALL
    SELECT
        CAST(month_start + INTERVAL '1 month' AS DATE),
        CAST(month_start + INTERVAL '2 month' - INTERVAL '1 day' AS DATE)
    FROM Months2024
    WHERE month_start < '2024-12-01'
)
SELECT
    m.month_start,
    COUNT(DISTINCT s.customer_id) AS active_subscribers
FROM Months2024 m
LEFT JOIN subscriptions s
    ON s.start_date <= m.month_end
   AND (s.end_date IS NULL OR s.end_date >= m.month_start)
GROUP BY m.month_start
ORDER BY m.month_start;
```

Section C: Behavioral Leadership & vConstruct Alignment (7 Minutes)

Interviewer: *"Why vConstruct? You have strong data science and engineering skills that could apply to finance or e-commerce. Why construction technology?"*

Candidate Spoken Response:

> *"Finance and e-commerce have spent the last two decades optimizing digital pipelines, but the Built Environment—which represents 13% of global GDP—is currently undergoing its foundational digital transformation.*
> *In construction, fragmented data between 3D BIM models, field sensors, and scheduling ERPs leads to massive real-world waste, material loss, and chronic project delays. Working at vConstruct gives me the rare opportunity to apply rigorous data science and data engineering to tangible, physical infrastructure.*
> *Building an automated pipeline that detects cost overruns weeks before concrete is poured, or training models that optimize equipment procurement schedules, creates direct economic and environmental impact. I want to*

> *build data infrastructure that shapes how global mega-projects are delivered."*

Part 3: Post-Mock Diagnostic Error Matrix

```
┌──────────────────────────────────────────────────────────────┐
│                  MOCK INTERVIEW PERFORMANCE MATRIX           │
├──────────────────────────────────────────────────────────────┤
│ CATEGORY 1: TECHNICAL CLARITY (Target: 95%+)                 │
│  - Did you explain WHY a tool was chosen rather than just WHAT it is?
│  - Did you state time and space complexity without being asked?
├──────────────────────────────────────────────────────────────┤
│ CATEGORY 2: CODE ROBUSTNESS & SYNTAX (Target: 95%+)          │
│  - Did Kadane's algorithm handle all-negative arrays correctly?
│  - Did your subscription query handle NULL end dates properly?
├──────────────────────────────────────────────────────────────┤
│ CATEGORY 3: PROJECT DEFENSE AUTHENTICITY (Target: 90%+)      │
│  - Did you share a genuine failure and technical pivot (Pneumothorax)?
│  - Were you able to defend the trade-offs under probing?     │
├──────────────────────────────────────────────────────────────┤
│ CATEGORY 4: CULTURAL & DOMAIN FIT (Target: 90%+)             │
│  - Did your answer to "Why vConstruct" cite specific AEC domain points
│    (BIM, VDC, project controls, material waste)?             │
└──────────────────────────────────────────────────────────────┘
```

Part 4: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────┐
│                  FULL MOCK INTERVIEW CHEAT SHEET             │
├──────────────────────────────────────────────────────────────┤
│ 1. EXECUTION ORDER:                                          │
│    FROM -> ON -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT ->
│    DISTINCT -> ORDER BY -> LIMIT.                            │
├──────────────────────────────────────────────────────────────┤
│ 2. COVERING INDEX:                                           │
│    - Non-key cols in INCLUDE live in leaf nodes. Zero Key Lookups!
├──────────────────────────────────────────────────────────────┤
│ 3. KADANE'S ALGORITHM:                                       │
│    - current_max = max(num, current_max + num); O(N) time, O(1) space.
├──────────────────────────────────────────────────────────────┤
│ 4. INTERVAL MATCHING:                                        │
│    - Start <= Period_End AND (End IS NULL OR End >= Period_Start).
├──────────────────────────────────────────────────────────────┤
│ 5. VCONSTRUCT HOOK:                                          │
│    - Transforming 13% of global GDP via BIM, VDC & predictive controls.
└──────────────────────────────────────────────────────────────┘
```

Part 5: 100-Mark Self-Assessment Examination

Section A: Technical Precision (25 Marks — 5 marks each)

Q1: In Kadane's algorithm for Maximum Subarray Sum, why does initializing `max_so_far = 0` produce an incorrect result when all array elements are negative (e.g., `[-5, -2, -8]`)?

(a) Causes integer division by zero.

(b) The algorithm will output 0, even though the true maximum subarray sum is `-2`.

(c) Causes an IndexError in Python.

(d) Kadane's algorithm only supports positive integers.

Q2: In an ANSI SQL query, which clause is logically evaluated immediately AFTER `HAVING`?

(a) WHERE

(b) SELECT

(c) GROUP BY

(d) FROM

Q3: What is the primary operational advantage of storing non-key columns in the `INCLUDE` clause of a secondary B-tree index rather than as part of the composite key?

(a) The index can be stored in text files.

(b) It keeps the B-tree internal navigation nodes lean and shallow while eliminating key lookups.

(c) It allows the column to store duplicate primary keys.

(d) It enables FULL OUTER JOIN.

Q4: In a subscription database where `end_date IS NULL` indicates an active subscription, what is the correct condition to identify subscriptions active during January 2024 ($[S_M, E_M]$)?

(a) `start_date = '2024-01-01'`

(b) `start_date <= '2024-01-31' AND (end_date IS NULL OR end_date >= '2024-01-01')`

(c) `end_date >= '2024-01-31'`

(d) `start_date BETWEEN '2024-01-01' AND '2024-01-31'`

Q5: What is the average time complexity of finding an element in a balanced B+ Tree of N records?

(a) $O(1)$

(b) $O(\log N)$

(c) $O(N)$

(d) $O(N \log N)$

Section B: Live Coding & Architecture Implementation (50 Marks)

Q6 (25 Marks - Live SQL Coding):

Given a table `employee_checkins (id, emp_id, checkin_date)`. Write a query to find all employees who checked in on at least 5 consecutive calendar days.

Q7 (25 Marks - Live Python Algorithmic Coding):

Write a Python function `find_peak_element(nums: list[int]) -> int` that finds a peak element (an element strictly greater than its neighbors) in strictly $O(\log n)$ runtime.

Section C: Systems & Behavioral Scenarios (25 Marks)

Q8 (25 Marks): Explain how you would architect an automated data pipeline that ingests daily material delivery manifests and 3D BIM model updates to calculate **Construction Schedule Slippage** at vConstruct. Detail the data sources, ingestion framework, warehouse modeling, and executive KPI output.

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** Outputs 0 instead of the true maximum negative element `-2`.
- **Q2: (b)** `SELECT`.
- **Q3: (b)** Keeps internal nodes lean; eliminates key lookups.
- **Q4: (b)** `start_date <= '2024-01-31' AND (end_date IS NULL OR end_date >= '2024-01-01')`.
- **Q5: (b)** $O(\log N)$.

Section B Solutions

Q6 Solution:

```
WITH DistinctDays AS (
    SELECT DISTINCT emp_id, CAST(checkin_date AS DATE) AS c_date
    FROM employee_checkins
),
Sequenced AS (
    SELECT
        emp_id,
```

```
        c_date,
        c_date - CAST(ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY c_date) AS INT) * INTERVAL '1
day' AS island_key
    FROM DistinctDays
)
SELECT DISTINCT emp_id
FROM Sequenced
GROUP BY emp_id, island_key
HAVING COUNT(*) >= 5;
```

Q7 Solution:

```
def find_peak_element(nums: list[int]) -> int:
    """
    Binary Search on Peak.
    If nums[mid] < nums[mid + 1], a peak MUST exist in the right half.
    Otherwise, a peak exists in the left half (including mid).
    Time Complexity: O(log N).
    Space Complexity: O(1).
    """
    left = 0
    right = len(nums) - 1

    while left < right:
        mid = left + (right - left) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid

    return left
```

Section C Solutions

Q8 Solution:

- **Data Ingestion:**

Source 1 (BIM/VDC): Autodesk BIM 360 / Revit API webhooks exporting planned element metadata (element ID, planned install date, spatial zone).
Source 2 (Field Tracking): Daily field installation logs and barcode scans from ERP / Procore.
Ingested via Azure Data Factory into an ADLS Gen2 Bronze data lake as JSON/Parquet.

- **Warehouse Modeling (Snowflake / Silver Layer):**

Staging models built via dbt reconciling `planned_install_date` with `actual_install_date` joined on `bim_element_uuid`.

- **Gold Marts & Executive KPIs:**

`fct_construction_progress (project_id, trade_id, date_key, planned_qty, actual_qty, variance_days)`.
Core Metric: **Schedule Performance Index (SPI)** = Earned Value / Planned Value.
Power BI executive dashboard alerting Project Managers when SPI < 0.85, predicting project completion date slippage weeks in advance.

Day 24: Targeted Gap Remediation & Weak Area Repair

Deep Diagnostic Framework, Advanced Problem-Solving Drills & High-Difficulty Reinforcement

---

Part 1: The Diagnostic Triad: Identifying Your Weakest Areas

In technical interview preparation, rereading familiar notes gives a false sense of mastery. Real improvement requires **active problem-solving on your exact points of failure**:

```
┌──────────────────────────────────────────────────────────────┐
│                  THE THREE TYPICAL FAILURE ARCHETYPES        │
├──────────────────────────────────────────────────────────────┤
│ ARCHETYPE A: SQL WINDOW FRAMING & COMPLEX STREAKS            │
│   - Symptoms: Struggles when gaps are multi-day, or when streaks
│     reset based on numerical thresholds rather than consecutive dates.
│   - Remedy: Master the running conditional sum & difference method.
├──────────────────────────────────────────────────────────────┤
│ ARCHETYPE B: DISTRIBUTED SYSTEMS, CONCURRENCY & LOCKING      │
│   - Symptoms: Confusion between Read Committed vs. Repeatable Read
│     anomalies; inability to trace deadlocks or query plan spills.
│   - Remedy: Draw physical transaction timelines & Wait-For graphs.
├──────────────────────────────────────────────────────────────┤
│ ARCHETYPE C: MEMORY SCALING & PRODUCTION CRASHES             │
│   - Symptoms: Writing code that loads entire datasets into RAM;
│     unfamiliarity with chunking, generators, and INT8 quantization.
│   - Remedy: Enforce strict O(1) memory streaming architectures.
└──────────────────────────────────────────────────────────────┘
```

Part 2: Deep Repair Area 1: Advanced SQL Streaks & Custom Gaps

---

Drill 1: Streaks Resetting on Thresholds (Non-Date Gaps)

Problem Statement:

Given a sensor telemetry table `server_metrics (server_id INT, log_time TIMESTAMP, cpu_utilization FLOAT)`.

Find all incidents where a server's CPU utilization exceeded **$90.0%$ for at least 3 consecutive readings**, returning the `server_id`, the start time of the incident, the end time, and the count of readings.

```
WITH AlertFlags AS (
    SELECT
        server_id,
        log_time,
        cpu_utilization,
        CASE WHEN cpu_utilization > 90.0 THEN 1 ELSE 0 END AS is_critical
    FROM server_metrics
),
```

```
IslandGroups AS (
    SELECT
        server_id,
        log_time,
        cpu_utilization,
        is_critical,
        -- Subtracting row number of critical items from overall row number forms island groups!
        ROW_NUMBER() OVER (PARTITION BY server_id ORDER BY log_time) -
        ROW_NUMBER() OVER (PARTITION BY server_id, is_critical ORDER BY log_time) AS island_id
    FROM AlertFlags
)
SELECT
    server_id,
    MIN(log_time) AS incident_start,
    MAX(log_time) AS incident_end,
    COUNT(*) AS consecutive_critical_readings
FROM IslandGroups
WHERE is_critical = 1
GROUP BY server_id, island_id
HAVING COUNT(*) >= 3
ORDER BY server_id, incident_start;
```

Drill 2: Non-Overlapping User Sessionization in Pure SQL

Problem Statement:

Given web clickstream events `web_events (event_id, user_id, event_time)`. A user session expires after **15 minutes of inactivity**. Assign a unique `session_number` to each user's events using pure ANSI SQL without procedural loops.

```
WITH LaggedEvents AS (
    SELECT
        user_id,
        event_time,
        LAG(event_time) OVER (
            PARTITION BY user_id
            ORDER BY event_time
        ) AS prev_event_time
    FROM web_events
),
SessionStarters AS (
    SELECT
        user_id,
        event_time,
        CASE
            WHEN prev_event_time IS NULL THEN 1
            WHEN event_time > prev_event_time + INTERVAL '15 minute' THEN 1
            ELSE 0
        END AS is_new_session
    FROM LaggedEvents
)
SELECT
    user_id,
    event_time,
    -- Running sum of session starters produces the session number!
    SUM(is_new_session) OVER (
        PARTITION BY user_id
        ORDER BY event_time
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS session_number
```

```
FROM SessionStarters
ORDER BY user_id, event_time;
```

Part 3: Deep Repair Area 2: Concurrency, Memory & Production Failures

---

Drill 3: Diagnosing & Eliminating a Financial Double-Spend Deadlock

Problem Statement:

Two transactions attempt to execute atomic transfers between Accounts A and B simultaneously:

- Txn 1: Transfers $100 from Account 1 to Account 2.
- Txn 2: Transfers $50 from Account 2 to Account 1.

In production under high load, this causes recurring `deadlock detected` crashes.

Explain why the deadlock occurs and write the production SQL stored procedure that permanently eliminates the deadlock.

```
DEADLOCK MECHANISM:
Txn 1 locks Account 1 (Exclusive Lock).
Txn 2 locks Account 2 (Exclusive Lock).
Txn 1 requests lock on Account 2 (BLOCKED by Txn 2).
Txn 2 requests lock on Account 1 (BLOCKED by Txn 1).
===> CIRCULAR WAIT: DEADLOCK!
```

Production Resolution: Deterministic Lock Ordering

```
CREATE OR REPLACE PROCEDURE transfer_balance(
    sender_id INT,
    receiver_id INT,
    transfer_amount DECIMAL
)
LANGUAGE plpgsql
AS $$
DECLARE
    first_id INT;
    second_id INT;
BEGIN
    -- ALWAYS lock accounts in the exact same numerical sequence!
    IF sender_id < receiver_id THEN
        first_id := sender_id;
        second_id := receiver_id;
    ELSE
        first_id := receiver_id;
        second_id := sender_id;
    END IF;

    -- Acquire locks in strict ascending order
    PERFORM 1 FROM accounts WHERE account_id = first_id FOR UPDATE;
    PERFORM 1 FROM accounts WHERE account_id = second_id FOR UPDATE;

    -- Execute transfer safely without risk of circular wait
    UPDATE accounts SET balance = balance - transfer_amount WHERE account_id = sender_id;
    UPDATE accounts SET balance = balance + transfer_amount WHERE account_id = receiver_id;
```

```
    COMMIT;
END;
$$;
```

Drill 4: Streaming Multi-Gigabyte CSV Parsing in Python with Zero Memory Overhead

Problem Statement:

A batch data pipeline receives a 25 GB CSV log file containing 100 million lines. The worker has strictly 2 GB of RAM. Write an idiomatic Python generator that parses, validates, and batches these records without crashing the process.

```
import csv
from typing import Generator

def stream_large_log_file(file_path: str, batch_size: int = 10_000) -> Generator[list[dict], None,
None]:
    """
    Streams and batches a multi-gigabyte file in O(1) memory.
    Reads line-by-line via iterator protocol, yielding memory-isolated chunks.
    """
    batch = []
    with open(file_path, mode="r", encoding="utf-8") as file:
        reader = csv.DictReader(file)
        for row in reader:
            # Inline data cleansing & validation
            if row.get("status_code") and row["status_code"].isdigit():
                clean_record = {
                    "user_id": int(row["user_id"]),
                    "endpoint": row["endpoint"].strip(),
                    "status_code": int(row["status_code"]),
                    "latency_ms": float(row["latency_ms"])
                }
                batch.append(clean_record)

            if len(batch) >= batch_size:
                yield batch
                batch = []  # Clear memory reference for garbage collection

        if batch:
            yield batch
```

Part 4: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  WEAK AREA REPAIR CHEAT SHEET                    │
├──────────────────────────────────────────────────────────────────┤
│ 1. CONDITIONAL ISLANDS:                                          │
│    - ROW_NUMBER() [overall] - ROW_NUMBER() [conditional] = Island Key.
│ 2. PURE SQL SESSIONIZATION:                                      │
│    - LAG() -> Diff > 15m -> Flag (1/0) -> SUM() OVER (Running Total)
│      = Sequential Session Number!                                │
├──────────────────────────────────────────────────────────────────┤
│ 3. DEADLOCK PREVENTION:                                          │
│    - Always acquire locks in STRICT SORTED ORDER (first_id < second_id)
│      to permanently break circular wait cycles.                  │
├──────────────────────────────────────────────────────────────────┤
│ 4. STREAMING GENERATORS:                                         │
```

```
    - csv.DictReader + yield batches prevents OutOfMemoryError.
```

Part 5: 100-Mark Self-Assessment Examination

Section A: Concurrency & Diagnostic Analysis (25 Marks — 5 marks each)

Q1: What condition is mathematically required for a deadlock to occur between two transactions?

(a) Both transactions must be read-only.

(b) A circular wait dependency in the Wait-For Graph ($T_1 \to T_2 \to T_1$).

(c) The database must be unindexed.

(d) Isolation level must be Read Uncommitted.

Q2: In pure SQL sessionization, how is the session number generated after identifying session start events?

(a) Using `DENSE_RANK()`.

(b) Computing a cumulative running sum of the session-start boolean flags: `SUM(is_new_session) OVER (ORDER BY event_time)`.

(c) Multiplying by 15.

(d) Using `AVG() OVER()`.

Q3: Why does deterministic lock ordering (e.g., sorting account IDs before `SELECT ... FOR UPDATE`) permanently prevent deadlocks?

(a) It increases transaction speed.

(b) It prevents circular wait: no transaction can request a lower-numbered resource while holding a higher-numbered resource.

(c) It forces all queries to execute sequentially on a single thread.

(d) It disables row-level locking.

Q4: What happens if an unpruned Decision Tree is trained on a dataset with 1,000 features and 1,000 rows?

(a) The model achieves high bias and underfits.

(b) The tree memorizes every row, achieving 100% training accuracy and massive test variance (overfitting).

(c) The tree cannot compile.

(d) Feature weights become zero.

Q5: In Python, what is the space complexity of an unconsumed generator expression `(x**2 for x in range(10_000_000))`?

(a) $O(N)$ (requires ≈ 80 MB of RAM)

(b) $O(1)$ (requires ≈ 200 bytes of RAM)

(c) $O(\log N)$

(d) $O(N^2)$

Section B: Advanced Problem Solving (50 Marks)

Q6 (25 Marks - Stock Price Volatility Streak):

Given a financial table `stock_prices (stock_id, price_date, closing_price)`. Write a query to find all stocks that experienced **at least 4 consecutive trading days of strictly increasing closing prices**. Output `stock_id`, streak start date, streak end date, and total days in the streak.

Q7 (25 Marks - Python LRU Cache):

Implement a Least Recently Used (LRU) Cache class `LRUCache(capacity: int)` with methods `get(key: int) -> int` and `put(key: int, value: int)` where both operations execute in strictly $O(1)$ average time complexity using `collections.OrderedDict`.

Section C: Enterprise Architecture Failure Recovery (25 Marks)

Q8 (25 Marks): A nightly ETL pipeline loading 20 million rows into an analytics database fails with:

`ERROR: out of memory for query result (disk spill quota exceeded in TempDB)`.

- Diagnose the query operator causing the spill (Hash Join, Sort, Window Function). (10 marks)
- Detail three architectural strategies to permanently fix the memory spill. (15 marks)

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** Circular wait in the Wait-For Graph.
- **Q2: (b)** Cumulative running sum of session start flags.
- **Q3: (b)** Breaks circular wait condition by enforcing global resource hierarchy.
- **Q4: (b)** 100% training accuracy with severe overfitting.

- **Q5: (b)** $O(1)$ constant memory (≈ 200 bytes).

Section B Solutions

Q6 Solution:

```
WITH PriceDeltas AS (
    SELECT
        stock_id,
        price_date,
        closing_price,
        CASE
            WHEN closing_price > LAG(closing_price) OVER (PARTITION BY stock_id ORDER BY price_date)
            THEN 1 ELSE 0
        END AS is_increase
    FROM stock_prices
),
IslandGroups AS (
    SELECT
        stock_id,
        price_date,
        closing_price,
        is_increase,
        ROW_NUMBER() OVER (PARTITION BY stock_id ORDER BY price_date) -
        ROW_NUMBER() OVER (PARTITION BY stock_id, is_increase ORDER BY price_date) AS island_id
    FROM PriceDeltas
)
SELECT
    stock_id,
    MIN(price_date) AS streak_start,
    MAX(price_date) AS streak_end,
    COUNT(*) AS consecutive_increasing_days
FROM IslandGroups
WHERE is_increase = 1
GROUP BY stock_id, island_id
HAVING COUNT(*) >= 4
ORDER BY stock_id, streak_start;
```

Q7 Solution:

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        # Move accessed key to end to represent most recent usage
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            # Pop first item (least recently used)
            self.cache.popitem(last=False)
```

Section C Solutions

Q8 Solution:

- **Diagnosis:** An unindexed `ORDER BY` or an analytical window function (`ROW_NUMBER() OVER (...)`) forced the execution engine to perform an external disk sort (`external merge disk`). Because the sort volume exceeded allocated `work_mem`, intermediate pages spilled to TempDB, exhausting storage quotas.
- **Remediations:**

Index Pre-sorting: Create a composite B-tree index matching the exact `PARTITION BY` and `ORDER BY` keys, streaming pre-sorted records directly into the accumulator and eliminating the Sort operator entirely.
Tuning Work Memory: Increase `work_mem` or `temp_buffers` for the pipeline user connection (`SET work_mem = '512MB'`).
Partitioned Chunking: Break the 20-million row load into daily date partition slices using incremental dbt models or ADF tumbling windows.

Day 25: Data Engineering & Business Intelligence Comprehensive Revision

End-to-End Modern Data Stack, Pipeline Blueprinting, Production Debugging & BI Architecture

---

Part 1: The Modern Data Stack Architectural Blueprint



```
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼ (dbt Transformation as Code)
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 4. TRANSFORMATION AS CODE (dbt - Data Build Tool)
│
│    - Silver Layer: Cleansed, deduplicated 3NF tables; enforces dbt tests (unique, not_null)
│
│    - Gold Layer: Star Schema Dimensional Models (fct_sales, dim_customers, dim_date)
│
│    - Lineage Graph: Directed Acyclic Graph compiled via {{ ref() }} macros
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼ (VertiPaq DirectQuery or Import Refresh)
┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 5. BUSINESS INTELLIGENCE & REPORTING (Power BI & Modern Excel)
│
│    - Star Schema: Single-direction 1:* relationships; denormalized dimensions
│
│    - DAX Analytical Measures: Dynamic Filter Context evaluation, CALCULATE(), Time Intelligence
│
│    - Executive Cockpit: Real-time KPIs, Pareto distribution, and automated anomaly alerts
│
└──────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Part 2: The 6-Step Systematic Pipeline Debugging Protocol

When an interviewer asks: *"Your nightly pipeline failed at 03:00 AM, and executive dashboards are displaying stale data. How do you investigate, fix, and communicate the incident?"*, follow this production protocol:

```
┌──────────────────────────────────────────────────────────────────────┐
│              THE 6-STEP PRODUCTION INCIDENT PROTOCOL                 │
├──────────────────────────────────────────────────────────────────────┤
│ STEP 1: TRIAGE & STAKEHOLDER COMMUNICATION (0 - 5 mins)              │
│  - Post status notice to Slack #data-incidents. Set report banner.   │
├──────────────────────────────────────────────────────────────────────┤
│ STEP 2: ISOLATE THE POINT OF FAILURE IN THE DAG (5 - 10 mins)        │
│  - Inspect Airflow Task Instance logs or ADF Activity run output.    │
│  - Classify failure: Ingestion vs. Storage vs. Transformation vs. BI.│
├──────────────────────────────────────────────────────────────────────┤
│ STEP 3: ANALYZE ERROR SIGNATURES & ROOT CAUSE (10 - 20 mins)         │
│  - Auth Error: Expired Key Vault secret or token refresh failure.    │
│  - Schema Drift: Upstream source dropped or renamed a critical column.│
│  - Memory/Spill: Sort operator exceeded work_mem; OOM kill on worker.│
│  - Constraint Failure: dbt test failed on duplicate natural keys.    │
├──────────────────────────────────────────────────────────────────────┤
│ STEP 4: APPLY DEFENSIVE REMEDIATION (20 - 35 mins)                   │
│  - Quarantine corrupted records to Dead Letter Queue (DLQ).          │
│  - Update dbt model or schema contract; hotfix configuration.        │
```

```
├──────────────────────────────────────────────────────────────────────┤
│ STEP 5: IDEMPOTENT BACKFILL REPLAY (35 - 45 mins)                    │
│  - Re-run the specific failed task slice using Tumbling Window rerun.  │
│  - Verify MERGE statement upserted records without duplication.      │
├──────────────────────────────────────────────────────────────────────┤
│ STEP 6: POST-MORTEM & PERMANENT PREVENTION (Within 24 hours)         │
│  - Publish formal 5-Whys root cause analysis (RCA).                  │
│  - Implement automated dbt source freshness tests and alert rules.   │
└──────────────────────────────────────────────────────────────────────┘
```

Part 3: Cross-Cutting Data Engineering & BI Interview Bank

Q1: Why does a Star Schema outperform a denormalized "One Big Table" (OBT) in Power BI, whereas OBT is often preferred in Snowflake/BigQuery?

Answer:

- **In Power BI (VertiPaq Engine):** VertiPaq uses columnar in-memory dictionary encoding. Storing descriptive strings (e.g., customer addresses) in a single normalized dimension table encodes the text once; repeating those strings across 50 million rows in an OBT destroys dictionary compression and balloons RAM usage. Furthermore, Star Schemas enable clean, intuitive DAX filter context propagation.
- **In Snowflake / BigQuery (Cloud OLAP):** Storage is cheap and compute is massively parallelized (MPP). Querying an OBT completely eliminates join overhead and join shuffle across distributed compute nodes, making OBT optimal for fast ad-hoc scanning.

Q2: What is "Context Transition" in DAX, and how does it relate to SQL window functions?

Answer:
In DAX, Context Transition occurs when `CALCULATE()` is executed within an active Row Context (such as inside a calculated column or iterator like `SUMX`). `CALCULATE()` converts all column values of the current row into an equivalent active Filter Context.

This is conceptually identical to a SQL window function `OVER (PARTITION BY current_row_values)`: it calculates an aggregated metric across the subset of records that share the current row's coordinates without collapsing the row itself.

Q3: How do you handle late-arriving dimension records in a real-time data warehouse?

Answer:
When a transactional fact record arrives (e.g., a sale made by `customer_id = 999`), but the corresponding customer profile has not yet replicated from the CRM:

- **Inferred Member Pattern:** The pipeline inserts a placeholder "dummy" record into `dim_customers` with `customer_id = 999`, setting attributes to `'Unknown / Pending'` and recording an `is_inferred = TRUE` flag.
- The fact record joins successfully with the inferred dimension record, preserving referential integrity and preventing pipeline failure.
- When the true customer record eventually arrives from the CRM, a `MERGE` statement overwrites the placeholder attributes while preserving the original surrogate key.

Part 4: 10-Minute Rapid Revision Sheet

```
┌──────────────────────────────────────────────────────────────────┐
│                  DATA ENGINEERING & BI CHEAT SHEET               │
├──────────────────────────────────────────────────────────────────┤
│ 1. PIPELINE BLUEPRINT:                                           │
│    - API/CDC -> ADF/Airflow -> ADLS/S3 Bronze -> Snowflake Silver ->
│      dbt Gold Star Schema -> Power BI VertiPaq Dashboard.        │
├──────────────────────────────────────────────────────────────────┤
│ 2. DEBUGGING CHECKLIST:                                          │
│    - Check Airflow task logs -> Identify error class (Auth, Schema,
│      Memory, Duplicates) -> Quarantine to DLQ -> Replay via MERGE.
├──────────────────────────────────────────────────────────────────┤
│ 3. SNOWFLAKE vs POWER BI MODELING:                               │
│    - Snowflake: OBT or Star Schema. Decoupled storage & compute. │
│    - Power BI : STRICT STAR SCHEMA. Minimize column cardinality! │
├──────────────────────────────────────────────────────────────────┤
│ 4. ESSENTIAL DAX:                                                │
│    - CALCULATE(): Only way to modify filter context.             │
│    - ALL(): Strips filters (Grand Total).                        │
│    - ALLSELECTED(): Strips visual filters, preserves slicers.    │
└──────────────────────────────────────────────────────────────────┘
```

Part 5: 100-Mark Self-Assessment Examination

Section A: Architectural Foundations (25 Marks — 5 marks each)

Q1: What is the primary operational purpose of a Self-Hosted Integration Runtime (SHIR) in Azure Data Factory?

(a) To execute Python scripts on GPUs.

(b) To establish a secure bridge for copying data between private on-premises corporate networks and the Azure cloud without opening inbound firewall ports.

(c) To host web APIs.

(d) To replace Snowflake.

Q2: In dbt, what does the macro `{{ source('raw_data', 'orders') }}` accomplish compared to hardcoding `raw_data.orders`?

(a) Encrypts the table with AES-256.

(b) Declares external source dependencies, enabling automated source freshness testing and graphical lineage tracing.

(c) Converts the table into a CSV file.

(d) Automatically deletes null values.

Q3: In Power BI, why is bi-directional cross-filtering between Fact and Dimension tables strongly discouraged?

(a) It causes syntax errors in DAX.

(b) It introduces ambiguous relationship paths, degrades VertiPaq query performance, and can cause unexpected filter context propagation.

(c) Power BI does not support bi-directional filtering.

(d) It requires Excel licenses.

Q4: In Snowflake, what happens to underlying storage costs when a 10 TB database is cloned using `CREATE DATABASE dev CLONE prod;`?

(a) Storage cost immediately doubles (incurs an extra 10 TB charge).

(b) Zero extra storage cost initially; both databases share the same immutable micro-partitions via metadata pointers until modifications occur (Copy-on-Write).

(c) The production database is permanently deleted.

(d) Cloning requires downloading data to local disk.

Q5: What type of measure is a bank account balance in a Kimball Dimensional Model?

(a) Fully Additive

(b) Semi-Additive (cannot be summed across the Time dimension)

(c) Non-Additive

(d) Degenerate

Section B: Failure Debugging Scenarios (45 Marks)

Q6 (25 Marks - The Broken Nightly Load):

At 06:00 AM, your team discovers that the automated nightly pipeline failed at the dbt transformation stage with the error:

`dbt test failure: 14 failures found on unique_order_id in model stg_orders.`

- Explain how duplicate natural keys in the raw source system caused this failure. (10 marks)
- Detail the exact deduplication SQL transformation using `ROW_NUMBER()` that must be implemented in the staging layer to resolve this issue. (15 marks)

Q7 (20 Marks - Power BI Memory Degradation):

A Power BI report connected to Snowflake via Import Mode takes 25 minutes to refresh and consumes 4.2 GB of RAM on the Power BI Premium capacity. Investigation reveals a calculated column `Event_UUID` exists on the fact table. Explain why removing this single column will reduce model size by over 60%.

Section C: Systems Integration Challenge (30 Marks)

Q8 (30 Marks): Draw the architectural component diagram and write the end-to-end data flow specifications for an **AEC Construction Equipment Telemetry Platform** at vConstruct:

- IoT field equipment transmits GPS and engine idle-time metrics via REST API.
- Azure Data Factory ingests data incrementally into an ADLS Gen2 Bronze lake.
- Snowflake loads raw data via `COPY INTO` and dbt cleanses and models it into an equipment utilization Star Schema.
- Power BI presents an executive dashboard displaying the **Equipment Utilization Rate %** and **Idle Fuel Waste Cost ($)**.

Assessment Solutions & Scoring Guide

Section A Solutions

- **Q1: (b)** Secure bridge for on-premises to cloud data movement without inbound ports.
- **Q2: (b)** External source declaration, freshness testing, and lineage graph generation.
- **Q3: (b)** Ambiguous relationship paths and severe performance degradation.
- **Q4: (b)** Zero extra storage cost initially (Metadata Zero-Copy Cloning).
- **Q5: (b)** Semi-Additive.

Section B Solutions

Q6 Solution:

- **Root Cause:** Upstream source transactional systems updated order records without enforcing uniqueness in the extract window (e.g., an order was updated twice in the same batch, or network retries inserted duplicate payloads into the raw landing zone). When dbt executed its generic `unique` test on `order_id`, it detected multiple rows sharing the same key, halting the pipeline to prevent corrupted downstream financial metrics.
- **Deduplication SQL Pattern:**

```
WITH RankedRaw AS (
    SELECT
        order_id,
        customer_id,
        order_date,
        total_amount,
        updated_at,
        ROW_NUMBER() OVER (
            PARTITION BY order_id
            ORDER BY updated_at DESC, _ingested_at DESC
        ) AS rn
    FROM {{ source('raw', 'orders') }}
)
```

```
SELECT
    order_id,
    customer_id,
    order_date,
    total_amount,
    updated_at
FROM RankedRaw
WHERE rn = 1;
```

Q7 Solution:

- VertiPaq memory compression depends strictly on **column cardinality** (the number of unique values).
- An `Event_UUID` on a 20-million row fact table contains 20 million unique strings.
- This completely destroys dictionary encoding (creating a 20-million-entry string hash table) and renders Run-Length Encoding (RLE) and bit-packing completely useless.
- Because a UUID is never used as an analytical dimension or aggregation metric in dashboards, dropping it eliminates the dictionary table, reclaiming 60–80% of model RAM.

Section C Solutions

Q8 Component Architecture & Data Flow:

- **Ingestion Layer:**
IoT Gateway buffers telemetry into Azure Event Hubs / REST API.
ADF pipeline triggers an hourly Tumbling Window copying raw JSON payloads into ADLS Gen2 Bronze (`/bronze/equipment_telemetry/year=2024/...`).

- **Storage & Warehousing (Snowflake):**
Snowflake executes `COPY INTO bronze.raw_telemetry FROM @adls_telemetry_stage FILE_FORMAT = (TYPE = 'JSON');`.

- **dbt Transformation (Silver $\to$ Gold):**
`stg_telemetry`: Extracts JSON keys (`equipment_id`, `engine_status`, `fuel_rate`, `timestamp`).
`fct_equipment_daily_activity`: Aggregates operating hours, idle hours, and fuel burned per equipment per day.
`dim_equipment`: Equipment metadata (Model, Trade, Job Site, Hourly Rental Cost).

- **Power BI Executive Dashboard:**
Star Schema model (`fct_equipment_daily_activity` connected $1:*$ to `dim_equipment` and `dim_date`).
DAX Measures:

```
Utilization Rate % =
DIVIDE(SUM(fct_equipment_daily_activity[operating_hours]),
       SUM(fct_equipment_daily_activity[total_logged_hours]))

Idle Fuel Cost ($) =
SUM(fct_equipment_daily_activity[idle_fuel_gallons]) * 4.25
```

Day 26: Final Timed Comprehensive Technical Assessment

90-Minute Closed-Book Examination Across SQL, Python, DBMS, Statistics, ML & System Design

---

Examination Instructions & Protocol

Duration: 90 Minutes

Format: Closed-Book Proctored Technical Benchmark (No notes, no web searches, no AI assistance, no pausing the timer)

Total Marks: 100

Passing Threshold: 75 Marks (Interview Ready) | 90+ Marks (Tier-1 Tech Ready)

Structure:

- **Section 1: Advanced Relational SQL (30 Marks — 3 Problems)**
- **Section 2: Algorithmic Python & Data Wrangling (25 Marks — 2 Problems)**
- **Section 3: Database Internals, Concurrency & Indexing (15 Marks — 3 Problems)**
- **Section 4: Applied Statistics, Probability & Machine Learning (15 Marks — 3 Problems)**
- **Section 5: End-to-End System Design & Architecture Defense (15 Marks — 1 Problem)**

The Examination Paper

Section 1: Advanced Relational SQL (30 Marks)

Problem 1.1: Relative Ranking with Multi-Attribute Tiebreak (10 Marks)

Given tables `employees (emp_id INT, emp_name VARCHAR, dept_id INT, salary DECIMAL)` and `departments (dept_id INT, dept_name VARCHAR)`.

Write an ANSI SQL query to find the employees who earn the **top 3 distinct salaries in each department**.

- If multiple employees share the exact same salary, include all of them.
- Output columns: `dept_name`, `emp_name`, `salary`, and `salary_rank`.
- Order the final output by `dept_name` ascending, `salary_rank` ascending, and `emp_name` ascending.

Problem 1.2: Advanced Gaps & Islands Activity Streaks (10 Marks)

Given a website user activity log `user_activity (user_id INT, activity_timestamp TIMESTAMP)`.

Find all `user_id`s who were active on **at least 3 consecutive calendar days**.

- Note: A user may log multiple events in a single calendar day.
- Output columns: `user_id`, `streak_start_date`, `streak_end_date`, and `total_consecutive_days`.
- Order by `user_id` ascending, `streak_start_date` ascending.

Problem 1.3: Month-over-Month (MoM) Customer Retention Rate (10 Marks)

Given an e-commerce orders table `orders (order_id INT, customer_id INT, order_date DATE, amount DECIMAL)`.

Write an ANSI SQL query to calculate the **Month-over-Month Customer Retention Rate** for every calendar month:

- For each month M, calculate:
  `active_customers`: Total distinct customers who placed an order in month M.
  `retained_customers`: Distinct customers who placed an order in month M **AND** also placed an order in the immediately preceding month ($M-1$).
  `retention_rate_pct`: Percentage of month $M-1$'s active customers who returned in month M, rounded to 2 decimal places.

Section 2: Algorithmic Python & Data Wrangling (25 Marks)

Problem 2.1: Longest Substring with At Most K Distinct Characters (12 Marks)

Write a Python function `length_of_longest_substring_k_distinct(s: str, k: int) -> int` that returns the length of the longest contiguous substring that contains at most k distinct characters.

- Constraints: $0 \le k \le 50$, s contains standard ASCII characters.
- Your solution must run in strictly **$O(N)$ time complexity** using the Sliding Window pattern with a hash map.

Problem 2.2: Advanced Clickstream Sessionization in Pandas (13 Marks)

You are given a Pandas DataFrame `clicks` with columns `['user_id', 'timestamp', 'page_url', 'action']`.

An active user session is defined as a sequence of clicks where the time elapsed between consecutive clicks is **less than 20 minutes**.

Write a Python function `sessionize_clicks(clicks: pd.DataFrame) -> pd.DataFrame` that:

- Assigns a unique `session_id` string (e.g., `'user101_s1'`) to every row.

- Returns an aggregated summary DataFrame grouped by `user_id` containing:
`total_sessions`: Count of unique sessions.
`avg_clicks_per_session`: Average number of clicks per session, rounded to 1 decimal place.
`bounced_sessions`: Count of sessions containing exactly 1 click.

Section 3: Database Internals, Concurrency & Indexing (15 Marks)

Problem 3.1: The Composite Index Left-Prefix Rule (5 Marks)

A database table `transactions` has 50 million rows and a composite B-tree index:
`CREATE INDEX idx_txns_composite ON transactions (account_id, txn_date, txn_type);`

Evaluate whether the following queries can utilize an **Index Seek** using this index, and justify your answer for each:

- `SELECT * FROM transactions WHERE account_id = 1045 AND txn_type = 'WIRE';`
- `SELECT * FROM transactions WHERE txn_date >= '2024-01-01' AND txn_type = 'WIRE';`

Problem 3.2: Isolation Level Read Phenomena Matrix (5 Marks)

Fill in the following concurrency matrix with **YES** (Permitted) or **NO** (Eliminated) according to ANSI SQL standards:

Isolation Level	Dirty Read Permitted?	Non-Repeatable Read Permitted?	Phantom Read Permitted?
Read Committed			
Repeatable Read			
Serializable			

Problem 3.3: Deadlock Detection & Deterministic Lock Ordering (5 Marks)

Explain how the database engine's **Wait-For Graph (WFG)** detects deadlocks. Provide the single application-level engineering practice that permanently eliminates circular wait deadlocks.

Section 4: Applied Statistics & Machine Learning (15 Marks)

Problem 4.1: Bayesian Diagnostic Paradox & Base Rate Fallacy (5 Marks)

A rare disease affects $0.05%$ (0.0005) of a population. A diagnostic blood test has:

- **Sensitivity (True Positive Rate):** $98%$ (0.98)

- **False Positive Rate:** $2%$ (0.02)

A randomly tested individual receives a positive result. Calculate the **exact probability** that they actually have the disease. Show all formula steps.

Problem 4.2: Asymmetric Cost Matrix & Decision Threshold Optimization (5 Marks)

In an enterprise credit card fraud model:

- Cost of a **False Negative (FN)** (uncaught fraud): $$250$
- Cost of a **False Positive (FP)** (declining a legitimate customer): $$10$

Explain why deploying this model with a default probability threshold of 0.50 is financially suboptimal, and explain the mathematical direction in which the threshold must be adjusted to minimize total expected dollar loss.

Problem 4.3: The Bias-Variance Trade-Off & Regularization (5 Marks)

Compare **Ridge (L2)** and **Lasso (L1)** regularization. Explain why Lasso is capable of performing automated feature selection by setting weights strictly to zero, whereas Ridge only shrinks weights asymptotically toward zero.

Section 5: End-to-End System Design & Architecture Defense (15 Marks)

Problem 5.1: The Modern Data Stack & Pipeline Resilience (15 Marks)

You are hired as a Lead Data Science Engineer to design a real-time analytics platform for **vConstruct**.

The system must ingest sensor telemetry and construction progress updates from 50 global job sites:

- **Draw the Architectural Diagram / Component Flow** from raw ingestion to executive dashboard (IoT/API $\to$ Ingestion Buffer $\to$ Raw Lake $\to$ Cloud Warehouse $\to$ dbt Modeling $\to$ Power BI). (5 marks)
- **Defend Storage & Compute Decoupling:** Why is Snowflake's virtual warehouse architecture superior to a monolithic PostgreSQL server for this workload? (5 marks)
- **Disaster Recovery & Failure Handling:** How does your architecture handle upstream schema changes and corrupted records without halting the entire daily reporting pipeline? (5 marks)

Official Answer Key & Scoring Guide

Section 1: SQL Solutions (30 Marks)

Solution 1.1 (10 Marks): Top-N per Department with Ties

```
WITH RankedEmployees AS (
    SELECT
        d.dept_name,
        e.emp_name,
        e.salary,
        DENSE_RANK() OVER (
            PARTITION BY e.dept_id
            ORDER BY e.salary DESC
        ) AS salary_rank
    FROM employees e
    INNER JOIN departments d ON e.dept_id = d.dept_id
)
SELECT
    dept_name,
    emp_name,
    salary,
    salary_rank
FROM RankedEmployees
WHERE salary_rank <= 3
ORDER BY dept_name ASC, salary_rank ASC, emp_name ASC;
```

Grading Rubric:

- 4 marks: Correct use of `DENSE_RANK()` with `PARTITION BY dept_id ORDER BY salary DESC`.
- 2 marks: Correct join between `employees` and `departments`.
- 2 marks: Proper outer `WHERE salary_rank <= 3` filter.
- 2 marks: Correct ordering clause.

Solution 1.2 (10 Marks): Gaps & Islands Activity Streaks

```
WITH DistinctDays AS (
    SELECT DISTINCT
        user_id,
        CAST(activity_timestamp AS DATE) AS act_date
    FROM user_activity
),
SequencedDays AS (
    SELECT
        user_id,
        act_date,
        act_date - CAST(ROW_NUMBER() OVER (
            PARTITION BY user_id
            ORDER BY act_date
        ) AS INT) * INTERVAL '1 day' AS island_key
    FROM DistinctDays
)
SELECT
    user_id,
    MIN(act_date) AS streak_start_date,
    MAX(act_date) AS streak_end_date,
    COUNT(*) AS total_consecutive_days
FROM SequencedDays
GROUP BY user_id, island_key
HAVING COUNT(*) >= 3
ORDER BY user_id ASC, streak_start_date ASC;
```

Grading Rubric:

- 3 marks: Deduplicating timestamps to calendar dates via `DISTINCT`.
- 4 marks: Applying the Difference of Sequences technique (`act_date - ROW_NUMBER()`).
- 3 marks: Grouping by `island_key` and filtering with `HAVING COUNT(*) >= 3`.

Solution 1.3 (10 Marks): MoM Customer Retention Rate

```sql
WITH MonthlyActiveUsers AS (
    SELECT DISTINCT
        customer_id,
        DATE_TRUNC('month', order_date) AS order_month
    FROM orders
),
MonthlyCohorts AS (
    SELECT
        m1.order_month,
        COUNT(DISTINCT m1.customer_id) AS active_customers,
        COUNT(DISTINCT m2.customer_id) AS retained_customers
    FROM MonthlyActiveUsers m1
    LEFT JOIN MonthlyActiveUsers m2
        ON m1.customer_id = m2.customer_id
       AND m1.order_month = m2.order_month + INTERVAL '1 month'
    GROUP BY m1.order_month
)
SELECT
    order_month,
    active_customers,
    retained_customers,
    ROUND(
        100.0 * retained_customers / NULLIF(LAG(active_customers) OVER (ORDER BY order_month), 0),
        2
    ) AS retention_rate_pct
FROM MonthlyCohorts
ORDER BY order_month;
```

Grading Rubric:

- 3 marks: Isolating unique monthly active users per month.
- 4 marks: Self-join on `m1.order_month = m2.order_month + INTERVAL '1 month'`.
- 3 marks: Using `LAG()` to compute percentage relative to previous month with `NULLIF` divide-by-zero protection.

Section 2: Python & Data Wrangling Solutions (25 Marks)

Solution 2.1 (12 Marks): Longest Substring with At Most K Distinct Characters

```python
def length_of_longest_substring_k_distinct(s: str, k: int) -> int:
    if k == 0 or not s:
        return 0

    char_counts = {}
    max_len = 0
    left = 0

    for right, char in enumerate(s):
        char_counts[char] = char_counts.get(char, 0) + 1

        # Contract window if distinct character count exceeds k
```

```
        while len(char_counts) > k:
            left_char = s[left]
            char_counts[left_char] -= 1
            if char_counts[left_char] == 0:
                del char_counts[left_char]
            left += 1

        max_len = max(max_len, right - left + 1)

    return max_len
```

Grading Rubric:

- 4 marks: Correct sliding window structure with `left` and `right` pointers.
- 4 marks: Proper hash map frequency updates and key deletion when count reaches 0.
- 4 marks: Accurate window length calculation and handling edge cases ($k=0$, empty string).

Solution 2.2 (13 Marks): Clickstream Sessionization

```
import pandas as pd

def sessionize_clicks(clicks: pd.DataFrame) -> pd.DataFrame:
    df = clicks.copy()
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.sort_values(["user_id", "timestamp"]).reset_index(drop=True)

    # Calculate gap to previous click
    time_diff = df.groupby("user_id")["timestamp"].diff()
    new_session = time_diff.isna() | (time_diff >= pd.Timedelta(minutes=20))

    # Sequential session numbering per user
    df["session_idx"] = df.groupby("user_id")[new_session.name].cumsum()
    df["session_id"] = df["user_id"].astype(str) + "_s" + df["session_idx"].astype(str)

    # Session level metrics
    session_stats = df.groupby(["user_id", "session_id"]).agg(
        click_count=("page_url", "count")
    ).reset_index()

    # User level report
    user_report = session_stats.groupby("user_id").agg(
        total_sessions=("session_id", "count"),
        avg_clicks_per_session=("click_count", lambda x: round(x.mean(), 1)),
        bounced_sessions=("click_count", lambda x: (x == 1).sum())
    ).reset_index()

    return user_report
```

Grading Rubric:

- 4 marks: Correctly identifying session boundaries via `.diff() >= 20 mins`.
- 4 marks: Generating cumulative session IDs via `.cumsum()`.
- 5 marks: Accurate two-stage aggregation producing `total_sessions`, `avg_clicks`, and `bounced_sessions`.

Section 3: Database Internals Solutions (15 Marks)

Solution 3.1 (5 Marks): Left-Prefix Rule

- **Query 1 (account_id = 1045 AND txn_type = 'WIRE'): PARTIAL SEEK.**
The engine uses `account_id` to seek directly to matching leaf pages ($O(\log N)$). However, because `txn_date` was skipped, it cannot seek on `txn_type`; it scans the remaining matching rows in memory to evaluate `txn_type`.

- **Query 2 (txn_date >= '2024-01-01' AND txn_type = 'WIRE'): NO SEEK (FULL SCAN).**
The query completely omits the leading column `account_id`. The B-tree is sorted primarily by `account_id`, so the engine cannot navigate the tree hierarchy and must resort to a full index or table scan.

Solution 3.2 (5 Marks): Isolation Level Matrix

Isolation Level	Dirty Read Permitted?	Non-Repeatable Read Permitted?	Phantom Read Permitted?
Read Committed	**NO**	**YES**	**YES**
Repeatable Read	**NO**	**NO**	**YES** (Standard ANSI) / **NO** (Postgres/InnoDB)
Serializable	**NO**	**NO**	**NO**

Solution 3.3 (5 Marks): Deadlock Detection & Resolution

- **Wait-For Graph:** The engine maintains a directed graph where transactions are nodes and locks are edges ($T_1 \to T_2$ means T_1 waits for T_2). A background thread traverses the graph; if a **directed cycle** is detected ($T_1 \to T_2 \to T_1$), a deadlock exists, and the engine aborts the lowest-cost victim transaction.
- **Prevention Practice: Deterministic Lock Ordering**: all application transactions must acquire locks on resources in the exact same sorted numerical sequence (e.g., sorting primary keys before executing `SELECT ... FOR UPDATE`), mathematically eliminating circular wait conditions.

Section 4: Statistics & Machine Learning Solutions (15 Marks)

Solution 4.1 (5 Marks): Bayes' Theorem Proof

$$P(D) = 0.0005, \quad P(H) = 0.9995$$
$$P(+ \mid D) = 0.98, \quad P(+ \mid H) = 0.02$$

$$P(+) = (0.98 \times 0.0005) + (0.02 \times 0.9995) = 0.00049 + 0.01999 = 0.02048$$

$$P(D \mid +) = \frac{0.98 \times 0.0005}{0.02048} = \frac{0.00049}{0.02048} \approx 0.0239 \quad (\mathbf{2.39\%})$$

- *Conclusion:* Even with a 98% sensitive test, an individual who tests positive has only a **2.39% chance** of actually having the disease due to the extremely rare base rate (0.05%).

Solution 4.2 (5 Marks): Asymmetric Cost Optimization

- **Why 0.5 is Suboptimal:** A default 0.5 threshold assumes symmetric error costs ($C(FN) = C(FP)$). In this scenario, missing a fraudulent transaction (FN) costs 250, which is **$25\times$ more expensive** than a false alarm ($FP = 10).
- **Direction of Adjustment:** The decision threshold must be **lowered significantly** (e.g., from 0.50 down to $0.08–0.15$). This forces the model to flag transactions as fraud even with low probability, deliberately trading off more false positives ($10) to eliminate catastrophic false negatives ($250).

Solution 4.3 (5 Marks): Ridge vs. Lasso Regularization

- **L1 (Lasso):** Uses an absolute value penalty ($\sum |w_j|$), which creates a **diamond-shaped constraint boundary** with sharp corners on the coordinate axes. Loss contours frequently hit these corners directly on the axes, setting non-informative feature weights **strictly to zero** (performing automatic feature selection).
- **L2 (Ridge):** Uses a squared penalty ($\sum w_j^2$), creating a smooth **circular/spherical constraint boundary**. Loss contours touch the circle along continuous curves, shrinking weights close to zero but never reaching exact zero.

Section 5: System Design & Architecture Solutions (15 Marks)

Solution 5.1 (15 Marks): Enterprise IoT Analytics Platform

- **Architecture Blueprint (5 marks):**

Ingestion: IoT Gateways buffer telemetry $\to$ Azure Event Hubs / Kafka $\to$ Azure Data Factory tumbling window jobs.
Storage: Raw JSON landed immutably in ADLS Gen2 Bronze lake.
Warehousing: Snowflake loads Bronze via `COPY INTO`.
Transformation: dbt executes scheduled runs compiling Silver conformed tables and Gold Star Schema models (`fct_equipment_telemetry`, `dim_job_site`, `dim_equipment`).
BI Layer: Power BI connects to Gold tables in Import Mode; DAX measures calculate utilization and idle fuel costs.

- **Storage & Compute Decoupling Defense (5 marks):**

Independent Virtual Warehouses eliminate resource contention: heavy batch transformations running on `ETL_WH` do not degrade interactive Power BI executive queries running on `BI_WH`.
Per-second billing with Auto-Suspend ensures compute clusters shut down when idle, saving up to 70% in cloud infrastructure costs compared to an always-on monolithic PostgreSQL instance.

- **Disaster Recovery & Failure Handling (5 marks):**
  Ingestion pipelines implement a **Quarantine Pattern / Dead Letter Queue (DLQ)**: schema-corrupted sensor payloads are routed to an error table while clean records process uninterrupted.
  Raw data in Bronze ADLS Gen2 is append-only and immutable, allowing full replay from scratch if transformation logic requires updates.

Day 27: Final Light Revision & Day-Before Interview Readiness

High-Yield Recall Sheets, Common Pitfalls, Behavioral Anchors & Environmental Setup

---

The Golden Rule of Day 27

> *[!IMPORTANT]*
> ***DO NOT START ANY NEW ADVANCED TOPICS TODAY.***
>
> *Today is dedicated entirely to* ***consolidation, high-yield pattern recall, behavioral anchoring, and logistical readiness****. Attempting to learn complex new algorithms on the day before an interview increases cognitive fatigue and erodes confidence. Review the distilled cheat sheets below, review your personal mistake log, verify your physical environment, and rest.*

Part 1: The Ultimate SQL & Relational DBMS Fast-Recall Sheet

1.1 The SQL Execution Pipeline

```
1. FROM / ON / JOIN ──> 2. WHERE ──> 3. GROUP BY ──> 4. HAVING ──>
5. SELECT (Projection + Window Functions) ──> 6. DISTINCT ──> 7. ORDER BY ──> 8. LIMIT
```

Why it matters: You cannot use aliases defined in `SELECT` inside `WHERE` or `GROUP BY` because those clauses execute earlier in the pipeline!

1.2 The Core SQL Patterns

- **Second-Highest Salary:**
```
WITH R AS (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) as rnk FROM employees)
SELECT COALESCE((SELECT DISTINCT salary FROM R WHERE rnk = 2), NULL);
```

- **Top-N per Group (with ties):**
```
DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) <= N
```

- **Gaps & Islands (Consecutive Days):**
```
-- Date - Sequential Integer = CONSTANT ANCHOR DATE for consecutive streaks!
activity_date - CAST(ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY activity_date) AS INT) * INTERVAL '1 day'
```

- **Left Anti-Join (Unmatched Records):**
```
SELECT a.* FROM table_a a LEFT JOIN table_b b ON a.id = b.id WHERE b.id IS NULL;
```

- **Running Total (Cumulative):**`SUM(val) OVER (ORDER BY dt ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`

- **7-Day Rolling Average:**`AVG(val) OVER (ORDER BY dt ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)`

1.3 Database Storage & Concurrency Anchors

- **B+ Tree Index:** $O(\log N)$. Internal nodes guide traversal; all records/pointers live in sequentially linked leaf disk pages. Excels at range scans and pre-sorted output (`ORDER BY`).
- **Composite Index Left-Prefix Rule:** An index on `(A, B, C)` supports queries on `A`, `(A, B)`, and `(A, B, C)`, but **cannot** seek on `B` or `C` alone!
- **Covering Index:** Non-key attributes in `INCLUDE` reside in leaf nodes, satisfying queries via **Index-Only Scans** and eliminating expensive Key Lookups.
- **Deadlock Resolution:** Enforce **Deterministic Lock Ordering**: always acquire row locks in strict ascending primary key order across all transactions.

Part 2: The Python, Pandas & Data Engineering Fast-Recall Sheet

2.1 Python Built-In Complexities

- `list`: `append()` $O(1)$ amortized, `pop()` from end $O(1)$, `insert(0, x)` $O(n)$, `search` $O(n)$.
- `dict` / `set`: Average lookup, insertion, and deletion are **$O(1)$**; worst-case $O(n)$ under hash collisions.
- `collections.deque`: $O(1)$ append and pop from **both ends**.
- `sorted()` / `.sort()`: **Timsort** ($O(n \log n)$ worst-case, $O(n)$ best-case, guaranteed stable).

2.2 Core Algorithmic Code Templates

- **Dynamic Sliding Window (Longest Substring):**`last_seen = {}`

```
left = max_len = 0
for right, char in enumerate(s):
    if char in last_seen and last_seen[char] >= left:
        left = last_seen[char] + 1
    last_seen[char] = right
    max_len = max(max_len, right - left + 1)
```

- **Subarray Sum Equals K (Prefix Sums):**`counts = {0: 1}`

```
running_sum = total = 0
for num in nums:
    running_sum += num
    total += counts.get(running_sum - k, 0)
```

```
    counts[running_sum] = counts.get(running_sum, 0) + 1
```

- **Two Pointers on Sorted Array:**
```
left, right = 0, len(nums) - 1
while left < right:
    s = nums[left] + nums[right]
    if s == target: return [left, right]
    elif s < target: left += 1
    else: right -= 1
```

2.3 Pandas Idioms

- **Avoid Chained Indexing:** Always use atomic assignment: `df.loc[mask, 'col'] = val`.
- **Emulating SQL Window Functions:** `df['dept_avg'] = df.groupby('dept')['salary'].transform('mean')`.
- **Sessionization:** `df['session_id'] = (df.groupby('user')['ts'].diff() > pd.Timedelta(minutes=30)).cumsum()`.

2.4 Cloud Warehousing & Orchestration

- **Snowflake:** Decoupled storage (micro-partitions) and compute (Virtual Warehouses). Zero-Copy Cloning duplicates metadata pointers without copying data blocks.
- **Airflow:** DAGs authored as Python. Always set `catchup=False` to prevent backfilling hundreds of historical runs. Bitshift operator: `task_a >> task_b`.
- **dbt:** Transforms raw data inside the warehouse (ELT). `{{ ref('model') }}` automatically compiles dependencies and environment routing.

Part 3: Statistics, Machine Learning & Power BI Fast-Recall Sheet

3.1 Statistics & Bayes' Theorem

- **Skewness:** Right Skew = $\text{Mean} > \text{Median} > \text{Mode}$. Outliers pull the mean toward the tail.
- **Bessel's Correction:** Divide sample variance by $n-1$ to produce an unbiased estimator of σ^2.
- **Bayes' Formula:**
$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$
- **Base Rate Fallacy:** When the base rate $P(\text{Fraud})$ is rare (e.g., $0.1%$), most positive test alarms are false positives, even with a 99% accurate model!

3.2 Machine Learning Metrics & Trade-Offs

- **Precision:** $\frac{TP}{TP + FP}$ (Cost of False Alarm is high: spam, recommendations).
- **Recall:** $\frac{TP}{TP + FN}$ (Cost of Missed Positive is high: cancer, fraud).
- **F_1-Score:** Harmonic mean $2 \cdot \frac{P \cdot R}{P + R}$. Penalizes extreme trade-offs.

- **Severe Class Imbalance: Use PR-AUC**, not ROC-AUC! High True Negatives artificially mask poor precision in ROC-AUC.
- **Data Leakage: NEVER** fit scalers or imputers on the full dataset before splitting! Encapsulate in `Pipeline`.

3.3 Power BI & DAX

- **Kimball Star Schema:** Central Fact table ($*$) connected via single-direction relationships to denormalized Dimension tables (1).
- **Calculated Column vs. Measure:** Calculated columns consume RAM; Measures are evaluated dynamically at query time based on visual **Filter Context**.
- **CALCULATE():** The only DAX function that modifies Filter Context; triggers **Context Transition** inside row contexts.
- **ALL() vs. ALLSELECTED():** `ALL()` strips all filters (Grand Total); `ALLSELECTED()` strips visual row headers while preserving external slicer selections.

Part 4: Behavioral & Executive Communication Anchors

4.1 Your 90-Second Professional Pitch

> *"I am a Data Science Engineer specializing in building scalable relational pipelines, analytical data warehouses, and applied machine learning systems.*
> *My core expertise lies in transforming fragmented, messy operational data into high-performance analytical assets using SQL, Python, Snowflake, and Apache Airflow. For example, I recently architected a real-time event aggregation engine processing over 50,000 hourly events with sub-second vector search retrieval, reducing compute costs by 62%.*
> *What brings me to vConstruct today is the opportunity to apply this rigorous data engineering foundation to the digital transformation of construction and virtual design workflows."*

4.2 Why vConstruct? (The 30-Second Punchline)

> *"vConstruct is digitizing one of the largest physical industries in the world: Architecture, Engineering, and Construction (13% of global GDP). While most tech roles optimize ad clicks, vConstruct solves tangible real-world problems—preventing schedule slippage and material waste through BIM and VDC integration. My background in resilient ELT pipelines and predictive modeling directly aligns with your data platform mission."*

Part 5: The 10 Most Fatal Technical Traps to Avoid Tomorrow

- **The = NULL Trap:** Never write `WHERE col = NULL`. Always write `WHERE col IS NULL`.
- **The NOT IN (NULL) Trap:** If a subquery returns a single `NULL`, `NOT IN` returns zero rows. Always use `NOT EXISTS` or `LEFT JOIN ... WHERE key IS NULL`.

- **The Outer Join WHERE Trap:** Placing a right-table filter in the `WHERE` clause of a `LEFT JOIN` silently converts it into an `INNER JOIN`.
- **The RANK() vs. DENSE_RANK() Trap:** If looking for the second-highest salary with ties, `RANK()` skips ($1, 1, 3$). Always use `DENSE_RANK()` ($1, 1, 2$).
- **The Chained Indexing Trap:** Never write `df[mask]['col'] = val`. Always write `df.loc[mask, 'col'] = val`.
- **The Mutable Default Argument Trap:** Never write `def f(x=[]):`. Always write `def f(x=None): if x is None: x = []`.
- **The Global Scaling Leakage Trap:** Never run `StandardScaler().fit_transform(X)` before `train_test_split()`.
- **The Silent Coding Trap:** Never write code in silence during live interviews. State your algorithm, constraints, and time complexity upfront.
- **The Defensive Answer Trap:** When an interviewer challenges your design, never become defensive. Acknowledge the constraint and reason from first principles.
- **The Unquantified STAR Trap:** Never end a behavioral story without a measurable metric ($62%$ latency reduction, zero downtime, $$2.4\text{M}$ GMV lift).

Part 6: The Day-Before Logistics & Environmental Checklist

Execute this physical and technical verification protocol today:

- [ ] **1. Computer Hardware & OS:** Reboot your laptop. Disable background OS updates, Slack notifications, and torrent/sync clients.
- [ ] **2. Web Camera & Lighting:** Position your camera at eye level. Ensure primary light is in front of you, not behind you (avoid backlighting).
- [ ] **3. Microphone & Audio:** Test your external headset/microphone. Confirm input/output devices are selected in Zoom/Teams/Google Meet.
- [ ] **4. Internet Failover:** Confirm primary Wi-Fi speed. Have your mobile phone configured as an instant **Personal Hotspot** ready to connect in 10 seconds if home internet drops.
- [ ] **5. Browser & Environment:** Close all personal tabs. Have a clean, distraction-free browser window open. Ensure your desktop is clear.
- [ ] **6. Paper & Pen:** Have a clean physical notepad and pen on your desk for taking notes during multi-part interviewer questions.
- [ ] **7. Documentation & Resume:** Have a physical printed copy or separate clean window with your resume, portfolio links, and government ID ready.
- [ ] **8. Sleep & Nutrition:** Stop studying by 20:00. Hydrate, eat a balanced dinner, and get at least 7.5 to 8 hours of sleep. A sharp, well-rested brain outperforms a tired, over-caffeinated brain every single time.

Part 7: Final Words of Encouragement

You have completed an exhaustive, rigorous 27-day preparation journey:

- You have mastered foundational to advanced ANSI SQL, window framing, and execution internals.
- You have conquered B-Tree storage, ACID guarantees, MVCC, and deadlock mechanics.
- You have written production Python algorithms, data structures, and streaming generators in $O(1)$ memory.
- You have designed end-to-end cloud ELT pipelines in Snowflake, Airflow, and dbt.
- You have modeled clinical AI in MediCLIP and real-time streaming in your AI News Engine.
- You have practiced under timed mock pressure and polished your executive communication.

Trust your preparation, breathe deeply, communicate clearly, and go claim your Data Science Engineer offer.